Contents

1 Programming Helpers **4**
1.1 Function Quick Reference . . . 4
1.2 C++17 Function Examples . . . 5
1.3 Map Operators to Functions . . . 8
1.4 Comparators . . . 8
1.5 Scientific Notation . . . 9
1.6 Formatted Output . . . 10
1.7 String Input and getline . . . 10
1.8 Plotting with Matplotlib (Python) . . . 12

2 Compile **13**
2.1 Compile . . . 13
2.2 Template . . . 13

3 Data Structures **13**
3.1 BIT . . . 13
3.2 Bitset . . . 13
3.3 Bit Trie . . . 14
3.4 CHMIN/CHMAX Rangesum GCD Queries . . . 14
3.5 Disjoint Sparse Table . . . 18
3.6 Disjoint Set Union Bipartite . . . 19
3.7 Disjoint Set Union . . . 19
3.8 Disjoint Set Union Queue Like Rollback . . . 20
3.9 Dynamic Conectivity . . . 23
3.10 Fenwick Tree . . . 25
3.11 Fenwick Tree 2D . . . 26
3.12 Li Chao Tree . . . 26
3.13 Link Cut Tree . . . 29
3.14 Linked List . . . 32
3.15 Merge Sort Tree . . . 33
3.16 Minimum Cartesian Tree . . . 33
3.17 Multi Ordered Set . . . 34
3.18 Ordered Set . . . 34
3.19 Palindromic Tree . . . 35
3.20 Persistent Array . . . 36
3.21 Persistent Segment Tree . . . 37
3.22 Polynomial Queries . . . 38
3.23 Segment Tree . . . 40
3.24 Segment Tree Examples . . . 40
3.25 Segment Tree 2D . . . 46
3.26 Segment Tree Dynamic . . . 47
3.27 Segment Tree Lazy Types . . . 48
3.28 Segment Tree Lazy . . . 48
3.29 Segment Tree Lazy Range Set . . . 49
3.30 Segment Tree Lazy Range Set 2 . . . 51
3.31 Segment Tree Max Subarray Sum . . . 53
3.32 Segment Tree Range Update . . . 53
3.33 Segment Tree Struct Types . . . 54
3.34 Segment Tree Struct . . . 54
3.35 Segment Tree Walk . . . 55
3.36 Sparse Table . . . 56
3.37 Sparse Table 2D . . . 57
3.38 Square Root Decomposition . . . 58
3.39 Tuple . . . 59
3.40 Treap . . . 60
3.41 Treap 2 . . . 61
3.42 Treap With Inversion . . . 62
3.43 Treap With Inversions and Range Updates . . . 63

4 Dynamic Programming **65**
4.1 Monotone CHT . . . 65
4.2 CHT with Counts . . . 67
4.3 CHT Deque . . . 68
4.4 Digit DP . . . 69
4.5 Divide and Conquer DP . . . 70
4.6 Edit Distance . . . 70
4.7 Knuth's Algorithm . . . 71
4.8 LCS . . . 71
4.9 Line Container . . . 72
4.10 Longest Increasing Subsequence . . . 73
4.11 SOS DP . . . 73
4.12 Subset Sum, Knapsack and Max-Plus: Six Variants . . . 74
4.12.1 1. All Subset Sums: Compress Repeated Weights . . . 74
4.12.2 2. One Subset-Sum Target: Balanced States in O(ND) . . . 75
4.12.3 3. Bounded Knapsack: Residues and a Monotone Deque . . . 76
4.12.4 4. Max-Plus Convolution with Concavity . . . 77
4.12.5 5. 0/1 Knapsack with Few Distinct Weights . . . 79
4.12.6 6. Unbounded Knapsack with Small Weights, Huge Capacity . . . 80
4.13 NDPC N: Exact-Fill Knapsack with Many Huge Queries . . . 81

5 Flow **83**
5.1 Dinic . . . 83
5.2 Hopcroft-Karp . . . 84
5.3 Hungarian . . . 86

5.4 Max Flow Min Cost . . . 86
5.5 Max Flow . . . 88
5.6 Min Cost Max Flow . . . 88
5.7 Push Relabel . . . 90

6 Geometry **91**
6.1 Point Struct . . . 91
6.2 Sort Points . . . 91
6.3 Point Struct2 . . . 91
6.4 Find Triangle With Given Area . . . 92
6.5 Antipodal Pairs . . . 94
6.6 Area and Perimeter . . . 94
6.7 Area Union Circles . . . 94
6.8 Centroid . . . 95
6.9 Circle Inside Circle . . . 95
6.10 Circle Outside Circle . . . 95
6.11 Closest Pair of Points . . . 96
6.12 Common Tangents . . . 96
6.13 Convex Hull . . . 96
6.14 Segment Intersection with Ray . . . 96
6.15 Cut Polygon . . . 96
6.16 Delaunay Triangulation . . . 97
6.17 Diameter and Width . . . 99
6.18 Distance Between Point and Circle . . . 99
6.19 Distance Between Point and Line . . . 99
6.20 Example of Geometry . . . 99
6.21 Half Plane Intersection . . . 100
6.22 Incircle . . . 101
6.23 Intersection of Two Circles . . . 101
6.24 Intersection Line and Circle . . . 101
6.25 Intersection Polygon and Circle . . . 102
6.26 Intersection Segment and Circle . . . 102
6.27 Line Intersection . . . 102
6.28 Minkowski Sum . . . 102
6.29 Point in Circle . . . 103
6.30 Point in Convex Hull . . . 103
6.31 Point in Line . . . 103
6.32 Point in Perimeter . . . 103
6.33 Point in Polygon . . . 103
6.34 Point in Segment . . . 104
6.35 Points Tangency . . . 104
6.36 Projection Point Circle . . . 104
6.37 Segment Intersection . . . 104
6.38 Smallest Enclosing Circle . . . 104
6.39 Smallest Enclosing Rectangle . . . 105
6.40 Vantage Point Tree . . . 105

7 Graphs **106**
7.1 2Sat . . . 106
7.2 Articulation Points . . . 107
7.3 Bellman-Ford . . . 108
7.4 Bipartite Checker . . . 109
7.5 Bipartite Maximum Matching . . . 109
7.6 Bipartite Minimum Maximum Matching2 . . . 110
7.7 Block Cut Tree . . . 111
7.8 Blossom . . . 112
7.9 Bridges . . . 115
7.10 Bridges Online . . . 115
7.11 Dijkstra . . . 117
7.12 Eulerian Path . . . 117
7.13 Floyd-Warshall . . . 118
7.14 Kruskal . . . 118
7.15 Marriage . . . 119
7.16 SCC . . . 120
7.17 Stoer-Wagner . . . 121

8 Linear Algebra **122**
8.1 Gaussian Elimination . . . 122
8.2 Gaussian Elimination Modulo . . . 122
8.3 Simplex . . . 123

9 Math **125**
9.1 BinPow . . . 125
9.2 Combination Rank . . . 125
9.3 Chinese Remainder Theorem . . . 126
9.4 Diophantine . . . 126
9.5 Discrete Logarithm . . . 127
9.6 Divisors . . . 127
9.7 Euler Totient (Phi) . . . 128
9.8 Fibonacci . . . 128
9.9 Fraction: Exact Rational Arithmetic . . . 128
9.10 Matrix Exponentiation . . . 130
9.11 Miller Rabin Deterministic . . . 131
9.12 Mobius . . . 131
9.13 Permutation Rank . . . 131
9.14 Prefix Sum Phi . . . 132

9.15 Sieve . . . 132
9.16 Identities . . . 133
9.17 Burnside's Lemma . . . 133
9.18 Recursion . . . 133
9.19 Theorems . . . 133
9.20 Sums . . . 134
9.21 Catalan numbers . . . 134
9.22 Cayley's formula . . . 134
9.23 Geometric series . . . 134
9.24 Estimates For Divisors . . . 134
9.25 Sum of divisors . . . 134
9.26 Pythagorean Triplets . . . 134
9.27 Derangements . . . 134

10 Game Theory **134**
10.1 Sprague-Grundy theorem . . . 134

11 Fórmulas y notas **135**
11.1 Números de Stirling del primer tipo . . . 135
11.2 Números de Stirling del segundo tipo . . . 135
11.3 Números de Euler . . . 135
11.4 Números de Catalan . . . 135
11.5 Números de Bell . . . 135
11.6 Números de Bernoulli . . . 135
11.7 Fórmula de Faulhaber . . . 136
11.8 Función Beta . . . 136
11.9 Función zeta de Riemann . . . 136
11.10Funciones generadoras . . . 136
11.11Números armónicos . . . 136
11.12Aproximación de Stirling . . . 136
11.13Ternas pitagóricas . . . 136
11.14Árbol de Stern–Brocot . . . 137
11.15Combinatoria . . . 137
11.16Grafos . . . 137
11.17Teoría de números . . . 138
11.18Primos . . . 139
11.19Números primos de Mersenne . . . 139
11.20Números primos de Fermat . . . 139
11.21Binomial Inversion . . . 139
11.22Generating Functions . . . 140

12 More Topics **144**
12.1 2D Prefix Sum . . . 144
12.2 CDQ D&C FFT . . . 144
12.3 CDQ Divide and Conquer . . . 144
12.4 Count Segments Around Index . . . 146
12.5 Day of the Week . . . 146
12.6 Directed MST . . . 146
12.7 GCD Convolution . . . 148
12.8 int128 . . . 149
12.9 Iterating Over All Subsets . . . 149
12.10LCM Convolution . . . 149
12.11Manhattan MST . . . 150
12.12Max Manhattan Distance . . . 151
12.13Matroid Intersection . . . 151
12.14Matroid Bipartite . . . 152
12.15Matroid Colorful . . . 152
12.16Matroid Graphic . . . 153
12.17Matroid Partition . . . 153
12.18Matroid Prime Field . . . 153
12.19Matroid Truncated . . . 154
12.20Matroid XOR . . . 155
12.21Mo . . . 155
12.22MOD INT . . . 156
12.23Next Permutation . . . 156
12.24Next and Previous Smaller/Greater Element . . . 157
12.25Parallel Binary Search . . . 157
12.26Random Number Generators . . . 158
12.27Ternary Search . . . 158
12.28Ternary Search Int . . . 158
12.29XOR Convolution . . . 158
12.30XOR Basis . . . 160
12.31XOR Basis Online . . . 160

13 Polynomials **164**
13.1 Berlekamp Massey . . . 164
13.2 Binary Exp FFT . . . 165
13.3 FFT . . . 166
13.4 NTT . . . 167
13.5 Roots NTT . . . 168

14 Strings **168**
14.1 Hashing Toolkit: Choosing the Right Hash . . . 168
14.1.1 Collision Guide: What the Probability Refers To . . . 169
14.2 Composable Double Hash . . . 170
14.3 Hashing Recipes: LCP, Comparison and Periods . . . 171

14.4 Set and Multiset Fingerprints . . . 172
14.5 Zobrist Hashing for Incremental States . . . 174
14.6 Dynamic String Hash: Replacements and Palindromes . . . 174
14.7 Rotation-Invariant Matrix Fingerprint . . . 175
14.8 Hash Toolkit: Runnable Examples . . . 177
14.9 Hashed String . . . 178
14.10KMP . . . 178
14.11Least Rotation String . . . 179
14.12Manacher . . . 180
14.13Suffix Array . . . 180
14.14Suffix Automaton . . . 182
14.15Suffix Automaton Extended . . . 185
14.16Trie Ahocorasick . . . 192
14.17Z Function . . . 193

15 Trees **194**
15.1 Centroid Decomposition . . . 194
15.2 HLD subtree query . . . 195
15.3 Heavy Light Decomposition . . . 195
15.4 Kruskal Reconstruction Tree . . . 197
15.5 Lowest Common Ancestor (LCA) . . . 198
15.6 Tree Diameter . . . 199
15.7 Virtual Tree . . . 199

16 Scripts **201**
16.1 build.sh . . . 201
16.2 stress.sh . . . 201
16.3 validate.sh . . . 201

1 Programming Helpers

1.1 Function Quick Reference

#include <algorithm> #include <numeric>

Algo	**Params**	**Funcion**
sort, stable_sort	f, l	ordena el intervalo
nth_element	f, nth, l	*void* ordena el n-esimo, y particiona el resto
fill, fill_n	f, l / n, elem	*void* llena [f, l) o [f, f+n) con elem
lower_bound, upper_bound	f, l, elem	*it* al primer / ultimo donde se puede insertar elem para que quede ordenada
binary_search	f, l, elem	*bool* esta elem en [f, l)
copy	f, l, resul	hace resul+i=f+i $\forall i$
find, find_if, find_first_of	f, l, elem / pred / f2, l2	*it* encuentra i ∈[f,l) tq. i=elem, pred(i), i∈[f2,l2)
count, count_if	f, l, elem/pred	cuenta elem, pred(i)
search	f, l, f2, l2	busca [f2,l2) ∈ [f,l)
replace, replace_if	f, l, old / pred, new	cambia old / pred(i) por new
reverse	f, l	da vuelta
partition, stable_partition	f, l, pred	pred(i) ad, !pred(i) atras
min_element, max_element	f, l, [comp]	*it* min, max de [f,l)
lexicographical_compare	f1,l1,f2,l2	*bool* con [f1,l1]¡[f2,l2]
next/prev_permutation	f,l	deja en [f,l) la perm sig, ant
set_intersection, set_difference, set_union, set_symmetric_difference,	f1, l1, f2, l2, res	[res, ...) la op. de conj
push_heap, pop_heap, make_heap	f, l, e / e /	mete/saca e en heap [f,l), hace un heap de [f,l)
is_heap	f,l	*bool* es [f,l) un heap
accumulate	f,l,i,[op]	$T = \sum$/oper de [f,l)
inner_product	f1, l1, f2, i	T = i + [f1, l1) . [f2, ...)
partial_sum	f, l, r, [op]	r+i = $\sum$/oper de [f,f+i] $\forall i$ ∈[f,l)
__builtin_ffs	unsigned int	Pos. del primer 1 desde la derecha
__builtin_clz	unsigned int	Cant. de ceros desde la izquierda.
__builtin_ctz	unsigned int	Cant. de ceros desde la derecha.
__builtin_popcount	unsigned int	Cant. de 1's en x.
__builtin_parity	unsigned int	1 si la cantidad de bits en 1 es impar.
__builtin_XXXXXXll	unsigned ll	= pero para long long's.

1.2 C++17 Function Examples

Runnable examples with expected results in comments. Iterator ranges use $[first, last)$: the end iterator is excluded. For an inclusive vector interval $[l, r]$, pass `begin()+l` and `begin()+r+1`. All indices below are zero-based.

- Reordering: reverse, sort, stable sort, rotate, partial sort, selection, and permutations.
- Searching: bounds, binary search, find, counts, predicates, minimum/maximum, and clamp.
- Editing: deduplication, erase-remove, replacement, fill, transform, and filtered copying.
- Numeric and string helpers: sums, prefixes, differences, dot products, gcd/lcm, substrings, parsing, and conversion.
- Sorted set operations and heap algorithms.

Each group is a separate function; `main` runs all examples.

```cpp
// C++17: useful library functions with runnable examples.
// Iterator ranges are [first, last): the last iterator is excluded.
// For a vector subrange [l, r], use begin()+l, begin()+r+1.
#include <bits/stdc++.h>
using namespace std;

// Only a helper for displaying vectors; the examples use standard functions.
template<class T>
void show(const char* name, const vector<T>& v) {
    cout << name << ':';
    for (const auto& x : v) cout << ' ' << x;
    cout << '\n';
}

void ordering_examples() {
    vector<int> v = {3, 1, 4, 2};
    reverse(v.begin(), v.end());
    show("reverse", v); // 2 4 1 3
    reverse(v.begin() + 1, v.begin() + 3);
    show("reverse subrange", v); // 2 1 4 3

    sort(v.begin(), v.end());
    show("sort", v); // 1 2 3 4
    sort(v.begin(), v.end(), greater<int>());
    show("sort descending", v); // 4 3 2 1

    // Comparator returns true when a must precede b; do not use <=.
    vector<pair<int, int>> p = {{3, 1}, {1, 2}, {2, 1}};
    stable_sort(p.begin(), p.end(), [](auto a, auto b) {
        return a.second < b.second;
    });
    // p = {{3, 1}, {2, 1}, {1, 2}}: ties keep their original order.
    cout << "stable_sort:";
    for (auto [a, b] : p) cout << ' ' << a << ',' << b;
    cout << '\n';

    v = {1, 2, 3, 4, 5};
    rotate(v.begin(), v.begin() + 2, v.end()); // left rotation by 2
    show("rotate left", v); // 3 4 5 1 2
    rotate(v.begin(), v.end() - 1, v.end()); // right rotation by 1
    show("rotate right", v); // 2 3 4 5 1

    v = {9, 2, 7, 1, 5};
    nth_element(v.begin(), v.begin() + 2, v.end());
    cout << "third smallest: " << v[2] << '\n'; // 5
    // Elements before/after v[2] are partitioned, not fully sorted.
    partial_sort(v.begin(), v.begin() + 3, v.end());
    show("three smallest", vector<int>(v.begin(), v.begin() + 3));
    // 1 2 5. partial_sort sorts only the chosen prefix.

    v = {1, 2, 3};
    next_permutation(v.begin(), v.end());
    show("next_permutation", v); // 1 3 2
    prev_permutation(v.begin(), v.end());
    show("prev_permutation", v); // 1 2 3
    // To enumerate all distinct permutations: sort first, then do/while.
    int permutations = 0;
    do { ++permutations; }
    while (next_permutation(v.begin(), v.end()));
    cout << "permutations: " << permutations << '\n'; // 6
    // Returning false also wraps to the first (sorted) permutation.
}

void searching_examples() {
    // These binary-search algorithms require the matching sorted order.
```

```
    vector<int> v = {1, 2, 2, 2, 5};
    auto lo = lower_bound(v.begin(), v.end(), 2); // first value >= 2
    auto hi = upper_bound(v.begin(), v.end(), 2); // first value > 2
    cout << "bounds: " << lo - v.begin() << ' ' << hi - v.begin();
    cout << ' ' << hi - lo << '\n'; // 1 4 3 (three copies of 2)
    cout << "binary_search: ";
    cout << binary_search(v.begin(), v.end(), 5) << '\n'; // 1
    auto [first, last] = equal_range(v.begin(), v.end(), 2);
    cout << "equal_range count: " << last - first << '\n'; // 3
    // A bound may return end(); check before dereferencing it.

    v = {7, 4, 9, 2}; // find/find_if do not require sorted input.
    auto it = find(v.begin(), v.end(), 9);
    if (it != v.end()) cout << "find index: " << it - v.begin() << '\n';
        // 2
    it = find_if(v.begin(), v.end(), [](int x) { return x % 2 == 0; });
    if (it != v.end()) cout << "first even: " << *it << '\n'; // 4

    cout << "count: " << count(v.begin(), v.end(), 4) << '\n'; // 1
    cout << "count_if: " << count_if(v.begin(), v.end(), [](int x) {
        return x % 2 == 0;
    }) << '\n'; // 2
    auto positive = [](int x) { return x > 0; };
    auto negative = [](int x) { return x < 0; };
    cout << "all/any/none: " << all_of(v.begin(), v.end(), positive);
    cout << ' ' << any_of(v.begin(), v.end(), negative);
    cout << ' ' << none_of(v.begin(), v.end(), negative) << '\n'; // 1 0
        1

    // Dereference min/max iterators only when the range is nonempty.
    auto [mn, mx] = minmax_element(v.begin(), v.end());
    cout << "min/max: " << *mn << ' ' << *mx << '\n'; // 2 9
    cout << "clamp: " << clamp(15, 0, 10) << '\n'; // 10
    cout << "is_sorted: " << is_sorted(v.begin(), v.end()) << '\n'; // 0
}

void editing_examples() {
    vector<int> v = {3, 1, 3, 2, 1};
    sort(v.begin(), v.end());
    v.erase(unique(v.begin(), v.end()), v.end());
    show("sort + unique", v); // 1 2 3
    // unique removes adjacent duplicates only; erase changes vector
        size.

    v = {1, 2, 3, 2, 4};
    v.erase(remove(v.begin(), v.end(), 2), v.end());
    show("erase-remove", v); // 1 3 4
    v.erase(remove_if(v.begin(), v.end(), [](int x) {
        return x % 2 != 0;
    }), v.end());
    show("erase-remove_if", v); // 4
    // remove/remove_if alone do not shrink the container.

    v = {1, 2, 1, 3};
    replace(v.begin(), v.end(), 1, 9);
    show("replace", v); // 9 2 9 3
    fill(v.begin(), v.begin() + 2, 4);
    show("fill prefix", v); // 4 4 9 3

    vector<int> doubled(v.size());
    transform(v.begin(), v.end(), doubled.begin(), [](int x) {
        return 2 * x;
    });
    show("transform", doubled); // 8 8 18 6
    vector<int> evens;
    copy_if(v.begin(), v.end(), back_inserter(evens), [](int x) {
        return x % 2 == 0;
    });
    show("copy_if even", evens); // 4 4
    // For copy/transform into begin(), allocate output elements first.
    // back_inserter grows a vector with push_back instead.
}

void numeric_examples() {
    vector<int> v = {2, 5, 3};
    cout << "sum: " << accumulate(v.begin(), v.end(), 0LL) << '\n'; //
        10
    // 0LL makes the accumulator long long; 0 would make it int.
    vector<int> ids(5);
    iota(ids.begin(), ids.end(), 10);
    show("iota", ids); // 10 11 12 13 14

    vector<long long> prefix(v.size()), difference(v.size());
    partial_sum(v.begin(), v.end(), prefix.begin());
    show("partial_sum", prefix); // 2 7 10
    // partial_sum accumulates in the input value type, here int.
```

```
    // For large sums, use vector<long long> as the input too.
    adjacent_difference(prefix.begin(), prefix.end(), difference.begin()
        );
    show("adjacent_difference", difference); // 2 5 3

    vector<int> weights = {4, 1, 2};
    cout << "dot product: ";
    cout << inner_product(v.begin(), v.end(), weights.begin(), 0LL);
    cout << '\n'; // 19 = 2*4 + 5*1 + 3*2
    // Multiplication still uses the element types; use long long inputs
    // if an individual product might overflow int.
    cout << "gcd/lcm: " << gcd(18, 24) << ' ' << lcm(6, 8) << '\n'; // 6
        24
    // For lcm, the resulting value must fit the argument type.
    int a = 3, b = 8;
    swap(a, b);
    cout << "swap: " << a << ' ' << b << '\n'; // 8 3
}

void string_examples() {
    string s = "abcdef";
    reverse(s.begin(), s.end());
    cout << "reverse string: " << s << '\n'; // fedcba
    s = "banana";
    cout << "substr: " << s.substr(1, 3) << '\n'; // ana (start, count)
    size_t pos = s.find("ana");
    if (pos != string::npos) cout << "string find: " << pos << '\n'; //
        1
    s.replace(1, 3, "XYZ"); // replace 3 characters starting at index 1.
    cout << "string replace: " << s << '\n'; // bXYZna
    s.erase(1, 3);
    cout << "string erase: " << s << '\n'; // bna

    cout << "stoll: " << stoll("1234567890123") << '\n';
    cout << "to_string: " << to_string(-42) << '\n'; // -42
    // stoi/stoll throw on invalid or out-of-range input.
    // They accept a numeric prefix; use the position argument to check
        all input.
    vector<int> numbers;
    istringstream in("10 20 -3");
    for (int x; in >> x;) numbers.push_back(x);
    show("istringstream", numbers); // 10 20 -3
}


void set_and_heap_examples() {
    // Set algorithms require both ranges sorted with the same
        comparator.
    vector<int> a = {1, 2, 4}, b = {2, 3, 4}, out;
    set_intersection(a.begin(), a.end(), b.begin(), b.end(),
        back_inserter(out));
    show("set_intersection", out); // 2 4
    out.clear();
    set_union(a.begin(), a.end(), b.begin(), b.end(), back_inserter(out)
        );
    show("set_union", out); // 1 2 3 4
    out.clear();
    set_difference(a.begin(), a.end(), b.begin(), b.end(), back_inserter
        (out));
    show("set_difference", out); // 1 (a minus b)
    // Duplicates are treated as multisets; deduplicate first for set
        semantics.

    vector<int> heap = {3, 1, 4};
    make_heap(heap.begin(), heap.end());
    cout << "heap maximum: " << heap.front() << '\n'; // 4
    heap.push_back(5);
    push_heap(heap.begin(), heap.end());
    cout << "after push_heap: " << heap.front() << '\n'; // 5
    pop_heap(heap.begin(), heap.end()); // moves maximum to back, size
        unchanged.
    cout << "popped maximum: " << heap.back() << '\n'; // 5
    heap.pop_back(); // actually remove it.
    cout << "remaining maximum: " << heap.front() << '\n'; // 4
    // priority_queue is usually simpler when only top/push/pop are
        needed.
}

int main() {
    ordering_examples();
    searching_examples();
    editing_examples();
    numeric_examples();
    string_examples();
    set_and_heap_examples();
}
```

1.3 Map Operators to Functions

`function<ll(ll,ll)>` stores a callable that takes two `long long` arguments and returns a `long long`. The map chooses the lambda by its operator character; the following parentheses call it. `at` requires the key to exist.

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
    map<char, function<ll(ll, ll)>> operation{
        {'+', [](ll a, ll b) { return a + b; }},
        {'*', [](ll a, ll b) { return a * b; }}
    };

    cout << operation.at('+')(3, 4) << '\n'; // 7
    cout << operation.at('*')(3, 4) << '\n'; // 12
}
```

1.4 Comparators

For `sort`, `set`, and `map`, `cmp(a,b)` is true when `a` comes before `b`. In a `priority_queue`, true means `a` has lower priority; the same comparator gives the reverse pop order.

Comparator	Sort / set order	Queue pop order
`less<int>` (default)	Smallest first	Largest first
`greater<int>`	Largest first	Smallest first

Use strict comparisons (`<` or `>`); `cmp(a,a)` must be false. Break ties explicitly when needed. Reverse an order by swapping arguments, not by using `!cmp`. Keep comparison keys unchanged while stored in a set, map, or heap.

1. Sorting with a lambda. `[](parameters) { body }` creates a small function. The first example sorts integers descending; the second sorts pairs by decreasing score, then increasing id. For a struct, use the same logic with fields such as `x.score` and `x.id`. Use `const T&` for larger objects.

```cpp
vector<int> v = {3, 1, 2};
sort(v.begin(), v.end(), [](int a, int b) { return a > b; });
// v: 3 2 1

using P = pair<int, int>; // (score, id)
vector<P> a = {{20,3}, {30,4}, {20,2}, {30,1}};
auto before = [](const P& x, const P& y) {
    if (x.first != y.first) return x.first > y.first;
    return x.second < y.second; // Tied score: smaller id first.
};
sort(a.begin(), a.end(), before);
// a: (30,1) (30,4) (20,2) (20,3)
```

2. A priority queue with the same desired order. A heap needs the opposite comparison, so return `before(y,x)`. `decltype(lower_priority)` is the lambda's type; pass the lambda object to the constructor (needed in C++17).

```cpp
// Reuse P, a, and before from the sorting example.
auto lower_priority = [before](const P& x, const P& y) {
    return before(y, x); // Reverse arguments for the heap.
};
using Queue = priority_queue<P, vector<P>,
                             decltype(lower_priority)>;
Queue q(lower_priority);
for (auto p : a) q.push(p);
// Repeated top()/pop(): (30,1) (30,4) (20,2) (20,3)
```

3. Sets and maps with a lambda. This orders strings by length descending, then alphabetically. A map compares its keys.

```
auto longer = [](const string& x, const string& y) {
    if (x.size() != y.size()) return x.size() > y.size();
    return x < y; // Equal length: alphabetical order.
};
set<string, decltype(longer)> s(longer);
map<string, int, decltype(longer)> freq(longer);
for (string w : {"fig", "pear", "kiwi", "pear"}) {
    s.insert(w);
    ++freq[w];
}
// Set iteration: kiwi pear fig
// Map iteration: (kiwi,1) (pear,2) (fig,1)
```

A set keeps one key when neither compares before the other; the alphabetical tie-break keeps different equal-length words. Use `multiset` to retain duplicates.

Captures and alternatives. `[]` captures nothing, `[x]` copies `x`, and `[&x]` refers to the existing object, which must stay alive. A comparator can also be a named `bool` function passed to `sort`, or a struct with `bool operator()(const T&, const T&) const`.

1.5 Scientific Notation

`aeb` and `aEb` mean $a \times 10^b$. A positive exponent shifts the decimal point right; a negative exponent shifts it left. The exponent is an integer.

Literal	Value	Type
`1e3`	1000	`double`
`2.5e-3`	0.0025	`double`
`-4.2E+2`	-420	`double`
`1e-9L`	10^{-9}	`long double`
`1000000000LL`	10^9 (exact integer)	`long long`

Scientific literals are floating-point values, even `1e9`. Use integer literals with `LL` when exact integer constants are needed. Read exponent-form input with floating-point types or `stod`/`stold`; integer parsers only consume the initial integer prefix. With `scientific` or `fixed`, `setprecision(p)` sets digits after the decimal point. With `defaultfloat`, it sets significant digits. Stream flags and precision persist; `defaultfloat` resets the float format, not the precision. `uppercase`/`nouppercase` choose `E`/`e`. Formatting rounds the display without changing the stored value.

Quick fixed-format snippet. Ten digits after the decimal point:

```
cout<<fixed<<setprecision(10);
```

```
// C++17. Scientific notation is coefficient * 10^exponent.
#include <bits/stdc++.h>
using namespace std;

int main() {
    double a = 1e3;       // 1 * 10^3 = 1000; type is double.
    double b = 2.5e-3;    // 2.5 * 10^-3 = 0.0025.
    double c = -4.2E+2;   // -4.2 * 10^2 = -420; E also works.
    long double eps = 1e-9L; // L selects long double; f selects float.
    cout << "literals: " << a << ' ' << b << ' ' << c << '\n';
    // 1000 0.0025 -420
    cout << "long double literal: " << scientific << setprecision(2);
    cout << eps << '\n'; // 1.00e-09
    cout << defaultfloat << setprecision(6);

    // Exact integer constants should use integer literals.
    const long long LIMIT = 1000000000LL;
    const long long LARGE = 10000000000000000001LL;
    cout << "integer constants: " << LIMIT << ' ' << LARGE << '\n';
    // 1000000000 10000000000000000001
    // 1e9 is a double even without a decimal point.
    // Large floating-point values can lose integer precision.
    // 10 ^ 3 is bitwise XOR in C++, not exponentiation; 1e3 means 1000.

    // Streams and stod/stold understand exponents when reading floats.
    istringstream input("1e3 2.5e-3 -4.2E+2");
    double x, y, z;
    input >> x >> y >> z;
    cout << "parsed input: " << x << ' ' << y << ' ' << z << '\n';
    // 1000 0.0025 -420
    cout << "stod: " << stod("2.5e3") << '\n'; // 2500
    // Integer parsers do not read the exponent: check the consumed
        length.
    size_t used = 0;
    long long prefix = stoll("1e3", &used);
    cout << "stoll prefix: " << prefix << " consumed " << used << '\n';
    // 1 consumed 1; this is NOT a way to parse the integer 1000.

    double value = 12345.6789;
    auto old_flags = cout.flags();
    auto old_precision = cout.precision();
    cout << "scientific p3: " << scientific << setprecision(3);
    cout << value << '\n'; // 1.235e+04 (3 fractional digits)
```

```
    cout << "uppercase p2: " << uppercase << setprecision(2);
    cout << value << '\n'; // 1.23E+04
    cout << "fixed p2: " << nouppercase << fixed << setprecision(2);
    cout << value << '\n'; // 12345.68 (2 fractional digits)
    cout << "defaultfloat p6: " << defaultfloat << setprecision(6);
    cout << value << '\n'; // 12345.7 (6 significant digits)
    // Formatting changes only the printed representation, not value.
    // Flags and precision persist; restore both when needed.
    cout.flags(old_flags);
    cout.precision(old_precision);
    cout << "restored: " << value << '\n'; // 12345.7

    // printf: %e/%E take double; %Le/%LE take long double.
    // Do not use %e for long double (or %Le for double).
    printf("printf p3: %.3e\n", value); // 1.235e+04
    printf("printf upper p2: %.2E\n", value); // 1.23E+04
    long double large_value = 12345.6789L;
    printf("printf long double p2: %.2Le\n", large_value); // 1.23e+04
    // This program keeps the default iostream/stdio synchronization.
}
```

1.6 Formatted Output

The table below is for `printf`. Use `%.3e` for three digits after the decimal point in scientific notation, or `%.2f` for two digits in fixed notation. `%e`/`%E` expect `double`; `%Le`/`%LE` expect `long double`. For `scanf`, use `%f` for `float*`, `%lf` for `double*`, and `%Lf` for `long double*`.

specifier	Output	Example
d or i	Signed decimal integer	392
u	Unsigned decimal integer	7235
o	Unsigned octal	610
x	Unsigned hexadecimal integer (lowercase)	7fa
X	Unsigned hexadecimal integer (uppercase)	7FA
f	Decimal floating point, lowercase	392.65
F	Decimal floating point, uppercase	392.65
e	Scientific notation (mantissa/exponent), lowercase	3.9265e+2
E	Scientific notation (mantissa/exponent), uppercase	3.9265E+2
g	General format (significant digits): %e or %f	392.65
G	General format (significant digits): %E or %F	392.65
a	Hexadecimal floating point, lowercase	-0xc.90fep-2
A	Hexadecimal floating point, uppercase	-0XC.90FEP-2
c	Character	a
s	String of characters	sample
p	Pointer address	b8000000
n	Nothing printed. The argument must be a pointer to a `signed int`. The number of characters written so far is stored.	
%	A % followed by another % prints a single %	%

1.7 String Input and getline

`cin >> s` reads a whitespace-separated word. `getline(cin, s)` reads a whole line, preserving leading and trailing spaces and accepting empty lines. It consumes the newline without storing it. Include `<string>`, `<iostream>`, `<sstream>` for parsing, and `<limits>` for `ignore`.

A number followed by lines. Formatted extraction leaves the newline in the stream. If the strings start on the next line, discard the rest of the number's line before calling `getline`:

```
int n;
string line;
if (cin >> n) {
```

```
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
    for (int i = 0; i < n && getline(cin, line); ++i) {
        // Process line; it may contain spaces or be empty.
    }
}
```

This intentionally discards any text after the number on that same line. To read that remainder instead, call `getline` directly.

Whitespace and EOF. `getline(cin >> ws, s)` is a shortcut only when all leading whitespace and blank lines may be skipped. Use plain `getline` to preserve them. `while (getline(cin, s))` reads until EOF, including a final nonempty line without a trailing newline. The same patterns work with `ifstream` for files and `istringstream` for strings. `cin.get(ch)` reads one character including whitespace; `cin >> ch` skips whitespace.

Runnable examples. The streams below contain sample input; replace them with `cin` for judge input. Quoted output makes spaces and empty strings visible.

```
// C++17. Replace each istringstream with cin for keyboard/judge input.
#include <bits/stdc++.h>
using namespace std;

int main() {
    // >> reads one token and skips leading whitespace.
    istringstream words("  red blue\n");
    string first, second;
    words >> first >> second;
    cout << "tokens: " << quoted(first) << ' '
         << quoted(second) << '\n';
    // tokens: "red" "blue"

    // getline removes '\n', preserves spaces, and accepts empty lines.
    // The last line below has no final newline; it is still read.
    istringstream lines("  Alice Smith  \n\nBob Jones");
    string line;
    int row = 0;
    while (getline(lines, line)) {
        cout << "line " << ++row << ": " << quoted(line) << '\n';
    }
    // line 1: "  Alice Smith  "
    // line 2: ""
    // line 3: "Bob Jones"
    // Use while (getline(...)), not while (!stream.eof()).

    // Mixing >> and getline: discard the REST of the count's line.
    istringstream names("2   \nAda Lovelace\nAlan Turing\n");
    int n;
    names >> n;
    names.ignore(numeric_limits<streamsize>::max(), '\n');
    for (int i = 0; i < n && getline(names, line); ++i) {
        cout << "name: " << quoted(line) << '\n';
    }
    // name: "Ada Lovelace"
    // name: "Alan Turing"
    // Without ignore, the first getline would read "   ".
    // ignore() alone discards just ONE character, not the whole line.

    // ws skips ALL leading whitespace, including blank lines.
    istringstream padded("\n   \n  Grace Hopper  \n");
    getline(padded >> ws, line);
    cout << "with ws: " << quoted(line) << '\n';
    // with ws: "Grace Hopper  " (trailing spaces remain).
    // Do not use ws when leading spaces or empty lines matter.

    // Parse a whole line separately with istringstream.
    istringstream data("10 20 30\n7 8\n");
    while (getline(data, line)) {
        istringstream fields(line);
        int x, sum = 0;
        while (fields >> x) sum += x;
        cout << "sum: " << sum << '\n';
    }
    // sum: 60
    // sum: 15

    // A custom delimiter also works. This is not a quoted-CSV parser.
    istringstream separated("red,,blue");
    string field;
    while (getline(separated, field, ',')) {
        cout << "field: " << quoted(field) << '\n';
    }
    // field: "red"
    // field: ""
    // field: "blue"
    // No extra field is read after a trailing delimiter.

    // >> char skips whitespace; get(char&) reads even spaces/newlines.
```

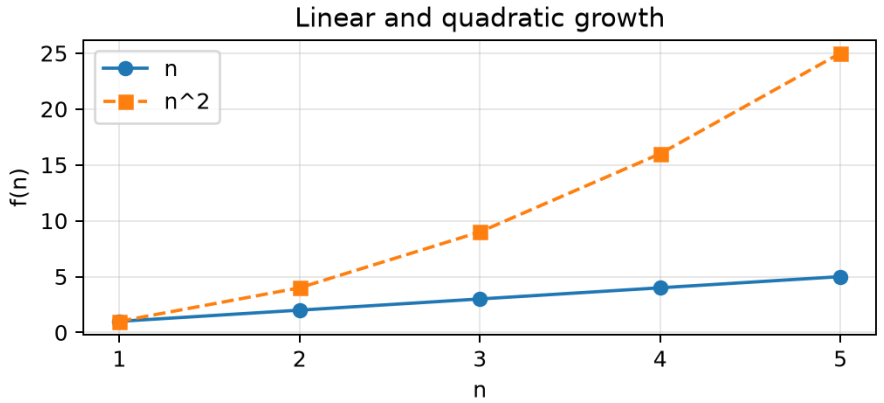
```
    istringstream chars(" A");
    char ch;
    chars.get(ch);
    cout << "get: " << quoted(string(1, ch)) << '\n'; // get: " "
    chars >> ch;
    cout << ">> char: " << quoted(string(1, ch)) << '\n'; // "A"
}
```

1.8 Plotting with Matplotlib (Python)

Install with `python3 -m pip install matplotlib`, then run `python3 MatplotlibExample.py`. Supply equally sized lists of x and y values; `plot` connects points in list order. `subplots` creates the figure and its plotting area.

```
# Python 3. Install once: python3 -m pip install matplotlib
# Run: python3 MatplotlibExample.py
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [n * n for n in x]
fig, ax = plt.subplots(figsize=(5.6, 2.6), layout="constrained")
ax.plot(x, x, "o-", label="n")   # Points connected by a solid line.
ax.plot(x, y, "s--", label="n^2")  # Square markers, dashed line.
ax.set(title="Linear and quadratic growth", xlabel="n", ylabel="f(n)")
ax.set_xticks(x)
ax.grid(True, alpha=0.3)
ax.legend()
# Save before show(); output is in the current working directory.
fig.savefig("matplotlib_example.png", dpi=180)
plt.show()  # Open the graph window; close it to finish the script.
# On a server without a display, omit show() and keep savefig().
```



Other plot types. Replace a `plot` call with `ax.scatter(x, y)` for unconnected points or `ax.bar(x, y)` for bars. `label` names a series; `legend()` displays those names. `fig.savefig("graph.pdf")` exports a vector PDF. Close saved figures with `plt.close(fig)` when generating many plots. Official Matplotlib quick start.

2 Compile

2.1 Compile

```
g++-13 nombre.cpp -o nombre (compilar)
./nombre (ejecutar)
g++ -std=c++23 -Wall -Wshadow -g -fsanitize=undefined -fsanitize=address
     -D_GLIBCXX_DEBUG nombre.cpp -o nombre
```

2.2 Template

```cpp
#include <bits/stdc++.h>
#pragma GCC optimize("O3,unroll-loops")
#pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt")
using namespace std;
#define pb push_back
#define ll long long
#define s second
#define f first
#define MOD 1000000007
#define INF 100000000000000

void solve(){

}

int main() {
  ios_base::sync_with_stdio(false);  cin.tie(0);  cout.tie(0);
  int t;cin>>t;for(int T=0;T<t;T++)
  solve();
}
```

3 Data Structures

3.1 BIT

```cpp
/*
  Binary Indexed Tree (Fenwick Tree) Fast Implementation
  ---------------------------------
  Indexing: 1-based
  Bounds: [1, MAXN)
  Time Complexity:
    - update(x, val): O(log n) -> adds val to index x
    - get(x): O(log n) -> prefix sum from 1 to x
  Space Complexity: O(n)
*/

const int MAXN = 10000;
int bit[MAXN];

// Add `val` to index `x`
void update(int x, int val) {
  for (; x < MAXN; x += x & -x) {
    bit[x] += val;
  }
}

// Get prefix sum from 1 to `x`
int get(int x) {
  int ans = 0;
  for (; x > 0; x -= x & -x) {
    ans += bit[x];
  }
  return ans;
}
```

3.2 Bitset

```cpp
bitset<3001> b[3001];

//set() Set the bit value at the given index to 1.
//reset() Set the bit value at the given index to 0.
//flip() Toggle the bit value at the given index.
//test() Check if the bit value at the given index is 1 or 0.
//count() Count the number of set bits.
//any() Checks if any bit is set
//all() Check if all bit is set.
//none() Check if no bit is set.
//to_string() Convert the bitset to a string representation.

#pragma GCC target("popcnt")
(int) __builtin_popcount(x);
(int) __builtin_popcountll(x);
__builtin_clz(x); // count leading zeros

// declare bitset
```

```
bitset<64> b;
```

3.3 Bit Trie

```
/*
  Bit Trie (Binary Trie for Integers)
  -----------------------------------
  Indexing: 0-based
  Bit Width: [0, MAX_BIT] inclusive (e.g., 31 for 32-bit integers)
  Time Complexity:
    - Insert: O(MAX_BIT)
    - Query: O(MAX_BIT)
  Space Complexity: O(N * MAX_BIT) nodes (in worst case, 1 per bit per number)
*/

const int K = 2; // Each node has 2 branches (bit 0 or 1)
const int MAX_BIT = 30; // Max bit position (for 32-bit integers)

struct Vertex {
  int next[K]; // next[0] = child for bit 0, next[1] = child for bit 1

  Vertex() {
    fill(begin(next), end(next), -1); // -1 means no child
  }
};

vector<Vertex> trie; // Trie nodes

// Inserts a number into the binary trie
void insert(int num) {
  int v = 0; // Start from root
  for (int j = MAX_BIT; j >= 0; --j) {
    int c = (num >> j) & 1; // Extract j-th bit (0 or 1)
    if (trie[v].next[c] == -1) {
      trie[v].next[c] = trie.size();
      trie.emplace_back(); // Add new node
    }
    v = trie[v].next[c]; // Move to next node
  }
}
```

3.4 CHMIN/CHMAX Rangesum GCD Queries

```
// Segment tree for chmin, chmax, add and query sum.
// Complexity is O (n (log n) ^ 2)

// Range chmin (l, r, v) -> a[l], ...., a[r] = min(a[i], v) call update with 1
// Range chmax (l, r, v) -> a[l], ...., a[r] = max(a[i], v) call update with 2
// Range set (l, r, v) -> a[l], ...., a[r] = v call update with 3 (this calls chmax and chmin on the interval)
// Range add (l, r, v) -> a[l], ...., a[r] += v call update with 4
// You can obtain sum, max, min, gcd in segment

typedef long long ll;

const ll inf = 1e17;

ll gcd(ll a, ll b){
  a = abs(a);
  b = abs(b);
  if(a == 0) return b;
  return gcd(b % a, a);
}

struct sum_t {
  ll sum, mx, mx2, cnt_mx, mn, mn2, cnt_mn, gcd_no_max_no_min;
  static const ll null_v = 0;
  sum_t(): sum(null_v), mx(-inf), mx2(-inf), cnt_mx(0), mn(inf), mn2(inf), cnt_mn(0), gcd_no_max_no_min(0) {}
  sum_t(ll v): sum(v), mx(v), mx2(-inf), cnt_mx(1), mn(v), mn2(inf), cnt_mn(1), gcd_no_max_no_min(0) {}
  sum_t(ll _sum, ll _mx, ll _mx2, ll _cnt_mx, ll _mn, ll _mn2, ll _cnt_mn, ll _gcd_no_max_no_min):
    sum(_sum), mx(_mx), mx2(_mx2), cnt_mx(_cnt_mx), mn(_mn), mn2(_mn2), cnt_mn(_cnt_mn), gcd_no_max_no_min(_gcd_no_max_no_min) {}
  sum_t op(sum_t& other) {
    sum_t res;
    res.sum = sum + other.sum;
    res.gcd_no_max_no_min = gcd(gcd_no_max_no_min, other.gcd_no_max_no_min);

    if(mx == other.mx){
      res.mx = mx;
      res.cnt_mx = cnt_mx + other.cnt_mx;
```

```
    res.mx2 = max(mx2, other.mx2);
  }
  else if(mx > other.mx){
    res.mx = mx;
    res.cnt_mx = cnt_mx;
    res.mx2 = max(mx2, other.mx);
  }
  else{
    res.mx = other.mx;
    res.cnt_mx = other.cnt_mx;
    res.mx2 = max(mx, other.mx2);
  }

  if(mn == other.mn){
    res.mn = mn;
    res.cnt_mn = cnt_mn + other.cnt_mn;
    res.mn2 = min(mn2, other.mn2);
  }
  else if(mn < other.mn){
    res.mn = mn;
    res.cnt_mn = cnt_mn;
    res.mn2 = min(mn2, other.mn);
  }
  else{
    res.mn = other.mn;
    res.cnt_mn = other.cnt_mn;
    res.mn2 = min(mn, other.mn2);
  }

  if(mx2 != -inf && mx2 != mn && other.mx2 != -inf && other.mx2 !=
      other.mn){
    res.gcd_no_max_no_min = gcd(res.gcd_no_max_no_min, mx2 - other.mx2
        );
  }

  ll any = -1;

  if(mx2 != -inf && mx2 != mn) any = mx2;
  if(other.mx2 != -inf && other.mx2 != other.mn) any = other.mx2;

  vector<ll> diff_max_min = {mx, mn, other.mx, other.mn};
  for(ll val: diff_max_min){
    if(val != res.mn && val != res.mx){
      if(any != -1) res.gcd_no_max_no_min = gcd(res.gcd_no_max_no_min,
          val - any);
      else any = val;
    }
  }

  return res;
 }
};

template <typename num_t>
struct segtree {
 int n;
 vector<num_t> tree, lazy;

 void init(int s, const vector<ll> &A) {
  n = s;
  tree.assign(4 * n, num_t());
  lazy.assign(4 * n, num_t());
  init(0, 0, n - 1, A);
 }

 num_t init(int i, int l, int r, const vector<ll> &A) {
  if (l == r) return tree[i] = num_t(A[l]);

  int mid = (l + r) / 2;
  num_t left = init(2 * i + 1, l, mid, A);
  num_t right = init(2 * i + 2, mid + 1, r, A);
  return tree[i] = left.op(right);
 }

 // update 3 is setting values to v
 void update(int l, int r, ll v, int op) {
  if (l > r) return;
  if(op == 1){
   update_chmin(0, 0, n - 1, l, r, v);
  }
  if(op == 2){
   update_chmax(0, 0, n - 1, l, r, v);
  }
  if(op == 3){
   update_chmin(0, 0, n - 1, l, r, v);
   update_chmax(0, 0, n - 1, l, r, v);
```

```
    }
    if(op == 4){
      update_add(0, 0, n - 1, l, r, v);
    }
  }

  num_t update_chmin(int i, int tl, int tr, int ql, int qr, ll v) {
    eval_lazy(i, tl, tr);

    if (tr < ql || qr < tl || tree[i].mx <= v) return tree[i];
    if (ql <= tl && tr <= qr && tree[i].mx2 < v) {
      push_min(i, v);
      eval_lazy(i, tl, tr);
      return tree[i];
    }

    int mid = (tl + tr) / 2;
    num_t a = update_chmin(2 * i + 1, tl, mid, ql, qr, v);
    num_t b = update_chmin(2 * i + 2, mid + 1, tr, ql, qr, v);
    return tree[i] = a.op(b);
  }

  num_t update_chmax(int i, int tl, int tr, int ql, int qr, ll v) {
    eval_lazy(i, tl, tr);

    if (tr < ql || qr < tl || tree[i].mn >= v) return tree[i];
    if (ql <= tl && tr <= qr && tree[i].mn2 > v) {
      push_max(i, v);
      eval_lazy(i, tl, tr);
      return tree[i];
    }

    int mid = (tl + tr) / 2;
    num_t a = update_chmax(2 * i + 1, tl, mid, ql, qr, v);
    num_t b = update_chmax(2 * i + 2, mid + 1, tr, ql, qr, v);
    return tree[i] = a.op(b);
  }

  num_t update_add(int i, int tl, int tr, int ql, int qr, ll v) {
    eval_lazy(i, tl, tr);

    if (tr < ql || qr < tl) return tree[i];
    if (ql <= tl && tr <= qr) {
      push_add(i, tl, tr, v);
      eval_lazy(i, tl, tr);
      return tree[i];
    }

    int mid = (tl + tr) / 2;
    num_t a = update_add(2 * i + 1, tl, mid, ql, qr, v);
    num_t b = update_add(2 * i + 2, mid + 1, tr, ql, qr, v);
    return tree[i] = a.op(b);
  }

  num_t query(int l, int r){
    if (l > r) return num_t();
    return query(0, 0, n - 1, l , r);
  }

  num_t query(int i, int tl, int tr, int ql, int qr) {
    eval_lazy(i, tl, tr);

    if (ql <= tl && tr <= qr) return tree[i];
    if (tr < ql || qr < tl) return num_t();

    int mid = (tl + tr) / 2;
    num_t a = query(2 * i + 1, tl, mid, ql, qr);
    num_t b = query(2 * i + 2, mid + 1, tr, ql, qr);
    return a.op(b);
  }

  ll query_gcd(int l, int r){
    if(l > r) return 0;
    return query_gcd(0, 0, n - 1, l, r);
  }

  ll query_gcd(int i, int tl, int tr, int ql, int qr){
    eval_lazy(i, tl, tr);

    if (ql <= tl && tr <= qr){
      ll res = gcd(tree[i].mx, tree[i].gcd_no_max_no_min);
      if(tree[i].mn2 != inf) res = gcd(res, tree[i].mn2 - tree[i].mn);
      if(tree[i].mx2 != -inf) res = gcd(res, tree[i].mx - tree[i].mx2);
      return res;
    }
    if (tr < ql || qr < tl) return 0;
```

```

    int mid = (tl + tr) / 2;
    ll a = query_gcd(2 * i + 1, tl, mid, ql, qr);
    ll b = query_gcd(2 * i + 2, mid + 1, tr, ql, qr);
    return gcd(a, b);
  }

  void eval_lazy(int i, int l, int r) {
    if (l != r) {
      int mid = (l + r) / 2;

      push_add(2 * i + 1, l, mid, lazy[i].sum);
      push_add(2 * i + 2, mid + 1, r, lazy[i].sum);

      push_min(2 * i + 1, tree[i].mx);
      push_min(2 * i + 2, tree[i].mx);

      push_max(2 * i + 1, tree[i].mn);
      push_max(2 * i + 2, tree[i].mn);

    }
    lazy[i] = num_t();
  }

  void push_add(int i, int l, int r, ll v){
    if(v == 0) return;
    tree[i].mx += v;
    tree[i].sum += v * (r - l + 1);
    tree[i].mx2 += (tree[i].mx2 == -inf ? 0 : v);
    tree[i].mn += v;
    tree[i].mn2 += (tree[i].mn2 == inf ? 0 : v);
    lazy[i].sum += v;
  }

  void push_min(int i, ll v){
    if (tree[i].mx <= v) return;
    if(tree[i].mn == tree[i].mx){
      tree[i].mn = v;
    }
    else if(tree[i].mn2 == tree[i].mx){
      tree[i].mn2 = v;
    }
    tree[i].sum -= tree[i].mx * tree[i].cnt_mx;
    tree[i].mx = v;
    tree[i].sum += tree[i].mx * tree[i].cnt_mx;
  }

  void push_max(int i, ll v){
    if (tree[i].mn >= v) return;
    if(tree[i].mx == tree[i].mn){
      tree[i].mx = v;
    }
    else if(tree[i].mx2 == tree[i].mn){
      tree[i].mx2 = v;
    }
    tree[i].sum -= tree[i].mn * tree[i].cnt_mn;
    tree[i].mn = v;
    tree[i].sum += tree[i].mn * tree[i].cnt_mn;
  }
};

int main(){
  ios_base::sync_with_stdio(false); cin.tie(NULL);
  int n; cin >> n;
  vector<ll> A(n);
  for(int i = 0; i < n; i++) cin >> A[i];
  segtree <sum_t> st;
  st.init(n, A);
  int q; cin >> q;
  while(q--){
    int t, l, r; cin >> t >> l >> r;
    l--; r--;
    if(t < 5){
      ll x; cin >> x;
      st.update(l, r, x, t);
    }
    else{
      if(t == 8){
        cout << st.query_gcd(l, r) << endl;
      }
      else{
        auto ans = st.query(l, r);
        if(t == 5) cout << ans.sum << endl;
        else if (t == 6) cout << ans.mn << endl;
        else if(t == 7) cout << ans.mx << endl;
      }
```

```
    }
  }
}
```

3.5 Disjoint Sparse Table

```
// queries are given weird because it is a lot of them but it is segment
    multiplication
#include<bits/stdc++.h>
using namespace std;

#define pii pair<int,int>
#define pll pair<ll,ll>
#define eb emplace_back
#define ll long long
#define nl '\n'
#define deb(x) cerr<<#x" = "<<x<<nl
#define in() ( { int a ; scanf("%d",&a); a; } )

const int N = 3e5 + 9;
int mod = 1e9 + 7;

struct DST {
  vector<vector<int>> left, right;
  int k, n;
  DST(vector<int> & a) {
    n = (int)a.size();
    k = log2(n) + 2;
    left.assign(k + 1, vector<int>(n));
    right.assign(k + 1, vector<int>(n));
    for(int j = 0; (1 << j) <= n; ++j) {
      int mask = (1 << j) - 1;
      int nw = 1; //neutral
      for(int i = 0; i < n; ++i) {
        nw = 1LL * nw * a[i] % mod;//prefix value
        left[j][i] = nw;
        if((i & mask) == mask) nw = 1; //neutral
      }
      nw = 1; //neutral
      for(int i = n - 1; i >= 0; --i) {
        nw = 1LL * nw * a[i] % mod;//prefix value
        right[j][i] = nw;
        if((i & mask) == 0) nw = 1; //neutral
      }
    }
  }
  int query(int l, int r) {
    if(l == r) return left[0][l];
    int i = 31 - __builtin_clz(l ^ r);
    int uno = left[i][r];
    int dos = right[i][l];
    return 1LL * uno * dos % mod;
  }
};

int32_t main() {
  int tc = in();
  while(tc--) {
    int n = in(), p = in(), q = in();
    mod = p;
    vector<int> a(n);
    for(int i = 0; i < n; i++) a[i] = in();
    DST t(a);
    vector<int> b((q >> 6) + 2);
    for(int i = 0; i < (int)b.size(); i++) b[i] = in();
    int x = 0, l = 1, r = 1;
    for(int i = 0; i < q; i++) {
      if(i % 64 == 0) {
        l = (b[i / 64] + x) % n;
        r = (b[(i / 64) + 1] + x) % n;
      } else {
        l = (l + x) % n;
        r = (r + x) % n;
      }
      if(l > r) swap(l, r);
      int ans = t.query(l, r);
      x = ans;
      x++;
      x %= mod;
    }
    printf("%d\n", x);
  }
  return 0;
}
```

3.6 Disjoint Set Union Bipartite

```cpp
/*
  DSU with Parity - Bipartite Checker
  -----------------------------------
  Indexing: 0-based
  Node Bounds: [0, n-1] inclusive

  Features:
    - Tracks parity (even/odd length) of paths in each component
    - Can be used to detect odd-length cycles (non-bipartite components)
    - Supports dynamic edge additions

  Functions:
    - make_set(v): initializes singleton component
    - find_set(v): returns root of v and its parity relative to root
    - add_edge(a, b): merges two components and checks for bipartite violation
    - is_bipartite(v): returns whether component containing v is bipartite
*/

const int MAXN = 100005; // Set according to constraints

pair<int, int> parent[MAXN]; // parent[v] = {root, parity_from_root_to_v}
int rank[MAXN];              // Union by rank
bool bipartite[MAXN];        // bipartite[root] = true if component is bipartite

// Create a new set for node v
void make_set(int v) {
  parent[v] = {v, 0};   // Self-rooted, even parity to self
  rank[v] = 0;
  bipartite[v] = true;
}

// Find the root of v and track parity along the path (0 = even, 1 = odd)
pair<int, int> find_set(int v) {
  if (v != parent[v].first) {
    auto [par, parity] = parent[v];
    auto root = find_set(par);
    parent[v] = {root.first, parity ^ root.second}; // Path compression with parity update
  }
  return parent[v];
}

// Adds an edge between a and b, merges components and checks for odd cycles
void add_edge(int a, int b) {
  auto [ra, pa] = find_set(a); // ra = root of a, pa = parity from root to a
  auto [rb, pb] = find_set(b); // rb = root of b, pb = parity from root to b

  if (ra == rb) {
    // Same component: edge (a, b) adds a cycle -> check parity
    if ((pa ^ pb) == 0) {
      bipartite[ra] = false; // Found odd-length cycle
    }
  } else {
    // Merge smaller rank under larger
    if (rank[ra] < rank[rb]) swap(ra, rb), swap(pa, pb);

    // Make rb child of ra; update parity of root
    parent[rb] = {ra, pa ^ pb ^ 1};

    bipartite[ra] &= bipartite[rb]; // Component is only bipartite if both were
    if (rank[ra] == rank[rb]) rank[ra]++;
  }
}

// Check if the component containing v is bipartite
bool is_bipartite(int v) {
  return bipartite[find_set(v).first];
}
```

3.7 Disjoint Set Union

```cpp
/*
  Disjoint Set Union (Union-Find) with Rollback
  ---------------------------------------------
  Indexing: 0-based
```

```
  Node Bounds: [0, N-1]

  Features:
    - Path compression + union by size
    - Optional rollback to previous state
    - Supports dynamic connectivity in offline algorithms (e.g., divide
        & conquer)

  Functions:
    - get(x): find root of x
    - connected(a, b): check if a and b are in same component
    - size(x): size of component containing x
    - unite(x, y): union x and y, returns true if merged
    - time(): current rollback timestamp
    - rollback(t): revert to state at timestamp t
*/

struct DSU {
  vector<int> e;                    // e[x] < 0 -> root; size = -e[x]; e
      [x] >= 0 -> parent
  vector<pair<int, int>> st;       // rollback stack: stores (index, old
       value)

  DSU(int N) : e(N, -1) {}

  // Find with path compression
  int get(int x) {
    return e[x] < 0 ? x : get(e[x]);
  }

  // Check if x and y belong to the same component
  bool connected(int a, int b) {
    return get(a) == get(b);
  }

  // Return size of component containing x
  int size(int x) {
    return -e[get(x)];
  }

  // Union by size, returns true if union happened
  bool unite(int x, int y) {
    x = get(x), y = get(y);
    if (x == y) return false;
    if (e[x] > e[y]) swap(x, y); // Ensure x has larger size (more
        negative)
    st.push_back({x, e[x]});
    st.push_back({y, e[y]});
    e[x] += e[y];
    e[y] = x;
    return true;
  }

  // Return current rollback timestamp
  int time() {
    return (int)st.size();
  }

  // Roll back to previous state at time t
  void rollback(int t) {
    for (int i = time(); i-- > t;) {
      e[st[i].first] = st[i].second;
    }
    st.resize(t);
  }
};
```

3.8 Disjoint Set Union Queue Like Rollback

```
#include <bits/stdc++.h>
    #define finish(x) return cout << x << endl, 0
    typedef long long ll;
    typedef long double ldb;
    const int md = 1e9 + 7;
    const ll inf = 2e18;
    const int OO = 1;
    const int OOO = 1;
    using namespace std;

    // used in the original data structure to represent a change in
        memory cell
    struct change {
      int t, value;
      int *cell;
      change() {}
      change(int _time, int _value, int *_cell) {
```

```
      t = _time;
      value = _value;
      cell = _cell;
    }
    void undo() {
      *cell = value;
    }
  };

  // dsu with normal undo, allowes to query bipartite-ness.
  struct dsu {
    int n;
    vector<int> p, r;
    vector<int> up;
    int nxtupd;
    vector<change> stack;
    int first_violate;
    dsu() {}
    dsu(int sz) {
      n = sz;
      p.resize(n);
      r.resize(n, 0);
      up.resize(n, 0);
      nxtupd = 0;
      first_violate = -1;
      for (int i = 0; i < n; i++) p[i] = i;
    }
    void upd(int *cell, int value) {
      stack.push_back(change(nxtupd, *cell, cell));
      *cell = value;
    }
    pair<int, int> find(int x) {
      if (x == p[x]) return{ x, 0 };
      pair<int, int> tmp = find(p[x]);
      tmp.second ^= up[x];
      return tmp;
    }
    bool is_bipartite() {
      return first_violate == -1; // no violation
    }
    void unite(int x, int y) {
      nxtupd++;
      int col = 1;
      pair<int, int> tmp = find(x);
      x = tmp.first, col ^= tmp.second;
      tmp = find(y);
      y = tmp.first, col ^= tmp.second;
      if (x == y) {
        if (col == 1 && first_violate == -1) first_violate = nxtupd;
        upd(&p[0], p[0]); // mark this update's existence.
        return;
      }
      if (r[x] < r[y]) {
        upd(&p[x], y);
        upd(&up[x], col);
      }
      else {
        upd(&p[y], x);
        upd(&up[y], col);
        if (r[x] == r[y]) {
          upd(&r[x], r[x] + 1);
        }
      }
      return;
    }
    void undo() {
      if (!stack.size()) return;
      int t = stack.back().t;
      if (first_violate == t) first_violate = -1;
      while (stack.size() && stack.back().t == t) {
        stack.back().undo();
        stack.pop_back();
      }
    }
  };

  // I'd want this 'struct update' to be more generic, and support
      maintaining different kinds of updates...
  // but for now I'm not sure how. So this is a specific impl for this
       problem ('update' only relates to 'unite').
  struct update {
    char type; // 'A' or 'B'
    int x, y;
    update() {}
    update(int xx, int yy) {
      x = xx;
```

```
      y = yy;
      type = 'B';
    }
  };

  struct dsuqueue {
    dsu D;
    vector<update> S;
    int bottom; // bottom of the stack: S[0..bottom-1] is entirely B's
        .
    dsuqueue() {}
    dsuqueue(int sz) {
      D = dsu(sz);
      bottom = 0;
    }
    // utility:
    void advance_bottom() {
      while (bottom < S.size() && S[bottom].type == 'B') bottom++;
    }
    void fix() {
      if (!S.size() || S.back().type == 'A') return;
      advance_bottom();
      vector<update> saveB, saveA;
      saveB.push_back(S.back());
      S.pop_back(), D.undo();
      while (saveA.size() != saveB.size() && S.size() > bottom) {
        if (S.back().type == 'A')
          saveA.push_back(S.back());
        else
          saveB.push_back(S.back());
        S.pop_back(), D.undo();
      }
      // reverse saveA and saveB so their relative order is maintained
      reverse(saveA.begin(), saveA.end());
      reverse(saveB.begin(), saveB.end());
      for (const update &u : saveB) {
        S.push_back(u);
        D.unite(u.x, u.y);
      }
      for (const update &u : saveA) {
        S.push_back(u);
        D.unite(u.x, u.y);
      }
      advance_bottom();
    }
    void reverse_updates() {
      for (int i = 0; i < S.size(); i++)
        D.undo();
      for (int i = (int)S.size() - 1; i >= 0; i--) {
        D.unite(S[i].x, S[i].y);
        S[i].type = 'A';
      }
      reverse(S.begin(), S.end());
      bottom = 0;
    }
    void undo() {
      advance_bottom();
      if (bottom == S.size()) {
        // no more A's, let's reverse and begin again.
        reverse_updates();
      }
      fix();
      D.undo();
      S.pop_back();
    }

    // begin copying all functions from the original dsu.
    bool is_bipartite() {
      return D.is_bipartite();
    }
    void unite(int x, int y) {
      D.unite(x, y);
      S.push_back(update(x, y));
    }
  };

  int n, m, q;
  vector<pair<int, int>> e;
  vector<int> R; // for each i, R[i] would be the minimum such that
      query (i, R[i]) is answered with NO (the graph is bipartite)

  int main() {
    ios::sync_with_stdio(0), cin.tie(0);
    cin >> n >> m >> q;
    dsuqueue D(n);
    e.resize(m);
```

```
    for (auto &i : e) {
      cin >> i.first >> i.second;
      --i.first, --i.second;
    }

    R.resize(m);
    for (int i = 0; i < m; i++)
      D.unite(e[i].first, e[i].second);
    int nxt = 0;
    for (int i = 0; i < m; i++) {
      while (!D.is_bipartite() && nxt < m) {
        D.undo();
        nxt++;
      }
      if (D.is_bipartite())
        R[i] = nxt - 1;
      else
        R[i] = md;

      D.unite(e[i].first, e[i].second);
    }

    while (q--) {
      int l, r;
      cin >> l >> r;
      --l, --r;
      if (R[l] <= r) cout << "NO\n";
      else cout << "YES\n";
    }
  }
```

3.9 Dynamic Conectivity

```
/*
  Offline Dynamic Connectivity - Segment Tree + Rollback DSU
  ------------------------------------------------------------
  Indexing: 0-based
  Node Bounds: [0, n-1]

  Features:
    - Handles dynamic edge insertions and deletions over time
    - Answers queries of type: "how many connected components at time t
      ?"
    - All operations are processed offline

  Components:
    - DSU with rollback (stores a stack of previous states)
    - Segment tree over time to store active edge intervals
*/

// Rollbackable Disjoint Set Union (Union-Find)
struct DSU {
  vector<int> e; // e[x] < 0 means x is a root, and size is -e[x]
  vector<pair<int, int>> st; // Stack for rollback (stores changed
      values)
  int cnt; // Current number of connected components

  DSU() {}
  DSU(int N) : e(N, -1), cnt(N) {}

  // Find root of x with path compression
  int get(int x) {
    return e[x] < 0 ? x : get(e[x]);
  }

  // Check if x and y are connected
  bool connected(int a, int b) {
    return get(a) == get(b);
  }

  // Size of component containing x
  int size(int x) {
    return -e[get(x)];
  }

  // Union two components; record state for rollback
  bool unite(int x, int y) {
    x = get(x), y = get(y);
    if (x == y) return false; // Already connected

    if (e[x] > e[y]) swap(x, y); // Union by size: ensure x is larger

    // Save state for rollback
    st.push_back({x, e[x]});
    st.push_back({y, e[y]});

```

```
    e[x] += e[y]; // Merge y into x
    e[y] = x;
    cnt--; // One fewer component
    return true;
  }

  // Undo last union
  void rollback() {
    auto [x1, y1] = st.back(); st.pop_back();
    e[x1] = y1;
    auto [x2, y2] = st.back(); st.pop_back();
    e[x2] = y2;
    cnt++;
  }
};

// Represents a single union operation (on edge u-v)
struct query {
  int v, u;
  bool united;
  query(int _v, int _u) : v(_v), u(_u), united(false) {}
};

// Segment Tree for storing edge intervals [l, r]
// be careful with the insertion time and sizes
struct QueryTree {
  vector<vector<query>> t; // Each node stores queries that are active
      in that time segment
  DSU dsu;
  int T; // Number of total operations (time steps)

  QueryTree() {}
  QueryTree(int _T, int n) : T(_T) {
    dsu = DSU(n);
    t.resize(4 * T + 4);
  }

  // Internal segment tree add function
  void add(int v, int l, int r, int ul, int ur, query& q) {
    if (ul > ur) return;
    if (l == ul && r == ur) {
      t[v].push_back(q);
      return;
    }
    int mid = (l + r) / 2;
    add(2 * v, l, mid, ul, min(ur, mid), q);
    add(2 * v + 1, mid + 1, r, max(ul, mid + 1), ur, q);
  }

  // Public wrapper: add a query in interval [l, r]
  void add_query(query q, int l, int r) {
    add(1, 0, T - 1, l, r, q);
  }

  // Traverse the segment tree and simulate unions with rollback
  void dfs(int v, int l, int r, vector<int>& ans) {
    // Apply all union operations in this segment node
    for (query& q : t[v]) {
      q.united = dsu.unite(q.v, q.u);
    }

    if (l == r) {
      ans[l] = dsu.cnt; // Save answer for time l
    } else {
      int mid = (l + r) / 2;
      dfs(2 * v, l, mid, ans);
      dfs(2 * v + 1, mid + 1, r, ans);
    }

    // Rollback all operations applied in this node
    for (query& q : t[v]) {
      if (q.united)
        dsu.rollback();
    }
  }
};

int main() {
  int n, k;
  cin >> n >> k; // n nodes, k operations
  if (k == 0) return 0;
  QueryTree st(k, n);
  map<pair<int, int>, int> mp; // Edge -> start time
  vector<int> ans(k);          // Answers for '?' queries
  vector<int> qmarks;          // Indices of '?' queries
  // Parse all k operations
```

```
  for (int i = 0; i < k; i++) {
    char c;
    cin >> c;
    if (c == '?') {
      qmarks.push_back(i); // Save query index
      continue;
    }
    int u, v;
    cin >> u >> v;
    u--; v--;
    if (u > v) swap(u, v); // Normalize edge direction

    if (c == '+') {
      mp[{u, v}] = i; // Mark time edge is added
    } else {
      // Edge removed: store active interval
      st.add_query(query(u, v), mp[{u, v}], i);
      mp[{u, v}] = -1;
    }
  }
  // Any edge still active is added until end of timeline
  for (auto [edge, start] : mp) {
    if (start != -1) {
      st.add_query(query(edge.first, edge.second), start, k - 1);
    }
  }
  // Process the tree to compute all '?'-query answers
  st.dfs(1, 0, k - 1, ans);
  // Output results of all '?'
  for (int x : qmarks) {
    cout << ans[x] << '\n';
  }
}
```

3.10 Fenwick Tree

```
/*
  Fenwick Tree (Binary Indexed Tree)
  ----------------------------------
  Indexing: 0-based
  Bounds: [0, n-1] inclusive
  Time Complexity:
    - add(x, v): O(log n)
    - sum(x): O(log n) -> prefix sum over [0, x)
    - rangeSum(l, r): O(log n) -> sum over [l, r)
    - select(k): O(log n) -> smallest x such that prefix sum >= k (works
        for monotonic cumulative sums)

  Space Complexity: O(n)
*/

template <typename T>
struct Fenwick {
  int n;
  std::vector<T> a;

  Fenwick(int n_ = 0) {
    init(n_);
  }

  // Initialize BIT of size n
  void init(int n_) {
    n = n_;
    a.assign(n, T{});
  }

  // Add value `v` to position `x`
  void add(int x, const T &v) {
    for (int i = x + 1; i <= n; i += i & -i) {
      a[i - 1] = a[i - 1] + v;
    }
  }

  // Compute prefix sum on range [0, x)
  T sum(int x) const {
    T ans{};
    for (int i = x; i > 0; i -= i & -i) {
      ans = ans + a[i - 1];
    }
    return ans;
  }

  // Compute sum over range [l, r)
  T rangeSum(int l, int r) const {
    return sum(r) - sum(l);
  }
```

```
50
51    // Find the smallest x such that sum[0, x) > k (if exists), or returns
          n
52    int select(const T &k) const {
53      int x = 0;
54      T cur{};
55      for (int i = 1 << std::__lg(n); i; i >>= 1) {
56        if (x + i <= n && cur + a[x + i - 1] <= k) {
57          cur = cur + a[x + i - 1];
58          x += i;
59        }
60      }
61      return x;
62    }
63  };
64  // Fenwick<int> bit(n);
```

3.11 Fenwick Tree 2D

```
1   /*
2     2D Fenwick Tree (Binary Indexed Tree)
3     -------------------------------------
4     Indexing: 1-based
5     Bounds: [1, n] inclusive
6     Time Complexity:
7       -update(x, y, v): O(log^2 n)
8       -get(x, y): sum of rectangle [1,1] to [x,y]
9       -get1(x1, y1, x2, y2): sum over rectangle [x1,y1] to [x2,y2]
10    Space Complexity: O(n^2)
11
12    -Can be adapted for rectangular grids by using n, m separately
13  */
14
15  struct Fenwick2D {
16    vector<vector<ll>> b; // 2D BIT array
17    int n;                // Grid size (1-based)
18
19    Fenwick2D(int _n) : b(_n + 5, vector<ll>(_n + 5, 0)), n(_n) {}
20
21    // Add 'val' to cell (x, y)
22    void update(int x, int y, int val) {
23      for (; x <= n; x += (x & -x)) {
24        for (int j = y; j <= n; j += (j & -j)) {
25          b[x][j] += val;
26        }
27      }
28    }
29
30    // Get sum of rectangle [(1,1) to (x,y)]
31    ll get(int x, int y) {
32      ll ans = 0;
33      for (; x > 0; x -= x & -x) {
34        for (int j = y; j > 0; j -= j & -j) {
35          ans += b[x][j];
36        }
37      }
38      return ans;
39    }
40
41    // Get sum over subrectangle [(x1,y1) to (x2,y2)]
42    ll get1(int x1, int y1, int x2, int y2) {
43      return get(x2, y2)-get(x1-1, y2)-get(x2, y1-1) + get(x1-1, y1-1);
44    }
45  };
46  // Usage example:
47  Fenwick2D fw(n);
48  fw.update(3, 4, 5);                 // add 5 to (3, 4)
49  ll sum = fw.get(3, 4);              // sum from (1,1) to (3,4)
50  ll range = fw.get1(2, 2, 5, 5);     // sum in rectangle [(2,2)-(5,5)]
```

3.12 Li Chao Tree

```
1   const ll inf = 2e18;
2
3   struct Line {
4     ll m, c;
5     ll eval(ll x) {
6       return m * x + c;
7     }
8   };
9   struct node {
10    Line line;
11    node* left = nullptr;
12    node* right = nullptr;
13    node(Line line) : line(line) {}
14    void add_segment(Line nw, int l, int r, int L, int R) {
```

```
    if (l > r || r < L || l > R) return;
    int m = (l + 1 == r ? l : (l + r) / 2);
    if (l >= L and r <= R) {
      bool lef = nw.eval(l) < line.eval(l);
      bool mid = nw.eval(m) < line.eval(m);
      if (mid) swap(line, nw);
      if (l == r) return;
      if (lef != mid) {
        if (left == nullptr) left = new node(nw);
        else left -> add_segment(nw, l, m, L, R);
      }
      else {
        if (right == nullptr) right = new node(nw);
        else right -> add_segment(nw, m + 1, r, L, R);
      }
      return;
    }
    if (max(l, L) <= min(m, R)) {
      if (left == nullptr) left = new node({0, inf});
      left -> add_segment(nw, l, m, L, R);
    }
    if (max(m + 1, L) <= min(r, R)) {
      if (right == nullptr) right = new node ({0, inf});
      right -> add_segment(nw, m + 1, r, L, R);
    }
  }
  ll query_segment(ll x, int l, int r, int L, int R) {
    if (l > r || r < L || l > R) return inf;
    int m = (l + 1 == r ? l : (l + r) / 2);
    if (l >= L and r <= R) {
      ll ans = line.eval(x);
      if (l < r) {
        if (x <= m && left != nullptr) ans = min(ans, left ->
            query_segment(x, l, m, L, R));
        if (x > m && right != nullptr) ans = min(ans, right ->
            query_segment(x, m + 1, r, L, R));
      }
      return ans;
    }
    ll ans = inf;
    if (max(l, L) <= min(m, R)) {
      if (left == nullptr) left = new node({0, inf});
      ans = min(ans, left -> query_segment(x, l, m, L, R));
    }
    if (max(m + 1, L) <= min(r, R)) {
      if (right == nullptr) right = new node ({0, inf});
      ans = min(ans, right -> query_segment(x, m + 1, r, L, R));
    }
    return ans;
  }
};

struct LiChaoTree {
  int L, R;
  node* root;
  LiChaoTree() : L(numeric_limits<int>::min() / 2), R(numeric_limits<int
      >::max() / 2), root(nullptr) {}
  LiChaoTree(int L, int R) : L(L), R(R) {
    root = new node({0, inf});
  }
  void add_line(Line line) {
    root -> add_segment(line, L, R, L, R);
  }
  // y = mx + b: x in [l, r]
  void add_segment(Line line, int l, int r) {
    root -> add_segment(line, L, R, l, r);
  }
  ll query(ll x) {
    return root -> query_segment(x, L, R, L, R);
  }
  ll query_segment(ll x, int l, int r) {
    return root -> query_segment(x, l, r, L, R);
  }
};

int32_t main() {
  ios_base::sync_with_stdio(0);
  cin.tie(0);
  LiChaoTree t = LiChaoTree((int)-1e9, (int) 1e9);
  int n, q; cin >> n >> q;
  for (int i = 0; i < n; i++) {
    ll l, r, a, b; cin >> l >> r >> a >> b;
    r--;
    t.add_segment({a, b}, l, r);
  }
  while (q--) {
```

```
    int ty; cin >> ty;
    if (ty == 0) {
      ll l, r, a, b; cin >> l >> r >> a >> b;
      r--;
      t.add_segment({a, b}, l, r);
    }
    else {
      ll x; cin >> x;
      ll ans = t.query(x);
      if (ans >= inf) cout << "INFINITY\n";
      else cout << ans << '\n';
    }
  }
}


// min of a * x^3 + b * x ^2 + c * x + d

#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

const int N = (int)1e5 + 5;
const int C = (int)1e5 + 5;
const int T = 10;
const int M = 350; // sqrt(10^5)
const ll inf = (ll)1e19;

namespace segtree {
  struct Poly {
    ll a, b, c, d = inf;
    ll operator()(ll x) { return a * x * x * x + b * x * x + c * x + d;
        }
  } a[T][C * 4];
  int t = -1;

  void insert(int l, int r, Poly poly, int o=0) {
    if(l + 1 == r) {
      if(poly(l) < a[t][o](l)) a[t][o] = poly;
      return;
    }
    int mid = (l + r) >> 1, lson = o * 2 + 1, rson = o * 2 + 2;
    bool b1 = poly(mid) < a[t][o](mid), b2 = poly(l) < a[t][o](l);
    if(b1) swap(poly, a[t][o]);
    if(b1 != b2) insert(l, mid, poly, lson);
    else insert(mid, r, poly, rson);
  }
  ll query(int l, int r, int x, int o=0) {
    if(l + 1 == r) return a[t][o](x);
    int mid = (l + r) >> 1, lson = o * 2 + 1, rson = o * 2 + 2;
    if(x < mid) return min(a[t][o](x), query(l, mid, x, lson));
    else return min(a[t][o](x), query(mid, r, x, rson));
  }
}

ll ans[M];

void init() {
  int n; cin >> n;
  fill(ans, ans + M, inf);
  segtree::t++;
  while(n--) {
    int a, b, c, d; cin >> d >> c >> b >> a;
    segtree::Poly p = {a, b, c, d};
    segtree::insert(M, C, p);
    for(int i = 0 ; i < M ; ++i) ans[i] = min(ans[i], p(i));
  }
}
void solve() {
  int q; cin >> q;
  while(q--) {
    int x; cin >> x;
    if(x < M) cout << ans[x] << '\n';
    else cout << segtree::query(M, C, x) << '\n';
  }
}

int main() {
  ios_base::sync_with_stdio(0), cin.tie(0);
  int t; cin >> t;
  while(t--) {
    init();
    solve();
  }
}
```

3.13 Link Cut Tree

```cpp
#include<bits/stdc++.h>
using namespace std;

const int N = 1e5 + 9;

struct node {
  int p = 0, c[2] = {0, 0}, pp = 0;
  bool flip = 0;
  int sz = 0, ssz = 0, vsz = 0; // sz -> aux tree size, ssz = subtree
      size in rep tree, vsz = virtual tree size
  long long val = 0, sum = 0, lazy = 0, subsum = 0, vsum = 0;
  node() {}
  node(int x) {
    val = x; sum = x;
    sz = 1; lazy = 0;
    ssz = 1; vsz = 0;
    subsum = x; vsum = 0;
  }
};
struct LCT {
  vector<node> t;
  LCT() {}
  LCT(int n) : t(n + 1) {}

  // <independant splay tree code>
  int dir(int x, int y) { return t[x].c[1] == y; }
  void set(int x, int d, int y) {
    if (x) t[x].c[d] = y, pull(x);
    if (y) t[y].p = x;
  }
  void pull(int x) {
    if (!x) return;
    int &l = t[x].c[0], &r = t[x].c[1];
    push(l); push(r);
    t[x].sum = t[l].sum + t[r].sum + t[x].val;
    t[x].sz = t[l].sz + t[r].sz + 1;
    t[x].ssz = t[l].ssz + t[r].ssz + t[x].vsz + 1;
    t[x].subsum = t[l].subsum + t[r].subsum + t[x].vsum + t[x].val;
  }
  void push(int x) {
    if (!x) return;
    int &l = t[x].c[0], &r = t[x].c[1];
    if (t[x].flip) {
      swap(l, r);
      if (l) t[l].flip ^= 1;
      if (r) t[r].flip ^= 1;
      t[x].flip = 0;
    }
    if (t[x].lazy) {
      t[x].val += t[x].lazy;
      t[x].sum += t[x].lazy * t[x].sz;
      t[x].subsum += t[x].lazy * t[x].ssz;
      t[x].vsum += t[x].lazy * t[x].vsz;
      if (l) t[l].lazy += t[x].lazy;
      if (r) t[r].lazy += t[x].lazy;
      t[x].lazy = 0;
    }
  }
  void rotate(int x, int d) {
    int y = t[x].p, z = t[y].p, w = t[x].c[d];
    swap(t[x].pp, t[y].pp);
    set(y, !d, w);
    set(x, d, y);
    set(z, dir(z, y), x);
  }
  void splay(int x) {
    for (push(x); t[x].p;) {
      int y = t[x].p, z = t[y].p;
      push(z); push(y); push(x);
      int dx = dir(y, x), dy = dir(z, y);
      if (!z) rotate(x, !dx);
      else if (dx == dy) rotate(y, !dx), rotate(x, !dx);
      else rotate(x, dy), rotate(x, dx);
    }
  }
  // </independant splay tree code>

  // making it a root in the rep. tree
  void make_root(int u) {
    access(u);
    int l = t[u].c[0];
    t[l].flip ^= 1;
    swap(t[l].p, t[l].pp);
    t[u].vsz += t[l].ssz;
```

```
    t[u].vsum += t[l].subsum;
    set(u, 0, 0);
  }
  // make the path from root to u a preferred path
  // returns last path-parent of a node as it moves up the tree
  int access(int _u) {
    int last = _u;
    for (int v = 0, u = _u; u; u = t[v = u].pp) {
      splay(u); splay(v);
      t[u].vsz -= t[v].ssz;
      t[u].vsum -= t[v].subsum;
      int r = t[u].c[1];
      t[u].vsz += t[r].ssz;
      t[u].vsum += t[r].subsum;
      t[v].pp = 0;
      swap(t[r].p, t[r].pp);
      set(u, 1, v);
      last = u;
    }
    splay(_u);
    return last;
  }
  void link(int u, int v) { // u -> v
    // assert(!connected(u, v));
    make_root(v);
    access(u); splay(u);
    t[v].pp = u;
    t[u].vsz += t[v].ssz;
    t[u].vsum += t[v].subsum;
  }
  void cut(int u) { // cut par[u] -> u, u is non root vertex
    access(u);
    assert(t[u].c[0] != 0);
    t[t[u].c[0]].p = 0;
    t[u].c[0] = 0;
    pull(u);
  }
  // parent of u in the rep. tree
  int get_parent(int u) {
    access(u); splay(u); push(u);
    u = t[u].c[0]; push(u);
    while (t[u].c[1]) {
      u = t[u].c[1]; push(u);
    }
    splay(u);
    return u;
  }
  // root of the rep. tree containing this node
  int find_root(int u) {
    access(u); splay(u); push(u);
    while (t[u].c[0]) {
      u = t[u].c[0]; push(u);
    }
    splay(u);
    return u;
  }
  bool connected(int u, int v) {
    return find_root(u) == find_root(v);
  }
  // depth in the rep. tree
  int depth(int u) {
    access(u); splay(u);
    return t[u].sz;
  }
  int lca(int u, int v) {
    // assert(connected(u, v));
    if (u == v) return u;
    if (depth(u) > depth(v)) swap(u, v);
    access(v);
    return access(u);
  }
  int is_root(int u) {
    return get_parent(u) == 0;
  }
  int component_size(int u) {
    return t[find_root(u)].ssz;
  }
  int subtree_size(int u) {
    int p = get_parent(u);
    if (p == 0) {
      return component_size(u);
    }
    cut(u);
    int ans = component_size(u);
    link(p, u);
    return ans;
```

```
  }
  long long component_sum(int u) {
    return t[find_root(u)].subsum;
  }
  long long subtree_sum(int u) {
    int p = get_parent(u);
    if (p == 0) {
      return component_sum(u);
    }
    cut(u);
    long long ans = component_sum(u);
    link(p, u);
    return ans;
  }
  // sum of the subtree of u when root is specified
  long long subtree_query(int u, int root) {
    int cur = find_root(u);
    make_root(root);
    long long ans = subtree_sum(u);
    make_root(cur);
    return ans;
  }
  // path sum
  long long query(int u, int v) {
    int cur = find_root(u);
    make_root(u); access(v);
    long long ans = t[v].sum;
    make_root(cur);
    return ans;
  }
  void upd(int u, int x) {
    access(u); splay(u);
    t[u].val += x;
  }
  // add x to the nodes on the path from u to v
  void upd(int u, int v, int x) {
    int cur = find_root(u);
    make_root(u); access(v);
    t[v].lazy += x;
    make_root(cur);
  }
}t[2];

```

```
vector<int> g[N];
int par[N];
void dfs(int u, int p = 0) {
  par[u] = p;
  for (auto v: g[u]) {
    if (v ^ p) {
      dfs(v, u);
    }
  }
}
int col[N];
int32_t main() {
  ios_base::sync_with_stdio(0);
  cin.tie(0);
  int n; cin >> n;
  for (int i = 1; i < n; i++) {
      int u, v; cin >> u >> v;
      g[u].push_back(v);
      g[v].push_back(u);
  }
  dfs(1, n + 1);
  t[0] = LCT(n + 1);
  t[1] = LCT(n + 1);
  for (int i = 1; i <= n + 1; i++) {
    t[0].t[i] = node(i);
    t[1].t[i] = node(i);
  }
  for (int i = 1; i <= n; i++) {
    t[0].link(par[i], i);
  }
  int q; cin >> q;
  while (q--) {
    int ty, u; cin >> ty >> u;
    if (ty == 0) {
      int z = t[col[u]].find_root(u);
      int c = t[col[u]].t[z].c[1];
      cout << t[col[u]].t[c].ssz << '\n';
    }
    else {
      t[col[u]].cut(u);
      col[u] ^= 1;
      t[col[u]].link(par[u], u);
    }
```

```
  }
  return 0;
}
```

3.14 Linked List

```
/*
  Linked list
  ------------------------------------
  Time Complexity:
    -insert: O(1)
  Space Complexity: O(n)

  -Can be adapted for rectangular grids by using n, m separately
*/
// === Constructors and Destructor ===
// list()                // Default constructor, creates an empty list.
// list(n, val)          // Creates a list with 'n' elements, all initialized to 'val'.
// list(first, last)     // Creates a list from another range (e.g., another list or array).
// list(const list& other) // Copy constructor.
// list(list&& other)    // Move constructor.
// ~list()               // Destructor, destroys the list.

// === Iterators ===
// begin()               // Returns an iterator to the first element of the list.
// end()                 // Returns an iterator to the position just past the last element.
// rbegin()              // Returns a reverse iterator to the last element.
// rend()                // Returns a reverse iterator to the position just before the first element.
// insert(it, val)       // Inserts 'val' before the position of the iterator 'it'.
// erase(it)             // Removes the element at the position of the iterator 'it'.
// erase(first, last)    // Removes a range of elements from 'first' to 'last'.
// push_back(val)        // Adds 'val' to the end of the list.
// push_front(val)       // Adds 'val' to the front of the list.
// pop_back()            // Removes the last element of the list.
// pop_front()           // Removes the first element of the list.
// clear()               // Removes all elements from the list.

// === Capacity ===
// empty()               // Returns true if the list is empty, otherwise false.
// size()                // Returns the number of elements in the list.
// max_size()            // Returns the maximum number of elements the list can hold.

// === Modifiers ===
// insert(it, n, val)    // Inserts 'n' copies of 'val' before the position of 'it'.
// insert(it, first, last) // Inserts a range of elements before the position of 'it'.
// remove(val)           // Removes all occurrences of 'val' from the list.
// remove_if(pred)       // Removes all elements that satisfy the predicate 'pred'.
// unique()              // Removes consecutive duplicate elements from the list.
// merge(other)          // Merges two sorted lists into one sorted list.
// merge(other, pred)    // Merges two sorted lists into one sorted list using custom comparison 'pred'.
// splice(it, other)     // Transfers elements from another list 'other' to the list at position 'it'.
// splice(it, other, it2) // Transfers an element from another list 'other' at position 'it2' to the list at position 'it'.
// splice(it, other, first, last) // Transfers a range from another list 'other' from 'first' to 'last' to the list at position 'it'.
// reverse()             // Reverses the order of elements in the list.
// sort()                // Sorts the elements in the list in ascending order.
// sort(comp)            // Sorts the elements in the list using custom comparison 'comp'.
// swap(other)           // Swaps the content of the list with another list 'other'.

// === Element Access ===
// front()               // Returns a reference to the first element in the list.
// back()                // Returns a reference to the last element in
```

```
    the list.

// === Search ===
// find(val)             // Returns an iterator to the first occurrence of 'val'.
// find_if(pred)         // Returns an iterator to the first element satisfying the predicate 'pred'.
// count(val)            // Returns the number of occurrences of 'val' in the list.
// count_if(pred)        // Returns the number of elements satisfying the predicate 'pred'.
// equal_range(val)      // Returns a pair of iterators representing the range of elements equal to 'val'.

// === Other Functions ===
// swap(other)           // Swaps the contents of the list with another list 'other'.
// emplace(it, args...)  // Constructs and inserts an element at the position 'it' using 'args...' (perfect forwarding).
// emplace_front(args...) // Constructs and inserts an element at the front using 'args...' (perfect forwarding).
// emplace_back(args...)  // Constructs and inserts an element at the back using 'args...' (perfect forwarding).
```

3.15 Merge Sort Tree

```
/*
  Merge Sort Tree (Segment Tree of Sorted Arrays)
  -----------------------------------------------
  Indexing: 0-based
  Node Bounds: [0, n-1] inclusive
  Time Complexity:
    - build(): O(n log n)
    - q(l, r, x): O(log^2 n) -> number of elements <= x in [l, r)
  Space Complexity: O(n log n)

  Features:
    - Supports frequency/count queries: "how many values <= x in range [l, r)?"
    - Static array (no point updates unless rebuilt)
*/

const int MAXN = 100005;        // size of original array
const int MAXT = 2 * MAXN;      // size of segment tree (2n)

vector<int> t[MAXT];    // Segment tree: each node holds sorted vector
int a[MAXN];            // Original array
int n;                  // Size of array

// Build merge sort tree (bottom-up)
void build() {
  // Fill leaves
  for (int i = 0; i < n; i++) {
    t[i + n].push_back(a[i]);
  }

  // Merge children into parent
  for (int i = n - 1; i > 0; i--) {
    auto &left = t[2 * i], &right = t[2 * i + 1];
    merge(left.begin(), left.end(), right.begin(), right.end(),
        back_inserter(t[i]));
  }
}

// Query how many elements <= `x` in range [l, r)
int q(int l, int r, int x) {
  int res = 0;
  for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
    if (l & 1) {
      res += upper_bound(t[l].begin(), t[l].end(), x) - t[l].begin();
      l++;
    }
    if (r & 1) {
      r--;
      res += upper_bound(t[r].begin(), t[r].end(), x) - t[r].begin();
    }
  }
  return res;
}
// Read n and array a, then call build()
```

3.16 Minimum Cartesian Tree

```
/*
  Min Cartesian Tree
  ------------------
```

```
  Indexing: 0-based
  Time Complexity: O(n)
  Space Complexity: O(n)
  Tree Properties:
    - Binary tree where in-order traversal = original array
    - Tree satisfies min-heap property: parent <= children
    - `par[i]`: parent of node i
    - `sons[i][0]`: left child, `sons[i][1]`: right child
    - `root`: index of root node

  Use cases:
    - RMQ construction
    - LCA over RMQ via Cartesian Tree
*/

struct min_cartesian_tree {
  vector<int> par;           // parent for each node
  vector<vector<int>> sons;  // left and right children
  int root;

  void init(int n, const vector<int> &arr) {
    par.assign(n, -1);
    sons.assign(n, vector<int>(2, -1)); // 0 = left, 1 = right
    stack<int> st;

    for (int i = 0; i < n; i++) {
      int last = -1;

      // Maintain increasing stack -> build min Cartesian Tree
      // Change > to < for max Cartesian Tree
      while (!st.empty() && arr[st.top()] > arr[i]) {
        last = st.top();
        st.pop();
      }

      if (!st.empty()) {
        par[i] = st.top();
        sons[st.top()][1] = i;
      }
      if (last != -1) {
        par[last] = i;
        sons[i][0] = last;
      }

      st.push(i);
    }

    for (int i = 0; i < n; i++) {
      if (par[i] == -1) {
        root = i;
      }
    }
  }
};
// Example usage:
vector<int> a = {4, 2, 6, 1, 3};
min_cartesian_tree ct;
ct.init(a.size(), a);
cout << "Root index: " << ct.root << '\n';
```

3.17 Multi Ordered Set

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
template <typename T> using oset = __gnu_pbds::tree<
    T, __gnu_pbds::null_type, less<T>, __gnu_pbds::rb_tree_tag,
    __gnu_pbds::tree_order_statistics_node_update
>;

//en main

oset<pair<int,int>> name;
    map<int,int> cuenta;
    function<void(int)> meter = [&] (int val) {
        name.insert({val,++cuenta[val]});
    };
    auto quitar = [&] (int val) {
        name.erase({val,cuenta[val]--});
    };

meter(x);
quitar(y);
multioset.order_of_key({y+1,-1})-multioset.order_of_key({x,0})
```

3.18 Ordered Set

```cpp
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
template <typename T> using oset = __gnu_pbds::tree<
    T, __gnu_pbds::null_type, less<T>, __gnu_pbds::rb_tree_tag,
    __gnu_pbds::tree_order_statistics_node_update
>;
// order_of_key() primero mayor o igual;
// find_by_order() apuntador al elemento k;
// oset<pair<int,int>> os;
// os.insert({1,2});
// os.insert({2,3});
// os.insert({5,6});
// ll k=os.order_of_key({2,0});
// cout<<k<<endl; // 1
// pair<int,int> p=*os.find_by_order(k);
// cout<<p.f<<" "<<p.s<<endl; // 2 3
// os.erase(p);
// p=*os.find_by_order(k);
// cout<<p.f<<" "<<p.s<<endl; // 5 6


// check if upperbound or lowerbound does what you want
// because they give better time.

// to allow repetitions
#define ordered_set tree<int, null_type,less_equal<int>, rb_tree_tag,
    tree_order_statistics_node_update>

// to not allow repetitions
#define ordered_set tree<int, null_type,less<int>, rb_tree_tag,
    tree_order_statistics_node_update>

//order_of_key(x): number of items are strictly smaller than x

//find_by_order(k) iterator to the kth element
```

3.19 Palindromic Tree

```cpp
/*
  Palindromic Tree (Eertree)
  --------------------------
  Indexing: 0-based
  Time Complexity:
    - extend(i): O(1) amortized
    - calc_occurrences(): O(n)
  Space Complexity: O(n)

  Features:
    - Each node represents a unique palindromic substring
    - Efficient online construction
    - `oc`: how many times this palindrome occurs as suffix
    - `cnt`: number of palindromic suffixes in its subtree
    - `link`: suffix link to longest proper palindromic suffix
*/

const int N = 3e5 + 9;

struct PalindromicTree {
  struct node {
    int nxt[26];      // transitions by character
    int len;          // length of palindrome
    int st, en;       // start and end indices in string
    int link;         // suffix link
    int cnt = 0;      // number of palindromic suffixes
    int oc = 0;       // occurrences of the palindrome
  };

  string s;
  vector<node> t;
  int sz, last;

  PalindromicTree() {}
  PalindromicTree(const string &_s) {
    s = _s;
    int n = s.size();
    t.clear();
    t.resize(n + 5); // up to n + 2 distinct palindromes
    sz = 2;
    last = 2;
    // Root 1: imaginary (-1 length), simplifies links
    t[1].len = -1;
    t[1].link = 1;
    // Root 2: length 0, link to root 1
    t[2].len = 0;
    t[2].link = 1;
```

```
  }

  // Extend tree with s[pos], returns 1 if a new node is created
  int extend(int pos) {
    int cur = last;
    int ch = s[pos] - 'a';
    // Find longest suffix palindrome that can be extended
    while (true) {
      int curlen = t[cur].len;
      if (pos - 1 - curlen >= 0 && s[pos - 1 - curlen] == s[pos]) break;
      cur = t[cur].link;
    }

    if (t[cur].nxt[ch]) {
      // Already exists
      last = t[cur].nxt[ch];
      t[last].oc++;
      return 0;
    }
    // Create new node
    sz++;
    last = sz;
    t[sz].oc = 1;
    t[sz].len = t[cur].len + 2;
    t[cur].nxt[ch] = sz;
    t[sz].en = pos;
    t[sz].st = pos - t[sz].len + 1;

    if (t[sz].len == 1) {
      t[sz].link = 2;
      t[sz].cnt = 1;
      return 1;
    }
    // Compute suffix link
    while (true) {
      cur = t[cur].link;
      int curlen = t[cur].len;
      if (pos - 1 - curlen >= 0 && s[pos - 1 - curlen] == s[pos]) {
        t[sz].link = t[cur].nxt[ch];
        break;
      }
    }
    t[sz].cnt = 1 + t[t[sz].link].cnt;
    return 1;
  }
  // Accumulate total occurrences for each palindrome node
  void calc_occurrences() {
    for (int i = sz; i >= 3; i--) {
      t[t[i].link].oc += t[i].oc;
    }
  }
};

int main() {
  string s;
  cin >> s;
  PalindromicTree t(s);
  for (int i = 0; i < s.size(); i++) {
    t.extend(i);
  }
  t.calc_occurrences();
  long long total = 0;
  for (int i = 3; i <= t.sz; i++) {
    total += t.t[i].oc;
  }
  cout << total << '\n'; // Total palindromic substrings
  return 0;
}
```

3.20 Persistent Array

```
/*
  Persistent Array (via Persistent Segment Tree)
  ----------------------------------------------
  Indexing: 0-based
  Bounds: [0, n-1]
  Time Complexity:
    - point update: O(log n)
    - point query: O(log n)
  Space Complexity: O(log n) per update/version

  Features:
    - Supports point updates with full version history
    - Allows querying any version at any index
*/

```

```
struct Node {
  int val;
  Node *l, *r;

  // Leaf node with value
  Node(int x) : val(x), l(nullptr), r(nullptr) {}

  // Internal node with children (value is not used)
  Node(Node *ll, Node *rr) : val(0), l(ll), r(rr) {}
};

int n;
int a[100001];         // Initial array
Node *roots[100001];  // Roots of all versions (0-based)

// Build version 0 from initial array
Node *build(int l = 0, int r = n - 1) {
  if (l == r) return new Node(a[l]);
  int mid = (l + r) / 2;
  return new Node(build(l, mid), build(mid + 1, r));
}

// Create new version with a[pos] = val
Node *update(Node *node, int pos, int val, int l = 0, int r = n - 1) {
  if (l == r) return new Node(val);
  int mid = (l + r) / 2;
  if (pos <= mid)
    return new Node(update(node->l, pos, val, l, mid), node->r);
  else
    return new Node(node->l, update(node->r, pos, val, mid + 1, r));
}

// Query value at position `pos` in a given version (node)
int query(Node *node, int pos, int l = 0, int r = n - 1) {
  if (l == r) return node->val;
  int mid = (l + r) / 2;
  if (pos <= mid) return query(node->l, pos, l, mid);
  else return query(node->r, pos, mid + 1, r);
}

// External helper: get value at index in version
int get_item(int index, int version) {
  return query(roots[version], index);
}

// External helper: make new version based on `prev_version`, updating
    one index
void update_item(int index, int value, int prev_version, int new_version
    ) {
  roots[new_version] = update(roots[prev_version], index, value);
}

// Initializes version 0 from given array
void init_arr(int nn, int *init) {
  n = nn;
  for (int i = 0; i < n; i++) a[i] = init[i];
  roots[0] = build();
}
```

3.21 Persistent Segment Tree

```
/*
  Persistent Segment Tree (Point Updates, Range Queries)
  ------------------------------------------------------
  Indexing: 1-based
  Bounds: [1, n]
  Time Complexity:
    - Build: O(n)
    - Point update: O(log n) -> returns new version
    - Range query: O(log n)
    - Copy version: O(1)

  Features:
    - Each update creates a new tree version with shared unchanged nodes
    - Supports querying over any version
    - Useful in rollback problems, version history, and functional
        programming
*/

struct Node {
  ll val;     // segment sum
  Node *l, *r;

  // Leaf node
  Node(ll x) : val(x), l(nullptr), r(nullptr) {}

```

```
  // Internal node with children
  Node(Node *_l, Node *_r) {
    l = _l;
    r = _r;
    val = 0;
    if (l) val += l->val;
    if (r) val += r->val;
  }

  // Version clone (used when copying tree version directly)
  Node(Node *cp) : val(cp->val), l(cp->l), r(cp->r) {}
};

int n, sz = 1;
ll a[200001];          // Input array (1-indexed)
Node *t[200001];       // Roots of different versions (t[version_id])

// Build initial segment tree from array a[1..n]
Node *build(int l = 1, int r = n) {
  if (l == r) return new Node(a[l]);
  int mid = (l + r) / 2;
  return new Node(build(l, mid), build(mid + 1, r));
}

// Update position `pos` with `val` in given `node` version
Node *update(Node *node, int pos, int val, int l = 1, int r = n) {
  if (l == r) return new Node(val); // replace leaf
  int mid = (l + r) / 2;
  if (pos <= mid)
    return new Node(update(node->l, pos, val, l, mid), node->r);
  else
    return new Node(node->l, update(node->r, pos, val, mid + 1, r));
}

// Query sum over range [a, b] in given version
ll query(Node *node, int a, int b, int l = 1, int r = n) {
  if (r < a || l > b) return 0;         // No overlap
  if (l >= a && r <= b) return node->val; // Total overlap
  int mid = (l + r) / 2;
  return query(node->l, a, b, l, mid) + query(node->r, a, b, mid + 1, r)
      ;
}

```

```
int main() {
  ios_base::sync_with_stdio(false); cin.tie(NULL);

  int q;
  cin >> n >> q;
  for (int i = 1; i <= n; i++) {
    cin >> a[i];
  }

  // Build version 0
  t[0] = build();
  sz = 1;

  while (q--) {
    int ty;
    cin >> ty;

    if (ty == 1) {
      // Point update: create new version from t[k] with a[pos] = x
      int k, pos, x;
      cin >> k >> pos >> x;
      t[k] = update(t[k], pos, x);

    } else if (ty == 2) {
      // Range query on version k over [l, r]
      int k, l, r;
      cin >> k >> l >> r;
      cout << query(t[k], l, r) << '\n';

    } else if (ty == 3) {
      // Clone version k into new version
      int k;
      cin >> k;
      t[sz++] = new Node(t[k]);
    }
  }
  return 0;
}
```

3.22 Polynomial Queries

```
// adds d * (i - l + 1) + a to range [l, r]
// d * (i - l + 1) = d*(1-l) + d*i
```

```
// Sum = A x segment_length + B x sum_of_indices_in_segment

struct sum_t {
  ll val;
  static const ll null_v = 0;
  sum_t(): val(null_v) {}
  sum_t(ll v): val(v) {}
  sum_t op(sum_t& other) {
    return sum_t(val + other.val);
  }
  sum_t lazy_op(sum_t& v, int size) {
    return val + v.val * size;
  }
  sum_t lazy_op(sum_t& v, ll l, ll r) {
    return val + v.val * ((l + r) * (r - l + 1)) / 2;
  }
};

template <typename num_t>
struct segtree {
  int n;
  vector<num_t> tree, lazy, lazy1;

  void init(int s) {
    n = s;
    tree.assign(4 * n, num_t(0));
    lazy.assign(4 * n, num_t(0));
    lazy1.assign(4 * n, num_t(0));
    init(0, 0, n - 1);
  }

  num_t init(int i, int l, int r) {
    if (l == r) return tree[i] = num_t(0);

    int mid = (l + r) / 2;
    num_t left = init(2 * i + 1, l, mid);
    num_t right = init(2 * i + 2, mid + 1, r);
    return tree[i] = left.op(right);
  }

  // do -l if you want it to add 0 to l. adds d * (i - l + 1) + a to
      range [l, r]
  void update(int l, int r, ll d, ll a) {
    if (l > r) return;
    ll dd = d;
    d = (1 - l) * d + a;
    a = dd;
    update(0, 0, n - 1, l, r, d, a);
  }

  num_t update(int i, int tl, int tr, int ql, int qr, ll D, ll A) {
    eval_lazy(i, tl, tr);

    if (tr < ql || qr < tl) return tree[i];
    if (ql <= tl && tr <= qr) {
      lazy[i].val += D;
      lazy1[i].val += A;
      eval_lazy(i, tl, tr);
      return tree[i];
    }

    int mid = (tl + tr) / 2;
    num_t a = update(2 * i + 1, tl, mid, ql, qr, D, A);
    num_t b = update(2 * i + 2, mid + 1, tr, ql, qr, D, A);
    return tree[i] = a.op(b);
  }

  num_t query(int l, int r) {
    if (l > r) return num_t::null_v;
    return query(0, 0, n - 1, l, r);
  }

  num_t query(int i, int tl, int tr, int ql, int qr) {
    eval_lazy(i, tl, tr);

    if (ql <= tl && tr <= qr) return tree[i];
    if (tr < ql || qr < tl) return num_t::null_v;

    int mid = (tl + tr) / 2;
    num_t a = query(2 * i + 1, tl, mid, ql, qr);
    num_t b = query(2 * i + 2, mid + 1, tr, ql, qr);
    return a.op(b);
  }

  void eval_lazy(int i, int l, int r) {
    tree[i] = tree[i].lazy_op(lazy1[i], l, r);
```

```
    tree[i] = tree[i].lazy_op(lazy[i], r - l + 1);
    if (l != r) {
      lazy[2 * i + 1].val += lazy[i].val;
      lazy[2 * i + 2].val += lazy[i].val;
      lazy1[2 * i + 1].val += lazy1[i].val;
      lazy1[2 * i + 2].val += lazy1[i].val;
    }
    lazy[i] = lazy1[i] = num_t(0);
  }
};

int main(){
  ios_base::sync_with_stdio(false); cin.tie(NULL);
  int n, q; cin >> n >> q;
  segtree <sum_t> st;
  st.init(n);
  while(q--){
    int t; cin >> t;
    if(t == 1){
      int l, r, a, d; cin >> l >> r >> a >> d;
      l--; r--;
      st.update(l, r, d, a);
    }
    else{
      int l; cin >> l;
      l--;
      cout << st.query(l, l).val << endl;
    }
  }
}
```

3.23 Segment Tree

```
/*
  Segment Tree (Iterative, Range Minimum Query)
  ---------------------------------------------
  Indexing: 0-based
  Bounds: [0, n-1] inclusive
  Time Complexity:
    - update(pos, val): O(log n)
    - get(l, r): O(log n) -> query min in range [l, r)
  Space Complexity: O(2n)
*/
struct SegmentTree {
  vector<ll> a; // segment tree array
  int n;        // number of elements in original array

  SegmentTree(int _n) : a(2 * _n, 1e18), n(_n) {}
  // Update position `pos` to value `val`
  void update(int pos, ll val) {
    pos += n;      // move to leaf
    a[pos] = val;  // set value
    for (pos /= 2; pos > 0; pos /= 2) {
      a[pos] = min(a[2 * pos], a[2 * pos + 1]); // update parent
    }
  }
  // Get minimum value in range [l, r)
  ll get(int l, int r) {
    ll res = 1e18;
    for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
      if (l & 1) res = min(res, a[l++]); // if l is right child
      if (r & 1) res = min(res, a[--r]); // if r is left child
    }
    return res;
  }
};
```

3.24 Segment Tree Examples

C++17 generic recursive tree with point assignment and inclusive, zero-based range queries. Includes 18 examples of associative merges, including noncommutative operations. Build $O(n)$; update/query $O(\log n)$ for constant-time merges and constant-size values.

```
/*
    RECURSIVE SEGMENT TREE - C++17
    ==============================

    Compile and run:
      g++ -std=c++17 -O2 -Wall -Wextra -pedantic segtree_examples.cpp -o
          segtree_examples
      ./segtree_examples

    Copy SegTree into your contest code, then choose:
      1. T: the value/summary stored in each node.
      2. merge(left, right): how to combine two consecutive segments.
      3. identity: the summary of an empty segment.
```

```
13
14      Requirements:
15        merge(merge(a, b), c) == merge(a, merge(b, c))    (associative)
16        merge(identity, x) == merge(x, identity) == x
17
18      Commutativity is NOT required. Left-to-right order is preserved.
19      Subtraction and division are not associative, so cannot be used
            directly.
20
21      API:
22        SegTree st(values, identity, merge);    // type deduction in C++17
23        st.update(pos, value);                  // point assignment, NOT
              increment
24        st.query(l, r);                         // 0-indexed, both endpoints
              included
25
26      Nonempty queries require 0 <= l <= r < n. Updates require 0 <= pos <
            n.
27      An empty query (l > r) returns identity; querying an empty tree does
            too.
28      This implementation has point updates only; there is no lazy
            propagation.
29
30      With constant-size T and constant-time merge:
31        Build O(n), update/query O(log n), storage O(n).
32      Otherwise multiply by the appropriate copy/merge cost; string
            concatenation
33      and variable-size matrix multiplication are not constant-time
            operations.
34
35      CONTENTS: 18 independent examples
36        01 sum                        10 minimum + occurrence count
37        02 minimum                    11 maximum + leftmost index
38        03 maximum                    12 sum + count (range average)
39        04 GCD                        13 maximum nonempty subarray sum
40        05 LCM                        14 balanced parentheses
41        06 product modulo MOD         15 string concatenation
42        07 bitwise OR                 16 matrix multiplication
43        08 bitwise AND                17 affine-function composition
44        09 bitwise XOR                18 permutation composition
45
46      Each example prints its result and shows the expected output in
            comments.
47      Arithmetic is assumed to fit its type unless modular arithmetic is
            used.
48  */
49
50  #include <bits/stdc++.h>
51  using namespace std;
52
53  template<class T, class F>
54  struct SegTree {
55      int n;
56      vector<T> tree;
57      T identity;
58      F merge;
59
60      SegTree(const vector<T>& a, T identity, F merge)
61          : n(static_cast<int>(a.size())), tree(4 * max(1, n), identity),
62            identity(identity), merge(merge) {
63          if (n > 0) build(1, 0, n - 1, a);
64      }
65
66      void build(int p, int l, int r, const vector<T>& a) {
67          if (l == r) {
68              tree[p] = a[l];
69              return;
70          }
71          int m = (l + r) / 2;
72          build(p * 2, l, m, a);
73          build(p * 2 + 1, m + 1, r, a);
74          pull(p);
75      }
76
77      void pull(int p) {
78          tree[p] = merge(tree[p * 2], tree[p * 2 + 1]);
79      }
80
81      void update(int pos, const T& value) {
82          update(1, 0, n - 1, pos, value);
83      }
84
85      void update(int p, int l, int r, int pos, const T& value) {
86          if (l == r) {
87              tree[p] = value;
88              return;
```

```
        }
        int m = (l + r) / 2;
        if (pos <= m)
            update(p * 2, l, m, pos, value);
        else
            update(p * 2 + 1, m + 1, r, pos, value);
        pull(p);
    }

    T query(int l, int r) const {
        if (n == 0 || l > r) return identity;
        return query(1, 0, n - 1, l, r);
    }

    T query(int p, int l, int r, int ql, int qr) const {
        if (qr < l || r < ql) return identity;
        if (ql <= l && r <= qr) return tree[p];
        int m = (l + r) / 2;
        return merge(
            query(p * 2, l, m, ql, qr),
            query(p * 2 + 1, m + 1, r, ql, qr)
        );
    }
};

constexpr long long MOD = 1'000'000'007;

// 01. SUM: identity = 0.
void example_sum() {
    vector<long long> a = {2, 5, 1, 4};
    SegTree st(a, 0LL, [](long long x, long long y) {
        return x + y;
    });
    cout << "01 sum: " << st.query(1, 3);  // 10
    st.update(2, 7);
    cout << " -> " << st.query(1, 3) << '\n';  // 16
}

// 02. MINIMUM: identity = largest representable value.
void example_minimum() {
    vector<long long> a = {8, 3, 6, 1};
    SegTree st(a, LLONG_MAX, [](long long x, long long y) {
        return min(x, y);
    });
    cout << "02 minimum: " << st.query(0, 2) << '\n';  // 3
}

// 03. MAXIMUM: identity = smallest representable value.
void example_maximum() {
    vector<long long> a = {-8, -3, -6, -1};
    SegTree st(a, LLONG_MIN, [](long long x, long long y) {
        return max(x, y);
    });
    cout << "03 maximum: " << st.query(0, 2) << '\n';  // -3
}

// 04. GCD: identity = 0, since gcd(0, x) = x for nonnegative x.
// Use nonnegative values; |LLONG_MIN| cannot be represented as long
    long.
void example_gcd() {
    vector<long long> a = {12, 18, 30, 42};
    SegTree st(a, 0LL, [](long long x, long long y) {
        return gcd(x, y);
    });
    cout << "04 gcd: " << st.query(0, 3) << '\n';  // 6
}

// 05. LCM: identity = 1. Every intermediate LCM must fit in long long.
// Do not take LCM modulo MOD: that would lose information needed for
    later merges.
void example_lcm() {
    vector<long long> a = {4, 6, 10};
    SegTree st(a, 1LL, [](long long x, long long y) {
        return lcm(x, y);
    });
    cout << "05 lcm: " << st.query(0, 2) << '\n';  // 60
}

// 06. MODULAR PRODUCT: identity = 1.
// Store residues in [0, MOD). This MOD allows two residues to multiply
    in long long.
void example_product() {
    vector<long long> a = {2, 3, 4, 5};
    SegTree st(a, 1LL, [](long long x, long long y) {
        return x * y % MOD;
    });
```

```
    cout << "06 product: " << st.query(1, 3) << '\n';  // 60
}

// 07. BITWISE OR: identity = 0.
void example_or() {
    vector<uint64_t> a = {1, 2, 4, 8};
    SegTree st(a, uint64_t{0}, [](uint64_t x, uint64_t y) {
        return x | y;
    });
    cout << "07 bitwise OR: " << st.query(0, 2) << '\n';  // 7
}

// 08. BITWISE AND: identity = all bits set.
void example_and() {
    vector<uint64_t> a = {15, 7, 3};
    SegTree st(a, ~uint64_t{0}, [](uint64_t x, uint64_t y) {
        return x & y;
    });
    cout << "08 bitwise AND: " << st.query(0, 2) << '\n';  // 3
}

// 09. BITWISE XOR: identity = 0.
void example_xor() {
    vector<uint64_t> a = {5, 3, 5};
    SegTree st(a, uint64_t{0}, [](uint64_t x, uint64_t y) {
        return x ^ y;
    });
    cout << "09 bitwise XOR: " << st.query(0, 2) << '\n';  // 3
}

// 10. MINIMUM + NUMBER OF OCCURRENCES.
struct MinCount {
    long long value;
    int count;
};

void example_min_count() {
    vector<long long> values = {4, 1, 3, 1, 1};
    vector<MinCount> a;
    for (long long x : values) a.push_back({x, 1});

    SegTree st(a, MinCount{LLONG_MAX, 0}, [](MinCount x, MinCount y) {
        if (x.value < y.value) return x;
        if (y.value < x.value) return y;
        return MinCount{x.value, x.count + y.count};
    });
    auto ans = st.query(0, 4);
    cout << "10 minimum + count: " << ans.value << ' ' << ans.count;
    st.update(1, MinCount{7, 1});
    ans = st.query(0, 4);
    cout << " -> " << ans.value << ' ' << ans.count << '\n';  // 1 3 ->
        1 2
}

// 11. MAXIMUM + LEFTMOST INDEX. index = -1 marks the empty summary.
struct MaxIndex {
    long long value;
    int index;
};

void example_max_index() {
    vector<long long> values = {4, 9, 2, 9};
    vector<MaxIndex> a;
    for (int i = 0; i < static_cast<int>(values.size()); i++)
        a.push_back({values[i], i});

    SegTree st(a, MaxIndex{0, -1}, [](MaxIndex x, MaxIndex y) {
        if (x.index == -1) return y;
        if (y.index == -1) return x;
        if (x.value != y.value) return x.value > y.value ? x : y;
        return x.index < y.index ? x : y;
    });
    auto ans = st.query(0, 3);
    cout << "11 maximum + index: " << ans.value << ' ' << ans.index <<
        '\n'; // 9 1
    st.update(1, MaxIndex{0, 1});  // Keep the updated position's index.
}

// 12. SUM + COUNT, then divide once to get the range average.
// Averaging two averages directly is not a valid associative merge.
struct SumCount {
    long long sum;
    int count;
};

void example_average() {
```

```
    vector<SumCount> a = {{2, 1}, {4, 1}, {9, 1}};
    SegTree st(a, SumCount{0, 0}, [](SumCount x, SumCount y) {
        return SumCount{x.sum + y.sum, x.count + y.count};
    });
    auto ans = st.query(0, 2);
    cout << "12 average: " << static_cast<double>(ans.sum) / ans.count
        << '\n'; // 5
}

// 13. MAXIMUM NONEMPTY SUBARRAY SUM.
// A node stores total sum, best nonempty prefix/suffix, and best
    nonempty subarray.
// len = 0 identifies the empty summary; this avoids fake infinity
    arithmetic.
struct MaxSubarray {
    long long sum, prefix, suffix, best;
    int len;
};

MaxSubarray subarray_leaf(long long x) {
    return {x, x, x, x, 1};
}

void example_max_subarray() {
    vector<long long> values = {-2, 3, -1, 4, -5};
    vector<MaxSubarray> a;
    for (long long x : values) a.push_back(subarray_leaf(x));

    SegTree st(a, MaxSubarray{0, 0, 0, 0, 0}, [](MaxSubarray x,
        MaxSubarray y) {
        if (x.len == 0) return y;
        if (y.len == 0) return x;
        return MaxSubarray{
            x.sum + y.sum,
            max(x.prefix, x.sum + y.prefix),
            max(y.suffix, y.sum + x.suffix),
            max({x.best, y.best, x.suffix + y.prefix}),
            x.len + y.len
        };
    });
    cout << "13 maximum subarray: " << st.query(0, 4).best; // 6: [3,
        -1, 4]
    st.update(2, subarray_leaf(-10));
    cout << " -> " << st.query(0, 4).best << '\n';  // 4
    // An all-negative, nonempty query returns its largest element, not
        0.
}

// 14. BALANCED PARENTHESES: '(' = +1, ')' = -1.
// minimum_prefix includes the empty prefix, whose sum is 0.
// A queried substring is balanced iff sum == 0 and minimum_prefix >= 0.
struct Brackets {
    int sum, minimum_prefix;
};

Brackets bracket_leaf(char c) {
    int x = c == '(' ? 1 : -1;
    return {x, min(0, x)};
}

void example_brackets() {
    string s = "(())()";
    vector<Brackets> a;
    for (char c : s) a.push_back(bracket_leaf(c));

    SegTree st(a, Brackets{0, 0}, [](Brackets x, Brackets y) {
        return Brackets{
            x.sum + y.sum,
            min(x.minimum_prefix, x.sum + y.minimum_prefix)
        };
    });
    auto balanced = [](Brackets x) {
        return x.sum == 0 && x.minimum_prefix >= 0;
    };
    cout << "14 balanced parentheses: " << balanced(st.query(0, 3)); //
        1: (())
    cout << ' ' << balanced(st.query(1, 4)) << '\n';                // 0:
         "())("
    st.update(0, bracket_leaf(')'));
}

// 15. STRING CONCATENATION: associative, but NOT commutative.
// Identity = empty string. Useful for demonstrating ordering.
// Concatenation/copying takes time proportional to string length: the
    usual
```

```cpp
// O(log n) bound counts merges, not character work. Avoid large strings in CP.
void example_strings() {
    vector<string> a = {"ab", "C", "de", "F"};
    SegTree st(a, string{}, [](const string& x, const string& y) {
        return x + y;
    });
    cout << "15 concatenation: " << st.query(1, 3); // CdeF
    st.update(2, string{"XY"});
    cout << " -> " << st.query(1, 3) << '\n';     // CXYF
}

// 16. MATRIX MULTIPLICATION: query(l, r) = A[l] * ... * A[r].
// Fixed 2x2 matrices, entries stored as residues in [0, MOD).
// For d x d matrices with naive multiplication: O(d^3 log n) per query/update.
struct Matrix {
    long long a[2][2]{};
};

Matrix multiply(const Matrix& x, const Matrix& y) {
    Matrix z;
    for (int i = 0; i < 2; i++)
        for (int j = 0; j < 2; j++)
            for (int k = 0; k < 2; k++)
                z.a[i][j] = (z.a[i][j] + x.a[i][k] * y.a[k][j]) % MOD;
    return z;
}

void example_matrices() {
    vector<Matrix> a = {
        {{{1, 2}, {3, 4}}},
        {{{5, 6}, {7, 8}}},
        {{{2, 0}, {0, 2}}}
    };
    Matrix identity;
    identity.a[0][0] = identity.a[1][1] = 1;

    SegTree st(a, identity, multiply);
    Matrix ans = st.query(0, 1);
    cout << "16 matrix product: "
         << ans.a[0][0] << ' ' << ans.a[0][1] << "; "
         << ans.a[1][0] << ' ' << ans.a[1][1] << '\n'; // 19 22; 43 50
    st.update(1, identity);
    ans = st.query(1, 2);  // I * 2I = 2I.
    cout << "   after update: "
         << ans.a[0][0] << ' ' << ans.a[0][1] << "; "
         << ans.a[1][0] << ' ' << ans.a[1][1] << '\n'; // 2 0; 0 2
}

// 17. AFFINE-FUNCTION COMPOSITION: f(x) = a*x + b modulo MOD.
// Query applies f[l] FIRST, then f[l+1], ..., then f[r].
// Thus merge(left, right) represents right(left(x)), not left(right(x)).
// Coefficients and inputs are stored as residues in [0, MOD).
struct Affine {
    long long a, b;
    long long apply(long long x) const {
        return (a * x + b) % MOD;
    }
};

void example_affine() {
    vector<Affine> a = {{2, 1}, {3, 4}}; // f0(x)=2x+1, f1(x)=3x+4
    SegTree st(a, Affine{1, 0}, [](Affine left, Affine right) {
        return Affine{
            right.a * left.a % MOD,
            (right.a * left.b + right.b) % MOD
        };
    });
    Affine result = st.query(0, 1); // f1(f0(x)) = 6x+7
    cout << "17 affine composition: " << result.apply(5); // 37
    st.update(0, Affine{1, 10});  // f0(x) = x+10
    cout << " -> " << st.query(0, 1).apply(5) << '\n'; // 49
}

// 18. PERMUTATION COMPOSITION on {0, 1, 2}.
// p[i] = where input i is sent. Apply the left permutation first, then right.
// Therefore merge(left, right)[i] = right[left[i]]. Identity sends i to i.
void example_permutations() {
    using Permutation = array<int, 3>;
    vector<Permutation> a = {
        {1, 0, 2},  // Swap 0 and 1.
        {0, 2, 1}   // Swap 1 and 2.
```

```
    };
    Permutation identity = {0, 1, 2};
    SegTree st(a, identity, [](const Permutation& left, const
        Permutation& right) {
        Permutation result{};
        for (int i = 0; i < 3; i++) result[i] = right[left[i]];
        return result;
    });
    auto ans = st.query(0, 1);
    cout << "18 permutation composition:";
    for (int x : ans) cout << ' ' << x;
    cout << '\n'; // 2 0 1
    st.update(0, identity);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    example_sum();
    example_minimum();
    example_maximum();
    example_gcd();
    example_lcm();
    example_product();
    example_or();
    example_and();
    example_xor();
    example_min_count();
    example_max_index();
    example_average();
    example_max_subarray();
    example_brackets();
    example_strings();
    example_matrices();
    example_affine();
    example_permutations();
}
```

3.25 Segment Tree 2D

```
/*
  2D Segment Tree (Sum over Rectangles)
  -------------------------------------
  Indexing: 0-based
  Grid Size: n * m
  Time Complexity:
    - build: O(nm log n log m)
    - point update: O(log n log m)
    - range query [x1..x2][y1..y2]: O(log n log m)
  Space Complexity: O(4n x 4m)
*/

const int MAXN = 505;
int n, m;              // grid dimensions
int a[MAXN][MAXN];     // input grid
int t[4 * MAXN][4 * MAXN]; // segment tree

// Build the tree along y-axis (internal to each x-interval)
void build_y(int vx, int lx, int rx, int vy, int ly, int ry) {
  if (ly == ry) {
    if (lx == rx)
      t[vx][vy] = a[lx][ly];
    else
      t[vx][vy] = t[vx * 2][vy] + t[vx * 2 + 1][vy];
  } else {
    int my = (ly + ry) / 2;
    build_y(vx, lx, rx, vy * 2, ly, my);
    build_y(vx, lx, rx, vy * 2 + 1, my + 1, ry);
    t[vx][vy] = t[vx][vy * 2] + t[vx][vy * 2 + 1];
  }
}

// Build the tree along x-axis and call build_y for each
void build_x(int vx, int lx, int rx) {
  if (lx != rx) {
    int mx = (lx + rx) / 2;
    build_x(vx * 2, lx, mx);
    build_x(vx * 2 + 1, mx + 1, rx);
  }
  build_y(vx, lx, rx, 1, 0, m - 1);
}

// Query along y-axis in a fixed x-node
int sum_y(int vx, int vy, int tly, int try_, int ly, int ry) {
```

```
  if (ly > ry) return 0;
  if (ly == tly && ry == try_) return t[vx][vy];
  int tmy = (tly + try_) / 2;
  return sum_y(vx, vy * 2, tly, tmy, ly, min(ry, tmy))
       + sum_y(vx, vy * 2 + 1, tmy + 1, try_, max(ly, tmy + 1), ry);
}

// Query sum in rectangle [lx..rx][ly..ry]
int sum_x(int vx, int tlx, int trx, int lx, int rx, int ly, int ry) {
  if (lx > rx) return 0;
  if (lx == tlx && trx == rx)
    return sum_y(vx, 1, 0, m - 1, ly, ry);
  int tmx = (tlx + trx) / 2;
  return sum_x(vx * 2, tlx, tmx, lx, min(rx, tmx), ly, ry)
       + sum_x(vx * 2 + 1, tmx + 1, trx, max(lx, tmx + 1), rx, ly, ry);
}

// Update along y-axis for fixed x-node
void update_y(int vx, int lx, int rx, int vy, int ly, int ry, int x, int
     y, int new_val) {
  if (ly == ry) {
    if (lx == rx)
      t[vx][vy] = new_val;
    else
      t[vx][vy] = t[vx * 2][vy] + t[vx * 2 + 1][vy];
  } else {
    int my = (ly + ry) / 2;
    if (y <= my)
      update_y(vx, lx, rx, vy * 2, ly, my, x, y, new_val);
    else
      update_y(vx, lx, rx, vy * 2 + 1, my + 1, ry, x, y, new_val);
    t[vx][vy] = t[vx][vy * 2] + t[vx][vy * 2 + 1];
  }
}

// Update point (x, y) to new_val
void update_x(int vx, int lx, int rx, int x, int y, int new_val) {
  if (lx != rx) {
    int mx = (lx + rx) / 2;
    if (x <= mx)
      update_x(vx * 2, lx, mx, x, y, new_val);
    else
      update_x(vx * 2 + 1, mx + 1, rx, x, y, new_val);
  }
  update_y(vx, lx, rx, 1, 0, m - 1, x, y, new_val);
}
```

3.26 Segment Tree Dynamic

```
/*
  Dynamic Segment Tree (Point Add, Range Sum)
  -------------------------------------------
  Indexing: [0, INF) or any large bounded range
  Time Complexity:
    - add(k, x): O(log U)
    - get_sum(l, r): O(log U)
      where U = range size (e.g., 1e9 if implicit bounds)

  Space Complexity: O(nodes visited or created) -> worst O(log U) per
      operation
*/

struct Vertex {
  int left, right;        // interval [left, right)
  int sum = 0;            // sum of elements in this interval
  Vertex *left_child = nullptr, *right_child = nullptr;

  Vertex(int lb, int rb) {
    left = lb;
    right = rb;
  }

  // Create children lazily only when needed
  void extend() {
    if (!left_child && left + 1 < right) {
      int mid = (left + right) / 2;
      left_child = new Vertex(left, mid);
      right_child = new Vertex(mid, right);
    }
  }

  // Add `x` to position `k`
  void add(int k, int x) {
    extend();
    sum += x;
    if (left_child) {
```

```
      if (k < left_child->right)
        left_child->add(k, x);
      else
        right_child->add(k, x);
    }
  }

  // Get sum over interval [lq, rq)
  int get_sum(int lq, int rq) {
    if (lq <= left && right <= rq)
      return sum;
    if (rq <= left || right <= lq)
      return 0;
    extend();
    return left_child->get_sum(lq, rq) + right_child->get_sum(lq, rq);
  }
};

Vertex *root = new Vertex(0, 1e9); // Range [0, 1e9)
root->add(5, 10);       // a[5] += 10
root->add(1000, 20);    // a[1000] += 20
cout << root->get_sum(0, 10) << '\n';     // sum of [0, 10) = 10
cout << root->get_sum(0, 2000) << '\n';   // sum of [0, 2000) = 30
```

3.27 Segment Tree Lazy Types

```
struct max_t {
  ll val;
  static const ll null_v = -1LL << 61;
  max_t(): val(0) {}
  max_t(ll v): val(v) {}
  max_t op(max_t& other) {
    return max_t(max(val, other.val));
  }
  max_t lazy_op(max_t& v, int size) {
    return max_t(val + v.val);
  }
};

struct min_t {
  ll val;
  static const ll null_v = 1LL << 61;
  min_t(): val(0) {}
  min_t(ll v): val(v) {}
  min_t op(min_t& other) {
    return min_t(min(val, other.val));
  }
  min_t lazy_op(min_t& v, int size) {
    return min_t(val + v.val);
  }
};

struct sum_t {
  ll val;
  static const ll null_v = 0;
  sum_t(): val(0) {}
  sum_t(ll v): val(v) {}
  sum_t op(sum_t& other) {
    return sum_t(val + other.val);
  }
  sum_t lazy_op(sum_t& v, int size) {
    return sum_t(val + v.val * size);
  }
};
```

3.28 Segment Tree Lazy

```
/*
  Lazy Segment Tree (Range Update, Range Query)
  ---------------------------------------------
  Indexing: 0-based
  Bounds: [0, n-1]
  Time Complexity:
    - build: O(n)
    - update(l, r, v): O(log n)
    - query(l, r): O(log n)
  Space Complexity: O(4n)
*/

// See SegTreeLazy_types for num_t structs

const num_t num_t::null_v = num_t(0);


template <typename num_t>
struct segtree {
```

```
  int n;
  vector<num_t> tree, lazy;

  // Initialize segment tree with array of size s
  void init(int s, long long* arr) {
    n = s;
    tree.assign(4 * n, num_t());
    lazy.assign(4 * n, num_t());
    init(0, 0, n - 1, arr);
  }

  // Build segment tree from array
  num_t init(int i, int l, int r, long long* arr) {
    if (l == r) return tree[i] = num_t(arr[l]);

    int mid = (l + r) / 2;
    num_t left = init(2 * i + 1, l, mid, arr);
    num_t right = init(2 * i + 2, mid + 1, r, arr);
    return tree[i] = left.op(right);
  }

  // Public wrapper: update range [l, r] with value v
  void update(int l, int r, num_t v) {
    if (l > r) return;
    update(0, 0, n - 1, l, r, v);
  }

  // Internal recursive update
  num_t update(int i, int tl, int tr, int ql, int qr, num_t v) {
    eval_lazy(i, tl, tr);

    if (tr < ql || qr < tl) return tree[i]; // no overlap
    if (ql <= tl && tr <= qr) {
      lazy[i].val += v.val;
      eval_lazy(i, tl, tr);
      return tree[i];
    }

    int mid = (tl + tr) / 2;
    num_t a = update(2 * i + 1, tl, mid, ql, qr, v);
    num_t b = update(2 * i + 2, mid + 1, tr, ql, qr, v);
    return tree[i] = a.op(b);
  }

  // Public wrapper: query sum in range [l, r]
  num_t query(int l, int r) {
    if (l > r) return num_t::null_v;
    return query(0, 0, n - 1, l, r);
  }

  // Internal recursive query
  num_t query(int i, int tl, int tr, int ql, int qr) {
    eval_lazy(i, tl, tr);

    if (ql <= tl && tr <= qr) return tree[i]; // total overlap
    if (tr < ql || qr < tl) return num_t::null_v; // no overlap

    int mid = (tl + tr) / 2;
    num_t a = query(2 * i + 1, tl, mid, ql, qr);
    num_t b = query(2 * i + 2, mid + 1, tr, ql, qr);
    return a.op(b);
  }

  // Push down pending lazy updates to children
  void eval_lazy(int i, int l, int r) {
    tree[i] = tree[i].lazy_op(lazy[i], r - l + 1);
    if (l != r) {
      lazy[2 * i + 1].val += lazy[i].val;
      lazy[2 * i + 2].val += lazy[i].val;
    }
    lazy[i] = num_t(); // reset lazy at this node
  }
};
```

3.29 Segment Tree Lazy Range Set

```
/*
  Lazy Segment Tree (Range Set + Range Add + Range Sum)
  ----------------------------------------------------
  Indexing: 0-based
  Bounds: [0, N-1]

  Features:
    - Supports range set (assign value), range add (increment), and
        range sum queries
    - Properly prioritizes lazy set > lazy add
```

```
*/

const int maxN = 1e5 + 5;
int N, Q;
int a[maxN];

struct node {
  ll val = 0;      // range sum
  ll lzAdd = 0;    // pending addition
  ll lzSet = 0;    // pending set (non-zero means active)
};

node tree[maxN << 2];

#define lc (p << 1)
#define rc ((p << 1) | 1)

// Update current node based on its children
inline void pushup(int p) {
  tree[p].val = tree[lc].val + tree[rc].val;
}

// Push lazy values down to children
void pushdown(int p, int l, int mid, int r) {
  // Range set overrides any pending add
  if (tree[p].lzSet != 0) {
    tree[lc].lzSet = tree[rc].lzSet = tree[p].lzSet;
    tree[lc].val = (mid - l + 1) * tree[p].lzSet;
    tree[rc].val = (r - mid) * tree[p].lzSet;
    tree[lc].lzAdd = tree[rc].lzAdd = 0;
    tree[p].lzSet = 0;
  }
  // Otherwise propagate add
  else if (tree[p].lzAdd != 0) {
    if (tree[lc].lzSet == 0) tree[lc].lzAdd += tree[p].lzAdd;
    else {
      tree[lc].lzSet += tree[p].lzAdd;
      tree[lc].lzAdd = 0;
    }
    if (tree[rc].lzSet == 0) tree[rc].lzAdd += tree[p].lzAdd;
    else {
      tree[rc].lzSet += tree[p].lzAdd;
      tree[rc].lzAdd = 0;
    }
    tree[lc].val += (mid - l + 1) * tree[p].lzAdd;
    tree[rc].val += (r - mid) * tree[p].lzAdd;
    tree[p].lzAdd = 0;
  }
}

// Build tree from array a[0..N-1]
void build(int p, int l, int r) {
  tree[p].lzAdd = tree[p].lzSet = 0;
  if (l == r) {
    tree[p].val = a[l];
    return;
  }
  int mid = (l + r) >> 1;
  build(lc, l, mid);
  build(rc, mid + 1, r);
  pushup(p);
}

// Add `val` to all elements in [a, b]
void add(int p, int l, int r, int a, int b, ll val) {
  if (a > r || b < l) return;
  if (a <= l && r <= b) {
    tree[p].val += (r - l + 1) * val;
    if (tree[p].lzSet == 0) tree[p].lzAdd += val;
    else tree[p].lzSet += val;
    return;
  }
  int mid = (l + r) >> 1;
  pushdown(p, l, mid, r);
  add(lc, l, mid, a, b, val);
  add(rc, mid + 1, r, a, b, val);
  pushup(p);
}

// Set all elements in [a, b] to `val`
void set(int p, int l, int r, int a, int b, ll val) {
  if (a > r || b < l) return;
  if (a <= l && r <= b) {
    tree[p].val = (r - l + 1) * val;
    tree[p].lzAdd = 0;
    tree[p].lzSet = val;
```

```
    return;
  }
  int mid = (l + r) >> 1;
  pushdown(p, l, mid, r);
  set(lc, l, mid, a, b, val);
  set(rc, mid + 1, r, a, b, val);
  pushup(p);
}

// Query sum over [a, b]
ll query(int p, int l, int r, int a, int b) {
  if (a > r || b < l) return 0;
  if (a <= l && r <= b) return tree[p].val;
  int mid = (l + r) >> 1;
  pushdown(p, l, mid, r);
  return query(lc, l, mid, a, b) + query(rc, mid + 1, r, a, b);
}
// Example usage
N = 5;
a[0] = 2, a[1] = 4, a[2] = 3, a[3] = 1, a[4] = 5;
build(1, 0, N - 1);
set(1, 0, N - 1, 1, 3, 7);     // a[1..3] = 7
add(1, 0, N - 1, 2, 4, 2);     // a[2..4] += 2
cout << query(1, 0, N - 1, 0, 4) << '\n'; // total sum
```

3.30 Segment Tree Lazy Range Set 2

```
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

struct sum_t {
  ll val;
  static const ll null_v = 0;
  sum_t(): val(null_v) {}
  sum_t(ll v): val(v) {}
  sum_t op(sum_t& other) {
    return sum_t(val + other.val);
  }
  sum_t lazy_op(sum_t& v, int size) {
    return v.val * size;
  }
  sum_t lazy_op1(sum_t& v, int size) {
    return val + v.val * size;
  }
};

template <typename num_t>
struct segtree {
  int n;
  vector<num_t> tree, lazy, lazy1;

  void init(int s) {
    n = s;
    tree.assign(4 * n, num_t(0));
    lazy.assign(4 * n, num_t(-1));
    lazy1.assign(4 * n, num_t(0));
    init(0, 0, n - 1);
  }

  num_t init(int i, int l, int r) {
    if (l == r) return tree[i] = num_t(0);

    int mid = (l + r) / 2;
    num_t left = init(2 * i + 1, l, mid);
    num_t right = init(2 * i + 2, mid + 1, r);
    return tree[i] = left.op(right);
  }

  void update(int l, int r, num_t v) {
    if (l > r) return;
    update(0, 0, n - 1, l, r, v);
  }

  num_t update(int i, int tl, int tr, int ql, int qr, num_t v) {
    eval_lazy(i, tl, tr);

    if (tr < ql || qr < tl) return tree[i];
    if (ql <= tl && tr <= qr) {
      lazy[i].val = v.val;
      lazy1[i].val = 0;
      eval_lazy(i, tl, tr);
      return tree[i];
    }

```

```
    int mid = (tl + tr) / 2;
    num_t a = update(2 * i + 1, tl, mid, ql, qr, v);
    num_t b = update(2 * i + 2, mid + 1, tr, ql, qr, v);
    return tree[i] = a.op(b);
  }

  void update1(int l, int r, num_t v) {
    if (l > r) return;
    update1(0, 0, n - 1, l, r, v);
  }

  num_t update1(int i, int tl, int tr, int ql, int qr, num_t v) {

    if (tr < ql || qr < tl){
      eval_lazy(i, tl, tr);
      return tree[i];
    }
    if (ql <= tl && tr <= qr) {
      lazy1[i].val += v.val;
      if(lazy[i].val != -1){
        lazy[i].val += lazy1[i].val;
        lazy1[i].val = 0;
      }
      eval_lazy(i, tl, tr);
      return tree[i];
    }

    eval_lazy(i, tl, tr);
    int mid = (tl + tr) / 2;
    num_t a = update1(2 * i + 1, tl, mid, ql, qr, v);
    num_t b = update1(2 * i + 2, mid + 1, tr, ql, qr, v);
    return tree[i] = a.op(b);
  }

  num_t query(int l, int r) {
    if (l > r) return num_t::null_v;
    return query(0, 0, n - 1, l, r);
  }

  num_t query(int i, int tl, int tr, int ql, int qr) {
    eval_lazy(i, tl, tr);

    if (ql <= tl && tr <= qr) return tree[i];
    if (tr < ql || qr < tl) return num_t::null_v;

    int mid = (tl + tr) / 2;
    num_t a = query(2 * i + 1, tl, mid, ql, qr);
    num_t b = query(2 * i + 2, mid + 1, tr, ql, qr);
    return a.op(b);
  }

  void eval_lazy(int i, int l, int r) {
    if(lazy[i].val != -1){
      tree[i] = tree[i].lazy_op(lazy[i], r - l + 1);
      if (l != r) {
        lazy[2 * i + 1].val = lazy[i].val;
        lazy[2 * i + 2].val = lazy[i].val;
        lazy1[2 * i + 1].val = lazy1[2 * i + 2].val = 0;
      }
      lazy[i] = num_t(-1);
      lazy1[i] = num_t(0);
    }
    else{
      tree[i] = tree[i].lazy_op1(lazy1[i], r - l + 1);
      if(l != r){
        if(lazy[2 * i + 1].val == -1) lazy1[2 * i + 1].val += lazy1[i].
            val;
        else{
          lazy[2 * i + 1].val += lazy1[i].val + lazy1[2 * i + 1].val;
          lazy1[2 * i + 1].val = 0;
        }
        if(lazy[2 * i + 2].val == -1) lazy1[2 * i + 2].val += lazy1[i].
            val;
        else{
          lazy[2 * i + 2].val += lazy1[i].val + lazy1[2 * i + 2].val;
          lazy1[2 * i + 2].val = 0;
        }
      }
      lazy1[i] = num_t(0);
    }
  }
};

int main(){
  ios_base::sync_with_stdio(false); cin.tie(NULL);
  int n, q; cin >> n >> q;
```

```
  segtree <sum_t> st;
  st.init(n);
  while(q--){
    int t; cin >> t;
    if(t == 1){
      int l, r, x; cin >> l >> r >> x;
      r--;
      st.update(l, r, x);
    }
    else if(t == 2){
      int l, r, x; cin >> l >> r >> x;
      r--;
      st.update1(l, r, x);
    }
    else{
      int l, r; cin >> l >> r;
      r--;
      cout << st.query(l, r).val << endl;
    }
  }
}
```

3.31 Segment Tree Max Subarray Sum

```
const ll inf=1e13;
struct Node {
  ll maxi, l_max, r_max, sum;
  Node(ll _maxi, ll _l_max, ll _r_max, ll _sum){
    maxi=_maxi;
    l_max=_l_max;
    r_max=_r_max;
    sum=_sum;
  }
  Node operator+(Node b) {
    return {max(max(maxi, b.maxi), max(r_max + b.l_max, 0LL)),
      max(l_max, max(sum + b.l_max, 0LL)), max(b.r_max, max(r_max + b.
          sum, 0LL)),
      sum + b.sum};
  }

};

struct SegmentTreeMaxSubSum{
  int n;
  vector<Node> t;
  SegmentTreeMaxSubSum(int _n) : n(_n), t(2 * _n, Node(-inf, -inf, -inf,
       -inf)) {}
  void update(int pos, ll val) {
    t[pos += n] = Node(val, val, val, val);
    for (pos>>=1; pos ; pos >>= 1) {
      t[pos] = t[2*pos]+t[2*pos+1];
    }
  }
  Node query(int l, int r) {
    Node node_l = Node(-inf, -inf, -inf, -inf);
    Node node_r = Node(-inf, -inf, -inf, -inf);
    for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
      if (l & 1) {
        node_l=node_l+t[l++];
      }
      if (r & 1) {
        node_r=t[--r]+node_r;
      }
    }
    return node_l+node_r;
  }
};
```

3.32 Segment Tree Range Update

```
/*
  Segment Tree (Range Min Update, Point Query)
  ----------------------------------------
  Indexing: 0-based
  Bounds: [0, n-1]
  Time Complexity:
    - update(l, r, val): O(log n) -> applies min(val) over [l, r)
    - get(pos): O(log n) -> minimum affecting position pos
  Space Complexity: O(2n)
*/

struct SegmentTree {
  vector<ll> a; // a[i] = min value affecting segment i
  int n;

  SegmentTree(int _n) : a(2 * _n, 1e18), n(_n) {}
```

```

  // Get the effective minimum at position ‘pos‘
  ll get(int pos) {
    ll res = 1e18;
    for (pos += n; pos > 0; pos >>= 1) {
      res = min(res, a[pos]);
    }
    return res;
  }

  // Apply min(val) to all positions in [l, r)
  void update(int l, int r, ll val) {
    for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
      if (l & 1) {
        a[l] = min(a[l], val);
        l++;
      }
      if (r & 1) {
        --r;
        a[r] = min(a[r], val);
      }
    }
  }
};
```

3.33 Segment Tree Struct Types

```
// Sum segment tree
struct sum_t{
  ll val;
  static const long long null_v = 0;

  sum_t(): val(null_v) {}
  sum_t(long long v): val(v) {}

  sum_t operator + (const sum_t &a) const {
    sum_t ans;
    ans.val = val + a.val;
    return ans;
  }
};
// Min segment tree
struct min_t{
  ll val;
  static const long long null_v = 1e18;

  min_t(): val(null_v) {}
  min_t(long long v): val(v) {}

  min_t operator + (const min_t &a) const {
    min_t ans;
    ans.val = min(val, a.val);
    return ans;
  }
};
// Max segment tree
struct max_t{
  ll val;
  static const long long null_v = -1e18;

  max_t(): val(null_v) {}
  max_t(long long v): val(v) {}

  max_t operator + (const max_t &a) const {
    max_t ans;
    ans.val = max(val, a.val);
    return ans;
  }
};
// GCD segment tree
struct gcd_t{
  ll val;
  static const long long null_v = 0;

  gcd_t(): val(null_v) {}
  gcd_t(long long v): val(v) {}

  gcd_t operator + (const gcd_t &a) const {
    gcd_t ans;
    ans.val = gcd(val, a.val);
    return ans;
  }
};
```

3.34 Segment Tree Struct

```cpp
// works as a 0-indexed segtree (not lazy)
template <typename num_t>
struct segtree
{
  int n, k;
  vector<num_t> tree;
  void init(int s, vector<ll> arr)
  {
    n = s;
    k = 0;
    while ((1 << k) < n)
      k++;
    tree = vector<num_t>(2 * (1 << k) + 1);
    for (int i = 0; i < n; i++)
    {
      tree[(1 << k) + i] = arr[i];
    }
    for (int i = (1 << k) - 1; i > 0; i--)
    {
      tree[i] = tree[i * 2] + tree[i * 2 + 1];
    }
  }
  void update(int a, ll b)
  {
    a += (1 << k);
    tree[a] = b;
    for (a /= 2; a >= 1; a /= 2)
    {
      tree[a] = tree[a * 2] + tree[a * 2 + 1];
    }
  }
  num_t find(int a, int b)
  {
    a += (1 << k);
    b += (1 << k);
    num_t s;
    while (a <= b)
    {
      if (a % 2 == 1)
        s = s + tree[a++];
      if (b % 2 == 0)
        s = s + tree[b--];
      a /= 2;
      b /= 2;
    }
    return s;
  }
};
```

3.35 Segment Tree Walk

```cpp
/*
  Segment Tree Walk - Find First Position >= val
  ----------------------------------------------
  Indexing: 0-based
  Bounds: [0, n-1]
  Time Complexity:
    - build: O(n)
    - update(pos, val): O(log n)
    - get(L, R): O(log n) -> min value in [L, R]
    - query(L, R, val): O(log n) -> find first index in [L, R] where a[i
      ] >= val

  Features:
    - Stores original value array in segment tree form
    - Maps original indices to tree positions for fast updates
    - Allows efficient walk to find constrained elements (e.g. lower
      bound >= val)
*/

struct SegmentTreeWalk {
  vector<ll> a;           // segment tree values
  vector<int> final_pos;  // maps index i to position in tree (leaf)
  int n;

  SegmentTreeWalk(int _n) : a(4 * _n, 1e18), final_pos(_n), n(_n) {}

  // Build segment tree from array `vals[0..n-1]`, start with node=1, l
    =0, r=n-1
  void build(int l, int r, int node, const vector<ll> &vals) {
    if (l == r) {
      final_pos[l] = node;
      a[node] = vals[l];
    } else {
      int mid = (l + r) / 2;
      build(l, mid, node * 2, vals);
```

```
      build(mid + 1, r, node * 2 + 1, vals);
      a[node] = min(a[node * 2], a[node * 2 + 1]);
    }
  }

  // Update value at original index ‘pos‘ to ‘val‘
  void update(int pos, ll val) {
    pos = final_pos[pos]; // leaf position
    a[pos] = val;
    for (pos /= 2; pos > 0; pos /= 2)
      a[pos] = min(a[pos * 2], a[pos * 2 + 1]);
  }

  // Get min value in [L, R], with current node interval [l, r] and root
      ‘node‘
  ll get(int l, int r, int L, int R, int node) {
    if (L > R) return 1e18;
    if (l == L && r == R) return a[node];
    int mid = (l + r) / 2;
    return min(
      get(l, mid, L, min(R, mid), node * 2),
      get(mid + 1, r, max(L, mid + 1), R, node * 2 + 1)
    );
  }

  // Find first position in [L, R] with a[i] >= val, starting from node
      interval [l, r]
  pair<ll, ll> query(int l, int r, int L, int R, int node, int val) {
    if (l > R || r < L) return {-1, 0};            // out of query
        bounds
    if (a[node] < val) return {-1, 0};             // all values < val
    if (l == r) return {a[node], l};               // leaf node that
        satisfies

    int mid = (l + r) / 2;
    auto left = query(l, mid, L, R, node * 2, val);
    if (left.first != -1) return left;
    return query(mid + 1, r, L, R, node * 2 + 1, val);
  }
};
// Example usage:
int n = 8;
vector<ll> vals = {4, 2, 7, 1, 9, 5, 6, 3};
SegmentTreeWalk st(n);
st.build(0, n - 1, 1, vals);
```

3.36 Sparse Table

```
/*
  Sparse Table (Range Minimum Query)
  ----------------------------------
  Indexing: 0-based
  Bounds: [0, n-1]
  Time Complexity:
    - Build: O(n log n)
    - Query: O(1)
  Space Complexity: O(n log n)

  Features:
    - Immutable RMQ (no updates)
    - Works for idempotent operations like min, max, gcd
*/

const int MAXN = 100005;
const int K = 30; // floor(log2(MAXN))
int lg[MAXN + 1]; // log base 2 of each i
int st[K + 1][MAXN]; // st[k][i] = min in [i, i + 2^k - 1]

vector<int> a; // input array
int n;

// Returns min in range [L, R]
int mini(int L, int R) {
  int len = R - L + 1;
  int i = lg[len];
  return min(st[i][L], st[i][R - (1 << i) + 1]);
}

int main() {
  cin >> n;
  a.resize(n);
  for (int i = 0; i < n; i++) cin >> a[i];
  // Precompute binary logs
  lg[1] = 0;
  for (int i = 2; i <= n; i++) {
    lg[i] = lg[i / 2] + 1;
```

```
  }
  // Initialize 2^0 intervals
  for (int i = 0; i < n; i++) {
    st[0][i] = a[i];
  }
  // Build sparse table
  for (int k = 1; k <= K; k++) {
    for (int i = 0; i + (1 << k) <= n; i++) {
      st[k][i] = min(st[k - 1][i], st[k - 1][i + (1 << (k - 1))]);
    }
  }
  // Example usage
  int q; cin >> q;
  while (q--) {
    int l, r;
    cin >> l >> r;
    cout << mini(l, r) << '\n';
  }
  return 0;
}
```

3.37 Sparse Table 2D

```
#include<bits/stdc++.h>
using namespace std;

const int N = 505, LG = 10;

int st[N][N][LG][LG];
int a[N][N], lg2[N];

int yo(int x1, int y1, int x2, int y2) {
  x2++;
  y2++;
  int a = lg2[x2 - x1], b = lg2[y2 - y1];
  return max(
         max(st[x1][y1][a][b], st[x2 - (1 << a)][y1][a][b]),
         max(st[x1][y2 - (1 << b)][a][b], st[x2 - (1 << a)][y2 - (1 << b
             )][a][b])
       );
}

void build(int n, int m) { // 0 indexed
  for (int i = 2; i < N; i++) lg2[i] = lg2[i >> 1] + 1;
  for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
      st[i][j][0][0] = a[i][j];
    }
  }
  for (int a = 0; a < LG; a++) {
    for (int b = 0; b < LG; b++) {
      if (a + b == 0) continue;
      for (int i = 0; i + (1 << a) <= n; i++) {
        for (int j = 0; j + (1 << b) <= m; j++) {
          if (!a) {
            st[i][j][a][b] = max(st[i][j][a][b - 1], st[i][j + (1 << (b
                - 1))][a][b - 1]);
          } else {
            st[i][j][a][b] = max(st[i][j][a - 1][b], st[i + (1 << (a -
                1))][j][a - 1][b]);
          }
        }
      }
    }
  }
}

string s[N];
int l[N][N], u[N][N];

int32_t main() {
  ios_base::sync_with_stdio(0);
  cin.tie(0);

  int n, m;
  cin >> n >> m;
  for (int i = 0; i < n; i++) {
    cin >> s[i];
  }
  for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
      if (!j) l[i][j] = 1;
      else l[i][j] = 1 + (s[i][j - 1] <= s[i][j] ? l[i][j - 1] : 0);
    }
  }
  for (int j = 0; j < m; j++) {
```

```
    for (int i = 0; i < n; i++) {
      if (!i) u[i][j] = 1;
      else u[i][j] = 1 + (s[i - 1][j] <= s[i][j] ? u[i - 1][j] : 0);
    }
  }
  for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
      int nw = 1, mnx = u[i][j], mny = l[i][j];
      for (int len = 1; len <= min(i, j); len++) {
        mnx = min(mnx, u[i][j - len]);
        mny = min(mny, l[i - len][j]);
        if (min(mnx, mny) >= len + 1) nw++;
        else break;
      }
      a[i][j] = nw;
    }
  }
  build(n, m);
  int q;
  cin >> q;
  while (q--) {
    int x1, y1, x2, y2;
    cin >> x1 >> y1 >> x2 >> y2;
    x1--, y1--;
    x2--;
    y2--;
    int l = 1, r = min(x2 - x1 + 1, y2 - y1 + 1), ans = 0;
    while (l <= r) {
      int mid = l + r >> 1;
      if (yo(x1 + mid - 1, y1 + mid - 1, x2, y2) >= mid) ans = mid, l =
          mid + 1;
      else r = mid - 1;
    }
    cout << ans << '\n';
  }
  return 0;
}
// https://www.codechef.com/problems/CENS20B
```

3.38 Square Root Decomposition

```
/*
  Sqrt Decomposition (String Block Cut and Move)
  ----------------------------------------------
  Operation:
    - Supports moving substrings using block cut logic
    - Rebuilds when too many blocks (for performance)

  Indexing: 0-based
  String Bounds: [0, n)
  Time Complexity:
    - cut(a, b): O(sqrt(n))
    - rebuildDecomp(): O(n)
  When to rebuild: after too many block splits

  Use case: performing multiple cut/paste operations efficiently on
      large strings
*/

const int MAXI = 350; // = sqrt(n), for n up to 1e5

int n, numBlocks;
string s;

struct Block {
  int l, r; // indices into string s
  int sz() const { return r - l; }
};

Block blocks[2 * MAXI];     // current block array
Block newBlocks[2 * MAXI];  // used temporarily during cutting

// Rebuilds the entire decomposition into 1 block (or balanced ones)
void rebuildDecomp() {
  string newS = s;
  int k = 0;
  // Flatten string using current block structure
  for (int i = 0; i < numBlocks; i++) {
    for (int j = blocks[i].l; j < blocks[i].r; j++) {
      newS[k++] = s[j];
    }
  }
  // Reset to one big block
  numBlocks = 1;
  blocks[0] = {0, n};
  s = newS;
```

```
}

// Cut [a, b) into a separate region and reorder it to the end
void cut(int a, int b) {
  int pos = 0, curBlock = 0;
  // Pass 1: Split blocks to isolate [a, b)
  for (int i = 0; i < numBlocks; i++) {
    Block B = blocks[i];
    bool containsA = (pos < a && pos + B.sz() > a);
    bool containsB = (pos < b && pos + B.sz() > b);
    int cutA = B.l + a - pos;
    int cutB = B.l + b - pos;

    if (containsA && containsB) {
      newBlocks[curBlock++] = {B.l, cutA};
      newBlocks[curBlock++] = {cutA, cutB};
      newBlocks[curBlock++] = {cutB, B.r};
    } else if (containsA) {
      newBlocks[curBlock++] = {B.l, cutA};
      newBlocks[curBlock++] = {cutA, B.r};
    } else if (containsB) {
      newBlocks[curBlock++] = {B.l, cutB};
      newBlocks[curBlock++] = {cutB, B.r};
    } else {
      newBlocks[curBlock++] = B;
    }

    pos += B.sz();
  }

  // Pass 2: Reorder - move [a, b) to the end
  pos = 0;
  numBlocks = 0;

  // First add all blocks not in [a, b)
  for (int i = 0; i < curBlock; i++) {
    if (pos < a || pos >= b)
      blocks[numBlocks++] = newBlocks[i];
    pos += newBlocks[i].sz();
  }

  // Then add blocks in [a, b)
  pos = 0;
  for (int i = 0; i < curBlock; i++) {
    if (pos >= a && pos < b)
      blocks[numBlocks++] = newBlocks[i];
    pos += newBlocks[i].sz();
  }
}
// Example usage
int main() {
  cin >> s;
  n = s.size();
  numBlocks = 1;
  blocks[0] = {0, n};

  int q; cin >> q;
  while (q--) {
    int a, b;
    cin >> a >> b;
    cut(a, b); // move [a, b) to the end

    if (numBlocks > MAXI) rebuildDecomp();
  }

  rebuildDecomp(); // flatten before output
  cout << s << '\n';
}
```

3.39 Tuple

```
#include <tuple>
#include <iostream>
#include <string>

int main() {
    // ------------------------------
    // DECLARING A TUPLE
    // ------------------------------
    std::tuple<int, double, std::string> t1;

    // ------------------------------
    // INITIALIZING A TUPLE
    // ------------------------------
    std::tuple<int, double, std::string> t2(42, 3.14, "hello");

```

```
    // ------------------------------
    // USING std::make_tuple
    // (automatic type deduction)
    // ------------------------------
    auto t3 = std::make_tuple(7, 2.71, "world");

    // ------------------------------
    // ACCESSING ELEMENTS USING std::get
    // ------------------------------
    int a = std::get<0>(t2);         // a = 42
    double b = std::get<1>(t2);      // b = 3.14
    std::string c = std::get<2>(t2); // c = "hello"

    // ------------------------------
    // MODIFYING ELEMENTS
    // ------------------------------
    std::get<1>(t2) = 5.67; // change second element

    // ------------------------------
    // STRUCTURED BINDINGS (C++17)
    // Unpacks the tuple into variables
    // ------------------------------
    auto [x, y, z] = t3;  // x=7, y=2.71, z="world"

    // ------------------------------
    // tuple_size and tuple_element
    // (using decltype to reference the type)
    // ------------------------------
    std::size_t n = std::tuple_size<decltype(t3)>::value; // n = 3
    using FirstType = std::tuple_element<0, decltype(t3)>::type;
    // FirstType is int

    // ------------------------------
    // SMALL EXAMPLE OF USE
    // ------------------------------
    auto person = std::make_tuple("Alice", 25, 1.70);

    // Access values
    std::cout << "Name: "  << std::get<0>(person) << "\n";
    std::cout << "Age: "   << std::get<1>(person) << "\n";
    std::cout << "Height: "<< std::get<2>(person) << "\n";

    // Modify a value
    std::get<1>(person) = 26;

    // Print updated value
    std::cout << "Updated age: " << std::get<1>(person) << "\n";

    return 0;
}
```

3.40 Treap

```
std::mt19937 rng(std::chrono::steady_clock::now().time_since_epoch().
    count());

struct Node {
  Node *l = 0, *r = 0;
  int val, y, c = 1;
  Node(int val) : val(val), y(rng()) {}
  void recalc();
};

int cnt(Node* n) { return n ? n->c : 0; }
void Node::recalc() { c = cnt(l) + cnt(r) + 1; }

template<class F> void each(Node* n, F f) {
  if (n) { each(n->l, f); f(n->val); each(n->r, f); }
}

pair<Node*, Node*> split(Node* n, int k) {
  if (!n) return {};
  if (cnt(n->l) >= k) { // "n->val >= k" for lower_bound(k)
    auto pa = split(n->l, k);
    n->l = pa.second;
    n->recalc();
    return {pa.first, n};
  } else {
    auto pa = split(n->r, k - cnt(n->l) - 1); // and just "k"
    n->r = pa.first;
    n->recalc();
    return {n, pa.second};
  }
}

Node* merge(Node* l, Node* r) {
```

```
  if (!l) return r;
  if (!r) return l;
  if (l->y > r->y) {
    l->r = merge(l->r, r);
    l->recalc();
    return l;
  } else {
    r->l = merge(l, r->l);
    r->recalc();
    return r;
  }
}

Node* ins(Node* t, Node* n, int pos) {
  auto pa = split(t, pos);
  return merge(merge(pa.first, n), pa.second);
}

// Example application: move the range [l, r) to index k
void move(Node*& t, int l, int r, int k) {
  Node *a, *b, *c;
  tie(a,b) = split(t, l); tie(b,c) = split(b, r - l);
  if (k <= l) t = merge(ins(a, b, k), c);
  else t = merge(a, ins(c, b, k - r));
}

// Usage
// create treap
// Node* name=nullptr;
// insert element
// name=ins(name, new Node(val), pos);
// Node* x = new Node(val);
// name = ins(name, x, pos);
// merge two treaps (name before x)
// name=merge(name, x);
// split treap (this will split treap in two treaps,
// first with elements [0, pos) and second with elements [pos, n))
// pa will be pair of two treaps
// auto pa = split(name, pos);
// move range [l, r) to index k
// move(name, l, r, k);
// iterate over treap
// each(name, [&](int val) {
//     cout << val << ' ';
// });
```

3.41 Treap 2

```
typedef struct item * pitem;
struct item {
    int prior, value, cnt;
    bool rev;
    pitem l, r;
};

int cnt (pitem it) {
    return it ? it->cnt : 0;
}

void upd_cnt (pitem it) {
    if (it)
        it->cnt = cnt(it->l) + cnt(it->r) + 1;
}

void push (pitem it) {
    if (it && it->rev) {
        it->rev = false;
        swap (it->l, it->r);
        if (it->l)  it->l->rev ^= true;
        if (it->r)  it->r->rev ^= true;
    }
}

void merge (pitem & t, pitem l, pitem r) {
    push (l);
    push (r);
    if (!l || !r)
        t = l ? l : r;
    else if (l->prior > r->prior)
        merge (l->r, l->r, r),  t = l;
    else
        merge (r->l, l, r->l),  t = r;
    upd_cnt (t);
}

void split (pitem t, pitem & l, pitem & r, int key, int add = 0) {
```

```
    if (!t)
        return void( l = r = 0 );
    push (t);
    int cur_key = add + cnt(t->l);
    if (key <= cur_key)
        split (t->l, l, t->l, key, add),  r = t;
    else
        split (t->r, t->r, r, key, add + 1 + cnt(t->l)),  l = t;
    upd_cnt (t);
}

void reverse (pitem t, int l, int r) {
    pitem t1, t2, t3;
    split (t, t1, t2, l);
    split (t2, t2, t3, r-l+1);
    t2->rev ^= true;
    merge (t, t1, t2);
    merge (t, t, t3);
}

void output (pitem t) {
    if (!t)  return;
    push (t);
    output (t->l);
    printf ("%d ", t->value);
    output (t->r);
}
```

3.42 Treap With Inversion

```
std::mt19937 rng(std::chrono::steady_clock::now().time_since_epoch().
    count());

struct Node {
  Node *l = 0, *r = 0;
  int val, y, c = 1;
  bool rev = 0;
  Node(int val) : val(val), y(rng()) {}
  void recalc();
  void push();
};

int cnt(Node* n) { return n ? n->c : 0; }
void Node::recalc() { c = cnt(l) + cnt(r) + 1; }
void Node::push() {
  if (rev) {
    rev = 0;
    swap(l, r);
    if (l) l->rev ^= 1;
    if (r) r->rev ^= 1;
  }
}

template<class F> void each(Node* n, F f) {
  if (n) { n->push(); each(n->l, f); f(n->val); each(n->r, f); }
}

pair<Node*, Node*> split(Node* n, int k) {
  if (!n) return {};
  n->push();
  if (cnt(n->l) >= k) {
    auto pa = split(n->l, k);
    n->l = pa.second;
    n->recalc();
    return {pa.first, n};
  } else {
    auto pa = split(n->r, k - cnt(n->l) - 1);
    n->r = pa.first;
    n->recalc();
    return {n, pa.second};
  }
}

Node* merge(Node* l, Node* r) {
  if (!l) return r;
  if (!r) return l;
  l->push();
  r->push();
  if (l->y > r->y) {
    l->r = merge(l->r, r);
    l->recalc();
    return l;
  } else {
    r->l = merge(l, r->l);
    r->recalc();
    return r;
```

```
  }
}

Node* ins(Node* t, Node* n, int pos) {
  auto pa = split(t, pos);
  return merge(merge(pa.first, n), pa.second);
}

// Example application: reverse the range [l, r]
void reverse(Node*& t, int l, int r) {
  Node *a, *b, *c;
  tie(a,b) = split(t, l);
  tie(b,c) = split(b, r - l + 1);
  b->rev ^= 1;
  t = merge(merge(a, b), c);
}

void move(Node*& t, int l, int r, int k) {
  Node *a, *b, *c;
  tie(a,b) = split(t, l);
  tie(b,c) = split(b, r - l);
  if (k <= l) t = merge(ins(a, b, k), c);
  else t = merge(a, ins(c, b, k - r));
}
```

3.43 Treap With Inversions and Range Updates

```
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

const int mod = 998244353;

template<int MOD>
struct ModInt {
  ll v;
  ModInt(ll _v = 0) {v = (-MOD < _v && _v < MOD) ? _v : _v % MOD; if (v < 0) v += MOD;}
  ModInt& operator += (const ModInt &other) {v += other.v; if (v >= MOD) v -= MOD; return *this;}
  ModInt& operator -= (const ModInt &other) {v -= other.v; if (v < 0) v += MOD; return *this;}
  ModInt& operator *= (const ModInt &other) {v = v * other.v % MOD; return *this;}
  ModInt& operator /= (const ModInt &other) {return *this *= inverse(other);}
  bool operator == (const ModInt &other) const {return v == other.v;}
  bool operator != (const ModInt &other) const {return v != other.v;}
  friend ModInt operator + (ModInt a, const ModInt &b) {return a += b;}
  friend ModInt operator - (ModInt a, const ModInt &b) {return a -= b;}
  friend ModInt operator * (ModInt a, const ModInt &b) {return a *= b;}
  friend ModInt operator / (ModInt a, const ModInt &b) {return a /= b;}
  friend ModInt operator - (const ModInt &a) {return 0 - a;}
  friend ModInt power(ModInt a, ll b) {ModInt ret(1); while (b > 0) {if (b & 1) ret *= a; a *= a; b >>= 1;} return ret;}
  friend ModInt inverse(ModInt a) {return power(a, MOD - 2);}
  friend istream& operator >> (istream &is, ModInt &m) {is >> m.v; m.v = (-MOD < m.v && m.v < MOD) ? m.v : m.v % MOD; if (m.v < 0) m.v += MOD; return is;}
  friend ostream& operator << (ostream &os, const ModInt &m) {return os << m.v;}
};

struct Line{
  ModInt<mod> b = 1, c = 0;
  ModInt<mod> operator()(ModInt<mod> x){ return b * x + c;}
  Line operator()(Line other) {return Line{b * other.b, other.c * b + c};}
  operator bool() const{
    return b != 1 || c != 0;
  };
};

std::mt19937 rng(std::chrono::steady_clock::now().time_since_epoch().count());

struct Node {
  Node *l = 0, *r = 0;
  int y, c = 1;
  ModInt<mod> val, sum;
  Line line;
  bool rev = 0;
  Node(int val) : val(val), sum(val), y(rng()) {}
  void recalc();
  void push();
```

```
};

int cnt(Node* n) { return n ? n->c : 0; }
ModInt<mod> sum1(Node* n) { return n ? n->sum : ModInt<mod>(0); }
void Node::recalc() { if(l) l -> push(); if(r) r -> push(); c = cnt(l) +
     cnt(r) + 1; sum = sum1(l) + sum1(r) + val;}
void Node::push(){
  if(rev){
    rev = 0;
    swap(l, r);
    if(l) l -> rev ^= 1;
    if(r) r -> rev ^= 1;
  }
  if(line){
    val = line(val);
    sum = sum * line.b + line.c * c;
    if(l){
      l -> line = line(l -> line);
    }
    if(r){
      r -> line = line(r -> line);
    }
    line = {1, 0};
  }
}

template<class F> void each(Node* n, F f) {
  if (n) { n -> push(); each(n->l, f); f(n->val); each(n->r, f); }
}

pair<Node*, Node*> split(Node* n, int k) {
  if (!n) return {};
  n -> push();
  if (cnt(n->l) >= k) { // "n->val >= k" for lower_bound(k)
    auto [L,R] = split(n->l, k);
    n->l = R;
    n->recalc();
    return {L, n};
  } else {
    auto [L,R] = split(n->r,k - cnt(n->l) - 1); // and just "k"
    n->r = L;
    n->recalc();
    return {n, R};
  }
}

Node* merge(Node* l, Node* r) {
  if (!l) return r;
  if (!r) return l;
  l -> push();
  r -> push();
  if (l->y > r->y) {
    l->r = merge(l->r, r);
    return l->recalc(), l;
  } else {
    r->l = merge(l, r->l);
    return r->recalc(), r;
  }
}

Node* ins(Node* t, Node* n, int pos) {
  auto [l,r] = split(t, pos);
  return merge(merge(l, n), r);
}

void move(Node*& t, int l, int r, int k){
  Node *a, *b, *c;
  tie(a, b) = split(t, l); tie(b, c) = split(b, r - l);
  if(k <= l) t = merge(ins(a, b, k), c);
  else t = merge(a, ins(c, b, k - r));
}

void revv(Node*& t, int l, int r){
  Node *a, *b, *c;
  tie(a, b) = split(t, l); tie(b, c) = split(b, r - l);
  b -> rev ^= 1;
  t = merge(merge(a, b), c);
}

ModInt<mod> query(Node*& t, int l, int r){
  Node *a, *b, *c;
  tie(a, b) = split(t, l); tie(b, c) = split(b, r - l);
  ModInt<mod> res = sum1(b);
  t = merge(merge(a, b), c);
  return res;
}
```

```

void remove(Node*& t, int x){
  Node *a, *b, *c;
  tie(a, b) = split(t, x); tie(b, c) = split(b, 1);
  t = merge(a, c);
}

void update(Node*& t, int l, int r, ll X, ll Y){
  Node *a, *b, *c;
  tie(a, b) = split(t, l); tie(b, c) = split(b, r - l);
  b -> line = Line{X, Y}(b -> line);
  t = merge(a, merge(b, c));
}

int main(){
  ios_base::sync_with_stdio(false); cin.tie(NULL);
  int n, q; cin >> n >> q;
  vector<int> a(n);
  for(int i = 0; i < n; i++){
    cin >> a[i];
  }
  Node* treap = nullptr;
  for(int i = 0; i < n; i++){
    treap = ins(treap, new Node(a[i]), i);
  }
  while(q--){
    int t; cin >> t;
    if(t == 0){
      int i, x; cin >> i >> x;
      treap = ins(treap, new Node(x), i);
    }
    else if(t == 1){
      int x; cin >> x;
      remove(treap, x);
    }
    else if(t == 2){
      int l, r; cin >> l >> r;
      revv(treap, l, r);
    }
    else if(t == 3){
      int l, r, b, c; cin >> l >> r >> b >> c;
      update(treap, l, r, b, c);
    }
    else{
      int l, r; cin >> l >> r;
      cout << query(treap, l, r) << endl;
    }
  }
}
```

4 Dynamic Programming

4.1 Monotone CHT

C++17. Insert lines $y = mx + b$ in the slope order selected by `INCREASING_SLOPES`; equal slopes are allowed. `MIN_QUERY` selects minimum (`true`) or maximum (`false`). Defaults: minimum, nonincreasing slopes.

Query	Slope order	`query(x)` order
Minimum	Nonincreasing	Nondecreasing x
Minimum	Nondecreasing	Nonincreasing x
Maximum	Nonincreasing	Nonincreasing x
Maximum	Nondecreasing	Nondecreasing x

`add` and `query`: amortized $O(1)$; `query_bs`: $O(\log n)$ with arbitrary x order, without changing the query pointer. Insertions and queries may be interleaved. Query only when nonempty. Inputs are signed 64-bit; results are `__int128_t`.

```
#include <bits/stdc++.h>

template<bool MIN_QUERY = true, bool INCREASING_SLOPES = false>
struct CHT {
    using i128 = __int128_t;
    using u128 = __uint128_t;

    static constexpr bool FLIP_X =
        MIN_QUERY == INCREASING_SLOPES;

    struct Line {
        i128 m, b;

        i128 eval(i128 x) const {
            return m * x + b;
        }
    };

    std::vector<Line> hull;
    int ptr = 0;
```

```

    void clear() {
        hull.clear();
        ptr = 0;
    }

    bool empty() const {
        return hull.empty();
    }

    static bool bad(const Line& a, const Line& b, const Line& c) {
        // Internally: decreasing slopes, minimum queries.
        i128 x = b.b - a.b;
        i128 y = c.b - b.b;

        if ((x < 0) != (y < 0))
            return x >= 0;

        // Unsigned magnitudes avoid signed cross-product overflow.
        u128 lhs = (u128)(x < 0 ? -x : x)
                 * (u128)(b.m - c.m);

        u128 rhs = (u128)(y < 0 ? -y : y)
                 * (u128)(a.m - b.m);

        return x < 0 ? lhs <= rhs : lhs >= rhs;
    }

    // Amortized O(1).
    void add(long long slope, long long intercept) {
        i128 m = slope, b = intercept;

        if constexpr (!MIN_QUERY) {
            m = -m;
            b = -b;
        }
        if constexpr (FLIP_X)
            m = -m;

        if (!hull.empty() && hull.back().m == m) {
            if (hull.back().b <= b)
                return;
            hull.pop_back();
        }

        Line line{m, b};

        while (hull.size() >= 2 &&
               bad(hull[hull.size() - 2], hull.back(), line)) {
            hull.pop_back();
        }

        hull.push_back(line);
        ptr = std::min(ptr, (int)hull.size() - 1);
    }

    // Amortized O(1). Requires the query order from the table.
    i128 query(long long x) {
        i128 X = x;

        if constexpr (FLIP_X)
            X = -X;

        while (ptr + 1 < (int)hull.size() &&
               hull[ptr + 1].eval(X) <= hull[ptr].eval(X)) {
            ++ptr;
        }

        i128 value = hull[ptr].eval(X);
        return MIN_QUERY ? value : -value;
    }

    // O(log n). Arbitrary query order; does not change ptr.
    i128 query_bs(long long x) const {
        i128 X = x;

        if constexpr (FLIP_X)
            X = -X;

        int l = 0, r = (int)hull.size() - 1;

        while (l < r) {
            int mid = l + (r - l) / 2;

            if (hull[mid + 1].eval(X) <= hull[mid].eval(X))
```

```
                l = mid + 1;
            else
                r = mid;
        }

        i128 value = hull[l].eval(X);
        return MIN_QUERY ? value : -value;
    }
};

/* Example (inside main): minimum, nondecreasing slopes.
CHT<true, true> cht;
cht.add(1, 0);  // y = x
cht.add(2, -1); // y = 2x - 1
cht.add(3, 0);  // y = 3x
auto a = cht.query(2);  // 2
auto b = cht.query(0);  // -1; nonincreasing x
auto c = cht.query_bs(4); // 4; arbitrary x; ptr unchanged
auto d = cht.query(-2); // -6
*/
```

4.2 CHT with Counts

C++17. Returns `(value, cnt)`: optimize the value and prefer smaller counts on ties, for both minimum and maximum queries. Uses the slope and query orders in the Monotone CHT table above; equal slopes are allowed. `add` and `query` take amortized $O(1)$. Query only when nonempty.

This version compares `long double` values directly; the count breaks ties only when computed values compare equal. Each line keeps the supplied count. Both variants define `CHT`; use one definition at a time.

```
#include <bits/stdc++.h>

template<bool MIN_QUERY = true, bool INCREASING_SLOPES = false>
struct CHT {
    using ld = long double;

    static constexpr bool FLIP_X =
        MIN_QUERY == INCREASING_SLOPES;

    struct Line {
        ld m, b;
        int cnt;

        std::pair<ld, int> eval(ld x) const {
            return {m * x + b, cnt};
        }
    };

    std::vector<Line> hull;
    int ptr = 0;

    void clear() {
        hull.clear();
        ptr = 0;
    }

    bool empty() const {
        return hull.empty();
    }

    static bool bad(const Line& a, const Line& b, const Line& c) {
        // Treat each intercept symbolically as b + eps * cnt.
        // Compare the cost products first, then count products.
        std::pair<ld, ld> lhs{
            (b.b - a.b) * (b.m - c.m),
            ((ld)b.cnt - a.cnt) * (b.m - c.m)
        };

        std::pair<ld, ld> rhs{
            (c.b - b.b) * (a.m - b.m),
            ((ld)c.cnt - b.cnt) * (a.m - b.m)
        };

        return lhs >= rhs;
    }

    // Slopes follow INCREASING_SLOPES; equal slopes are allowed.
    void add(ld slope, ld intercept, int cnt) {
        ld m = slope, b = intercept;

        if constexpr (!MIN_QUERY) {
            m = -m;
            b = -b;
        }

```

```
        if constexpr (FLIP_X)
            m = -m;

        if (!hull.empty() && hull.back().m == m) {
            if (std::pair<ld, int>{hull.back().b, hull.back().cnt}
                <= std::pair<ld, int>{b, cnt}) {
                return;
            }

            hull.pop_back();
        }

        Line line{m, b, cnt};

        while (hull.size() >= 2 &&
               bad(hull[hull.size() - 2], hull.back(), line)) {
            hull.pop_back();
        }

        hull.push_back(line);
        ptr = std::min(ptr, (int)hull.size() - 1);
    }

    // Requires a nonempty hull.
    // x must be nonincreasing if FLIP_X, nondecreasing otherwise.
    std::pair<ld, int> query(ld x) {
        ld X = x;

        if constexpr (FLIP_X)
            X = -X;

        while (ptr + 1 < (int)hull.size() &&
               hull[ptr + 1].eval(X) <= hull[ptr].eval(X)) {
            ++ptr;
        }

        auto [value, cnt] = hull[ptr].eval(X);

        // Counts remain unchanged: both modes prefer fewer on ties.
        if constexpr (!MIN_QUERY)
            value = -value;

        return {value, cnt};
    }
};

/* Example (inside main): minimum, nonincreasing slopes.
CHT<true, false> cht;
cht.add(3, 0, 4);
cht.add(2, 0, 1);
cht.add(1, 0, 3);
auto a = cht.query(-1); // {-3, 4}
auto b = cht.query(0);  // {0, 1}; all values tie, fewer wins
auto c = cht.query(1);  // {1, 3}
*/
```

4.3 CHT Deque

```
/*
  Convex Hull Trick (CHT) - Min Query with Increasing Slopes
  ----------------------------------------------------------
  Indexing: 1-based for ‘a‘, ‘dp‘, ‘s‘
  Bounds:
    - i from 1 to m // number of elements in the array
    - j from 1 to p // number of transitions
  Time Complexity: O(m * p)
  Requires:
    - Lines inserted in increasing slope order for min query
    - Queries made with increasing x values

  dp[i][j] = min over k < i of { dp[k][j-1] + a[i] * (i - k) + s[i] - s[
      k+1] }
  dp[i][j] = min over k < i of { dp[k][j-1] + cost(k, i) }

  We reformulate:
    y = m * x + c
    line: m = -k - 1, c = dp[k][j-1] - s[k+1]
    eval: m * a[i] + c + a[i] * i + s[i]
*/

struct Line {
  ll a, b; // y = ax + b
  Line(ll A, ll B) : a(A), b(B) {}
  ll eval(ll x) const {
    return a * x + b;
```

```
  }
  // Returns intersection x-coordinate with another line
  double intersect(const Line& other) const {
    return (double)(other.b - b) / (a - other.a);
  }
};

// this finds the minimum and slope in increasing
// Deques for each dp stage
deque<Line> cht[p+1];
// Fill dp
cht[0].push_back(Line(-1, -s[1])); // base case
for (int i = 1; i <= m; i++) {
  for (int j = p; j >= 1; j--) {
    if (j > i) continue;
    // Maintain front of deque to find minimum
    while (cht[j - 1].size() >= 2 && cht[j - 1][1].eval(a[i]) <= cht[j -
        1][0].eval(a[i])) {
      cht[j - 1].pop_front();
    }
    // Evaluate best line
    dp[i][j] = cht[j - 1].front().eval(a[i]) + a[i] * i + s[i];
    // Create new line for current i
    Line curr(-i - 1, dp[i][j] - s[i + 1]);
    // Maintain convexity: remove worse lines from back
    while (cht[j].size() >= 2) {
      Line& l1 = cht[j][cht[j].size() - 2];
      Line& l2 = cht[j].back();
      if (curr.intersect(l1) <= l2.intersect(l1)) {
        cht[j].pop_back();
      } else break;
    }
    cht[j].push_back(curr);
  }
}
```

4.4 Digit DP

```
/*
  Digit Dynamic Programming (Digit DP)
  ------------------------------------
  Goal: Count numbers in range [0, x] that do not have two adjacent
      equal digits.
  State:
    - pos: current digit position
    - last: digit placed at previous position (0 to 9)
    - f: tight flag (0 = must match prefix of x, 1 = already below x)
    - z: leading zero flag (1 = still in leading zero zone)
  Notes:
    - Solve up to x using `solve(x)`
    - Can be modified to count palindromes, digits divisible by 3, etc.
*/

vector<int> num;
ll DP[20][20][2][2]; // pos, last digit, f (tight), z (leading zero)

ll g(int pos, int last, int f, int z) {
  if (pos == num.size()) return 1; // reached end, valid number
  if (DP[pos][last][f][z] != -1) return DP[pos][last][f][z];
  ll res = 0;
  int limit = f ? 9 : num[pos]; // upper digit bound based on tight flag
  for (int dgt = 0; dgt <= limit; dgt++) {
    // Skip if digit equals last (unless it's a leading zero)
    if (dgt == last && !(dgt == 0 && z == 1)) continue;
    int nf = f;
    if (!f && dgt < limit) nf = 1;
    if (z && dgt == 0) res += g(pos + 1, dgt, nf, 1); // still leading
        zeros
    else res += g(pos + 1, dgt, nf, 0); // now in significant digits
  }
  return DP[pos][last][f][z] = res;
}

ll solve(ll x) {
  if (x == -1) return 0;
  num.clear();
  while (x > 0) {
    num.push_back(x % 10);
    x /= 10;
  }
  reverse(num.begin(), num.end());
  memset(DP, -1, sizeof(DP));
  return g(0, 0, 0, 1);
}
```

4.5 Divide and Conquer DP

```cpp
/*
  Divide and Conquer DP Optimization
  ----------------------------------
  Problem:
    - dp[i][j] = min over k <= j of { dp[i-1][k] + C(k, j) }
    - C(k, j) must satisfy the quadrangle inequality:
        - C(a, c) + C(b, d) <= C(a, d) + C(b, c) for a <= b <= c <= d
    - or monotonicity of opt[i][j] <= opt[i][j+1]

  Indexing: 0-based
  Time Complexity: O(m * n * log n)
  Space Complexity: O(n)

  dp_cur[j]: current dp[i][j] layer
  dp_before[j]: previous dp[i-1][j] layer
*/

int n, m;
vector<ll> dp_before, dp_cur;
ll C(int i, int j); // Cost function defined by user

// Recursively compute dp_cur[l..r] with optimal k in [optl, optr]
void compute(int l, int r, int optl, int optr) {
  if (l > r) return;
  int mid = (l + r) / 2;
  pair<ll, int> best = {LLONG_MAX, -1};
  for (int k = optl; k <= min(mid, optr); k++) {
    ll val = (k > 0 ? dp_before[k - 1] : 0) + C(k, mid);
    if (val < best.first) best = {val, k};
  }
  dp_cur[mid] = best.first;
  int opt = best.second;
  compute(l, mid - 1, optl, opt);
  compute(mid + 1, r, opt, optr);
}

// Entry point: computes dp[m-1][n-1]
ll solve() {
  dp_before.assign(n, 0);
  dp_cur.assign(n, 0);
  for (int i = 0; i < n; i++) {
    dp_before[i] = C(0, i);
  }
  for (int i = 1; i < m; i++) {
    compute(0, n - 1, 0, n - 1);
    dp_before = dp_cur;
  }
  return dp_before[n - 1];
}
```

4.6 Edit Distance

```cpp
/*
  Given strings s and t, compute the minimum number of operations
  (insert, delete, substitute) to convert s into t.
  Indexing: 0-based (strings), DP is 1-based with offset
  dp[i][j] = cost to convert s[0..i-1] into t[0..j-1]
  Time Complexity: O(n * m)
  Transitions:
    - insert: dp[i][j-1] + 1
    - delete: dp[i-1][j] + 1
    - replace/match: dp[i-1][j-1] + (s[i-1] != t[j-1])
*/
const int MAXN = 5005;
int dp[MAXN][MAXN];

string s, t; cin >> s >> t;
int n = s.length(), m = t.length();
// Initialize all to a large number
for (int i = 0; i <= n; i++) {
  fill(dp[i], dp[i] + m + 1, 1e9);
}
dp[0][0] = 0;
for (int i = 0; i <= n; i++) {
  for (int j = 0; j <= m; j++) {
    if (j) { // insert
      dp[i][j] = min(dp[i][j], dp[i][j - 1] + 1);
    }
    if (i) { // delete
      dp[i][j] = min(dp[i][j], dp[i - 1][j] + 1);
    }
    if (i && j) { // replace or match
      int cost = (s[i - 1] != t[j - 1]) ? 1 : 0;
      dp[i][j] = min(dp[i][j], dp[i - 1][j - 1] + cost);
```

```
    }
  }
}
```

4.7 Knuth's Algorithm

```
#include<bits/stdc++.h>
using namespace std;

const int N = 1010;
using ll = long long;
/*
Knuths optimization works for optimization over sub arrays
for which optimal middle point depends monotonously on the end points.
Let mid[l,r] be the first middle point for (l,r) sub array which gives
    optimal result.
It can be proven that mid[l,r-1] <= mid[l,r] <= mid[l+1,r]
- this means monotonicity of mid by l and r.
Applying this optimization reduces time complexity from O(k^3) to O(k^2)
because with fixed s (sub array length) we have m_right(l) = mid[l+1][r]
     = m_left(l+1).
That's why nested l and m loops require not more than 2k iterations
    overall.
*/

int n, k;
int a[N], mid[N][N];
ll res[N][N];
ll solve() {
  for (int s = 0; s <= k; s++) {          // s - length of the subarray
    for (int l = 0; l + s <= k; l++) {  // l - left point
      int r = l + s;                   // r - right point
      if (s < 2) {
        res[l][r] = 0;              // base case- nothing to break
        mid[l][r] = l;              // mid is equal to left border
        continue;
      }
      int mleft = mid[l][r - 1];
      int mright = mid[l + 1][r];
      res[l][r] = 2e18;
      for (int m = mleft; m <= mright; m++) {    // iterating for m in
          the bounds only
        ll tmp = res[l][m] + res[m][r] + (a[r] - a[l]);
        if (res[l][r] > tmp) {                // relax current solution
          res[l][r] = tmp;
          mid[l][r] = m;
        }
      }
    }
  }
  ll ans = res[0][k];
  return ans;
}
int main() {
  int i, j, m;
  while(cin >> n >> k) {
    for(i = 1; i <= k; i++) cin >> a[i];
    a[0] = 0;
    a[k + 1] = n;
    k++;
    cout << solve() << endl;
  }
  return 0;
}
// https://vjudge.net/problem/ZOJ-2860
```

4.8 LCS

```
string s, t; cin >> s >> t;
int n=s.length(), m=t.length();
int dp[n+1][m+1];
memset(dp, 0, sizeof(dp));
for(int i=1;i<=n;i++){
    for(int j=1;j<=m;j++){
        dp[i][j]=max(dp[i-1][j], dp[i][j-1]);
        if(s[i-1]==t[j-1]){
            dp[i][j]=dp[i-1][j-1]+1;
        }
    }
}
int i=n, j=m;
string ans="";
while(i && j){
    if(s[i-1]==t[j-1]){
        ans+=s[i-1];
        i--; j--;
```

```
    }
    else if(dp[i][j-1]>=dp[i-1][j]){
        j--;
    }
    else{
        i--;
    }
}
reverse(all(ans));
cout << ans << endl;

// For two permutations one can create new array that will map each element from the first permutation to the second.
// For each element a[i] in the first permutatio, you find which j is a[i] == b[j].
// After creating this new array, run LIS (Longest Increasing subsequence).
```

4.9 Line Container

```
/*
  Line Container (Dynamic Convex Hull Trick)
  ------------------------------------------
  Supports:
    - Adding lines: y = k * x + m
    - Querying maximum y at given x
  Indexing: arbitrary, supports any x
  Time Complexity:
    - add(): amortized O(log n)
    - query(x): O(log n)
  Space Complexity: O(n)
  For min queries: negate slopes and intercepts on insert and result on query
  Structure:
    - Stores lines in slope-sorted order (k)
    - Each line keeps its intersection point `p` with the next line
    - Binary search on `p` to answer queries
*/
struct Line {
  mutable ll k, m, p;
  bool operator<(const Line& o) const { return k < o.k; }  // Sort by slope
  bool operator<(ll x) const { return p < x; }             // Query comparator
};

struct LineContainer : multiset<Line, less<>> {
  // (for doubles, use inf = 1/.0, div(a,b) = a/b)
  static const ll inf = LLONG_MAX;
  ll div(ll a, ll b) { // Floored division
    return a / b - ((a ^ b) < 0 && a % b);
  }
  // Update intersection point x->p with y
  bool isect(iterator x, iterator y) {
    if (y == end()) return x->p = inf, false;
    if (x->k == y->k)
      x->p = (x->m > y->m ? inf : -inf); // higher line wins
    else
      x->p = div(y->m - x->m, x->k - y->k);
    return x->p >= y->p;
  }
  // Add new line: y = k * x + m
  void add(ll k, ll m) {
    auto z = insert({k, m, 0}), y = z++, x = y;
    // Remove dominated lines after y
    while (isect(y, z)) z = erase(z);
    // Remove dominated lines before y
    if (x != begin() && isect(--x, y))
      isect(x, y = erase(y));
    // Further cleanup to preserve order
    while ((y = x) != begin() && (--x)->p >= y->p)
      isect(x, erase(y));
  }
  // Query max y at given x
  ll query(ll x) {
    assert(!empty());
    auto l = *lower_bound(x);
    return l.k * x + l.m;
  }
};
// Example usage:
LineContainer cht;
cht.add(3, 5);      // y = 3x + 5
cht.add(2, 7);      // y = 2x + 7
cout << cht.query(4) << '\n'; // max y at x = 4
```

4.10 Longest Increasing Subsequence

```cpp
/*
  Longest Increasing Subsequence + (Recover Sequence) O(n log n)
  ----------------------------------------------
  If no recovery is needed, use dp[] only.
  dp.size() gives the length of LIS.
  For non-decreasing use upper_bound instead of lower_bound.
*/
vector<int> dp;    // smallest tail values of LIS length i+1
vector<int> dp_index;    // index in original array
vector<int> parent(n, -1);    // parent[i] = index of previous element in LIS
vector<int> last_pos(n + 1);  // last_pos[len] = index in v[] ending LIS of length len
for (int i = 0; i < n; i++) {
  auto it = lower_bound(dp.begin(), dp.end(), v[i]);
  int len = it - dp.begin();
  if (it == dp.end()) {
    dp.push_back(v[i]);
    dp_index.push_back(i); // Ignore if no recovery
  } else {
    *it = v[i];
    dp_index[len] = i; // Ignore if no recovery
  }
  if (len > 0) parent[i] = dp_index[len - 1]; // Ignore if no recovery
}
// Reconstruct LIS
vector<int> lis;
int pos = dp_index.back();
while (pos != -1) {
  lis.push_back(v[pos]);
  pos = parent[pos];
}
reverse(lis.begin(), lis.end());
```

4.11 SOS DP

```cpp
vector<int> SOS_DP(vector<int> A, int k) {
  vector<int> Ap = A;
  for (int i = k - 1; i >= 0; i--)
    for (int mk = 0; mk < (1 << k); mk++)
      if (mk & (1 << i))
        Ap[mk] += Ap[mk - (1 << i)];
  return Ap;
}

vector<int> SOS_DP_inv (vector<int> Ap, int k) {
  vector<int> A = Ap;
  for (int i = 0; i < k; i++)
      for (int mk = 0; mk < (1 << k); mk++)
          if (mk & (1 << i))
              A[mk] -= A[mk - (1 << i)];
  return A;
}

vector<int> SUPER_SOS_DP(vector<int> A, int k) {
  vector<int> Ap = A;
  for (int i = k - 1; i >= 0; i--)
    for (int mk = 0; mk < (1 << k); mk++)
      if (~mk & (1 << i))
        Ap[mk] += Ap[mk + (1 << i)];
  return Ap;
}

vector<int> SUPER_SOS_DP_inv (vector<int> Ap, int k) {
  vector<int> A = Ap;
  for (int i = 0; i < k; i++)
    for (int mk = (1 << k) - 1; mk >= 0; mk--)
          if (~mk & (1 << i))
              A[mk] -= A[mk + (1 << i)];
  return A;
}

// or convolution
Ap = SOS_DP(A, k);
Bp = SOS_DP(B, k);

for (int i = 0; i < (1 << k); i++)
    Cp[i] = Ap[i] * Bp[i];

C = SOS_DP_inv(Cp, k);

// and convolution
Ap = SUPER_SOS_DP(A, k);
Bp = SUPER_SOS_DP(B, k);
```

```
49
50 for (int i = 0; i < (1 << k); i++)
51     Cp[i] = Ap[i] * Bp[i];
52
53 C = SUPER_SOS_DP_inv(Cp, k);
```

4.12 Subset Sum, Knapsack and Max-Plus: Six Variants

These are six useful parameter regimes from errorgorn's tutorial, with independently written implementations, derivations and examples. Choose the method by its assumptions, not just by the word "knapsack". All weights below are *positive integers*. Subset sum asks for an *exact* target. The bounded, distinct-weight and window knapsack implementations return the best value with weight *at most* the capacity; choosing nothing is allowed. NDPC exact-fill knapsack is a separate subsection afterward.

Notation and implementation conventions. N is the input length, C a target/capacity, $S = \sum_i w_i$, D a maximum weight, and d a number of distinct weights. These meanings are restated where needed. The C++17 headers use namespace `knapsack_ref`; include the header corresponding to the method. Only the NDPC listing contains `main`. Array sizes and integer index arithmetic must fit `int` and available memory. Knapsack values, sums, differences and products in these generic headers must fit `long long`. For larger values, change the value type and its numeric limits together.

Assumption / goal	Method / time
All subset sums; total S manageable	Coalesce equal weights, then bitsets; $O(N + S + (S/64 + 1)\sqrt{S})$
One subset-sum target; $w_i \le D$	Balanced states; $O(ND)$
Bounded copies per type	Residues and a monotone deque; $O(NC)$
One concave max-plus operand	SMAWK; $O(n + m)$
0/1 items; d distinct weights	Group and convolve; $O(N \log N + dC)$
Unlimited copies; $w_i \le D$, huge C	Repeatedly double a short window; $O(N + D^2 \log(C + 2))$

4.12.1 1. All Subset Sums: Compress Repeated Weights

Problem. Each of N items may be taken once. Determine every attainable sum from 0 to $S = \sum_i w_i$. A boolean array costs $O(NS)$. A bitset packs 64 states into a word: initially only bit zero is set, and one item of weight w performs

$$B \leftarrow B \mid (B \ll w).$$

The shift uses the *old* B, so the item is used at most once. For a manually stored bitset, update words from high to low for the same reason. A shift by an exact multiple of 64 needs a separate case: shifting a 64-bit word by 64 is undefined.

Reduce the number of shifts. Let c_w be the number of remaining copies of weight w. Process weights in increasing order. For odd c_w, retain one copy of w and replace the other copies by $(c_w - 1)/2$ bundles of weight $2w$. For positive even c_w, retain two copies and create $(c_w - 2)/2$ bundles of weight $2w$. Add these bundles to the count for $2w$, which will be processed later. A bundle is one binary choice representing two copies, not an unbounded item.

Why all sums survive. With one retained copy, selecting $e \in \{0, 1\}$ singles and t pairs gives $e + 2t$ original copies, covering every count from zero through the original odd c_w. With two retained copies, $e \in \{0, 1, 2\}$ covers both parities and every count through the original even c_w. Conversely, every new selection uses at most c_w original copies. Thus the replacement preserves exactly the attainable sums; applying it repeatedly does too.

Why only $O(\sqrt{S})$ shifts. Total weight is conserved. If k distinct positive weights remain, their total is at least $1 + 2 + \cdots + k = k(k + 1)/2 \le S$. At most two copies of each survive, so there are at most $2k = O(\sqrt{S})$ bitset updates. Counting/scanning takes $O(N + S)$; the shifts take $O((S/64 + 1)\sqrt{S})$. The implementation uses $O(S)$ integer counters and $O(S/64 + 1)$ bitset words. This is a bound in terms of the *total sum*, not an arbitrary small target.

Example. Eight copies of weight two become two copies of two, three copies of four; processing the latter leaves one four and one eight. The final weights $[2, 2, 4, 8]$ attain exactly $0, 2, 4, \ldots, 16$. Add an original weight three and one bitset shift also gives the corresponding odd sums. Zero-weight items may be ignored for feasibility; negative weights require another method.

```
1  #pragma once
2  #include <bits/stdc++.h>
3
4  namespace knapsack_ref {
5  // Positive weights; total sum S must fit int and memory.
6  // Bit s of the returned words says whether sum s is possible.
7  std::vector<uint64_t> subset_sum_all(
8      const std::vector<int>& weights) {
9      int S = std::accumulate(weights.begin(), weights.end(), 0);
10     std::vector<int> count(S + 1);
11     for (int w : weights) ++count[w];
12     std::vector<uint64_t> bits(S / 64 + 1);
13     bits[0] = 1;
14     auto shift_or = [&](int w) {
15         int whole = w / 64, part = w % 64;
16         // Descend: every source still belongs to the old bitset.
17         for (int j = (int)bits.size() - 1; j >= whole; --j) {
18             uint64_t moved = bits[j - whole] << part;
19             if (part && j > whole)
```

```
                moved |= bits[j - whole - 1] >> (64 - part);
            bits[j] |= moved;
        }
    };
    for (int w = 1; w <= S; ++w) {
        int c = count[w];
        if (!c) continue;
        int keep = (c % 2 ? 1 : 2);
        int pairs = (c - keep) / 2;
        if (pairs) count[2 * w] += pairs;
        for (int j = 0; j < keep; ++j) shift_or(w);
    }
    return bits;
}
```

```
auto B = knapsack_ref::subset_sum_all({2, 2, 3});
int target = 7;
bool possible = (B[target / 64] >> (target % 64)) & 1;
// Check 0 <= target <= total sum before indexing.
// Attainable sums: 0, 2, 3, 4, 5, 7.
```

4.12.2 2. One Subset-Sum Target: Balanced States in O(ND)

Problem. Each item may be used once, $1 \le w_i \le D$, and we only need to decide whether target C is attainable. Here D is *maximum item weight*, not the number of distinct weights. The running time is $O(ND)$, independent of the numerical size of C. Return true for $C = 0$, and false when $C > S$.

Start near the target. Use zero-based indices. Take the longest prefix $[0, k)$ whose sum P is strictly smaller than C. Because the next item has weight at most D, $C - D \le P < C$. Any target solution can be reached from this prefix by removing some prefix items and adding some suffix items.

There is an order of these operations that stays in the narrow band $[C - D, C + D]$. Below C, add a required suffix item; above C, remove a prefix item that is absent from the desired solution. The required kind of operation always exists: otherwise the remaining operations could not bring the current sum to C. Every step changes the total by at most D. We may choose additions in increasing index order and removals in decreasing index order; the same argument still applies because any remaining item of the required kind is suitable. Stop immediately if the total is C.

Compress the choice of removed prefix. After processing suffix indices through r, store a boundary $l \in [0, k]$ for each band total s. Indices $[0, l)$ of the initial prefix are still fixed as selected. The last removed prefix index, if any, is l; future removals may use only indices smaller than l. Initially $l = k$. All already processed suffix choices and prefix indices at least l are accounted for in the state's witness.

For the same (s, r), retain only the *largest* feasible l: it permits every future removal allowed by a smaller boundary. Future suffix choices depend only on r, not on the history. Equivalently, one can define feasibility with $[0, l)$ fixed and the processed interval after it flexible; feasibility for a larger boundary implies feasibility for a smaller one. This is the dominance that replaces a third DP dimension by one integer.

Transitions and their order.

1. Copy the previous suffix layer, representing skipping item r. From each old total below C, also add w_r, keeping its boundary. Only old-layer states are used, so item r cannot be added twice.

2. Scan totals above C from high to low. From boundary l, removing fixed prefix item $j < l$ creates total $s - w_j$ and boundary j. Descending totals ensure that if this new total is still above C, its own removals are processed later in the same scan.

3. At total s, let l_{old} be the boundary before adding this suffix item. Try only $\max(0, l_{\text{old}}) \le j < l_{\text{new}}$. Indices smaller than l_{old} were already tried from this total in an earlier layer, and their results remain available because skipping later suffix items is permitted.

For a fixed total, boundaries never decrease as suffix items are processed. The newly tried ranges of j therefore never overlap. There are $2D + 1$ totals and at most k removals per total over the *whole run*, giving $O(ND)$ time. Two layers use $O(D)$ space. The array index for total s is $s - C + D$; index D is the target. A value -1 means unreachable, while boundary zero is reachable.

Example. Weights $[4, 3, 5]$, target eight: the initial prefix has total seven. Add five to obtain twelve, then remove the prefix item four to obtain eight. A DP that kept only totals at most eight would discard the necessary intermediate total twelve.

```
// Positive weights; target >= 0; returns exact feasibility.
// O(N*D) time, O(D) extra space, D = maximum weight.
bool subset_sum_small_weights(const std::vector<int>& w,
                              long long target) {
    if (target == 0) return true;
    long long total = std::accumulate(w.begin(), w.end(), 0LL);
    if (target > total || w.empty()) return false;
    int n = (int)w.size();
    int D = *std::max_element(w.begin(), w.end());
    int k = 0;
    long long base = 0;
    while (k < n && base + w[k] < target) base += w[k++];
    // Index t represents total target + t - D.
    // dp[t] = largest boundary l in [0,k], or -1.
    std::vector<int> dp(2 * D + 1, -1);
```

```
    dp[(int)(base - target + D)] = k;
    for (int r = k; r < n; ++r) {
        auto old = dp;
        // Add item r only from totals below the target.
        for (int t = 0; t < D; ++t)
            if (old[t] >= 0)
                dp[t + w[r]] = std::max(dp[t + w[r]], old[t]);
        // Remove a still-fixed prefix item only above target.
        // Descending totals close all chains of removals.
        for (int t = 2 * D; t > D; --t) {
            int stop = std::max(0, old[t]);
            for (int j = dp[t] - 1; j >= stop; --j)
                dp[t - w[j]] = std::max(dp[t - w[j]], j);
        }
        if (dp[D] >= 0) return true;
    }
    return false;
}
} // namespace knapsack_ref
```

```
using knapsack_ref::subset_sum_small_weights;
bool yes = subset_sum_small_weights({4, 3, 5}, 8); // true
bool no  = subset_sum_small_weights({4, 3, 5}, 6); // false
```

This is an exact deterministic algorithm. Randomly shuffling signed items and retaining an experimentally chosen band is a different, heuristic approach without this guarantee. The balancing technique originates with Pisinger; see Linear Time Algorithms for Knapsack Problems with Bounded Weights (1999).

4.12.3 3. Bounded Knapsack: Residues and a Monotone Deque

Problem. Type i has weight w_i, value v_i and at most K_i copies; maximize value with weight at most C. Here N counts *types*, not all expanded copies. Let $F[c]$ be the best value with budget c before this type, and $G[c]$ the answer afterward. Initially $F[c] = 0$ for every c, since unused budget is permitted. The ordinary transition is

$$G[c] = \max_{0 \le t \le K_i,\ tw_i \le c} \{F[c - tw_i] + tv_i\}.$$

Trying every copy count costs too much when K_i is large.
Fix one residue. Write $c = r + tw_i$, with $0 \le r < w_i$. If s is the number of weight steps assigned to the old DP, then $t - s$ is the number of new copies:

$$G[r + tw_i] = tv_i + \max_{\max(0,t-K_i) \le s \le t} \{F[r + sw_i] - sv_i\}.$$

For increasing t, this is a sliding maximum over a window of $K_i + 1$ candidates. Store pairs $(s, F[r + sw_i] - sv_i)$ in a deque. Before querying, remove expired indices $s < t - K_i$ from the front. When inserting a new candidate, remove back entries with no larger value: the newer candidate is at least as valuable and expires later. The front is then the maximum. Insert $s = t$ before querying so taking zero copies is included. Every index enters and leaves at most once.
Why use an old layer? Reading partially updated G would allow extra copies of this type. Copy F before the residue loops. All residues together contain $C + 1$ states, so one type takes $O(C)$, all types $O(NC)$, with $O(C)$ extra space. Replace K_i by $\min(K_i, \lfloor C/w_i \rfloor)$ first. Types with $w_i > C$ cannot be taken.
Example. Suppose the old DP is all zero, a type has $(w, v, K) = (3, 5, 2)$, and $C = 8$. In residue two, budgets $2, 5, 8$ give $t = 0, 1, 2$. The candidates are $0, -5, -10$, and the resulting values are $0, 5, 10$. At budget eleven, the old candidate $s = 0$ would expire: three copies are forbidden even though they would fit the budget.

```
#pragma once
#include <bits/stdc++.h>

namespace knapsack_ref {
struct BoundedItem {
    int weight;
    long long value, copies;
};

// dp[c] = maximum value using weight AT MOST c.
// Positive weights, nonnegative copy counts; signed values allowed.
// Every intermediate value/product must fit long long.
std::vector<long long> bounded_knapsack(
    int C, const std::vector<BoundedItem>& items) {
    using ll = long long;
    std::vector<ll> dp(C + 1, 0);
    for (auto [w, v, copies] : items) {
        if (w > C || copies == 0) continue;
        int K = (int)std::min(copies, (ll)(C / w));
        auto old = dp;
        for (int r = 0; r < w && r <= C; ++r) {
            std::deque<std::pair<int, ll>> dq;
            for (int t = 0; r + t * w <= C; ++t) {
                while (!dq.empty() && dq.front().first < t - K)
                    dq.pop_front();
                ll candidate = old[r + t * w] - (ll)t * v;
                while (!dq.empty() && dq.back().second <= candidate)
                    dq.pop_back();
```

```
                dq.emplace_back(t, candidate);
                dp[r + t * w] = (ll)t * v + dq.front().second;
            }
        }
    }
    return dp;
}
} // namespace knapsack_ref
```

```
auto dp = knapsack_ref::bounded_knapsack(
    8, {{3, 5, 2}, {2, 4, 1}});
// dp[8] = 14: two weight-3 copies and one weight-2 copy.
```

Optional pruning before the DP. For a fixed weight w, no solution uses more than $\lfloor C/w \rfloor$ copies in total, even if many types have that weight. Sort those types by decreasing value and retain only the best that many copies, reducing the final type's count if necessary. Do this using counts; do not expand huge multiplicities. Replacing a retained low-value copy by a higher-value copy of the same weight can only improve a solution, so the pruning is safe. Across all weights there are at most

$$\sum_{w=1}^{C} \lfloor C/w \rfloor = O(C \log(C+1))$$

retained copies, hence at most that many nonempty types. Combining pruning with the deque DP gives $O(N \log N + C \min(N, C \log(C+1)))$ time, including a simple sort. The displayed function is the $O(NC)$ core; apply this pruning separately when it reduces the number of types.

4.12.4 4. Max-Plus Convolution with Concavity

Problem. Given arrays A of length n and B of length m, compute

$$H[k] = \max_{\substack{0 \le i < n \\ 0 \le k-i < m}} (A[i] + B[k-i]), \quad 0 \le k < n+m-1.$$

This combines two independent DP choices whose sizes add to k. The generic double loop costs $O(nm)$. Ordinary FFT convolution uses addition and multiplication, so it does not directly compute this maximum-and-addition operation.

Use the correct curvature. A sequence B is discretely *concave* if $B[t+1] - B[t]$ is nonincreasing. For example, $[0, 9, 15, 16]$ has differences $[9, 6, 1]$ and is concave. A convex sequence has nondecreasing differences. The fast maximization methods here require concavity; negating both operands gives the corresponding *min-plus* method for convex operands. A finite sequence of length one is both convex and concave.

Case A: both operands are concave. Start with $H[0] = A[0] + B[0]$. Merge the two already sorted lists of differences in decreasing order and append each difference to the running value. For any k, the largest k differences of their union maximize the sum. Since each list is decreasing, the selected differences can be chosen as a prefix of each list; such prefixes correspond to legal indices i and $k-i$. This proves the merge rule in $O(n+m)$ time. For $A = [0, 8, 11]$, $B = [0, 6, 7]$, merge $[8, 3]$ and $[6, 1]$ to obtain $[8, 6, 3, 1]$, hence $H = [0, 8, 14, 17, 18]$.

Case B: only B is concave. Build an implicit matrix with row k and column i:

$$M[k, i] = A[i] + B[k-i], \qquad i \le k < i+m.$$

A row maximum is exactly $H[k]$. We evaluate entries on demand; materializing the whole matrix would lose the speedup. For columns $i < j$ where both entries exist, define the advantage of j over i as

$$g(k) = A[j] - A[i] + B[k-j] - B[k-i].$$

Let $\Delta B[t] = B[t] - B[t-1]$. Then

$$g(k+1) - g(k) = \Delta B[k+1-j] - \Delta B[k+1-i] \ge 0,$$

since the earlier difference of a concave sequence is no smaller. Thus once a right column is at least as good as a left column, it stays so in later rows. This property holds for every submatrix: the matrix is *totally monotone for row maxima*. With ties resolved to the right, maximizing columns are nondecreasing. No concavity assumption is made on A.

Boundary comparisons matter. Legal columns of row k form $[\max(0, k-m+1), \min(n-1, k)]$. Assigning all illegal cells the same numeric minus infinity can destroy total monotonicity in submatrices containing only illegal entries. Instead compare ordered columns $i < j$ as follows:

1. If $j > k$, prefer i: j is too far right.
2. Otherwise, if $k - i \ge m$, prefer j: i is too far left.
3. Otherwise both entries are valid; compare their values, preferring j on equality.

For two fixed columns this preference still switches only from left to right as the row increases. Every full row has a legal column, so its reported optimum is legal. The implementation accepts finite values; its knapsack caller uses budget states initialized to zero, so unreachable-state sentinels are unnecessary.

How SMAWK finds all maxima in linear time.

1. *Reduce columns.* Process columns from left to right, keeping a stack of candidates. Compare a new column against the stack top at the row corresponding to the stack's current height. If the new column wins, pop the old one. Otherwise retain it when there is space. Total monotonicity makes a popped column unnecessary: later rows prefer the new column, and earlier rows have their candidates protected by earlier stack positions. Keep at most one candidate per row. Each column is pushed/popped at most once.

2. *Recurse on odd-positioned rows.* Solve rows at positions $1, 3, 5, \ldots$ using the reduced columns. Their maximizing columns are in nondecreasing order.

3. *Interpolate even-positioned rows.* A row between two solved rows searches only between their maximizing columns. At an end, use the first/last candidate as the missing bound. These search intervals overlap only at endpoints, so the total scan is linear.

The first reduction costs $O(n + m)$. Recursion halves the row count and has at most the previous row count in columns; remaining work forms a geometric series. Time and memory are $O(n + m)$. The following header includes both the SMAWK version and the shorter two-concave-array merge version.

```cpp
#pragma once
#include <bits/stdc++.h>

namespace knapsack_ref {
// Return row maxima of a totally monotone implicit matrix.
// better(row, old_col, new_col) chooses new_col on ties.
// All column comparisons below have old_col < new_col.
template<class Better>
void smawk(const std::vector<int>& rows,
           const std::vector<int>& cols,
           std::vector<int>& arg, const Better& better) {
    if (rows.empty()) return;
    std::vector<int> reduced;
    for (int col : cols) {
        while (!reduced.empty() &&
               better(rows[reduced.size() - 1], reduced.back(), col))
            reduced.pop_back();
        if (reduced.size() < rows.size()) reduced.push_back(col);
    }
    std::vector<int> odd;
    for (int i = 1; i < (int)rows.size(); i += 2)
        odd.push_back(rows[i]);
    smawk(odd, reduced, arg, better);
    int left = 0;
    for (int i = 0; i < (int)rows.size(); i += 2) {
        int right = (int)reduced.size() - 1;
        if (i + 1 < (int)rows.size()) {
            right = left;
            while (reduced[right] != arg[rows[i + 1]]) ++right;
        }
        int best = reduced[left];
        for (int j = left + 1; j <= right; ++j)
            if (better(rows[i], best, reduced[j])) best = reduced[j];
        arg[rows[i]] = best;
        left = right;
    }
}

// Finite entries only. B's successive differences must not increase.
// Empty input -> empty output. Arithmetic must fit long long.
// O(A.size()+B.size()) time and space.
std::vector<long long> max_plus_concave(
    const std::vector<long long>& A,
    const std::vector<long long>& B) {
    if (A.empty() || B.empty()) return {};
    int n = (int)A.size(), m = (int)B.size();
    std::vector<int> rows(n + m - 1), cols(n), arg(n + m - 1);
    std::iota(rows.begin(), rows.end(), 0);
    std::iota(cols.begin(), cols.end(), 0);
    auto better = [&](int k, int i, int j) {
        // i < j. Invalid cells have a directional ordering.
        if (k < j) return false;      // j is too far right
        if (k - i >= m) return true;  // i is too far left
        return A[i] + B[k - i] <= A[j] + B[k - j];
    };
    smawk(rows, cols, arg, better);
    std::vector<long long> out(n + m - 1);
    for (int k = 0; k < n + m - 1; ++k)
        out[k] = A[arg[k]] + B[k - arg[k]];
    return out;
}

// Both arrays concave: merge their nonincreasing differences.
std::vector<long long> max_plus_both_concave(
    const std::vector<long long>& A,
    const std::vector<long long>& B) {
    if (A.empty() || B.empty()) return {};
    std::vector<long long> out{A[0] + B[0]};
    int i = 1, j = 1;
    while (i < (int)A.size() || j < (int)B.size()) {
        if (j == (int)B.size() ||
            (i < (int)A.size() && A[i] - A[i - 1] >= B[j] - B[j - 1])) {
            out.push_back(out.back() + A[i] - A[i - 1]);
            ++i;
```

```
        } else {
            out.push_back(out.back() + B[j] - B[j - 1]);
            ++j;
        }
    }
    return out;
}
} // namespace knapsack_ref
```

```
using knapsack_ref::max_plus_concave;
auto H = max_plus_concave({0, 4, 10}, {0, 7, 10});
// B differences: 7, 3. A need not be concave.
// H = {0, 7, 11, 17, 20}.
```

For comparison, the same monotone maximizing indices also allow a simpler divide-and-conquer search: solve the middle row, restrict left rows to columns no larger than its optimum, and right rows to columns no smaller. That is generally logarithmically slower; SMAWK is what gives the linear bound used in the next variant.

4.12.5 5. 0/1 Knapsack with Few Distinct Weights

Problem. There are N individual items, each available once, capacity C, but only d distinct usable weights. Processing every item separately costs $O(NC)$; group by weight to obtain $O(N \log N + dC)$ including sorting.

Summarize one weight group. Suppose its weight is w and its values in descending order are $v_1 \geq v_2 \geq \cdots \geq v_K$. If exactly t copies from this group are taken, the best choice is the t largest values. Define $P[0] = 0$ and $P[t] = \sum_{j=1}^{t} v_j$. Since $P[t] - P[t-1] = v_t$ is nonincreasing, P is concave. Keep only $\min(K, \lfloor C/w \rfloor)$ values: additional copies cannot fit. Values may be negative because choosing zero copies remains an option.

Derive the convolution. Let $F[c]$ be the previous best value with budget c and $G[c]$ the new answer. For residue $r \in [0, w)$, form $A[s] = F[r + sw]$. Then

$$G[r + tw] = \max_{0 \leq j \leq \min(t,K)} \{A[t-j] + P[j]\} = (A \star_{\max,+} P)[t].$$

Thus call the one-concave-operand convolution from variant 4 and retain only entries corresponding to budgets at most C. Build every residue from an *old* copy of F so the group cannot be reused. $F[c] = 0$ initially for every budget, making all A entries finite and implementing "weight at most" consistently.

Account for all work. The lengths of all A arrays for one weight sum to $C + 1$. Repeating P in w residue calls costs at most $w(K+1) \leq C + w = O(C)$ because $w \leq C$. The full convolution outputs are also $O(C)$ in total. Therefore one weight group costs $O(C)$ even though it makes several calls. The arbitrary old array A usually is not concave; use SMAWK, not the merge of two sorted difference lists. Memory is $O(N + C)$ for the groups and DP, with $O(C)$ temporary convolution storage.

Example. Capacity eight; weight-two values $[9, 6, 1]$ give $P = [0, 9, 15, 16]$. After this group, the budget DP is $[0, 0, 9, 9, 15, 15, 16, 16, 16]$. Now process a weight-three item worth eight. At budget eight, take the two best weight-two items plus the weight-three item: weight seven, value $9 + 6 + 8 = 23$. Budget states correctly allow the one unused unit.

```
#pragma once
#include "ConcaveMaxPlus.hpp"

namespace knapsack_ref {
// 0/1 items (weight,value), capacity C. Positive weights.
// O(N log N + D*C) time, D = #distinct usable weights.
// dp[c] uses weight AT MOST c; negative values may be skipped.
std::vector<long long> distinct_weight_knapsack(
    int C, const std::vector<std::pair<int, long long>>& items) {
    using ll = long long;
    std::map<int, std::vector<ll>> groups;
    for (auto [w, v] : items)
        if (w <= C) groups[w].push_back(v);
    std::vector<ll> dp(C + 1, 0);
    for (auto& [w, values] : groups) {
        std::sort(values.rbegin(), values.rend());
        int K = std::min((int)values.size(), C / w);
        std::vector<ll> prefix(K + 1, 0);
        for (int t = 1; t <= K; ++t)
            prefix[t] = prefix[t - 1] + values[t - 1];
        auto old = dp;
        for (int r = 0; r < w && r <= C; ++r) {
            std::vector<ll> A;
            for (int c = r; c <= C; c += w) A.push_back(old[c]);
            auto merged = max_plus_concave(A, prefix);
            for (int t = 0; t < (int)A.size(); ++t)
                dp[r + t * w] = merged[t];
        }
    }
    return dp;
}
} // namespace knapsack_ref
```

```cpp
auto dp = knapsack_ref::distinct_weight_knapsack(
    8, {{2, 9}, {2, 6}, {2, 1}, {3, 8}});
// dp[8] == 23. Each listed item can be taken only once.
```

4.12.6 6. Unbounded Knapsack with Small Weights, Huge Capacity

Problem. Item types have $1 \le w_i \le D$, unlimited copies, and values v_i. Capacity C can be far too large for an array of length $C+1$. Return the optimum with total weight at most C in $O(N + D^2 \log(C+2))$ time. No concavity is assumed here. For duplicate weights, keep only the greatest value.

Turn a budget into an exact sum safely. Add a dummy item of weight one and value zero, available without limit. Any unused capacity can now be filled with dummy items, preserving value. Conversely, removing dummy items leaves a feasible budget solution. Let $F[s]$ be this padded exact-sum optimum. Every $s \ge 0$ is reachable, $F[0] = 0$, and $F[C]$ is the desired answer. The dummy is appropriate here because unused capacity is allowed; it must not be added to the NDPC exact-fill problem.

Balanced partition lemma. Take any multiset of items, dummy items included, and assign its items one at a time to the lighter of two piles. If the difference in pile weights was at most D, adding a weight at most D to the lighter pile leaves a difference at most D. Starting from two empty piles proves that a total s can be partitioned into $a + b = s$ with $|a - b| \le D$. Apply this to an optimal multiset. Each part's value is at most $F[a]$ or $F[b]$, and combining two optimal parts is feasible. Therefore

$$F[s] = \max_{\substack{a+b=s,\ a,b\ge 0 \\ |a-b|\le D}} \{F[a] + F[b]\}.$$

Unlimited copies are essential: combining independently optimal parts may repeat an item type arbitrarily often.

Compute only a window. Define a routine for center T returning $F[s]$ for $\max(0, T - D) \le s \le T + D$. If $T \le 2D$, all these indices are at most $3D$; obtain them from a normal unbounded DP precomputed through $3D$ in $O(D^2)$. Otherwise recursively compute the window centered at $h = \lfloor T/2 \rfloor$. For any $s \in [T - D, T + D]$, the balanced split obeys

$$\left\lceil \frac{s-D}{2} \right\rceil \le a, b \le \left\lfloor \frac{s+D}{2} \right\rfloor,$$

so both indices lie in $[h - D, h + D]$, including when T is odd. Convolve the child's window with itself using the ordinary double loop. If its first absolute index is L, entries i, j produce absolute total $s = 2L + i + j$. Keep only totals in the parent's window. Taking *all* child pairs is safe: they are feasible solutions, and the balanced partition lemma ensures an optimal pair is present.

Why logarithmic in capacity. The centers are $C, \lfloor C/2 \rfloor, \lfloor C/4 \rfloor, \ldots$ until at most $2D$. Each window has at most $2D + 1$ entries, each double loop costs $O(D^2)$, and there is only one recursive child at each level. This gives the claimed time. The code releases the child at each return, using $O(D + \log(C+2))$ extra memory rather than storing all windows. It supports $D \ge 1$, $C \le 10^{18}$ and integer index sizes large enough for $3D + 1$.

Example. For types $(w, v) = (2, 5), (3, 8)$, capacity ten has optimum 26 from $3 + 3 + 2 + 2$. For capacity 41 and $D = 3$, the parent needs totals $[38, 44]$ and its child, centered at 20, stores $[17, 23]$. Every total in the parent has a balanced split entirely inside the child. This avoids computing all answers up to 41 and works the same way when 41 is replaced by 10^{18}.

```cpp
#pragma once
#include <bits/stdc++.h>

namespace knapsack_ref {
// Unbounded items (weight,value); positive integer weights <= D.
// Capacity 0 <= C <= 10^18. Intermediate values fit long long.
// O(N + D^2 log(C+2)) time, O(D + log(C+2)) extra space.
long long unbounded_knapsack_window(
    long long C, int D,
    const std::vector<std::pair<int, long long>>& items) {
    using ll = long long;
    const ll NEG = std::numeric_limits<ll>::lowest();
    std::vector<ll> value(D + 1, NEG);
    value[1] = 0; // Dummy unit item fills unused capacity.
    for (auto [w, v] : items) value[w] = std::max(value[w], v);
    std::vector<ll> base(3 * D + 1, 0);
    for (int c = 1; c <= 3 * D; ++c)
        for (int w = 1; w <= D && w <= c; ++w)
            if (value[w] != NEG)
                base[c] = std::max(base[c], base[c - w] + value[w]);
    // A block stores exact padded answers at lo,lo+1,... .
    struct Block { ll lo; std::vector<ll> a; };
    std::function<Block(ll)> solve = [&](ll T) -> Block {
        ll lo = std::max(0LL, T - D), hi = T + D;
        if (T <= 2LL * D)
            return {lo, std::vector<ll>(base.begin() + lo,
                                        base.begin() + hi + 1)};
        Block child = solve(T / 2);
        std::vector<ll> out(hi - lo + 1, NEG);
        for (int i = 0; i < (int)child.a.size(); ++i)
            for (int j = 0; j < (int)child.a.size(); ++j) {
                ll s = 2 * child.lo + i + j;
                if (lo <= s && s <= hi)
```

```
                    out[s - lo] = std::max(out[s - lo],
                                           child.a[i] + child.a[j]);
            }
        return {lo, std::move(out)};
    };
    Block answer = solve(C);
    return answer.a[C - answer.lo];
}
} // namespace knapsack_ref
```

```
long long ans = knapsack_ref::unbounded_knapsack_window(
    10, 3, {{2, 5}, {3, 8}}); // 26
```

The grouped-weight and short-window techniques are also developed in Axiotis and Tzamos, Capacitated Dynamic Programming: Faster Knapsack and Graph Algorithms. The implementations here use explicit integer window endpoints and budget-state conventions throughout.

4.13 NDPC N: Exact-Fill Knapsack with Many Huge Queries

Problem and bounds. Next DP Contest, N – Knapsack: type i has weight i and value v_i, for $1 \le i \le n$; every type has unlimited copies. Each query asks for maximum value at *exactly* weight W. Official bounds are $n \le 4000$, $q \le 2 \cdot 10^5$, $0 \le v_i \le 10^9$, $1 \le W \le 10^9$. A weight-one type exists, so every nonnegative exact weight is reachable. The listing below implements the supplied algorithm, with explicit-width density comparisons and a guarded unreachable state. Under these bounds, 64-bit values suffice because an answer is at most $W \max_i v_i \le 10^{18}$. Outside these bounds, recheck both arithmetic and the $O(n^2)$ memory requirement.

Step 1: Sort Types by Value per Unit Weight

An item a precedes b when $v_a/a > v_b/b$. Compare $v_a b$ and $v_b a$ using integer arithmetic; do not divide or compare floating-point ratios. Break equal-density ties by smaller original index. Let `order` contain the original indices in this order, and let $p =$ `order[0]`. The weight of the best-density type is p, not its position zero in the sorted list. Ties cause no difficulty: replacements need only preserve value, not strictly increase it.

Step 2: Reduce Every Large Query to at Most n Squared

Let $L = n^2$. For every exact weight $W \ge L$, there exists an optimum containing type p. To prove this, an optimum uses at least n items because every item weighs at most n. Among any p of those items, the $p+1$ prefix sums have two equal residues modulo p. The nonempty block between them weighs a positive multiple of p. Replace that block by copies of type p. Its weight stays equal, and its value does not decrease because p has maximum density. Thus an optimum containing p exists, even if a different optimum originally omitted it.

Writing $F[W]$ for the exact optimum, this proves $F[W] = F[W-p] + v_p$ whenever $W \ge L$. The forward inequality follows by removing p from that optimum; the reverse follows by adding p to an optimum for $W - p$. Repeatedly apply the equality until the remaining weight is at most L. For $W > L$, the number of removed copies is

$$t = \left\lceil \frac{W-L}{p} \right\rceil, \qquad R = W - tp.$$

Then $L - p < R \le L$, and the answer is $F[R] + tv_p$. For $W \le L$, set $t = 0, R = W$. This explains both the ceiling and why its argument is $W - L$, rather than W itself. The large part can use the best-density item, but greedily filling *all* of a small query with it is not valid.

Step 3: Avoid n Transitions at Every Small Weight

The ordinary exact-fill DP is

$$F[0] = 0, \qquad F[w] = \max_{1 \le i \le \min(n,w)} \{F[w-i] + v_i\}.$$

Computing it through L costs $O(n^3)$. A density-rank lemma lets us restrict the candidate final item. For exact weight w, some optimum uses a type among the first

$$K(w) = \min\left(n, \left\lceil \frac{2n^2}{w} \right\rceil\right)$$

entries of `order`. Remove that item from this optimum; its remainder is an optimum of smaller exact weight, by the usual optimal-substructure argument. Consequently it suffices to try these $K(w)$ types in the recurrence, ignoring any whose weight exceeds w. Increasing w makes every needed predecessor already available. This is a statement about *at least one item* in an optimum; it does not say all its items have high density.

Why the rank lemma holds. The nontrivial number-theoretic ingredient is the following classical Frobenius bound: for $h \ge 2$ distinct positive integers with gcd one, maximum M, and second largest M_2, the largest unrepresentable integer is at most $2\lfloor M/h \rfloor M_2 - M$ (Erdős–Graham). In particular it is at most $2M^2/h - M$. Here is how that bound yields the exact statement used by the code.

Fix a rank $k < n$ and assume $w \ge 2n^2/k$. Take any optimum omitting the first k types; if none exists the claim already holds. Let q be its highest-density used type, and let $g = \gcd(w_1, \ldots, w_k, w_q)$, where these w_j denote the weights of the sorted types. Divide those $k+1$ distinct weights by g, obtaining a gcd-one set A with maximum $M \le n/g$. For any chosen top-k weight $a \in A$, the Frobenius bound gives

$$F_A + a \le \frac{2M^2}{k+1} < \frac{2M^2}{k}.$$

Thus any integer $x \geq 2M^2/k$ can be represented using A while including a top-k weight: first reserve a, then represent $x - a$. Here F_A means the Frobenius number, not the knapsack DP.

We need a sufficiently large weight divisible by g to replace. As long as at least g items remain in the optimum, equal residues of $g+1$ prefix sums identify a nonempty block divisible by g. Remove such blocks repeatedly and let their combined weight be B. At most $g-1$ items remain, so

$$B \geq w - (g-1)n \geq \frac{2n^2}{gk}.$$

For the last inequality, subtract the right side from $2n^2/k - (g-1)n$ to get $(g-1)n(2n/(gk) - 1) \geq 0$. Indeed, $k+1$ distinct positive multiples of g are at most n, so $g(k+1) \leq n$. Therefore $B/g \geq 2n^2/(g^2k) \geq 2M^2/k$, and B/g is an integer. Represent it by A while including a top-k weight, then multiply back by g. This replaces exactly weight B using only top-k types and type q. Every replacement type has density at least that of q, which was highest among the removed items, so value cannot decrease. We have constructed an optimum using a top-k type, proving the claim. If $k = n$, it is immediate. Taking $k = K(w)$ gives the restricted recurrence above.

This proof uses the classical Frobenius theorem as an external lemma; the algorithmic reduction and its inequalities are shown explicitly. See Chan and He, More on Change-Making and Related Problems, Sections 4–5. The distinction between exact weight and an at-most capacity is necessary: the same factor two must not be applied blindly to other knapsack definitions.

Step 4: Complexity, Code and a Query Walkthrough

The transition count is

$$\sum_{w=1}^{n^2} \min\left(n, \left\lceil \frac{2n^2}{w} \right\rceil\right) = O(n^2 \log(n+1)).$$

For small w use the cap n; for the rest use the harmonic-series bound $\sum_{w=a}^{b} 1/w = O(\log(b/a) + 1)$. Sorting costs $O(n \log n)$, queries $O(1)$ each, and memory is $O(n^2)$. At $n = 4000$, the 64-bit DP alone occupies about 128 MB (decimal), so this is not a small-memory method.

Example. Let $n = 3$ and $[v_1, v_2, v_3] = [2, 5, 7]$. Densities are $2, 2.5, 7/3$, hence `order` is $[2, 3, 1]$, $p = 2$ and $L = 9$. For query $W = 20$, $t = \lceil (20-9)/2 \rceil = 6$ and $R = 8$. The precomputed optimum is $F[8] = 20$, giving $20 + 6 \cdot 5 = 50$. For the small query $W = 3$, the best answer is seven; taking one best-density item and leaving one unit empty would violate exact fill. The DP supplies the correct remainder.

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
// NDPC N: 1 <= n <= 4000, 0 <= v[i] <= 10^9,
// 1 <= query weight <= 10^9. Fill the weight EXACTLY.
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<ll> v(n + 1);
    for (int i = 1; i <= n; ++i) cin >> v[i];
    vector<int> order(n);
    iota(order.begin(), order.end(), 1);
    sort(order.begin(), order.end(), [&](int a, int b) {
        __int128 lhs = (__int128)v[a] * b;
        __int128 rhs = (__int128)v[b] * a;
        if (lhs != rhs) return lhs > rhs;
        return a < b;
    });
    int p = order[0], limit = n * n;
    const ll NEG = -(1LL << 60);
    vector<ll> dp(limit + 1, NEG);
    dp[0] = 0;
    for (int w = 1; w <= limit; ++w) {
        int k = (int)min((ll)n, (2LL * limit + w - 1) / w);
        for (int j = 0; j < k; ++j) {
            int i = order[j];
            if (i <= w && dp[w - i] != NEG)
                dp[w] = max(dp[w], dp[w - i] + v[i]);
        }
    }
    while (q--) {
        ll W;
        cin >> W;
        ll take = 0;
        if (W > limit) {
            take = (W - limit + p - 1) / p;
            W -= take * p;
        }
        cout << dp[(int)W] + take * v[p] << '\n';
    }
}
```

```
// Input:              Output:
// 3 4                 7
// 2 5 7               20
// 3                   22
// 8                   50
// 9
// 20
```

5 Flow

5.1 Dinic

```
// Si en el grafo todos los vertices distintos
// de s y t cumplen que solo tienen una arista
// de entrada o una de salida la y dicha arista
// tiene capacidad 1 entonces la complejidad es
// O(E sqrt(v))

// si todas las aristas tinen capacidad 1
// el algoritmo tiene complejidad O(E sqrt(E))

// to find min cut run bfs from source and find all vertices that can be reached
// edges between vertices that can be reached and the ones that cant are the min cut
struct FlowEdge {
  int v, u;
  long long cap, flow = 0;
  FlowEdge(int v, int u, long long cap) : v(v), u(u), cap(cap) {}
};

struct Dinic {
  const long long flow_inf = 1e18;
  vector<FlowEdge> edges;
  vector<vector<int>> adj;
  vector<int> sz;
  int n, m = 0;
  int s, t;
  vector<int> level, ptr;
  queue<int> q;

  Dinic(int n, int s, int t) : n(n), s(s), t(t) {
      adj.resize(n);
      level.resize(n);
      ptr.resize(n);
      sz.resize(n);
  }

  void add_edge(int v, int u, long long cap) {
      edges.emplace_back(v, u, cap);
      edges.emplace_back(u, v, 0);
      adj[v].push_back(m);
      adj[u].push_back(m + 1);
      m += 2;
  }

  bool bfs() {
      while (!q.empty()) {
          int v = q.front();
          q.pop();
          for (int id : adj[v]) {
              if (edges[id].cap - edges[id].flow < 1)
                  continue;
              if (level[edges[id].u] != -1)
                  continue;
              level[edges[id].u] = level[v] + 1;
              q.push(edges[id].u);
          }
      }
      return level[t] != -1;
  }

  long long dfs(int v, long long pushed) {
      if (pushed == 0)
          return 0;
      if (v == t)
          return pushed;
      for (int& cid = ptr[v]; cid < (int)adj[v].size(); cid++) {
          int id = adj[v][cid];
          int u = edges[id].u;
          if (level[v] + 1 != level[u] || edges[id].cap - edges[id].flow < 1)
              continue;
          long long tr = dfs(u, min(pushed, edges[id].cap - edges[id].flow));
```

```
        if (tr == 0)
            continue;
        edges[id].flow += tr;
        edges[id ^ 1].flow -= tr;
        return tr;
    }
    return 0;
}

long long flow() {
    long long f = 0;
    while (true) {
        fill(level.begin(), level.end(), -1);
        level[s] = 0;
        q.push(s);
        if (!bfs())
            break;
        fill(ptr.begin(), ptr.end(), 0);
        while (long long pushed = dfs(s, flow_inf)) {
            f += pushed;
        }
    }
    return f;
}


vector<pair<int, int>> minCut(){
  vector<pair<int, int>> ans;
  queue<int> q;
  q.push(s);
  vector<bool> vis(n, 0);
  while((int)q.size()){
    int cur = q.front();
    q.pop();
    vis[cur] = 1;
    for(int id: adj[cur]){
      if(edges[id].cap - edges[id].flow <= 0) continue;
      if(vis[edges[id].u]) continue;
      vis[edges[id].u] = 1;
      q.push(edges[id].u);
    }
  }
  for(auto edge: edges){
    if(vis[edge.v] && !vis[edge.u] && edge.flow > 0) {
      ans.push_back({edge.v, edge.u});
    }
  }
  return ans;
}

bool getPath(int cur, vector<int> &path){
  path.push_back(cur);
  for(int &i = sz[cur]; i >= 0; i--){
    int edge_index = adj[cur][i];
    if(edge_index & 1) continue;
    if(edges[edge_index].cap - edges[edge_index].flow == 0){
      i--;
      getPath(edges[edge_index].u, path);
      return true;
    }
  }
  return false;
}

vector<vector<int>> getPaths(){
  vector<vector<int>> ans;
  vector<int> path;
  for(int i = 0; i < n; i++) sz[i] = (int)adj[i].size() - 1;
  while(getPath(s, path)){
    ans.push_back(path);
    path.clear();
  }
  return ans;
}
};
```

5.2 Hopcroft-Karp

```
// maximum matching in bipartite graph
vector<int> match, dist;
vector<vector<int>> g;
int n, m, k;
bool bfs()
{
  queue<int> q;
  // The alternating path starts with unmatched nodes
```

```
  for (int node = 1; node <= n; node++)
  {
    if (!match[node])
    {
      q.push(node);
      dist[node] = 0;
    }
    else
    {
      dist[node] = INF;
    }
  }

  dist[0] = INF;

  while (!q.empty())
  {
    int node = q.front();
    q.pop();
    if (dist[node] >= dist[0])
    {
      continue;
    }
    for (int son : g[node])
    {
      // If the match of son is matched
      if (dist[match[son]] == INF)
      {
        dist[match[son]] = dist[node] + 1;
        q.push(match[son]);
      }
    }
  }
  // Returns true if an alternating path has been found
  return dist[0] != INF;
}

// Returns true if an augmenting path has been found starting from
    vertex node
bool dfs(int node)
{
  if (node == 0)
  {
    return true;
  }
  for (int son : g[node])
  {
    if (dist[match[son]] == dist[node] + 1 && dfs(match[son]))
    {
      match[node] = son;
      match[son] = node;
      return true;
    }
  }
  dist[node] = INF;
  return false;
}

int hopcroft_karp()
{
  int cnt = 0;
  // While there is an alternating path
  while (bfs())
  {
    for (int node = 1; node <= n; node++)
    {
      // If node is unmatched but we can match it using an augmenting
          path
      if (!match[node] && dfs(node))
      {
        cnt++;
      }
    }
  }
  return cnt;
}
// usage
// n numero de puntos en la izquierda
// m numero de puntos en la derecha
// las aristas se guardan en g
// los puntos estan 1 indexados
// el punto 1 de m es el punto n+1 de g
// hopcroft_karp() devuelve el tamano del maximo matching
// match contiene el match de cada punto
// si match de i es 0, entonces i no esta matcheado
//
```

```
// https://judge.yosupo.jp/submission/247277
```

5.3 Hungarian

```
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

int ckmin(int &a, int b) { return a > b ? ((a = b), true) : false; }

template <class T> vector<int> hungarian(const vector<vector<T>> &C) {
  int J = C.size();
  int W = C[0].size();
  assert(J <= W);
  // job[w] = job assigned to w-th worker, or -1 if no job assigned
  // note: a W-th worker was added for convenience
  vector<int> job(W + 1, -1);
  vector<T> h(W);  // Johnson potentials
  const T inf = numeric_limits<T>::max();
  // assign j_cur-th job using Dijkstra with potentials
  for (int j_cur = 0; j_cur < J; j_cur++) {
    int w_cur = W;  // unvisited worker with minimum distance
    job[w_cur] = j_cur;
    vector<T> dist(W + 1, inf);  // Johnson-reduced distances
    dist[W] = 0;
    vector<bool> vis(W + 1);     // whether visited yet
    vector<int> prv(W + 1, -1);  // previous worker on shortest path
    while (job[w_cur] != -1) {   // Dijkstra step: pop min worker from
        heap
      T min_dist = inf;
      vis[w_cur] = true;
      int w_next = -1;  // next unvisited worker with minimum distance
      // consider extending shortest path by w_cur -> job[w_cur] -> w
      for (int w = 0; w < W; w++) {
        if (!vis[w]) {
          // sum of reduced edge weights w_cur -> job[w_cur] -> w
          T edge = C[job[w_cur]][w] - h[w];
          if (w_cur != W) {
            edge -= C[job[w_cur]][w_cur] - h[w_cur];
            assert(edge >= 0);
          }
          if (ckmin(dist[w], dist[w_cur] + edge)) { prv[w] = w_cur; }
          if (ckmin(min_dist, dist[w])) { w_next = w; }
        }
      }
      w_cur = w_next;
    }
    for (int w = 0; w < W; w++) {  // update potentials
      ckmin(dist[w], dist[w_cur]);
      h[w] += dist[w];
    }
    while (w_cur != W) {  // update job assignment
      job[w_cur] = job[prv[w_cur]];
      w_cur = prv[w_cur];
    }
  }
  return job;
}
int main(){
  ios_base::sync_with_stdio(false); cin.tie(NULL);
  int n; cin >> n;
  vector<vector<int>> a(n, vector<int>(n));
  for(int i = 0; i < n; i++){
    for(int j = 0; j < n; j++){
      cin >> a[i][j];
    }
  }
  vector<int> m = hungarian(a);
  ll ans = 0;
  for(int i = 0; i < n; i++){
    ans += a[m[i]][i];
  }
  cout << ans << endl;
  for(int i = 0; i < n; i++) cout << i + 1 << " " << m[i] + 1 << endl;
}
```

5.4 Max Flow Min Cost

```
// dado un acomodo de flujos con costos
// devuelve el costo minimo para un flujo especificado

struct Edge
{
  int from, to, capacity, cost;
  Edge(int _from, int _to, int _capacity, int _cost)
```

```
  {
    from = _from;
    to = _to;
    capacity = _capacity;
    cost = _cost;
  }
};

vector<vector<int>> adj, cost, capacity;

const int INF = 1e9;

void shortest_paths(int n, int v0, vector<int> &d, vector<int> &p)
{
  d.assign(n, INF);
  d[v0] = 0;
  vector<bool> inq(n, false);
  queue<int> q;
  q.push(v0);
  p.assign(n, -1);

  while (!q.empty())
  {
    int u = q.front();
    q.pop();
    inq[u] = false;
    for (int v : adj[u])
    {
      if (capacity[u][v] > 0 && d[v] > d[u] + cost[u][v])
      {
        d[v] = d[u] + cost[u][v];
        p[v] = u;
        if (!inq[v])
        {
          inq[v] = true;
          q.push(v);
        }
      }
    }
  }
}

int min_cost_flow(int N, vector<Edge> edges, int K, int s, int t)
{
  adj.assign(N, vector<int>());
  cost.assign(N, vector<int>(N, 0));
  capacity.assign(N, vector<int>(N, 0));
  for (Edge e : edges)
  {
    adj[e.from].push_back(e.to);
    adj[e.to].push_back(e.from);
    cost[e.from][e.to] = e.cost;
    cost[e.to][e.from] = -e.cost;
    capacity[e.from][e.to] = e.capacity;
  }

  int flow = 0;
  int cost = 0;
  vector<int> d, p;
  while (flow < K)
  {
    shortest_paths(N, s, d, p);
    if (d[t] == INF)
      break;

    // find max flow on that path
    int f = K - flow;
    int cur = t;
    while (cur != s)
    {
      f = min(f, capacity[p[cur]][cur]);
      cur = p[cur];
    }

    // apply flow
    flow += f;
    cost += f * d[t];
    cur = t;
    while (cur != s)
    {
      capacity[p[cur]][cur] -= f;
      capacity[cur][p[cur]] += f;
      cur = p[cur];
    }
  }

```

```
  if (flow < K)
    return -1;
  else
    return cost;
}
```

5.5 Max Flow

```
long long max_flow(vector<vector<int>> adj, vector<vector<long long>>
    capacity,
                   int source, int sink)
{
  int n = adj.size();
  vector<int> parent(n, -1);
  // Find a way from the source to sink on a path with non-negative
      capacities
  auto reachable = [&]() -> bool
  {
    queue<int> q;
    q.push(source);
    while (!q.empty())
    {
      int node = q.front();
      q.pop();
      for (int son : adj[node])
      {
        long long w = capacity[node][son];
        if (w <= 0 || parent[son] != -1)
          continue;
        parent[son] = node;
        q.push(son);
      }
    }
    return parent[sink] != -1;
  };

  long long flow = 0;
  // While there is a way from source to sink with non-negative
      capacities
  while (reachable())
  {
    int node = sink;
    // The minimum capacity on the path from source to sink
    long long curr_flow = LLONG_MAX;
    while (node != source)
    {
      curr_flow = min(curr_flow, capacity[parent[node]][node]);
      node = parent[node];
    }
    node = sink;
    while (node != source)
    {
      // Subtract the capacity from capacity edges
      capacity[parent[node]][node] -= curr_flow;
      // Add the current flow to flow backedges
      capacity[node][parent[node]] += curr_flow;
      node = parent[node];
    }
    flow += curr_flow;
    fill(parent.begin(), parent.end(), -1);
  }

  return flow;
}



//vector<vector<long long>> capacity(n, vector<long long>(n));
//vector<vector<int>> adj(n);
//adj[a].push_back(b);
//adj[b].push_back(a);
//capacity[a][b] += c;
```

5.6 Min Cost Max Flow

```
/**
 *  If costs can be negative, call setpi before maxflow, but note that
     negative cost cycles are not supported.
 *  To obtain the actual flow, look at positive values only
 * Time: $O(F E \log(V))$ where F is max flow. $O(VE)$ for setpi.
 */
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

```

```cpp
#include <ext/pb_ds/priority_queue.hpp>
using namespace __gnu_pbds;

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;


const ll INF = numeric_limits<ll>::max() / 4;

struct MCMF {
  struct edge {
    int from, to, rev;
    ll cap, cost, flow;
  };
  int N;
  vector<vector<edge>> ed;
  vi seen;
  vector<ll> dist, pi;
  vector<edge*> par;

  MCMF(int N) : N(N), ed(N), seen(N), dist(N), pi(N), par(N) {}

  void addEdge(int from, int to, ll cap, ll cost) {
    if (from == to) return;
    ed[from].push_back(edge{ from,to,sz(ed[to]),cap,cost,0 });
    ed[to].push_back(edge{ to,from,sz(ed[from])-1,0,-cost,0 });
  }

  void path(int s) {
    fill(all(seen), 0);
    fill(all(dist), INF);
    dist[s] = 0; ll di;

    __gnu_pbds::priority_queue<pair<ll, int>> q;
    vector<decltype(q)::point_iterator> its(N);
    q.push({ 0, s });

    while (!q.empty()) {
      s = q.top().second; q.pop();
      seen[s] = 1; di = dist[s] + pi[s];
      for (edge& e : ed[s]) if (!seen[e.to]) {
        ll val = di - pi[e.to] + e.cost;
        if (e.cap - e.flow > 0 && val < dist[e.to]) {
          dist[e.to] = val;
          par[e.to] = &e;
          if (its[e.to] == q.end())
            its[e.to] = q.push({ -dist[e.to], e.to });
          else
            q.modify(its[e.to], { -dist[e.to], e.to });
        }
      }
    }
    rep(i,0,N) pi[i] = min(pi[i] + dist[i], INF);
  }

  pair<ll, ll> maxflow(int s, int t) {
    ll totflow = 0, totcost = 0;
    while (path(s), seen[t]) {
      ll fl = INF;
      for (edge* x = par[t]; x; x = par[x->from])
        fl = min(fl, x->cap - x->flow);

      totflow += fl;
      for (edge* x = par[t]; x; x = par[x->from]) {
        x->flow += fl;
        ed[x->to][x->rev].flow -= fl;
      }
    }
    rep(i,0,N) for(edge& e : ed[i]) totcost += e.cost * e.flow;
    return {totflow, totcost/2};
  }

  // If some costs can be negative, call this before maxflow:
  void setpi(int s) { // (otherwise, leave this out)
    fill(all(pi), INF); pi[s] = 0;
    int it = N, ch = 1; ll v;
    while (ch-- && it--)
      rep(i,0,N) if (pi[i] != INF)
        for (edge& e : ed[i]) if (e.cap)
          if ((v = pi[i] + e.cost) < pi[e.to])
            pi[e.to] = v, ch = 1;
    assert(it >= 0); // negative cost cycle
```

```
  }


  // Might need stress testing
  vector<vector<int>> getPaths(int s, int t){
    vector<vector<int>> paths;
    while(1){
      paths.push_back({});
      if(getPath(s, t, paths.back()) == false){
        paths.pop_back();
        break;
      }
    }
    return paths;
  }

  bool getPath(int cur, int t, vector<int>& path){
    path.push_back(cur);
    if(cur == t) return true;
    for(auto &e: ed[cur]){
      if(e.flow == 1 && e.cap == 1){
        e.flow = 0;
        return getPath(e.to, t, path);
      }
    }
    return false;
  }
};
```

5.7 Push Relabel

```
const int inf = 1000000000;

int n;
vector<vector<int>> capacity, flow;
vector<int> height, excess, seen;
queue<int> excess_vertices;

void push(int u, int v) {
  int d = min(excess[u], capacity[u][v] - flow[u][v]);
  flow[u][v] += d;
  flow[v][u] -= d;
  excess[u] -= d;
  excess[v] += d;
  if (d && excess[v] == d)
    excess_vertices.push(v);
}

void relabel(int u) {
  int d = inf;
  for (int i = 0; i < n; i++) {
    if (capacity[u][i] - flow[u][i] > 0)
      d = min(d, height[i]);
  }
  if (d < inf)
    height[u] = d + 1;
}

void discharge(int u) {
  while (excess[u] > 0) {
    if (seen[u] < n) {
      int v = seen[u];
      if (capacity[u][v] - flow[u][v] > 0 && height[u] > height[v])
        push(u, v);
      else
        seen[u]++;
    } else {
      relabel(u);
      seen[u] = 0;
    }
  }
}

int max_flow(int s, int t) {
  height.assign(n, 0);
  height[s] = n;
  flow.assign(n, vector<int>(n, 0));
  excess.assign(n, 0);
  excess[s] = inf;
  for (int i = 0; i < n; i++) {
    if (i != s)
      push(s, i);
  }
  seen.assign(n, 0);

  while (!excess_vertices.empty()) {
```

```
    int u = excess_vertices.front();
    excess_vertices.pop();
    if (u != s && u != t)
      discharge(u);
  }

  int max_flow = 0;
  for (int i = 0; i < n; i++)
    max_flow += flow[i][t];
  return max_flow;
}
```

6 Geometry

6.1 Point Struct

```
typedef long long T;
struct pt {
  T x,y;
  pt operator+(pt p) {return {x+p.x, y+p.y};}
  pt operator-(pt p) {return {x-p.x, y-p.y};}
  pt operator*(T d) {return {x*d, y*d};}
  pt operator/(T d) {return {x/d, y/d};}
};

// cross product
// positivo si el segundo esta en sentido antihorario
// 0 si el angulo es 180
// negativo si el segundo esta en sentido horario
T cross(pt v, pt w) {return v.x*w.y - v.y*w.x;}

// dot product
// positivo si el angulo entre los vectores es agudo
// 0 si son perpendiculares
// negativo si el angulo es obtuso
T dot(pt v, pt w) {return v.x*w.x + v.y*w.y;}

T orient(pt a, pt b, pt c) {return cross(b-a,c-a);}

T dist(pt a,pt b){
  pt aux=b-a;
  return sqrtl(aux.x*aux.x+aux.y*aux.y);
}
```

6.2 Sort Points

```
// This comparator sorts the points clockwise
// starting from the first quarter

bool getQ(Point a){
  if(a.y!=0){
    if(a.y>0)return 0;
    return 1;
  }
  if(a.x>0)return 0;
  return 1;
}
bool comp(Point a, Point b){
  if(getQ(a)!=getQ(b))return getQ(a)<getQ(b);
  return a*b>0;
}
```

6.3 Point Struct2

```
#include <bits/stdc++.h>
using namespace std;
using ld = long double;
const ld eps = 1e-9, inf = numeric_limits<ld>::max(), pi = acos(-1);
// For use with integers, just set eps=0 and everything remains the same
bool geq(ld a, ld b){return a-b >= -eps;}     //a >= b
bool leq(ld a, ld b){return b-a >= -eps;}     //a <= b
bool ge(ld a, ld b){return a-b > eps;}        //a > b
bool le(ld a, ld b){return b-a > eps;}        //a < b
bool eq(ld a, ld b){return abs(a-b) <= eps;}  //a == b
bool neq(ld a, ld b){return abs(a-b) > eps;}  //a != b

struct point{
  ld x, y;
  point(): x(0), y(0){}
  point(ld x, ld y): x(x), y(y){}

  point operator+(const point & p) const{return point(x + p.x, y + p.y)
      ;}
  point operator-(const point & p) const{return point(x - p.x, y - p.y)
      ;}
  point operator*(const ld & k) const{return point(x * k, y * k);}
  point operator/(const ld & k) const{return point(x / k, y / k);}
```

```
22
23    point operator+=(const point & p){*this = *this + p; return *this;}
24    point operator-=(const point & p){*this = *this - p; return *this;}
25    point operator*=(const ld & p){*this = *this * p; return *this;}
26    point operator/=(const ld & p){*this = *this / p; return *this;}
27
28    point rotate(const ld & a) const{return point(x*cos(a) - y*sin(a), x*
          sin(a) + y*cos(a));}
29    point perp() const{return point(-y, x);}
30    ld ang() const{
31      ld a = atan2l(y, x); a += le(a, 0) ? 2*pi : 0; return a;
32    }
33    ld dot(const point & p) const{return x * p.x + y * p.y;}
34    ld cross(const point & p) const{return x * p.y - y * p.x;}
35    ld norm() const{return x * x + y * y;}
36    ld length() const{return sqrtl(x * x + y * y);}
37    point unit() const{return (*this) / length();}
38
39    bool operator==(const point & p) const{return eq(x, p.x) && eq(y, p.y)
          ;}
40    bool operator!=(const point & p) const{return !(*this == p);}
41    bool operator<(const point & p) const{return le(x, p.x) || (eq(x, p.x)
           && le(y, p.y));}
42    bool operator>(const point & p) const{return ge(x, p.x) || (eq(x, p.x)
           && ge(y, p.y));}
43    bool half(const point & p) const{return le(p.cross(*this), 0) || (eq(p
          .cross(*this), 0) && le(p.dot(*this), 0));}
44  };
45
46  istream &operator>>(istream &is, point & p){return is >> p.x >> p.y;}
47  ostream &operator<<(ostream &os, const point & p){return os << "(" << p.
        x << ",␣" << p.y << ")";}
48
49  int sgn(ld x){
50    if(ge(x, 0)) return 1;
51    if(le(x, 0)) return -1;
52    return 0;
53  }
54
55  void polarSort(vector<point> & P, const point & o, const point & v){
56    //sort points in P around o, taking the direction of v as first angle
57    sort(P.begin(), P.end(), [&](const point & a, const point & b){
58      return point((a - o).half(v), 0) < point((b - o).half(v), (a - o).
          cross(b - o));
59    });
60  }
```

6.4 Find Triangle With Given Area

```
1   #include <algorithm>
2   #include <cassert>
3   #include <iostream>
4   #include <vector>
5   using namespace std;
6
7   struct point {
8       int x, y;
9       int index;
10
11      point(int _x = 0, int _y = 0) : x(_x), y(_y) {
12          index = -1;
13      }
14
15      point& operator+=(const point &other) {
16          x += other.x;
17          y += other.y;
18          return *this;
19      }
20
21      point& operator-=(const point &other) {
22          x -= other.x;
23          y -= other.y;
24          return *this;
25      }
26
27      point operator+(const point &other) const {
28          return point(*this) += other;
29      }
30
31      point operator-(const point &other) const {
32          return point(*this) -= other;
33      }
34
35      bool operator<(const point &other) const {
36          return make_pair(y, x) > make_pair(other.y, other.x);
37      }
```

```
};

long long cross(const point &a, const point &b) {
    return (long long) a.x * b.y - (long long) b.x * a.y;
}

long long triangle_area(const point &a, const point &b, const point &c)
    {
    return abs(cross(a, b) + cross(b, c) + cross(c, a));
}

struct segment {
    point p;
    int index1, index2;

    segment(const point &_p, int _index1, int _index2) : p(_p), index1(
        _index1), index2(_index2) {}

    bool operator<(const segment &other) const {
        return cross(p, other.p) > 0;
    }
};

int N;
long long S;
vector<point> points;
vector<segment> segments;
vector<int> location;

void yes(point a, point b, point c) {
    cout << "Yes" << '\n';
    cout << a.x << ' ' << a.y << '\n';
    cout << b.x << ' ' << b.y << '\n';
    cout << c.x << ' ' << c.y << '\n';
    assert(triangle_area(a, b, c) == 2 * S);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cin >> N >> S;
    points.resize(N);


    for (int i = 0; i < N; i++)
        cin >> points[i].x >> points[i].y;

    sort(points.begin(), points.end());
    location.resize(N);

    for (int i = 0; i < N; i++) {
        points[i].index = i;
        location[i] = i;
    }

    for (int i = 0; i < N; i++)
        for (int j = 0; j < i; j++)
            segments.emplace_back(points[j] - points[i], i, j);

    sort(segments.begin(), segments.end());

    for (segment &s : segments) {
        int &a = location[s.index1];
        int &b = location[s.index2];
        assert(b == a - 1);

        int low = 0, high = b - 1;

        while (low <= high) {
            int mid = (low + high) / 2;
            long long area = triangle_area(points[mid], points[a],
                points[b]);

            if (area == 2 * S) {
                yes(points[mid], points[a], points[b]);
                return 0;
            }

            if (area > 2 * S)
                low = mid + 1;
            else
                high = mid - 1;
        }

        low = a + 1;
        high = N - 1;
```

```
        while (low <= high) {
            int mid = (low + high) / 2;
            long long area = triangle_area(points[mid], points[a],
                points[b]);

            if (area == 2 * S) {
                yes(points[mid], points[a], points[b]);
                return 0;
            }

            if (area > 2 * S)
                high = mid - 1;
            else
                low = mid + 1;
        }

        swap(points[a], points[b]);
        swap(a, b);
    }

    cout << "No" << '\n';
    return 0;
}
```

6.5 Antipodal Pairs

```
vector<pair<int, int>> antipodalPairs(vector<point> & P){
  vector<pair<int, int>> ans;
  int n = P.size(), k = 1;
  auto f = [&](int u, int v, int w){return abs((P[v%n]-P[u%n]).cross(P[w
      %n]-P[u%n]));};
  while(ge(f(n-1, 0, k+1), f(n-1, 0, k))) ++k;
  for(int i = 0, j = k; i <= k && j < n; ++i){
    ans.emplace_back(i, j);
    while(j < n-1 && ge(f(i, i+1, j+1), f(i, i+1, j)))
      ans.emplace_back(i, ++j);
  }
  return ans;
}
```

6.6 Area and Perimeter

```
ld perimeter(vector<point> & P){
  int n = P.size();
  ld ans = 0;
  for(int i = 0; i < n; i++){
    ans += (P[i] - P[(i + 1) % n]).length();
  }
  return ans;
}

ld area(vector<point> & P){
  int n = P.size();
  ld ans = 0;
  for(int i = 0; i < n; i++){
    ans += P[i].cross(P[(i + 1) % n]);
  }
  return abs(ans / 2);
}
```

6.7 Area Union Circles

```
struct circ{
  point c;
  ld r;
  circ() {}
  circ(const point & c, ld r): c(c), r(r) {}
  set<pair<ld, ld>> ranges;

  void disable(ld l, ld r){
    ranges.emplace(l, r);
  }

  auto getActive() const{
    vector<pair<ld, ld>> ans;
    ld maxi = 0;
    for(const auto & dis : ranges){
      ld l, r;
      tie(l, r) = dis;
      if(l > maxi){
        ans.emplace_back(maxi, l);
      }
      maxi = max(maxi, r);
    }
    if(!eq(maxi, 2*pi)){
      ans.emplace_back(maxi, 2*pi);
```

```
    }
    return ans;
  }
};

ld areaUnionCircles(const vector<circ> & circs){
  vector<circ> valid;
  for(const circ & curr : circs){
    if(eq(curr.r, 0)) continue;
    circ nuevo = curr;
    for(circ & prev : valid){
      if(circleInsideCircle(prev.c, prev.r, nuevo.c, nuevo.r)){
        nuevo.disable(0, 2*pi);
      }else if(circleInsideCircle(nuevo.c, nuevo.r, prev.c, prev.r)){
        prev.disable(0, 2*pi);
      }else{
        auto cruce = intersectionCircles(prev.c, prev.r, nuevo.c, nuevo.
            r);
        if(cruce.size() == 2){
          ld a1 = (cruce[0] - prev.c).ang();
          ld a2 = (cruce[1] - prev.c).ang();
          ld b1 = (cruce[1] - nuevo.c).ang();
          ld b2 = (cruce[0] - nuevo.c).ang();
          if(a1 < a2){
            prev.disable(a1, a2);
          }else{
            prev.disable(a1, 2*pi);
            prev.disable(0, a2);
          }
          if(b1 < b2){
            nuevo.disable(b1, b2);
          }else{
            nuevo.disable(b1, 2*pi);
            nuevo.disable(0, b2);
          }
        }
      }
    }
    valid.push_back(nuevo);
  }
  ld ans = 0;
  for(const circ & curr : valid){
    for(const auto & range : curr.getActive()){
      ld l, r;
      tie(l, r) = range;
      ans += curr.r*(curr.c.x * (sin(r) - sin(l)) - curr.c.y * (cos(r) -
          cos(l))) + curr.r*curr.r*(r-l);
    }
  }
  return ans/2;
};
```

6.8 Centroid

```
point centroid(vector<point> & P){
  point num;
  ld den = 0;
  int n = P.size();
  for(int i = 0; i < n; ++i){
    ld cross = P[i].cross(P[(i + 1) % n]);
    num += (P[i] + P[(i + 1) % n]) * cross;
    den += cross;
  }
  return num / (3 * den);
}
```

6.9 Circle Inside Circle

```
int circleInsideCircle(const point & c1, ld r1, const point & c2, ld r2)
    {
  //test if circle 2 is inside circle 1
  //returns "-1" if 2 touches internally 1, "1" if 2 is inside 1, "0" if
      they overlap
  ld l = r1 - r2 - (c1 - c2).length();
  return (ge(l, 0) ? 1 : (eq(l, 0) ? -1 : 0));
}
```

6.10 Circle Outside Circle

```
int circleOutsideCircle(const point & c1, ld r1, const point & c2, ld r2
    ){
  //test if circle 2 is outside circle 1
  //returns "-1" if they touch externally, "1" if 2 is outside 1, "0" if
      they overlap
  ld l = (c1 - c2).length() - (r1 + r2);
  return (ge(l, 0) ? 1 : (eq(l, 0) ? -1 : 0));
}
```

6.11 Closest Pair of Points

```
bool comp1(const point & a, const point & b){
  return le(a.y, b.y);
}
pair<point, point> closestPairOfPoints(vector<point> P){
  sort(P.begin(), P.end(), comp1);
  set<point> S;
  ld ans = inf;
  point p, q;
  int pos = 0;
  for(int i = 0; i < P.size(); ++i){
    while(pos < i && geq(P[i].y - P[pos].y, ans)){
      S.erase(P[pos++]);
    }
    auto lower = S.lower_bound({P[i].x - ans - eps, -inf});
    auto upper = S.upper_bound({P[i].x + ans + eps, -inf});
    for(auto it = lower; it != upper; ++it){
      ld d = (P[i] - *it).length();
      if(le(d, ans)){
        ans = d;
        p = P[i];
        q = *it;
      }
    }
    S.insert(P[i]);
  }
  return {p, q};
}
```

6.12 Common Tangents

```
vector<vector<point>> tangents(const point & c1, ld r1, const point & c2
    , ld r2, bool inner){
  //returns a vector of segments or a single point
  if(inner) r2 = -r2;
  point d = c2 - c1;
  ld dr = r1 - r2, d2 = d.norm(), h2 = d2 - dr*dr;
  if(eq(d2, 0) || le(h2, 0)) return {};
  point v = d*dr/d2;
  if(eq(h2, 0)) return {{c1 + v*r1}};
  else{
    point u = d.perp()*sqrt(h2)/d2;
    return {{c1 + (v - u)*r1, c2 + (v - u)*r2}, {c1 + (v + u)*r1, c2 + (
        v + u)*r2}};
  }
}
```

6.13 Convex Hull

```
vector<point> convexHull(vector<point> P){
  sort(P.begin(), P.end());
  vector<point> L, U;
  for(int i = 0; i < P.size(); i++){
    while(L.size() >= 2 && leq((L[L.size() - 2] - P[i]).cross(L[L.size()
        - 1] - P[i]), 0)){
      L.pop_back();
    }
    L.push_back(P[i]);
  }
  for(int i = P.size() - 1; i >= 0; i--){
    while(U.size() >= 2 && leq((U[U.size() - 2] - P[i]).cross(U[U.size()
        - 1] - P[i]), 0)){
      U.pop_back();
    }
    U.push_back(P[i]);
  }
  L.pop_back();
  U.pop_back();
  L.insert(L.end(), U.begin(), U.end());
  return L;
}
```

6.14 Segment Intersection with Ray

```
bool crossesRay(const point & a, const point & b, const point & p){
  return (geq(b.y, p.y) - geq(a.y, p.y)) * sgn((a - p).cross(b - p)) >
      0;
}
```

6.15 Cut Polygon

```
vector<point> cutPolygon(const vector<point> & P, const point & a, const
    point & v){
  //returns the part of the convex polygon P on the left side of line a+
      tv
  int n = P.size();
```

```
  vector<point> lhs;
  for(int i = 0; i < n; ++i){
    if(geq(v.cross(P[i] - a), 0)){
      lhs.push_back(P[i]);
    }
    if(intersectLineSegmentInfo(a, v, P[i], P[(i+1)%n]) == 1){
      point p = intersectLines(a, v, P[i], P[(i+1)%n] - P[i]);
      if(p != P[i] && p != P[(i+1)%n]){
        lhs.push_back(p);
      }
    }
  }
  return lhs;
}
```

6.16 Delaunay Triangulation

```
//Delaunay triangulation in O(n log n)
const point inf_pt(inf, inf);

struct QuadEdge{
  point origin;
  QuadEdge* rot = nullptr;
  QuadEdge* onext = nullptr;
  bool used = false;
  QuadEdge* rev() const{return rot->rot;}
  QuadEdge* lnext() const{return rot->rev()->onext->rot;}
  QuadEdge* oprev() const{return rot->onext->rot;}
  point dest() const{return rev()->origin;}
};

QuadEdge* make_edge(const point & from, const point & to){
  QuadEdge* e1 = new QuadEdge;
  QuadEdge* e2 = new QuadEdge;
  QuadEdge* e3 = new QuadEdge;
  QuadEdge* e4 = new QuadEdge;
  e1->origin = from;
  e2->origin = to;
  e3->origin = e4->origin = inf_pt;
  e1->rot = e3;
  e2->rot = e4;
  e3->rot = e2;
  e4->rot = e1;
  e1->onext = e1;
  e2->onext = e2;
  e3->onext = e4;
  e4->onext = e3;
  return e1;
}

void splice(QuadEdge* a, QuadEdge* b){
  swap(a->onext->rot->onext, b->onext->rot->onext);
  swap(a->onext, b->onext);
}

void delete_edge(QuadEdge* e){
  splice(e, e->oprev());
  splice(e->rev(), e->rev()->oprev());
  delete e->rot;
  delete e->rev()->rot;
  delete e;
  delete e->rev();
}

QuadEdge* connect(QuadEdge* a, QuadEdge* b){
  QuadEdge* e = make_edge(a->dest(), b->origin);
  splice(e, a->lnext());
  splice(e->rev(), b);
  return e;
}

bool left_of(const point & p, QuadEdge* e){
  return ge((e->origin - p).cross(e->dest() - p), 0);
}

bool right_of(const point & p, QuadEdge* e){
  return le((e->origin - p).cross(e->dest() - p), 0);
}

ld det3(ld a1, ld a2, ld a3, ld b1, ld b2, ld b3, ld c1, ld c2, ld c3) {
  return a1 * (b2 * c3 - c2 * b3) - a2 * (b1 * c3 - c1 * b3) + a3 * (b1
      * c2 - c1 * b2);
}

bool in_circle(const point & a, const point & b, const point & c, const
    point & d) {
```

```
  ld det = -det3(b.x, b.y, b.norm(), c.x, c.y, c.norm(), d.x, d.y, d.
      norm());
  det += det3(a.x, a.y, a.norm(), c.x, c.y, c.norm(), d.x, d.y, d.norm()
      );
  det -= det3(a.x, a.y, a.norm(), b.x, b.y, b.norm(), d.x, d.y, d.norm()
      );
  det += det3(a.x, a.y, a.norm(), b.x, b.y, b.norm(), c.x, c.y, c.norm()
      );
  return ge(det, 0);
}

pair<QuadEdge*, QuadEdge*> build_tr(int l, int r, vector<point> & P){
  if(r - l + 1 == 2){
    QuadEdge* res = make_edge(P[l], P[r]);
    return {res, res->rev()};
  }
  if(r - l + 1 == 3){
    QuadEdge *a = make_edge(P[l], P[l + 1]), *b = make_edge(P[l + 1], P[
        r]);
    splice(a->rev(), b);
    int sg = sgn((P[l + 1] - P[l]).cross(P[r] - P[l]));
    if(sg == 0)
      return {a, b->rev()};
    QuadEdge* c = connect(b, a);
    if(sg == 1)
      return {a, b->rev()};
    else
      return {c->rev(), c};
  }
  int mid = (l + r) / 2;
  QuadEdge *ldo, *ldi, *rdo, *rdi;
  tie(ldo, ldi) = build_tr(l, mid, P);
  tie(rdi, rdo) = build_tr(mid + 1, r, P);
  while(true){
    if(left_of(rdi->origin, ldi)){
      ldi = ldi->lnext();
      continue;
    }
    if(right_of(ldi->origin, rdi)){
      rdi = rdi->rev()->onext;
      continue;
    }
    break;
  }
  QuadEdge* basel = connect(rdi->rev(), ldi);
  auto valid = [&basel](QuadEdge* e){return right_of(e->dest(), basel)
      ;};
  if(ldi->origin == ldo->origin)
    ldo = basel->rev();
  if(rdi->origin == rdo->origin)
    rdo = basel;
  while(true){
    QuadEdge* lcand = basel->rev()->onext;
    if(valid(lcand)){
      while(in_circle(basel->dest(), basel->origin, lcand->dest(), lcand
          ->onext->dest())){
        QuadEdge* t = lcand->onext;
        delete_edge(lcand);
        lcand = t;
      }
    }
    QuadEdge* rcand = basel->oprev();
    if(valid(rcand)){
      while(in_circle(basel->dest(), basel->origin, rcand->dest(), rcand
          ->oprev()->dest())){
        QuadEdge* t = rcand->oprev();
        delete_edge(rcand);
        rcand = t;
      }
    }
    if(!valid(lcand) && !valid(rcand))
      break;
    if(!valid(lcand) || (valid(rcand) && in_circle(lcand->dest(), lcand
        ->origin, rcand->origin, rcand->dest())))
      basel = connect(rcand, basel->rev());
    else
      basel = connect(basel->rev(), lcand->rev());
  }
  return {ldo, rdo};
}

vector<tuple<point, point, point>> delaunay(vector<point> & P){
  sort(P.begin(), P.end());
  auto res = build_tr(0, (int)P.size() - 1, P);
  QuadEdge* e = res.first;
  vector<QuadEdge*> edges = {e};
```

```
  while(le((e->dest() - e->onext->dest()).cross(e->origin - e->onext->
      dest()), 0))
    e = e->onext;
  auto add = [&P, &e, &edges](){
    QuadEdge* curr = e;
    do{
      curr->used = true;
      P.push_back(curr->origin);
      edges.push_back(curr->rev());
      curr = curr->lnext();
    }while(curr != e);
  };
  add();
  P.clear();
  int kek = 0;
  while(kek < (int)edges.size())
    if(!(e = edges[kek++])->used)
      add();
  vector<tuple<point, point, point>> ans;
  for(int i = 0; i < (int)P.size(); i += 3){
    ans.emplace_back(P[i], P[i + 1], P[i + 2]);
  }
  return ans;
}
```

6.17 Diameter and Width

```
pair<ld, ld> diameterAndWidth(vector<point> & P){
  int n = P.size(), k = 0;
  auto dot = [&](int a, int b){return (P[(a+1)%n]-P[a]).dot(P[(b+1)%n]-P
      [b]);};
  auto cross = [&](int a, int b){return (P[(a+1)%n]-P[a]).cross(P[(b+1)%
      n]-P[b]);};
  ld diameter = 0;
  ld width = inf;
  while(ge(dot(0, k), 0)) k = (k+1) % n;
  for(int i = 0; i < n; ++i){
    while(ge(cross(i, k), 0)) k = (k+1) % n;
    //pair: (i, k)
    diameter = max(diameter, (P[k] - P[i]).length());
    width = min(width, distancePointLine(P[i], P[(i+1)%n] - P[i], P[k]))
        ;
  }
  return {diameter, width};
}
```

6.18 Distance Between Point and Circle

```
ld distancePointCircle(const point & c, ld r, const point & p){
  //point p, circle with center c and radius r
  return max((ld)0, (p - c).length() - r);
}
```

6.19 Distance Between Point and Line

```
ld distancePointLine(const point & a, const point & v, const point & p){
  //line: a + tv, point p
  return abs(v.cross(p - a)) / v.length();
}
```

6.20 Example of Geometry

```
int main(){
  /*vector<pair<point, point>> centers = {{point(-2, 5), point(-8, -7)},
       {point(14, 4), point(18, 6)}, {point(9, 20), point(9, 28)},
                       {point(21, 20), point(21, 29)}, {point(8, -10),
                           point(14, -10)}, {point(24, -6), point(34, -6)
                           },
                       {point(34, 8), point(36, 9)}, {point(50, 20),
                           point(56, 24.5)}};
  vector<pair<ld, ld>> radii = {{7, 4}, {3, 5}, {4, 4}, {4, 5}, {3, 3},
      {4, 6}, {5, 1}, {10, 2.5}};
  int n = centers.size();
  for(int i = 0; i < n; ++i){
    cout << "\n" << centers[i].first << " " << radii[i].first << " " <<
        centers[i].second << " " << radii[i].second << "\n";
    auto extLines = tangents(centers[i].first, radii[i].first, centers[i
        ].second, radii[i].second, false);
    cout << "Exterior tangents:\n";
    for(auto par : extLines){
      for(auto p : par){
        cout << p << " ";
      }
      cout << "\n";
    }
    auto intLines = tangents(centers[i].first, radii[i].first, centers[i
        ].second, radii[i].second, true);
```

```
    cout << "Interior tangents:\n";
    for(auto par : intLines){
      for(auto p : par){
        cout << p << " ";
      }
      cout << "\n";
    }
  }*/

  /*int n;
  cin >> n;
  vector<point> P(n);
  for(auto & p : P) cin >> p;
  auto triangulation = delaunay(P);
  for(auto triangle : triangulation){
    cout << get<0>(triangle) << " " << get<1>(triangle) << " " << get
        <2>(triangle) << "\n";
  }*/

  /*int n;
  cin >> n;
  vector<point> P(n);
  for(auto & p : P) cin >> p;
  auto ans = smallestEnclosingCircle(P);
  cout << ans.first << " " << ans.second << "\n";*/

  /*vector<point> P;
  srand(time(0));
  for(int i = 0; i < 1000; ++i){
    P.emplace_back(rand() % 1000000000, rand() % 1000000000);
  }
  point o(rand() % 1000000000, rand() % 1000000000), v(rand() %
      1000000000, rand() % 1000000000);
  polarSort(P, o, v);
  auto ang = [&](point p){
    ld th = atan2(p.y, p.x);
    if(th < 0) th += acosl(-1)*2;
    ld t = atan2(v.y, v.x);
    if(t < 0) t += acosl(-1)*2;
    if(th < t) th += acosl(-1)*2;
    return th;
  };
  for(int i = 0; i < P.size()-1; ++i){
    assert(leq(ang(P[i] - o), ang(P[i+1] - o)));
  }*/
  return 0;
}
```

6.21 Half Plane Intersection

```
struct plane{
  point a, v;
  plane(): a(), v(){}
  plane(const point& a, const point& v): a(a), v(v){}

  point intersect(const plane& p) const{
    ld t = (p.a - a).cross(p.v) / v.cross(p.v);
    return a + v*t;
  }

  bool outside(const point& p) const{ // test if point p is strictly
      outside
    return le(v.cross(p - a), 0);
  }

  bool inside(const point& p) const{ // test if point p is inside or in
      the boundary
    return geq(v.cross(p - a), 0);
  }

  bool operator<(const plane& p) const{ // sort by angle
    auto lhs = make_tuple(v.half({1, 0}), ld(0), v.cross(p.a - a));
    auto rhs = make_tuple(p.v.half({1, 0}), v.cross(p.v), ld(0));
    return lhs < rhs;
  }

  bool operator==(const plane& p) const{ // paralell and same directions
      , not really equal
    return eq(v.cross(p.v), 0) && ge(v.dot(p.v), 0);
  }
};

vector<point> halfPlaneIntersection(vector<plane> planes){
  planes.push_back({{0, -inf}, {1, 0}});
  planes.push_back({{inf, 0}, {0, 1}});
  planes.push_back({{0, inf}, {-1, 0}});
```

```
  planes.push_back({{-inf, 0}, {0, -1}});
  sort(planes.begin(), planes.end());
  planes.erase(unique(planes.begin(), planes.end()), planes.end());
  deque<plane> ch;
  deque<point> poly;
  for(const plane& p : planes){
    while(ch.size() >= 2 && p.outside(poly.back())) ch.pop_back(), poly.pop_back();
    while(ch.size() >= 2 && p.outside(poly.front())) ch.pop_front(), poly.pop_front();
    if(p.v.half({1, 0}) && poly.empty()) return {};
    ch.push_back(p);
    if(ch.size() >= 2) poly.push_back(ch[ch.size()-2].intersect(ch[ch.size()-1]));
  }
  while(ch.size() >= 3 && ch.front().outside(poly.back())) ch.pop_back(), poly.pop_back();
  while(ch.size() >= 3 && ch.back().outside(poly.front())) ch.pop_front(), poly.pop_front();
  poly.push_back(ch.back().intersect(ch.front()));
  return vector<point>(poly.begin(), poly.end());
}

vector<point> halfPlaneIntersectionRandomized(vector<plane> planes){
  point p = planes[0].a;
  int n = planes.size();
  random_shuffle(planes.begin(), planes.end());
  for(int i = 0; i < n; ++i){
    if(planes[i].inside(p)) continue;
    ld lo = -inf, hi = inf;
    for(int j = 0; j < i; ++j){
      ld A = planes[j].v.cross(planes[i].v);
      ld B = planes[j].v.cross(planes[j].a - planes[i].a);
      if(ge(A, 0)){
        lo = max(lo, B/A);
      }else if(le(A, 0)){
        hi = min(hi, B/A);
      }else{
        if(ge(B, 0)) return {};
      }
      if(ge(lo, hi)) return {};
    }
    p = planes[i].a + planes[i].v*lo;
  }
  return {p};
}
```

6.22 Incircle

```
pair<point, ld> getCircle(const point & m, const point & n, const point & p){
  //find circle that passes through points p, q, r
  point c = intersectLines((n + m) / 2, (n - m).perp(), (p + n) / 2, (p - n).perp());
  ld r = (c - m).length();
  return {c, r};
}
```

6.23 Intersection of Two Circles

```
vector<point> intersectionCircles(const point & c1, ld r1, const point & c2, ld r2){
  //circle 1 with center c1 and radius r1
  //circle 2 with center c2 and radius r2
  point d = c2 - c1;
  ld d2 = d.norm();
  if(eq(d2, 0)) return {}; //concentric circles
  ld pd = (d2 + r1*r1 - r2*r2) / 2;
  ld h2 = r1*r1 - pd*pd/d2;
  point p = c1 + d*pd/d2;
  if(eq(h2, 0)) return {p}; //circles touch at one point
  else if(le(h2, 0)) return {}; //circles don't intersect
  else{
    point u = d.perp() * sqrt(h2/d2);
    return {p - u, p + u};
  }
}
```

6.24 Intersection Line and Circle

```
vector<point> intersectLineCircle(const point & a, const point & v, const point & c, ld r){
  //line a+tv, circle with center c and radius r
  ld h2 = r*r - v.cross(c - a) * v.cross(c - a) / v.norm();
  point p = a + v * v.dot(c - a) / v.norm();
  if(eq(h2, 0)) return {p}; //line tangent to circle
  else if(le(h2, 0)) return {}; //no intersection
```

```
  else{
    point u = v.unit() * sqrt(h2);
    return {p - u, p + u}; //two points of intersection (chord)
  }
}
```

6.25 Intersection Polygon and Circle

```
ld signed_angle(const point & a, const point & b){
  return sgn(a.cross(b)) * acosl(a.dot(b) / (a.length() * b.length()));
}

ld intersectPolygonCircle(const vector<point> & P, const point & c, ld r
    ){
  //Gets the area of the intersection of the polygon with the circle
  int n = P.size();
  ld ans = 0;
  for(int i = 0; i < n; ++i){
    point p = P[i], q = P[(i+1)%n];
    bool p_inside = (pointInCircle(c, r, p) != 0);
    bool q_inside = (pointInCircle(c, r, q) != 0);
    if(p_inside && q_inside){
      ans += (p - c).cross(q - c);
    }else if(p_inside && !q_inside){
      point s1 = intersectSegmentCircle(p, q, c, r)[0];
      point s2 = intersectSegmentCircle(c, q, c, r)[0];
      ans += (p - c).cross(s1 - c) + r*r * signed_angle(s1 - c, s2 - c);
    }else if(!p_inside && q_inside){
      point s1 = intersectSegmentCircle(c, p, c, r)[0];
      point s2 = intersectSegmentCircle(p, q, c, r)[0];
      ans += (s2 - c).cross(q - c) + r*r * signed_angle(s1 - c, s2 - c);
    }else{
      auto info = intersectSegmentCircle(p, q, c, r);
      if(info.size() <= 1){
        ans += r*r * signed_angle(p - c, q - c);
      }else{
        point s2 = info[0], s3 = info[1];
        point s1 = intersectSegmentCircle(c, p, c, r)[0];
        point s4 = intersectSegmentCircle(c, q, c, r)[0];
        ans += (s2 - c).cross(s3 - c) + r*r * (signed_angle(s1 - c, s2 -
            c) + signed_angle(s3 - c, s4 - c));
      }
    }
  }
  return abs(ans)/2;
}
```

6.26 Intersection Segment and Circle

```
vector<point> intersectSegmentCircle(const point & a, const point & b,
    const point & c, ld r){
  //segment ab, circle with center c and radius r
  vector<point> P = intersectLineCircle(a, b - a, c, r), ans;
  for(const point & p : P){
    if(pointInSegment(a, b, p)) ans.push_back(p);
  }
  return ans;
}
```

6.27 Line Intersection

```
int intersectLinesInfo(const point & a1, const point & v1, const point &
     a2, const point & v2){
  //lines a1+tv1 and a2+tv2
  ld det = v1.cross(v2);
  if(eq(det, 0)){
    if(eq((a2 - a1).cross(v1), 0)){
      return -1; //infinity points
    }else{
      return 0; //no points
    }
  }else{
    return 1; //single point
  }
}

point intersectLines(const point & a1, const point & v1, const point &
    a2, const point & v2){
  //lines a1+tv1, a2+tv2
  //assuming that they intersect
  ld det = v1.cross(v2);
  return a1 + v1 * ((a2 - a1).cross(v2) / det);
}
```

6.28 Minkowski Sum

```
vector<point> minkowskiSum(vector<point> A, vector<point> B){
```

```
  int na = (int)A.size(), nb = (int)B.size();
  if(A.empty() || B.empty()) return {};

  rotate(A.begin(), min_element(A.begin(), A.end()), A.end());
  rotate(B.begin(), min_element(B.begin(), B.end()), B.end());

  int pa = 0, pb = 0;
  vector<point> M;

  while(pa < na && pb < nb){
    M.push_back(A[pa] + B[pb]);
    ld x = (A[(pa + 1) % na] - A[pa]).cross(B[(pb + 1) % nb] - B[pb]);
    if(leq(x, 0)) pb++;
    if(geq(x, 0)) pa++;
  }

  while(pa < na) M.push_back(A[pa++] + B[0]);
  while(pb < nb) M.push_back(B[pb++] + A[0]);

  return M;
}
```

6.29 Point in Circle

```
int pointInCircle(const point & c, ld r, const point & p){
  //test if point p is inside the circle with center c and radius r
  //returns "0" if it's outside, "-1" if it's in the perimeter, "1" if
      it's inside
  ld l = (p - c).length() - r;
  return (le(l, 0) ? 1 : (eq(l, 0) ? -1 : 0));
}
```

6.30 Point in Convex Hull

```
//point in convex polygon in O(log n)
//make sure that P is convex and in ccw
//before the queries, do the preprocess on P:
// rotate(P.begin(), min_element(P.begin(), P.end()), P.end());
// int right = max_element(P.begin(), P.end()) - P.begin();
//returns 0 if p is outside, 1 if p is inside, -1 if p is in the
    perimeter
int pointInConvexPolygon(const vector<point> & P, const point & p, int
    right){
  if(p < P[0] || P[right] < p) return 0;
  int orientation = sgn((P[right] - P[0]).cross(p - P[0]));
  if(orientation == 0){
    if(p == P[0] || p == P[right]) return -1;
    return (right == 1 || right + 1 == P.size()) ? -1 : 1;
  }else if(orientation < 0){
    auto r = lower_bound(P.begin() + 1, P.begin() + right, p);
    int det = sgn((p - r[-1]).cross(r[0] - r[-1])) - 1;
    if(det == -2) det = 1;
    return det;
  }else{
    auto l = upper_bound(P.rbegin(), P.rend() - right - 1, p);
    int det = sgn((p - l[0]).cross((l == P.rbegin() ? P[0] : l[-1]) - l
        [0])) - 1;
    if(det == -2) det = 1;
    return det;
  }
}
```

6.31 Point in Line

```
bool pointInLine(const point & a, const point & v, const point & p){
  //line a+tv, point p
  return eq((p - a).cross(v), 0);
}
```

6.32 Point in Perimeter

```
bool pointInPerimeter(const vector<point> & P, const point & p){
  int n = P.size();
  for(int i = 0; i < n; i++){
    if(pointInSegment(P[i], P[(i + 1) % n], p)){
      return true;
    }
  }
  return false;
}
```

6.33 Point in Polygon

```
int pointInPolygon(const vector<point> & P, const point & p){
  if(pointInPerimeter(P, p)){
    return -1; //point in the perimeter
  }
  int n = P.size();
```

```
  int rays = 0;
  for(int i = 0; i < n; i++){
    rays += crossesRay(P[i], P[(i + 1) % n], p);
  }
  return rays & 1; //0: point outside, 1: point inside
}
```

6.34 Point in Segment

```
bool pointInSegment(const point & a, const point & b, const point & p){
  //segment ab, point p
  return pointInLine(a, b - a, p) && leq((a - p).dot(b - p), 0);
}
```

6.35 Points Tangency

```
pair<point, point> pointsOfTangency(const point & c, ld r, const point &
     p){
  //point p (outside the circle), circle with center c and radius r
  point v = (p - c).unit() * r;
  ld d2 = (p - c).norm(), d = sqrt(d2);
  point v1 = v * (r / d), v2 = v.perp() * (sqrt(d2 - r*r) / d);
  return {c + v1 - v2, c + v1 + v2};
}
```

6.36 Projection Point Circle

```
point projectionPointCircle(const point & c, ld r, const point & p){
  //point p (outside the circle), circle with center c and radius r
  return c + (p - c).unit() * r;
}
```

6.37 Segment Intersection

```
int intersectLineSegmentInfo(const point & a, const point & v, const
    point & c, const point & d){
  //line a+tv, segment cd
  point v2 = d - c;
  ld det = v.cross(v2);
  if(eq(det, 0)){
    if(eq((c - a).cross(v), 0)){
      return -1; //infinity points
    }else{
      return 0; //no point
    }
  }else{
    return sgn(v.cross(c - a)) != sgn(v.cross(d - a)); //1: single point
        , 0: no point
  }
}

int intersectSegmentsInfo(const point & a, const point & b, const point
    & c, const point & d){
  //segment ab, segment cd
  point v1 = b - a, v2 = d - c;
  int t = sgn(v1.cross(c - a)), u = sgn(v1.cross(d - a));
  if(t == u){
    if(t == 0){
      if(pointInSegment(a, b, c) || pointInSegment(a, b, d) ||
          pointInSegment(c, d, a) || pointInSegment(c, d, b)){
        return -1; //infinity points
      }else{
        return 0; //no point
      }
    }else{
      return 0; //no point
    }
  }else{
    return sgn(v2.cross(a - c)) != sgn(v2.cross(b - c)); //1: single
        point, 0: no point
  }
}
```

6.38 Smallest Enclosing Circle

```
pair<point, ld> mec2(vector<point> & S, const point & a, const point & b
    , int n){
  ld hi = inf, lo = -hi;
  for(int i = 0; i < n; ++i){
    ld si = (b - a).cross(S[i] - a);
    if(eq(si, 0)) continue;
    point m = getCircle(a, b, S[i]).first;
    ld cr = (b - a).cross(m - a);
    if(le(si, 0)) hi = min(hi, cr);
    else lo = max(lo, cr);
  }
  ld v = (ge(lo, 0) ? lo : le(hi, 0) ? hi : 0);
```

```
  point c = (a + b) / 2 + (b - a).perp() * v / (b - a).norm();
  return {c, (a - c).norm()};
}

pair<point, ld> mec(vector<point> & S, const point & a, int n){
  random_shuffle(S.begin(), S.begin() + n);
  point b = S[0], c = (a + b) / 2;
  ld r = (a - c).norm();
  for(int i = 1; i < n; ++i){
    if(ge((S[i] - c).norm(), r)){
      tie(c, r) = (n == S.size() ? mec(S, S[i], i) : mec2(S, a, S[i], i)
          );
    }
  }
  return {c, r};
}

pair<point, ld> smallestEnclosingCircle(vector<point> S){
  assert(!S.empty());
  auto r = mec(S, S[0], S.size());
  return {r.first, sqrt(r.second)};
}
```

6.39 Smallest Enclosing Rectangle

```
pair<ld, ld> smallestEnclosingRectangle(vector<point> & P){
  int n = P.size();
  auto dot = [&](int a, int b){return (P[(a+1)%n]-P[a]).dot(P[(b+1)%n]-P
      [b]);};
  auto cross = [&](int a, int b){return (P[(a+1)%n]-P[a]).cross(P[(b+1)%
      n]-P[b]);};
  ld perimeter = inf, area = inf;
  for(int i = 0, j = 0, k = 0, m = 0; i < n; ++i){
    while(ge(dot(i, j), 0)) j = (j+1) % n;
    if(!i) k = j;
    while(ge(cross(i, k), 0)) k = (k+1) % n;
    if(!i) m = k;
    while(le(dot(i, m), 0)) m = (m+1) % n;
    //pairs: (i, k) , (j, m)
    point v = P[(i+1)%n] - P[i];
    ld h = distancePointLine(P[i], v, P[k]);
    ld w = distancePointLine(P[j], v.perp(), P[m]);
    perimeter = min(perimeter, 2 * (h + w));
    area = min(area, h * w);
  }
  return {area, perimeter};
}
```

6.40 Vantage Point Tree

```
struct vantage_point_tree{
  struct node
  {
    point p;
    ld th;
    node *l, *r;
  }*root;

  vector<pair<ld, point>> aux;

  vantage_point_tree(vector<point> &ps){
    for(int i = 0; i < ps.size(); ++i)
      aux.push_back({ 0, ps[i] });
    root = build(0, ps.size());
  }

  node *build(int l, int r){
    if(l == r)
      return 0;
    swap(aux[l], aux[l + rand() % (r - l)]);
    point p = aux[l++].second;
    if(l == r)
      return new node({ p });
    for(int i = l; i < r; ++i)
      aux[i].first = (p - aux[i].second).dot(p - aux[i].second);
    int m = (l + r) / 2;
    nth_element(aux.begin() + l, aux.begin() + m, aux.begin() + r);
    return new node({ p, sqrt(aux[m].first), build(l, m), build(m, r) })
        ;
  }

  priority_queue<pair<ld, node*>> que;

  void k_nn(node *t, point p, int k){
    if(!t)
      return;
```

```
    ld d = (p - t->p).length();
    if(que.size() < k)
      que.push({ d, t });
    else if(ge(que.top().first, d)){
      que.pop();
      que.push({ d, t });
    }
    if(!t->l && !t->r)
      return;
    if(le(d, t->th)){
      k_nn(t->l, p, k);
      if(leq(t->th - d, que.top().first))
        k_nn(t->r, p, k);
    }else{
      k_nn(t->r, p, k);
      if(leq(d - t->th, que.top().first))
        k_nn(t->l, p, k);
    }
  }

  vector<point> k_nn(point p, int k){
    k_nn(root, p, k);
    vector<point> ans;
    for(; !que.empty(); que.pop())
      ans.push_back(que.top().second->p);
    reverse(ans.begin(), ans.end());
    return ans;
  }
};
```

7 Graphs

7.1 2Sat

```
struct TwoSatSolver {
  int n_vars;                    // Number of boolean variables
  int n_vertices;                // Total vertices in the implication
      graph (2 per variable)
  vector<vector<int>> adj;       // Implication graph: adj[i] contains
      edges from node i
  vector<vector<int>> adj_t;     // Transposed graph for Kosaraju's
      algorithm
  vector<bool> used;             // Visited marker for DFS
  vector<int> order;             // Finishing order of vertices (DFS1)
  vector<int> comp;              // Component ID for each node (DFS2)
  vector<bool> assignment;       // Final truth assignment for each
      variable

  // Constructor initializes all data structures
  TwoSatSolver(int _n_vars)
    : n_vars(_n_vars),
      n_vertices(2 * _n_vars),
      adj(n_vertices),
      adj_t(n_vertices),
      used(n_vertices),
      comp(n_vertices, -1),
      assignment(n_vars) {
    order.reserve(n_vertices); // Pre-allocate memory for efficiency
  }

  // First DFS pass for Kosaraju's algorithm (on original graph)
  void dfs1(int v) {
    used[v] = true;
    for (int u : adj[v]) {
      if (!used[u])
        dfs1(u);
    }
    order.push_back(v); // Save the vertex post-DFS for reverse ordering
  }

  // Second DFS pass on the transposed graph to label components
  void dfs2(int v, int cl) {
    comp[v] = cl;
    for (int u : adj_t[v]) {
      if (comp[u] == -1)
        dfs2(u, cl);
    }
  }

  // Solves the 2-SAT problem using Kosaraju's algorithm
  bool solve_2SAT() {
    // 1st pass: fill the order vector
    order.clear();
    used.assign(n_vertices, false);
    for (int i = 0; i < n_vertices; ++i) {
      if (!used[i])
```

```
        dfs1(i);
    }

    // 2nd pass: find SCCs in reverse postorder
    comp.assign(n_vertices, -1);
    for (int i = 0, j = 0; i < n_vertices; ++i) {
      int v = order[n_vertices - i - 1]; // Reverse postorder
      if (comp[v] == -1)
        dfs2(v, j++);
    }

    // Assign values to variables based on component comparison
    assignment.assign(n_vars, false);
    for (int i = 0; i < n_vertices; i += 2) {
      if (comp[i] == comp[i + 1])
        return false; // Contradiction: variable and its negation are in
            the same SCC
      assignment[i / 2] = comp[i] > comp[i + 1]; // True if var's
          component comes after its negation
    }
    return true;
  }

  // Adds a disjunction (a v b) to the implication graph
  // 'na' and 'nb' indicate negation: if true means !a or !b
  // Variables are 0-indexed. Bounds are inclusive for each literal (i.e
      ., 0 to n_vars - 1)
  void add_disjunction(int a, bool na, int b, bool nb) {
    // Each variable `x` has two nodes:
    // x => 2*x, !x => 2*x + 1
    // We encode (a v b) as (!a -> b) and (!b -> a)
    a = 2 * a ^ na;
    b = 2 * b ^ nb;
    int neg_a = a ^ 1;
    int neg_b = b ^ 1;

    adj[neg_a].push_back(b);
    adj[neg_b].push_back(a);
    adj_t[b].push_back(neg_a);
    adj_t[a].push_back(neg_b);
  }
};
```

7.2 Articulation Points

```
/*
  Articulation Points (Cut Vertices) in an Undirected Graph
  ---------------------------------------------------------
  Indexing: 0-based
  Node Bounds: [0, n-1] inclusive
  Time Complexity: O(V + E)
  Space Complexity: O(V)

  Use Case:
    - Identifies vertices whose removal increases the number of
        connected components.
    - Works on undirected graphs (connected or disconnected).
*/

int n; // Number of nodes in the graph
vector<vector<int>> adj; // Adjacency list of the undirected graph

vector<bool> visited; // Marks if a node was visited during DFS
vector<int> tin, low; // tin[v]: discovery time; low[v]: lowest
    discovery time reachable from subtree
int timer; // Global time counter for DFS

// DFS traversal to identify articulation points
void dfs(int v, int p = -1) {
  visited[v] = true;
  tin[v] = low[v] = timer++;
  int children = 0;
  for (int to : adj[v]) {
    if (to == p) continue; // Skip the parent edge
    if (visited[to]) {
      // Back edge
      low[v] = min(low[v], tin[to]);
    } else {
      dfs(to, v);
      low[v] = min(low[v], low[to]);
      // Articulation point condition for non-root
      if (low[to] >= tin[v] && p != -1) {
        // be careful as this may make the same articulation point
            appear many times
        // v is an articulation point
        // handle_cutpoint(v);
```

```
      }
      ++children;
    }
  }
  // Articulation point condition for root
  if (p == -1 && children > 1) {
    // v is an articulation point
    // handle_cutpoint(v);
  }
}

// Initializes structures and launches DFS
void find_cutpoints() {
  timer = 0;
  visited.assign(n, false);
  tin.assign(n, -1);
  low.assign(n, -1);

  for (int i = 0; i < n; ++i) {
    if (!visited[i])
      dfs(i);
  }
}
```

7.3 Bellman-Ford

```
/*
  Bellman-Ford (SPFA variant) for Shortest Paths
  ----------------------------------------------
  Indexing: 0-based
  Node Bounds: [0, n-1] inclusive
  Time Complexity: O(V * E) worst-case (amortized better)
  Space Complexity: O(V + E)

  Features:
    - Handles negative edge weights
    - Detects negative weight cycles (returns false if one exists)
    - Works on directed or undirected graphs

  Path Reconstruction:
    - To recover the path from source `s` to any node `u`:
        vector<int> path;
        for (int v = u; v != -1; v = parent[v])
          path.push_back(v);
        reverse(path.begin(), path.end());
*/

const int INF = 1<<30; // Large value to represent "infinity"
vector<vector<pair<int, int>>> adj; // adj[v] = list of (neighbor,
    weight) pairs
vector<int> parent; // parent(n, -1) for path reconstruction

// SPFA implementation to find shortest paths from source s
// d[i] will contain shortest distance from s to i
// Returns false if a negative cycle is detected
// For path reconstruction add vector<int>& parent as parameter
bool spfa(int s, vector<int>& d, vector<int>& parent) {
  int n = adj.size();
  d.assign(n, INF);
  vector<int> cnt(n, 0);       // Count how many times each node has
      been relaxed
  vector<bool> inqueue(n, false); // Tracks if a node is currently in
      queue
  queue<int> q;

  d[s] = 0;
  q.push(s);
  inqueue[s] = true;

  while (!q.empty()) {
    int v = q.front();
    q.pop();
    inqueue[v] = false;

    for (auto edge : adj[v]) {
      int to = edge.first;
      int len = edge.second;

      if (d[v] + len < d[to]) {
        parent[to] = v; // For path reconstruction
        d[to] = d[v] + len;
        if (!inqueue[to]) {
          q.push(to);
          inqueue[to] = true;
          cnt[to]++;
          if (cnt[to] > n)
```

```
            return false; // Negative weight cycle detected
          }
        }
      }
    }

  return true; // No negative cycles; shortest paths computed
}
```

7.4 Bipartite Checker

```
/*
  Bipartite Graph Checker (BFS-based)
  -----------------------------------
  Indexing: 0-based
  Time Complexity: O(V + E)
  Space Complexity: O(V)

  Handles disconnected graphs
*/

int n; // Number of nodes
vector<vector<int>> adj; // Adjacency list of the undirected graph

vector<int> side(n, -1); // -1 = unvisited, 0/1 = sides of bipartition
bool is_bipartite = true;
queue<int> q;

for (int st = 0; st < n; ++st) {
  if (side[st] == -1) {
    q.push(st);
    side[st] = 0; // Start with side 0
    while (!q.empty()) {
      int v = q.front();
      q.pop();
      for (int u : adj[v]) {
        if (side[u] == -1) {
          // Assign opposite side to neighbor
          side[u] = side[v] ^ 1;
          q.push(u);
        } else {
          // Conflict: adjacent nodes on same side
          is_bipartite &= side[u] != side[v];
        }
      }
    }
  }
}

cout << (is_bipartite ? "YES" : "NO") << endl;
```

7.5 Bipartite Maximum Matching

```
/*
  Maximum Bipartite Matching (Kuhn's Algorithm)
  ---------------------------------------------
  Indexing: 0-based
  Time Complexity: O(N * (E + N)) worst case
  Space Complexity: O(N + K + E)

  Input:
    - n: number of nodes on the left side
    - k: number of nodes on the right side
    - g: adjacency list where g[v] contains all right nodes adjacent to
       left node v

  Output:
    - Prints the pairs (left, right) in the matching
    - mt[r] = l means right node r is matched to left node l
*/

int n, k; // n: number of left nodes, k: number of right nodes
vector<vector<int>> g; // g[l]: list of right-side neighbors of left
    node l
vector<int> mt;        // mt[r]: matched left node for right node r (or
    -1 if unmatched)
vector<bool> used;     // used[l]: visited status for left node l during
     DFS

// Try to find an augmenting path from left node v
bool try_kuhn(int v) {
  if (used[v])
    return false;
  used[v] = true;
  for (int to : g[v]) {
    if (mt[to] == -1 || try_kuhn(mt[to])) {
```

```
      mt[to] = v;
      return true;
    }
  }
  return false;
}

int main() {
  //... reading the graph ...

  mt.assign(k, -1); // Right-side nodes initially unmatched
  for (int v = 0; v < n; ++v) {
    used.assign(n, false); // Reset visited for each left node
    try_kuhn(v);
  }
  // Output matched pairs (left+1, right+1 for 1-based output)
  for (int i = 0; i < k; ++i) {
    if (mt[i] != -1)
      printf("%d %d\n", mt[i] + 1, i + 1);
  }
  return 0;
}
```

7.6 Bipartite Minimum Maximum Matching2

```
/*
  THIS CODE HAS NOT BEEN TESTED BEFOREHAND
  Maximum Bipartite Matching - Hopcroft-Karp Algorithm
  -----------------------------------------------------
  Indexing: 0-based
  Time Complexity: O(sqrt(V) * E)
  Space Complexity: O(V + E)

  Input:
    - n: number of nodes on the left
    - m: number of nodes on the right
    - g[l]: adjacency list for left node l (list of right-side neighbors
        )

  Output:
    - match_right[r] = l means right node r is matched to left node l
    - match_left[l] = r means left node l is matched to right node r
    - Function returns total number of matching pairs
*/

const int INF = 1e9;

int n, m; // Number of nodes on left and right
vector<vector<int>> g;       // Adjacency list: g[l] contains neighbors
    of left node l
vector<int> match_left, match_right; // match_left[l], match_right[r]
vector<int> dist;            // Distance levels for BFS

// BFS to build layer graph, returns true if there's at least one
    unmatched node on the right reachable from left
bool bfs() {
  queue<int> q;
  dist.assign(n, INF);

  for (int l = 0; l < n; ++l) {
    if (match_left[l] == -1) {
      dist[l] = 0;
      q.push(l);
    }
  }

  bool found = false;

  while (!q.empty()) {
    int l = q.front(); q.pop();
    for (int r : g[l]) {
      int matched_l = match_right[r];
      if (matched_l == -1) {
        found = true; // Free right node reachable -> potential
            augmenting path
      } else if (dist[matched_l] == INF) {
        dist[matched_l] = dist[l] + 1;
        q.push(matched_l);
      }
    }
  }

  return found;
}

// DFS to find augmenting paths in the layered graph
```

```cpp
bool dfs(int l) {
  for (int r : g[l]) {
    int matched_l = match_right[r];
    if (matched_l == -1 || (dist[matched_l] == dist[l] + 1 && dfs(
        matched_l))) {
      match_left[l] = r;
      match_right[r] = l;
      return true;
    }
  }
  dist[l] = INF;
  return false;
}

// Main function to compute maximum matching
int hopcroft_karp() {
  match_left.assign(n, -1);
  match_right.assign(m, -1);

  int matching = 0;
  while (bfs()) {
    for (int l = 0; l < n; ++l) {
      if (match_left[l] == -1 && dfs(l)) {
        ++matching;
      }
    }
  }
  return matching;
}

int main() {
  cin >> n >> m;
  g.assign(n, {});
  int e; cin >> e;
  while (e--) {
    int u, v;
    cin >> u >> v;
    g[u].push_back(v); // u in [0, n-1], v in [0, m-1]
  }

  int res = hopcroft_karp();
  cout << "Maximum matching size: " << res << '\n';
  for (int r = 0; r < m; ++r) {
    if (match_right[r] != -1) {
      cout << match_right[r] + 1 << " " << r + 1 << '\n'; // 1-based
          output
    }
  }
}
```

7.7 Block Cut Tree

```cpp
/*
  Block-Cut Tree from Biconnected Components
  ------------------------------------------
  Indexing: 0-based
  Node Bounds: [0, n-1] inclusive
  Time Complexity: O(V + E)
  Space Complexity: O(V + E)

  Features:
    - Identifies articulation points (cut vertices)
    - Extracts all biconnected components (BCCs)
    - Constructs the Block-Cut Tree:
        - Each BCC becomes a node in the tree
        - Each articulation point becomes its own node
        - An edge connects a BCC-node to each cutpoint in it

  Output:
    - `is_cutpoint`: true if node is an articulation point
    - `id[v]`: node ID of `v` in the block-cut tree
    - Returns the block-cut tree as an adjacency list
*/

vector<vector<int>> biconnected_components(vector<vector<int>> &g, //
    Adjacency list of the undirected graph
                                           vector<bool> &is_cutpoint, //
                                                Output vector (resized
                                               internally)
                                           vector<int> &id) { // Output
                                               vector (resized
                                               internally)

  int n = g.size();
  vector<vector<int>> comps; // Stores all biconnected components
  vector<int> stk;           // Stack of visited nodes for current
```

```
    component
vector<int> num(n), low(n); // DFS discovery time and low-link values
is_cutpoint.assign(n, false);

// DFS to find BCCs and articulation points
function<void(int, int, int&)> dfs = [&](int node, int parent, int &
    timer) {
  num[node] = low[node] = ++timer;
  stk.push_back(node);
  for (int son : g[node]) {
    if (son == parent) continue;
    if (num[son]) {
      // Back edge
      low[node] = min(low[node], num[son]);
    } else {
      dfs(son, node, timer);
      low[node] = min(low[node], low[son]);
      // Check articulation point condition
      if (low[son] >= num[node]) {
        is_cutpoint[node] = (num[node] > 1 || num[son] > 2); // For
            root and non-root
        comps.push_back({node});
        while (comps.back().back() != son) {
          comps.back().push_back(stk.back());
          stk.pop_back();
        }
      }
    }
  }
};

int timer = 0;
dfs(0, -1, timer);

id.resize(n); // Maps each original node to its block-cut tree node ID

// Build block-cut tree using articulation points and BCCs
function<vector<vector<int>>()> build_tree = [&]() {
  vector<vector<int>> t(1); // Dummy index 0 (not used)
  int node_id = 1; // Start assigning block-cut tree IDs from 1
  // Assign unique tree node IDs to cutpoints
  for (int node = 0; node < n; ++node) {
    if (is_cutpoint[node]) {
      id[node] = node_id++;
      t.push_back({});
    }
  }
  // Assign each component a new node and connect it to its cutpoints
  for (auto &comp : comps) {
    int bcc_node = node_id++;
    t.push_back({});
    for (int u : comp) {
      if (!is_cutpoint[u]) {
        id[u] = bcc_node;
      } else {
        t[bcc_node].push_back(id[u]);
        t[id[u]].push_back(bcc_node);
      }
    }
  }
  return t;
};

return build_tree(); // Return the block-cut tree
}
```

7.8 Blossom

```
/*
  Edmonds' Blossom Algorithm (Maximum Matching in General Graphs)
  ---------------------------------------------------------------
  Indexing: 1-based
  Node Bounds: [1, n]
  Time Complexity: O(n^3) in worst case
  Space Complexity: O(n^2)

  Features:
    - Handles odd-length cycles (blossoms)
    - Works on any undirected graph (not just bipartite)
    - Uses BFS with blossom contraction and path augmentation

  Input:
    - n: number of vertices
    - add_edge(u, v): undirected edges between nodes (1 <= u,v <= n)

  Output:
```

```
    - maximum_matching(): returns size of max matching
    - match[u]: matched vertex for node u (or 0 if unmatched)
*/

const int N = 2009;
mt19937 rnd(chrono::steady_clock::now().time_since_epoch().count());

struct Blossom {
  int vis[N];        // vis[u]: -1 = unvisited, 0 = in queue, 1 = outer layer
  int par[N];        // par[u]: parent in alternating tree
  int orig[N];       // orig[u]: base of blossom u belongs to
  int match[N];      // match[u]: matched partner of u (0 if unmatched)
  int aux[N];        // aux[u]: visit marker for LCA
  int t;             // global timestamp for LCA markers
  int n;             // number of nodes
  bool ad[N];        // ad[u]: whether u is reachable in an alternating path
  vector<int> g[N];  // g[u]: adjacency list
  queue<int> Q;      // BFS queue

  // Constructor: initializes data for n nodes
  Blossom() {}
  Blossom(int _n) {
    n = _n;
    t = 0;
    for (int i = 0; i <= n; ++i) {
      g[i].clear();
      match[i] = par[i] = vis[i] = aux[i] = ad[i] = orig[i] = 0;
    }
  }

  void add_edge(int u, int v) {
    g[u].push_back(v);
    g[v].push_back(u);
  }

  // Augment the matching along the alternating path from u to v
  void augment(int u, int v) {
    int pv = v, nv;
    do {
      pv = par[v];
      nv = match[pv];
      match[v] = pv;
      match[pv] = v;
      v = nv;
    } while (u != pv);
  }

  int lca(int v, int w) {
    ++t; // Increment timestamp for LCA markers
    while (true) {
      if (v) {
        if (aux[v] == t) return v;
        aux[v] = t;
        v = orig[par[match[v]]]; // Move to the parent in the alternating tree
      }
      swap(v, w);
    }
  }

  // Contract a blossom from v and w with common ancestor a
  void blossom(int v, int w, int a) {
    while (orig[v] != a) {
      par[v] = w;
      w = match[v];
      ad[v] = true;
      if (vis[w] == 1) Q.push(w), vis[w] = 0;
      orig[v] = orig[w] = a;
      v = par[w];
    }
  }

  // Find augmenting path starting from unmatched node u
  bool bfs(int u) {
    fill(vis + 1, vis + n + 1, -1);
    iota(orig + 1, orig + n + 1, 1);
    Q = queue<int>();
    Q.push(u);
    vis[u] = 0;

    while (!Q.empty()) {
      int v = Q.front(); Q.pop();
      ad[v] = true;
      for (int x : g[v]) {
```

```
      if (vis[x] == -1) {
        par[x] = v;
        vis[x] = 1;
        if (!match[x]) {
          augment(u, x);
          return true;
        }
        Q.push(match[x]);
        vis[match[x]] = 0;
      } else if (vis[x] == 0 && orig[v] != orig[x]) {
        int a = lca(orig[v], orig[x]);
        blossom(x, v, a);
        blossom(v, x, a);
      }
    }
  }
  return false;
}

// Computes maximum matching and returns the size
int maximum_matching() {
  int ans = 0;
  vector<int> p(n - 1);
  iota(p.begin(), p.end(), 1);
  shuffle(p.begin(), p.end(), rnd);
  for (int i = 1; i <= n; ++i) {
    shuffle(g[i].begin(), g[i].end(), rnd);
  }

  // Greedy matching: try to match unmatched nodes directly
  for (int u : p) {
    if (!match[u]) {
      for (int v : g[u]) {
        if (!match[v]) {
          match[u] = v;
          match[v] = u;
          ++ans;
          break;
        }
      }
    }
  }

  // Augmenting path phase
  for (int i = 1; i <= n; ++i) {
    if (!match[i] && bfs(i)) ++ans;
  }

  return ans;
 }
} M;

int main() {
 ios_base::sync_with_stdio(0);
 cin.tie(0);

 int t;
 cin >> t;
 while (t--) {
  int n, m;
  cin >> n >> m;
  M = Blossom(n);
  // Read all edges
  for (int i = 0; i < m; i++) {
   int u, v;
   cin >> u >> v;
   M.add_edge(u, v);
  }
  // Compute max matching
  int matched = M.maximum_matching();
  if (matched * 2 == n) {
   // Perfect matching
   cout << 0 << '\n';
  } else {
   // Find reachable unmatched nodes in alternating trees
   memset(M.ad, 0, sizeof M.ad);
   for (int i = 1; i <= n; i++) {
    if (M.match[i] == 0) M.bfs(i);
   }
   int unmatched_reachable = 0;
   for (int i = 1; i <= n; i++) {
    unmatched_reachable += M.ad[i];
   }
   cout << unmatched_reachable << '\n';
  }
 }
```

```
  return 0;
}
```

7.9 Bridges

```
/*
  Bridge-Finding in an Undirected Graph
  -------------------------------------
  Indexing: 0-based
  Node Bounds: [0, n-1] inclusive
  Time Complexity: O(V + E)
  Space Complexity: O(V)

  Input:
    n    - Number of nodes in the graph
    adj  - Adjacency list of the undirected graph

  Output:
    - Call `find_bridges()` to populate bridge information.
    - Modify the DFS `Bridge` section to store or print the bridges.
      A bridge is an edge (v, to) such that removing it increases the
          number of connected components.
*/

int n; // Number of nodes
vector<vector<int>> adj; // Adjacency list

vector<bool> visited; // Marks visited nodes
vector<int> tin, low; // tin[v]: discovery time; low[v]: lowest ancestor
     reachable
int timer; // Global DFS timer

// DFS to detect bridges
void dfs(int v, int p = -1) {
  visited[v] = true;
  tin[v] = low[v] = timer++;
  for (int to : adj[v]) {
    if (to == p) continue; // Skip edge to parent
    if (visited[to]) {
      // Back edge
      low[v] = min(low[v], tin[to]);
    } else {
      dfs(to, v);
      low[v] = min(low[v], low[to]);
      // Bridge condition: if no back edge connects subtree rooted at `
          to` to ancestors of `v`
      if (low[to] > tin[v]) {
        // (v, to) is a bridge
        // Example: bridges.push_back({v, to});
      }
    }
  }
}

// Initialize tracking structures and run DFS
void find_bridges() {
  timer = 0;
  visited.assign(n, false);
  tin.assign(n, -1);
  low.assign(n, -1);
  for (int i = 0; i < n; ++i) {
    if (!visited[i])
      dfs(i);
  }
}
```

7.10 Bridges Online

```
/*
  Online Bridge-Finding (Dynamic Edge Insertion)
  ----------------------------------------------
  Indexing: 0-based
  Node Bounds: [0, n-1] inclusive
  Time Complexity:
    - Amortized O(log^2N) per edge addition
  Space Complexity: O(V)

  Features:
    - Maintains the number of bridges dynamically as edges are added one
        by one.
    - Detects if adding an edge merges different 2-edge-connected
      components.
    - No deletions supported.

  Input:
    init(n)     - Initializes the data structure for a graph with n
```

```
        nodes.
    add_edge(a, b) - Adds an undirected edge between nodes a and b.

  Output:
    'bridges' - Global variable representing the current number of
        bridges.
*/

vector<int> par, dsu_2ecc, dsu_cc, dsu_cc_size;
int bridges; // Number of bridges in the graph
int lca_iteration;
vector<int> last_visit;

// Initializes the data structures
void init(int n) {
  par.resize(n);
  dsu_2ecc.resize(n);
  dsu_cc.resize(n);
  dsu_cc_size.resize(n);
  last_visit.assign(n, 0);
  lca_iteration = 0;
  bridges = 0;

  for (int i = 0; i < n; ++i) {
    par[i] = -1;
    dsu_2ecc[i] = i;
    dsu_cc[i] = i;
    dsu_cc_size[i] = 1;
  }
}

// Finds the representative of the 2-edge-connected component of node v
int find_2ecc(int v) {
  if (v == -1) return -1;
  return dsu_2ecc[v] == v ? v : dsu_2ecc[v] = find_2ecc(dsu_2ecc[v]);
}

// Finds the connected component representative of the component
    containing v
int find_cc(int v) {
  v = find_2ecc(v);
  return dsu_cc[v] == v ? v : dsu_cc[v] = find_cc(dsu_cc[v]);
}

// Makes node v the root of its tree, rerouting parent pointers upward
void make_root(int v) {
  int root = v;
  int child = -1;
  while (v != -1) {
    int p = find_2ecc(par[v]);
    par[v] = child;
    dsu_cc[v] = root;
    child = v;
    v = p;
  }
  dsu_cc_size[root] = dsu_cc_size[child];
}

// Merges paths from a and b to their lowest common ancestor in the 2ECC
    forest
void merge_path(int a, int b) {
  ++lca_iteration;
  vector<int> path_a, path_b;
  int lca = -1;

  while (lca == -1) {
    if (a != -1) {
      a = find_2ecc(a);
      path_a.push_back(a);
      if (last_visit[a] == lca_iteration) {
        lca = a;
        break;
      }
      last_visit[a] = lca_iteration;
      a = par[a];
    }
    if (b != -1) {
      b = find_2ecc(b);
      path_b.push_back(b);
      if (last_visit[b] == lca_iteration) {
        lca = b;
        break;
      }
      last_visit[b] = lca_iteration;
      b = par[b];
    }
```

```
  }

  // Merge all nodes on path_a and path_b into the same 2ECC
  for (int v : path_a) {
    dsu_2ecc[v] = lca;
    if (v == lca) break;
    --bridges;
  }
  for (int v : path_b) {
    dsu_2ecc[v] = lca;
    if (v == lca) break;
    --bridges;
  }
}

// Adds an undirected edge between a and b and updates bridge count
void add_edge(int a, int b) {
  a = find_2ecc(a);
  b = find_2ecc(b);
  if (a == b) return; // Already in the same 2ECC

  int ca = find_cc(a);
  int cb = find_cc(b);

  if (ca != cb) {
    // Bridge found - connects two different components
    ++bridges;
    // Union by size
    if (dsu_cc_size[ca] > dsu_cc_size[cb]) {
      swap(a, b);
      swap(ca, cb);
    }
    make_root(a);
    par[a] = b;
    dsu_cc[a] = b;
    dsu_cc_size[cb] += dsu_cc_size[a];
  } else {
    // No new bridge, but must merge paths to unify 2ECCs
    merge_path(a, b);
  }
}

// Example usage
int main() {
  init(n);
  for (auto [u, v] : edges) {
    add_edge(u, v);
    cout << "Current bridge count: " << bridges << '\n';
  }
}
```

7.11 Dijkstra

```
vector<vector<pair<int, int>>> adj(n); // Adjacency list (node, weight)
vector<ll> dist(n, 1LL << 61); // Distance array initialized to infinity

priority_queue<pair<ll, int>> q; // Max-heap, so we push negative
    weights to simulate min-heap
dist[0] = 0; // Starting node distance
q.push({0, 0}); // (distance, vertex)

while (!q.empty()) {
  auto [w, v] = q.top(); q.pop();
  w = -w; // Convert back to positive
  if (w > dist[v]) continue; // Skip outdated entry
  for (auto [u, cost] : adj[v]) {
    if (dist[v] + cost < dist[u]) {
      dist[u] = dist[v] + cost;
      q.push({-dist[u], u}); // Push updated distance (negated)
    }
  }
}
```

7.12 Eulerian Path

An Eulerian Path is a path that passes through every edge once. For an undirected graph an eulerian path exists if the degree of every node is even or the degree of exactly two nodes is odd. In the first case, the eulerian path is also an eulerian circuit or cycle. In a directed graph, an eulerian path exists if at most one noda has $out_i - in_i = 1$ and at most one node has $in_i - out_i = 1$. A cycle exists if $in_i - out_i = 0$ for all i.

```
/*
  Eulerian Path (Hierholzer's Algorithm)
  This will find an Eurelian Circuit.
  For directed graph eulerian path must start on vertex with outdegree[i
      ] == indegree[i] + 1 and ends on indegree[i] = outdegree[i] + 1.
```

```
  For directed graph eulerian Cycle only exists if indegree[i] =
      outdegree[i] for all i.
  For undirected graph eulerian path must start on vertex with odd
      degree and will end in the other vertex with odd degree.
  For undirected graph eulerian Cycle only exists degree is even for all
       vertices.

  ------------------------------------
  Time Complexity: O(E)
  Space Complexity: O(V + E)

  Input:
    - g: adjacency list of the graph
        * Directed: vector<vector<pair<int, int>>> g
            where g[v] = list of {to, edge_index}
        * Undirected: vector<vector<int>> g
            where g[v] = list of neighbors
    - seen: vector<bool> seen(E) - only needed for directed version
    - path: vector<int> path - will be filled in reverse order of
        traversal
            reverse(path.begin(), path.end());
*/

// Directed Version //
void dfs_directed(int node) {
  while (!g[node].empty()) {
    auto [son, idx] = g[node].back();
    g[node].pop_back();
    if (seen[idx]) continue;  // Skip if edge already visited
    seen[idx] = true;
    dfs_directed(son);
  }
  path.push_back(node); // Post-order insertion (reverse of actual path)
}

// Undirected Version //
// Be careful with graph not being connected. Add set to not repeat
    edges(going from u to v and then from v to u)
void dfs_undirected(int node) {
  while (!g[node].empty()) {
    int son = g[node].back();
    g[node].pop_back();
    dfs_undirected(son);
  }
  path.push_back(node); // Post-order insertion
}
```

7.13 Floyd-Warshall

```
/*
  Floyd-Warshall Algorithm (All-Pairs Shortest Paths)
  ---------------------------------------------------
  Indexing: 0-based
  Time Complexity: O(V^3)
  Space Complexity: O(V^2)

  Input:
    - d: distance matrix of size n x n
        * d[i][j] should be initialized as:
            - 0 if i == j
            - weight of edge (i, j) if exists
            - INF (e.g. 1e18) otherwise
*/

vector<vector<ll>> d(n, vector<ll>(n, 1e18)); // distance matrix

// This version is by default adapted for UNDIRECTED graphs.
for (int k = 0; k < n; k++) {
  for (int i = 0; i < n; i++) {
    for (int j = i + 1; j < n; j++) { // For directed graphs, use j = 0;
         j < n; j++
      long long new_dist = d[i][k] + d[k][j];
      if (new_dist < d[i][j]) {
        d[i][j] = d[j][i] = new_dist; // update both directions for
            undirected graph
      }
    }
  }
}
```

7.14 Kruskal

```
/*
  Kruskal's Algorithm (Minimum Spanning Tree - MST)
  -------------------------------------------------
```

```
  Indexing: 0-based for nodes in edges
  Time Complexity: O(E log E)
  Space Complexity: O(N)

  Input:
    - N: number of nodes
    - edges: list of weighted edges in form {weight, {u, v}}

  Output:
    - Returns total weight of the MST if the graph is connected
    - Returns -1 if MST cannot be formed (i.e., graph is disconnected)

  Note:
    - Requires a Disjoint Set Union (DSU) / Union-Find data structure
        with:
        - unite(a, b): merges components, returns true if successful
        - size(v): returns size of component containing v
*/

template <class T>
T kruskal(int N, vector<pair<T, pair<int, int>>> edges) {
  sort(edges.begin(), edges.end()); // Sort by weight (non-decreasing)
  T ans = 0;
  DSU D(N); // Disjoint Set Union for N nodes
  for (auto &[w, uv] : edges) {
    int u = uv.first, v = uv.second;
    if (D.unite(u, v)) {
      ans += w; // Add edge to MST if u and v are in different
          components
    }
  }
  // Check if MST spans all nodes (i.e., one component of size N)
  return (D.size(0) == N ? ans : -1);
}
```

7.15 Marriage

```
/*
  Male-Optimal Stable Marriage Problem (Gale-Shapley Algorithm)
  ------------------------------------------------------------
  Indexing: 0-based
  Bounds: 0 <= i, j < n
  Time Complexity: O(n^2)
  Space Complexity: O(n^2)

  Input:
    - n: Number of men/women (equal)
    - gv[i][j]: j-th most preferred woman for man i
    - om[i][j]: j-th most preferred man for woman i
        * Both are permutations of {0, ..., n-1}
        * om must be inverted to get om[w][m] = woman w's ranking of man
            m

  Output:
    - pm[i]: Woman matched to man i (i.e. pairings)
    - pv[i]: Man matched to woman i
*/

#define MAXN 1000
int gv[MAXN][MAXN], om[MAXN][MAXN]; // Male and female preference lists
int pv[MAXN], pm[MAXN];             // pv[woman] = man, pm[man] = woman
int pun[MAXN];                      // pun[man] = next woman to propose
    to

void stableMarriage(int n) {
  fill_n(pv, n, -1);   // All women initially unmatched
  fill_n(pm, n, -1);   // All men initially unmatched
  fill_n(pun, n, 0);   // Each man starts at his top preference

  int unmatched = n;   // Number of free men
  int i = n - 1;       // Current man index (rotates over all men)

  #define engage pm[j] = i; pv[i] = j;

  while (unmatched) {
    while (pm[i] == -1) {
      int j = gv[i][pun[i]++]; // Next woman on man i's list

      if (pv[j] == -1) {
        // Woman j is free -> engage with man i
        unmatched--;
        engage;
      } else if (om[j][i] < om[j][pv[j]]) {
        // Woman j prefers i over her current partner
        int loser = pv[j];
        pm[loser] = -1;
```

```
        engage;
        i = loser; // Reconsider the rejected man
      }
    }

    // Move to next unmatched man
    i--;
    if (i < 0) i = n - 1;
  }

  #undef engage
}
```

7.16 SCC

```
/*
  Strongly Connected Components (Kosaraju's Algorithm)
  ----------------------------------------------------
  Indexing: 0-based
  Time Complexity: O(V + E)
  Space Complexity: O(V + E)

  Input:
    - n: number of nodes
    - m: number of directed edges
    - adj: original graph
    - adjr: reversed graph

  Output:
    - comp[i]: component ID of node i
    - order[]: nodes in reverse post-order (1st DFS)
    - nc: is the number of unique comp values
*/

vector<vector<int>> adj, adjr;
vector<bool> vis;
vector<int> order, comp;

// First DFS: post-order on original graph
void dfs(int v) {
  vis[v] = true;
  for (int u : adj[v]) {
    if (!vis[u])
      dfs(u);
  }
  order.push_back(v); // Record post-order
}

// Second DFS: assign component IDs on reversed graph
void dfsr(int v, int k) {
  vis[v] = true;
  comp[v] = k;
  for (int u : adjr[v]) {
    if (!vis[u])
      dfsr(u, k);
  }
}

void solve() {
  int n, m;
  cin >> n >> m;
  adj.assign(n, vector<int>());
  adjr.assign(n, vector<int>());
  comp.resize(n);
  // Read edges and build both original and reversed graphs
  for (int i = 0; i < m; i++) {
    int a, b;
    cin >> a >> b;
    a--; b--;
    adj[a].push_back(b);
    adjr[b].push_back(a);
  }
  // First pass: DFS on original graph to get order
  vis.assign(n, false);
  order.clear();
  for (int i = 0; i < n; i++) {
    if (!vis[i]) dfs(i);
  }
  // Second pass: DFS on reversed graph using reverse post-order
  vis.assign(n, false);
  int nc = 0;
  for (int i = n - 1; i >= 0; i--) {
    int v = order[i];
    if (!vis[v]) {
      dfsr(v, nc++);
    }
```

```
  }
  // comp[i] now holds the component ID for node i (0-based)
  // nc = number of strongly connected components
}
```

7.17 Stoer-Wagner

```
/*
                                    Stoer - Wagner
  ------------------------------------------------------------------------------

  Solves the minimum cut problem in undirected weighted graphs with non-
      negative weights.

  Time Complexity: O(V ^ 3)
  Space Complexity: O(V ^ 2)
*/

mt19937 rnd(chrono::steady_clock::now().time_since_epoch().count());
struct StoerWagner {
  int n;
  long long G[N][N], dis[N];
  int idx[N];
  bool vis[N];
  const long long inf = 1e18;
  StoerWagner() {}
  StoerWagner(int _n) {
    n = _n;
    memset(G, 0, sizeof G);
  }
  void add_edge(int u, int v, long long w) { //undirected edge, multiple
      edges are merged into one edge
    if (u != v) {
      G[u][v] += w;
      G[v][u] += w;
    }
  }
  long long solve() {
    long long ans = inf;
    for (int i = 0; i < n; ++ i) idx[i] = i + 1;
    shuffle(idx, idx + n, rnd);
    while (n > 1) {
      int t = 1, s = 0;
      for (int i = 1; i < n; ++ i) {
        dis[idx[i]] = G[idx[0]][idx[i]];
        if (dis[idx[i]] > dis[idx[t]]) t = i;
      }
      memset(vis, 0, sizeof vis);
      vis[idx[0]] = true;
      for (int i = 1; i < n; ++ i) {
        if (i == n - 1) {
          if (ans > dis[idx[t]]) ans = dis[idx[t]]; //idx[s] - idx[t] is
              ---------- in two halves of the  mincut
          if (ans == 0) return 0;
          for (int j = 0; j < n; ++ j) {
            G[idx[s]][idx[j]] += G[idx[j]][idx[t]];
            G[idx[j]][idx[s]] += G[idx[j]][idx[t]];
          }
          idx[t] = idx[-- n];
        }
        vis[idx[t]] = true;
        s = t;
        t = -1;
        for (int j = 1; j < n; ++ j) {
          if (!vis[idx[j]]) {
            dis[idx[j]] += G[idx[s]][idx[j]];
            if (t == -1 || dis[idx[t]] < dis[idx[j]]) t = j;
          }
        }
      }
    }
    return ans;
  }
};

int32_t main() {
  ios_base::sync_with_stdio(0);
  cin.tie(0);
  int t, cs = 0;
  cin >> t;
  while (t--) {
    int n, m;
    cin >> n >> m;
    StoerWagner st(n);
    while (m--) {
      int u, v, w;
```

```
      cin >> u >> v >> w;
      st.add_edge(u, v, w);
    }
    cout << "Case #" << ++cs << ": " << st.solve() << '\n';
  }
  return 0;
}
```

8 Linear Algebra

8.1 Gaussian Elimination

```
/*
  Gaussian elimination
  ----------------------------------
  Utilized for solving linear systems of equations.

  If the system is modulo 2, use GaussianEliminationModulo instead.

  Time Complexity: O(n ^ 3)
*/


const double eps = 1e-9;
int Gauss(vector<vector<double>> a, vector<double> &ans) {
  int n = (int)a.size(), m = (int)a[0].size() - 1;
  vector<int> pos(m, -1);
  double det = 1; int rank = 0;
  for(int col = 0, row = 0; col < m && row < n; ++col) {
    int mx = row;
    for(int i = row; i < n; i++) if(fabs(a[i][col]) > fabs(a[mx][col]))
        mx = i;
    if(fabs(a[mx][col]) < eps) {det = 0; continue;}
    for(int i = col; i <= m; i++) swap(a[row][i], a[mx][i]);
    if (row != mx) det = -det;
    det *= a[row][col];
    pos[col] = row;
    for(int i = 0; i < n; i++) {
      if(i != row && fabs(a[i][col]) > eps) {
        double c = a[i][col] / a[row][col];
        for(int j = col; j <= m; j++) a[i][j] -= a[row][j] * c;
      }
    }
    ++row; ++rank;
  }
  ans.assign(m, 0);
  for(int i = 0; i < m; i++) {
    if(pos[i] != -1) ans[i] = a[pos[i]][m] / a[pos[i]][i];
  }
  for(int i = 0; i < n; i++) {
    double sum = 0;
    for(int j = 0; j < m; j++) sum += ans[j] * a[i][j];
    if(fabs(sum - a[i][m]) > eps) return -1; //no solution
  }
  for(int i = 0; i < m; i++) if(pos[i] == -1) return 2; //infinte
      solutions
  return 1; //unique solution
}
int main() {
  int n, m; cin >> n >> m;
  vector< vector<double> > v(n);
  for(int i = 0; i < n; i++) {
    for(int j = 0; j <= m; j++) {
      double x; cin >> x; v[i].push_back(x);
    }
  }
  vector<double> ans;
  int k = Gauss(v, ans);
  if(k) for(int i = 0; i < n; i++) cout << fixed << setprecision(5) <<
      ans[i] << ' ';
  else cout << "no solution\n";
  return 0;
}
```

8.2 Gaussian Elimination Modulo

```
/*
  Gaussian elimination Modulo 2
  ----------------------------------
  Utilized for solving linear systems of equations modulo2.

  Time Complexity: O(n ^ 3)
*/

//n = number of equations, m = number of variables
int Gauss(int n, int m, vector<bitset<N>> a, bitset<N> &ans) {
```

```
  //reversing for lexocgraphically largest solution
  for (int i = 0; i < n; i++) {
      bitset<N> tmp;
      for (int j = 0; j < m; j++) tmp[j] = a[i][m - j - 1];
      tmp[m] = a[i][m];
      a[i] = tmp;
  }
  int rank = 0, det = 1;
  vector<int> pos(N, -1);
  for(int col = 0, row = 0; col < m && row < n; ++col) {
    int mx = row;
    for(int i = row; i < n; ++i) if(a[i][col]) { mx = i; break; }
    if(!a[mx][col]) { det = 0; continue; }
    swap(a[mx], a[row]);
    if(row != mx) det = det == 0 ? 0 : 1;
    det &= a[row][col];
    pos[col] = row;
    //forward elimination
    for(int i = row + 1; i < n; ++i) if (i != row && a[i][col]) a[i] ^=
        a[row];
    ++row, ++rank;
  }
  ans.reset();
  //backward substituition
  for (int i = m - 1; i >= 0; i--) {
      if (pos[i] == -1) ans[i] = true;
      else {
        int k = pos[i];
        for (int j = i + 1; j < m; j++) if (a[k][j]) ans[i] = ans[i] ^
            ans[j];
        ans[i] = ans[i] ^ a[k][m];
      }
  }
  for(int i = rank; i < n; ++i) if(a[i][m]) return -1; //no solution
  //reversing again beacuse we reversed earlier
  bitset<N> tmp;
  for (int j = 0; j < m; j++) tmp[j] = ans[m - j - 1];
  ans = tmp;
  int free_var = 0;
  for(int i = 0; i < m; ++i) if(pos[i] == -1) free_var++;
  return free_var; //has solution
}
string read() {
   string t;
   if(!(cin >> t)) return "";
   if(t.empty() || t == "and") return "";
   while(t[0] == '(') t.erase(t.begin());
   while(t.back() == ')') t.pop_back();
   return t;
}
bool is_var(string t) { return t.size() > 0 && t[0] == 'x'; }
int get_var(string t) { return atoi(t.substr(1).c_str()) - 1; }
int32_t main() {
  ios_base::sync_with_stdio(0);
  cin.tie(0);
  int n, m; cin >> n >> m;
  vector<bitset<N>> bs(n, bitset<N>(0));
  for(int i = 0; i < n; i++) {
     string s;
     bool eq = 1;
     while((s = read()).size() > 0) {
        if(is_var(s)) {
          int x = get_var(s);
          bs[i][x] = bs[i][x] ^ 1;
        }
        else if(s == "not") eq ^= 1;
     }
     bs[i][m] = eq;
  }
  bitset<N> ans;
  int ok = Gauss(n, m, bs, ans);
  if (ok == -1) cout << "impossible\n";
  else {
    for (int i = 0; i < m; i++) cout << "FT"[ans[i]]; cout << '\n';
  }
   return 0;
}
//https://codeforces.com/gym/101908/problem/M
```

8.3 Simplex

```
/*
Parametric Self-Dual Simplex method
Solve a canonical LP:
  min or max. c x
  s.t. A x <= b
```

```
    x >= 0
*/
#include <bits/stdc++.h>
using namespace std;
const double eps = 1e-9, oo = numeric_limits<double>::infinity();

typedef vector<double> vec;
typedef vector<vec> mat;

pair<vec, double> simplexMethodPD(const mat &A, const vec &b, const vec
    &c, bool mini = true){
  int n = c.size(), m = b.size();
  mat T(m + 1, vec(n + m + 1));
  vector<int> base(n + m), row(m);

  for(int j = 0; j < m; ++j){
    for(int i = 0; i < n; ++i)
      T[j][i] = A[j][i];
    row[j] = n + j;
    T[j][n + j] = 1;
    base[n + j] = 1;
    T[j][n + m] = b[j];
  }

  for(int i = 0; i < n; ++i)
    T[m][i] = c[i] * (mini ? 1 : -1);

  while(true){
    int p = 0, q = 0;
    for(int i = 0; i < n + m; ++i)
      if(T[m][i] <= T[m][p])
        p = i;

    for(int j = 0; j < m; ++j)
      if(T[j][n + m] <= T[q][n + m])
        q = j;

    double t = min(T[m][p], T[q][n + m]);

    if(t >= -eps){
      vec x(n);
      for(int i = 0; i < m; ++i)
        if(row[i] < n) x[row[i]] = T[i][n + m];
      return {x, T[m][n + m] * (mini ? -1 : 1)}; // optimal
    }

    if(t < T[q][n + m]){
      // tight on c -> primal update
      for(int j = 0; j < m; ++j)
        if(T[j][p] >= eps)
          if(T[j][p] * (T[q][n + m] - t) >= T[q][p] * (T[j][n + m] - t))
            q = j;

      if(T[q][p] <= eps)
        return {vec(n), oo * (mini ? 1 : -1)}; // primal infeasible
    }else{
      // tight on b -> dual update
      for(int i = 0; i < n + m + 1; ++i)
        T[q][i] = -T[q][i];

      for(int i = 0; i < n + m; ++i)
        if(T[q][i] >= eps)
          if(T[q][i] * (T[m][p] - t) >= T[q][p] * (T[m][i] - t))
            p = i;

      if(T[q][p] <= eps)
        return {vec(n), oo * (mini ? -1 : 1)}; // dual infeasible
    }

    for(int i = 0; i < m + n + 1; ++i)
      if(i != p) T[q][i] /= T[q][p];

    T[q][p] = 1; // pivot(q, p)
    base[p] = 1;
    base[row[q]] = 0;
    row[q] = p;

    for(int j = 0; j < m + 1; ++j){
      if(j != q){
        double alpha = T[j][p];
        for(int i = 0; i < n + m + 1; ++i)
          T[j][i] -= T[q][i] * alpha;
      }
    }
  }

```

```
  return {vec(n), oo};
}

int main(){
  int m, n;
  bool mini = true;
  cout << "Numero de restricciones: ";
  cin >> m;
  cout << "Numero de incognitas: ";
  cin >> n;
  mat A(m, vec(n));
  vec b(m), c(n);
  for(int i = 0; i < m; ++i){
    cout << "Restriccion #" << (i + 1) << ": ";
    for(int j = 0; j < n; ++j){
      cin >> A[i][j];
    }
    cin >> b[i];
  }
  cout << "[0]Max o [1]Min?: ";
  cin >> mini;
  cout << "Coeficientes de" << (mini ? "min" : "max") << " z: ";
  for(int i = 0; i < n; ++i){
    cin >> c[i];
  }
  cout.precision(6);
  auto ans = simplexMethodPD(A, b, c, mini);
  cout << (mini ? "Min" : "Max") << " z = " << ans.second << ", cuando: \n";
  for(int i = 0; i < ans.first.size(); ++i){
    cout << "x_" << (i + 1) << " = " << ans.first[i] << "\n";
  }
  return 0;
}
```

9 Math

9.1 BinPow

```
ll binpow(ll a, ll b){
    ll r=1;
    while(b){
        if(b%2)
            r=(r*a)%MOD;
        a=(a*a)%MOD;
        b/=2;
    }
    return r;
}

ll divide(ll a, ll b){
    return ((a%MOD)*binpow(b, MOD-2))%MOD;
}
void inverses(long long p) {
  inv[MAXN] = exp(fac[MAXN], p - 2, p);
  for (int i = MAXN; i >= 1; i--) { inv[i - 1] = inv[i] * i % p; }
}
```

9.2 Combination Rank

```
// Compute nCr (binomial coefficient)
long long nCr(int n, int r) {
    if (r > n) return 0;
    if (r == 0 || r == n) return 1;
    long long res = 1;
    for (int i = 1; i <= r; ++i) {
        res = res * (n - i + 1) / i;
    }
    return res;
}

// Compute 0-indexed lexicographic rank of a combination
long long combinationRank0(const vector<int>& comb) {
    long long rank = 0;
    for (int i = 0; i < (int)comb.size(); ++i) {
        rank += nCr(comb[i] - 1, i + 1);
    }
    return rank; // 0-indexed
}

// Given n, r, and rank (0-indexed), reconstruct the combination
vector<int> combinationUnrank0(int n, int r, long long rank) {
    vector<int> comb;
    int x = 1;

    for (int i = r; i >= 1; --i) {
```

```
        while (nCr(n - x, i) > rank) {
            x++;
        }
        comb.push_back(x);
        rank -= nCr(n - x, i);
        x++;
    }

    return comb;
}
```

9.3 Chinese Remainder Theorem

```
ll euclid(ll a, ll b, ll &x, ll &y) {
  if (!b) return x = 1, y = 0, a;
  ll d = euclid(b, a % b, y, x);
  return y -= a/b * x, d;
}

ll crt(ll a, ll m, ll b, ll n) {
  if (n > m) swap(a, b), swap(m, n);
  ll x, y, g = euclid(m, n, x, y);
  if((a - b) % g != 0)return -1;
  x = (b - a) % n * x % n / g * m + a;
  return x < 0 ? x + m*n/g : x;
}
```

9.4 Diophantine

If one solution is (x_0, y_0) all solutions can be obtained by $x = x_0 + k * \frac{b}{gcd(a,b)}$ and $y = y_0 - k * \frac{a}{gcd(a,b)}$.

```
int gcd(int a, int b, int& x, int& y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    int x1, y1;
    int d = gcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}

bool find_any_solution(int a, int b, int c, int &x0, int &y0, int &g) {
    g = gcd(abs(a), abs(b), x0, y0);
    if (c % g) {
        return false;
    }

    x0 *= c / g;
    y0 *= c / g;
    if (a < 0) x0 = -x0;
    if (b < 0) y0 = -y0;
    return true;
}



//n variables
vector<ll> find_any_solution(vector<ll> a, ll c) {
  int n = a.size();
  vector<ll> x;
  bool all_zero = true;
  for (int i = 0; i < n; i++) {
    all_zero &= a[i] == 0;
  }
  if (all_zero) {
    if (c) return {};
    x.assign(n, 0);
    return x;
  }
  ll g = 0;
  for (int i = 0; i < n; i++) {
    g = __gcd(g, a[i]);
  }
  if (c % g != 0) return {};
  if (n == 1) {
    return {c / a[0]};
  }
  vector<ll> suf_gcd(n);
  suf_gcd[n - 1] = a[n - 1];
  for (int i = n - 2; i >= 0; i--) {
    suf_gcd[i] = __gcd(suf_gcd[i + 1], a[i]);
  }
  ll cur = c;
```

```
  for (int i = 0; i + 1 < n; i++) {
    ll x0, y0, g;
    // solve for a[i] * x + suf_gcd[i + 1] * (y / suf_gcd[i + 1]) = cur
    bool ok = find_any_solution(a[i], suf_gcd[i + 1], cur, x0, y0, g);
    assert(ok);
    {
      // trying to minimize x0 in case x0 becomes big
      // it is needed for this problem, not needed in general
      ll shift = abs(suf_gcd[i + 1] / g);
      x0 = (x0 % shift + shift) % shift;
    }
    x.push_back(x0);

    // now solve for the next suffix
    cur -= a[i] * x0;
  }
  x.push_back(a[n - 1] == 0 ? 0 : cur / a[n - 1]);
  return x;
}
```

9.5 Discrete Logarithm

Finds discrete logarithm in $O(\sqrt{(m)})$.

```
// Returns minimum x for which a ^ x % m = b % m, a and m are coprime.
int solve(int a, int b, int m) {
    a %= m, b %= m;
    int n = sqrt(m) + 1;

    int an = 1;
    for (int i = 0; i < n; ++i)
        an = (an * 1ll * a) % m;

    unordered_map<int, int> vals;
    for (int q = 0, cur = b; q <= n; ++q) {
        vals[cur] = q;
        cur = (cur * 1ll * a) % m;
    }

    for (int p = 1, cur = 1; p <= n; ++p) {
        cur = (cur * 1ll * an) % m;
        if (vals.count(cur)) {
            int ans = n * p - vals[cur];
            return ans;
        }
    }
    return -1;
}

// Returns minimum x for which a ^ x % m = b % m.
int solve(int a, int b, int m) {
    a %= m, b %= m;
    int k = 1, add = 0, g;
    while ((g = gcd(a, m)) > 1) {
        if (b == k)
            return add;
        if (b % g)
            return -1;
        b /= g, m /= g, ++add;
        k = (k * 1ll * a / g) % m;
    }

    int n = sqrt(m) + 1;
    int an = 1;
    for (int i = 0; i < n; ++i)
        an = (an * 1ll * a) % m;

    unordered_map<int, int> vals;
    for (int q = 0, cur = b; q <= n; ++q) {
        vals[cur] = q;
        cur = (cur * 1ll * a) % m;
    }

    for (int p = 1, cur = k; p <= n; ++p) {
        cur = (cur * 1ll * an) % m;
        if (vals.count(cur)) {
            int ans = n * p - vals[cur] + add;
            return ans;
        }
    }
    return -1;
}
```

9.6 Divisors

```cpp
ll numberOfDivisors(ll num) {
  ll total = 1;
  for (int i = 2; (ll)i * i <= num; i++) {
    if (num % i == 0){
      int e = 0;
      do{
        e++;
        num /= i;
      } while (num % i == 0);
      total *= e + 1;
    }
  }
  if (num > 1) total *= 2;
  return total;
}

ll SumOfDivisors(ll num) {
  ll total = 1;
  for (int i = 2; (ll)i * i <= num; i++) {
    if (num % i == 0) {
      int e = 0;
      do {
        e++;
        num /= i;
      } while (num % i == 0);
      ll sum = 0, pow = 1;
      do {
        sum += pow;
        pow *= i;
      } while (e-- > 0);
      total *= sum;
    }
  }
  if (num > 1) total *= (1 + num);
  return total;
}
```

9.7 Euler Totient (Phi)

```cpp
//counts coprimes to each number from 1 to n
vector<int> phi1(int n) {
  vector<int> phi(n + 1);
  for (int i = 0; i <= n; i++) phi[i] = i;
  for (int i = 2; i <= n; i++) {
      if (phi[i] == i) {
        for (int j = i; j <= n; j += i)
          phi[j] -= phi[j] / i;
      }
  }
  return phi;
}
```

9.8 Fibonacci

```cpp
// Fibonacci in O(log n)
void fib(ll n, ll&x, ll&y){
  if(n==0){
    x = 0;
    y = 1;
    return ;
  }
  if(n&1){
    fib(n-1, y, x);
    y=(y+x)%MOD;
  } else {
    ll a, b;
    fib(n>>1, a, b);
    y = (a*a+b*b)%MOD;
    x = (a*b + a*(b-a+MOD))%MOD;
  }
}
// Usage
ll x, y;
fib(10, x, y);
cout << x << "␣" << y << endl;
// This will output 55 89
```

9.9 Fraction: Exact Rational Arithmetic

C++17 (GCC/Clang). `Fraction(n,d)` represents n/d; `Fraction(n)` is the integer n. Every result is reduced by gcd, its denominator is positive, and zero is stored as 0/1. Supports `+`, `-`, `*`, `/`, unary minus, compound assignments and all six comparisons. Comparisons use exact cross products, so the class also works with `sort`, `set` and `map` keys.

Numerator and denominator are signed 64-bit integers. Arithmetic and cross products use `__int128` intermediates, then reduce before checking that the result fits; this is not arbitrary precision. An unrepresentable reduced result throws

`overflow_error`; a zero constructor denominator throws `invalid_argument`, and division by a zero fraction throws `domain_error`. Access parts with `num()` and `den()`. Output uses `n/d`; an explicit `long double` conversion is available when an approximation is wanted. Each arithmetic operation costs $O(\log M)$ for gcd reduction; comparisons use a constant number of integer operations. Here M bounds the intermediate integer magnitudes.

```cpp
#pragma once
#include <bits/stdc++.h>

class Fraction {
    using ll = long long;
    using i128 = __int128_t;
    ll n_, d_;

    void set(i128 n, i128 d) {
        if (d == 0) throw std::invalid_argument("zero denominator");
        if (d < 0) { n = -n; d = -d; }
        i128 a = n < 0 ? -n : n, b = d;
        while (b) { i128 r = a % b; a = b; b = r; }
        n /= a;
        d /= a;
        if (n < std::numeric_limits<ll>::min() ||
            n > std::numeric_limits<ll>::max() ||
            d > std::numeric_limits<ll>::max())
            throw std::overflow_error("fraction overflow");
        n_ = (ll)n;
        d_ = (ll)d;
    }
    static Fraction make(i128 n, i128 d) {
        Fraction f;
        f.set(n, d);
        return f;
    }

public:
    // Always reduced, positive denominator; zero is 0/1.
    Fraction(ll n = 0, ll d = 1) { set(n, d); }
    ll num() const { return n_; }
    ll den() const { return d_; }

    friend Fraction operator+(const Fraction& a, const Fraction& b) {
        return make((i128)a.n_ * b.d_ + (i128)b.n_ * a.d_,
                    (i128)a.d_ * b.d_);
    }
    friend Fraction operator-(const Fraction& a, const Fraction& b) {
        return make((i128)a.n_ * b.d_ - (i128)b.n_ * a.d_,
                    (i128)a.d_ * b.d_);
    }
    friend Fraction operator*(const Fraction& a, const Fraction& b) {
        return make((i128)a.n_ * b.n_, (i128)a.d_ * b.d_);
    }
    friend Fraction operator/(const Fraction& a, const Fraction& b) {
        if (b.n_ == 0) throw std::domain_error("divide by zero");
        return make((i128)a.n_ * b.d_, (i128)a.d_ * b.n_);
    }
    Fraction operator-() const { return make(-(i128)n_, d_); }
    Fraction& operator+=(const Fraction& b) { return *this = *this + b; }
    Fraction& operator-=(const Fraction& b) { return *this = *this - b; }
    Fraction& operator*=(const Fraction& b) { return *this = *this * b; }
    Fraction& operator/=(const Fraction& b) { return *this = *this / b; }

    friend bool operator==(const Fraction& a, const Fraction& b) {
        return a.n_ == b.n_ && a.d_ == b.d_;
    }
    friend bool operator<(const Fraction& a, const Fraction& b) {
        return (i128)a.n_ * b.d_ < (i128)b.n_ * a.d_;
    }
    friend bool operator!=(const Fraction& a, const Fraction& b) {
        return !(a == b);
    }
    friend bool operator>(const Fraction& a, const Fraction& b) {
        return b < a;
    }
    friend bool operator<=(const Fraction& a, const Fraction& b) {
        return !(b < a);
    }
    friend bool operator>=(const Fraction& a, const Fraction& b) {
        return !(a < b);
    }
    explicit operator long double() const {
        return (long double)n_ / d_;
```

```
    }
    friend std::ostream& operator<<(std::ostream& out,
                                    const Fraction& f) {
        return out << f.n_ << '/' << f.d_;
    }
};
```

```
Fraction a(2, 4), b(1, 3); // a is reduced to 1/2.
std::cout << a << '\n';                  // 1/2
std::cout << a + b << ' ' << a - b << '\n'; // 5/6 1/6
std::cout << a * b << ' ' << a / b << '\n'; // 1/6 3/2
std::cout << (b < a) << '\n';             // 1
Fraction c(3, -6);                       // -1/2
c += 1;                                  // 1/2
```

9.10 Matrix Exponentiation

```
struct Mat {
  int n, m;
  vector<vector<int>> a;
  Mat() { }
  Mat(int _n, int _m) {n = _n; m = _m; a.assign(n, vector<int>(m, 0)); }
  Mat(vector< vector<int> > v) { n = v.size(); m = n ? v[0].size() : 0;
      a = v; }
  inline void make_unit() {
    assert(n == m);
    for (int i = 0; i < n; i++)  {
      for (int j = 0; j < n; j++) a[i][j] = i == j;
    }
  }
  inline Mat operator + (const Mat &b) {
    assert(n == b.n && m == b.m);
    Mat ans = Mat(n, m);
    for(int i = 0; i < n; i++) {
      for(int j = 0; j < m; j++) {
        ans.a[i][j] = (a[i][j] + b.a[i][j]) % mod;
      }
    }
    return ans;
  }
  inline Mat operator - (const Mat &b) {
    assert(n == b.n && m == b.m);
    Mat ans = Mat(n, m);
    for(int i = 0; i < n; i++) {
      for(int j = 0; j < m; j++) {
        ans.a[i][j] = (a[i][j] - b.a[i][j] + mod) % mod;
      }
    }
    return ans;
  }
  inline Mat operator * (const Mat &b) {
    assert(m == b.n);
    Mat ans = Mat(n, b.m);
    for(int i = 0; i < n; i++) {
      for(int j = 0; j < b.m; j++) {
        for(int k = 0; k < m; k++) {
          ans.a[i][j] = (ans.a[i][j] + 1LL * a[i][k] * b.a[k][j] % mod)
              % mod;
        }
      }
    }
    return ans;
  }
  inline Mat pow(long long k) {
    assert(n == m);
    Mat ans(n, n), t = a; ans.make_unit();
    while (k) {
      if (k & 1) ans = ans * t;
      t = t * t;
      k >>= 1;
    }
    return ans;
  }
  inline Mat& operator += (const Mat& b) { return *this = (*this) + b; }
  inline Mat& operator -= (const Mat& b) { return *this = (*this) - b; }
  inline Mat& operator *= (const Mat& b) { return *this = (*this) * b; }
  inline bool operator == (const Mat& b) { return a == b.a; }
  inline bool operator != (const Mat& b) { return a != b.a; }
};

// Usage
// Mat a(n, n);
// Mat b(n, n);
// Mat c = a * b;
// Mat d = a + b;
```

```
// Mat e = a - b;
// Mat f = a.pow(k);
// a.a[i][j] = x;
```

9.11 Miller Rabin Deterministic

```
using u64 = uint64_t;
using u128 = __uint128_t;

u64 binpower(u64 base, u64 e, u64 mod) {
    u64 result = 1;
    base %= mod;
    while (e) {
        if (e & 1)
            result = (u128)result * base % mod;
        base = (u128)base * base % mod;
        e >>= 1;
    }
    return result;
}

bool check_composite(u64 n, u64 a, u64 d, int s) {
    u64 x = binpower(a, d, n);
    if (x == 1 || x == n - 1)
        return false;
    for (int r = 1; r < s; r++) {
        x = (u128)x * x % n;
        if (x == n - 1)
            return false;
    }
    return true;
};


bool MillerRabin(ll n) {
    if (n < 2)
        return false;

    int r = 0;
    ll d = n - 1;
    while ((d & 1) == 0) {
        d >>= 1;
        r++;
    }

    for (int a : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}) {
        if (n == a)
            return true;
        if (check_composite(n, a, d, r))
            return false;
    }
    return true;
}
```

9.12 Mobius

```
int mob[N];
void mobius() {
  mob[1] = 1;
  for (int i = 2; i < N; i++){
    mob[i]--;
    for (int j = i + i; j < N; j += i) {
      mob[j] -= mob[i];
    }
  }
}
```

9.13 Permutation Rank

```
// Compute factorial (up to reasonable limits)
long long fact(int n) {
    static vector<long long> f(21, -1); // cache up to 20!
    if (n <= 1) return 1;
    if (f[n] != -1) return f[n];
    return f[n] = n * fact(n - 1);
}

// Compute 0-indexed lexicographic rank of a permutation
long long permutationRank0(const vector<int>& perm) {
    int n = perm.size();
    long long rank = 0;
    vector<int> used(n + 1, 0);

    for (int i = 0; i < n; ++i) {
        int smaller_unused = 0;
        for (int j = 1; j < perm[i]; ++j) {
            if (!used[j]) smaller_unused++;
```

```
        }
        rank += smaller_unused * fact(n - i - 1);
        used[perm[i]] = 1;
    }
    return rank;
}

// Given n and rank (0-indexed), reconstruct the permutation
vector<int> permutationUnrank0(int n, long long rank) {
    vector<int> elems(n);
    iota(elems.begin(), elems.end(), 1); // {1, 2, 3, ..., n}
    vector<int> perm;

    for (int i = n; i >= 1; --i) {
        long long f = fact(i - 1);
        int idx = rank / f;
        rank %= f;
        perm.push_back(elems[idx]);
        elems.erase(elems.begin() + idx);
    }

    return perm;
}
```

9.14 Prefix Sum Phi

```
vector<ll> sieve(kMaxV + 1,0);
vector<ll> phi(kMaxV + 1,0);

void primes()
{
  phi[1]=1;
  vector<ll> pr;
  for(int i=2;i<kMaxV;i++){
    if(sieve[i]==0){
      sieve[i]=i;
      pr.pb(i);
      phi[i]=i-1;
    }
    for(auto p:pr){
      if(p>sieve[i]||i*p>=kMaxV)break;
      sieve[i*p]=p;
      phi[i*p]=(p==sieve[i]?p:p-1)*phi[i];
    }
  }
  for(int i=1;i<kMaxV;i++){
    phi[i]+=phi[i-1];
    phi[i]%=MOD;
  }
}

map<ll,ll> m;
ll PHI(ll a){
  if(a<kMaxV)return phi[a];
  if(m.count(a))return m[a];
  // if(a<3)return 1;
  m[a]=((((a%MOD)*((a+1)%MOD))%MOD)*inverse(2));
  m[a]%=MOD;
  long long i=2;
  while(i<=a){
      long long j=a/i;
      j=a/j;
      m[a]+=MOD;
      m[a]-=((j-i+1)*PHI(a/i))%MOD;
      m[a]%=MOD;
      i=j+1;
  }
  m[a]%=MOD;
  return m[a];
}
```

9.15 Sieve

```
const int kMaxV = 1e6;

int sieve[kMaxV + 1];

//stores some prime (not necessarly the minimum one)
void primes()
{
  for (int i = 4; i <= kMaxV; i += 2)
    sieve[i] = 2;
  for (int i = 3; i <= kMaxV / i; i += 2)
  {
    if (sieve[i])
      continue;
```

```
    for (int j = i * i; j <= kMaxV; j += i)
      sieve[j] = i;
    }
}

vector<int> PrimeFactors(int x)
{
  if (x == 1)
    return {};

  unordered_set<int> primes;
  while (sieve[x])
  {
    primes.insert(sieve[x]);
    x /= sieve[x];
  }
  primes.insert(x);
  return {primes.begin(), primes.end()};
}
```

9.16 Identities

$$C_n = \frac{2(2n-1)}{n+1} C_{n-1}$$
$$C_n = \frac{1}{n+1}\binom{2n}{n}$$
$$C_n \sim \frac{4^n}{n^{3/2}\sqrt{\pi}}$$
$\sigma(n) = O(\log(\log(n)))$ (number of divisors of n)
$$F_{2n+1} = F_n^2 + F_{n+1}^2$$
$$F_{2n} = F_{n+1}^2 - F_{n-1}^2$$
$$\sum_{i=1}^{n} F_i = F_{n+2} - 1$$
$$F_{n+i}F_{n+j} - F_nF_{n+i+j} = (-1)^n F_iF_j$$
(Möbius Inv. Formula) $\mu(p^k) = [k=0] - [k=1]$ Let $g(n) = \sum_{d|n} f(d)$, then $f(n) = \sum_{d|n} g(d)\mu\left(\frac{n}{d}\right)$.
(Dirichlet Convolution) Let f, g be arithmetic functions, then $(f * g)(n) = \sum_{d|n} f(d)g\left(\frac{n}{d}\right)$. If f, g are multiplicative, then so is $f * g$.
$$n = \sum_{d|n} \phi(d)$$
Lucas' Theorem: $\binom{m}{n} \equiv \prod_{i=0}^{k} \binom{m_i}{n_i} \pmod p$ where $m = \sum_{i=0}^{k} m_i p^i$ and $n = \sum_{i=0}^{k} n_i p^i$.

9.17 Burnside's Lemma

Dado un grupo G de permutaciones y un conjunto X de n elementos, el número de órbitas de X bajo la acción de G es igual al promedio del número de puntos fijos de las permutaciones en G.
Formalmente, el número de órbitas es $\frac{1}{|G|}\sum_{g\in G} f(g)$ donde $f(g)$ es el número de puntos fijos de g.
Ejemplo: Dado un collar con n cuentas y 2 colores, el número de collares diferentes que se pueden formar es $\frac{1}{n}\sum_{i=0}^{n} f(i)$ donde $f(i)$ es el número de collares que quedan fijos bajo una rotación de i posiciones.
Para contar el número de collares que quedan fijos bajo una rotación de i posiciones, se puede usar la fórmula $f(i) = 2^{\gcd(i,n)}$.
Para un collar de n cuentas y k colores, el número de collares diferentes que se pueden formar es $\frac{1}{n}\sum_{i=0}^{n} k^{\gcd(i,n)}$
Ejemplo: Dado un cubo con 6 caras y k colores, el número de cubos diferentes que se pueden formar es $\frac{1}{24}\sum_{i=0}^{24} f(i)$ donde $f(i)$ es el número de cubos que quedan fijos bajo una rotación de i posiciones. Esta formula es igual a $\frac{1}{24}(n^6 + 3n^4 + 12n^3 + 8n^2)$

9.18 Recursion

Sea $f(n) = \sum_{i=1}^{k} a_i f(n-i)$ entonces podemos considerar la matriz:

$$\begin{bmatrix} f(n) \\ f(n-1) \\ \vdots \\ f(n-k+1) \end{bmatrix} = \begin{bmatrix} a_1 & a_2 & \cdots & a_{k-1} & a_k \\ 1 & 0 & \cdots & 0 & 0 \\ 0 & 1 & \cdots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & 1 & 0 \end{bmatrix} \begin{bmatrix} f(n-1) \\ f(n-2) \\ \vdots \\ f(n-k) \end{bmatrix}$$

De aqui podemos calcular $f(n)$ con exponenciación de matrices.

$$\begin{bmatrix} f(n) \\ f(n-1) \\ \vdots \\ f(n-k+1) \end{bmatrix} = \begin{bmatrix} a_1 & a_2 & \cdots & a_{k-1} & a_k \\ 1 & 0 & \cdots & 0 & 0 \\ 0 & 1 & \cdots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & 1 & 0 \end{bmatrix}^{n-k} \begin{bmatrix} f(k) \\ f(k-1) \\ \vdots \\ f(1) \end{bmatrix}$$

9.19 Theorems

Koeing's Theorem: La cardinalidad del emparejamiento maximo de una grafica bipartita es igual al minimum vertex cover.
Hall's Theorem: Una grafica bipartita G tiene un emparejamiento que cubre todos los nodos de G si y solo si para todo subconjunto S de nodos de G, el número de vecinos de S es mayor o igual a $|S|$.
Kuratowski's Theorem: Una grafica es plana si y solo si no contiene un subgrafo homeomorfo a $K_{3,3}$ o K_5.

9.20 Sums

$$c^a + c^{a+1} + \cdots + c^b = \frac{c^{b+1} - c^a}{c-1}, c \neq 1$$

$$1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}$$
$$1^2 + 2^2 + 3^2 + \cdots + n^2 = \frac{n(2n+1)(n+1)}{6}$$
$$1^3 + 2^3 + 3^3 + \cdots + n^3 = \frac{n^2(n+1)^2}{4}$$
$$1^4 + 2^4 + 3^4 + \cdots + n^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$

9.21 Catalan numbers

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1} = \frac{(2n)!}{(n+1)!n!}$$

$$C_0 = 1,\ C_{n+1} = \frac{2(2n+1)}{n+2}C_n,\ C_{n+1} = \sum C_i C_{n-i}$$

$$C_n = 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, \ldots$$

- sub-diagonal monotone paths in an $n \times n$ grid.
- strings with n pairs of parenthesis, correctly nested. If prefix is given, number of ways is $\binom{n}{remaining_c losed}$-$\binom{n}{remaining_c losed+1}$.
- binary trees with with $n+1$ leaves (0 or 2 children).
- ordered trees with $n+1$ vertices.
- ways a convex polygon with $n+2$ sides can be cut into triangles by connecting vertices with straight lines.
- permutations of $[n]$ with no 3-term increasing subseq.

9.22 Cayley's formula

Number of labeled trees of n vertices is n^{n-2}. Number of rooted forest of n vertices is $(n+1)^{n-1}$.

9.23 Geometric series

Infinite
$a + ar + ar^2 + ar^3 + ... + \sum_{k=0}^{\infty} ar^k$
Sum=$\frac{a}{1-r}$
Finite
$a + ar + ar^2 + ar^3 + ... + \sum_{k=0}^{n} ar^k$
Sum=$\frac{a(1-r^{n+1})}{1-r}$

9.24 Estimates For Divisors

$\sum_{d|n} d = O(n \log\log n)$.
The number of divisors of n is at most around 100 for $n < 5e4$, 500 for $n < 1e7$, 2000 for $n < 1e10$, 200 000 for $n < 1e19$.

9.25 Sum of divisors

$$\sum d|n = \frac{{p_1}^{\alpha_1+1}-1}{p_1-1} + \frac{{p_2}^{\alpha_2+1}-1}{p_2-1} + ... + \frac{{p_n}^{\alpha_n+1}-1}{p_n-1}$$

9.26 Pythagorean Triplets

The Pythagorean triples are uniquely generated by

$$a = k \cdot (m^2 - n^2),\ \ b = k \cdot (2mn),\ \ c = k \cdot (m^2 + n^2),$$

with $m > n > 0$, $k > 0$, $m \perp n$, and either m or n even.

9.27 Derangements

Permutations of a set such that none of the elements appear in their original position.

$$D(n) = (n-1)(D(n-1) + D(n-2)) = nD(n-1) + (-1)^n = \left\lfloor \frac{n!}{e} \right\rceil$$

10 Game Theory

10.1 Sprague-Grundy theorem

`https://codeforces.com/blog/entry/66040` Dado un juego con pilas p_1, p_2, ..., p_n sea $g(p)$ el nimber de la pila p, entonces el nimber del juego es $g(p_1) \oplus g(p_2) \oplus ... \oplus g(p_n)$. Para calcular el nimber de una pila, se puede usar la fórmula $g(r) = mex(\{g(r_1), g(r_2), ..., g(r_k)\})$ donde $r_1, r_2, ..., r_k$ son los posibles estados a los que se puede llegar desde r y $g(r) = 0$ si r es un estado perdedor.

11 Fórmulas y notas

11.1 Números de Stirling del primer tipo

$\begin{bmatrix} n \\ k \end{bmatrix}$ representa el número de permutaciones de n elementos en exactamente k ciclos disjuntos.

$$\begin{bmatrix} 0 \\ 0 \end{bmatrix} = 1$$

$$\begin{bmatrix} 0 \\ n \end{bmatrix} = \begin{bmatrix} n \\ 0 \end{bmatrix} = 0 \qquad , \quad n > 0$$

$$\begin{bmatrix} n \\ k \end{bmatrix} = (n-1)\begin{bmatrix} n-1 \\ k \end{bmatrix} + \begin{bmatrix} n-1 \\ k-1 \end{bmatrix} \qquad , \quad k > 0$$

$$\sum_{k=0}^{n} \begin{bmatrix} n \\ k \end{bmatrix} = n!$$

$$\sum_{k=0}^{\infty} \begin{bmatrix} n \\ k \end{bmatrix} x^k = \prod_{k=0}^{n-1} (x+k)$$

11.2 Números de Stirling del segundo tipo

$\begin{Bmatrix} n \\ k \end{Bmatrix}$ representa el número de formas de particionar un conjunto de n objetos distinguibles en k subconjuntos no vacíos.

$$\begin{Bmatrix} 0 \\ 0 \end{Bmatrix} = 1$$

$$\begin{Bmatrix} 0 \\ n \end{Bmatrix} = \begin{Bmatrix} n \\ 0 \end{Bmatrix} = 0 \qquad , \quad n > 0$$

$$\begin{Bmatrix} n \\ k \end{Bmatrix} = k\begin{Bmatrix} n-1 \\ k \end{Bmatrix} + \begin{Bmatrix} n-1 \\ k-1 \end{Bmatrix} \qquad , \quad k > 0$$

$$= \sum_{j=0}^{k} \frac{j^n}{j!} \cdot \frac{(-1)^{k-j}}{(k-j)!}$$

11.3 Números de Euler

$\left\langle\!\left\langle \begin{smallmatrix} n \\ k \end{smallmatrix} \right\rangle\!\right\rangle$ representa el número de permutaciones de 1 a n en donde exactamente k números son mayores que el número anterior, es decir, cuántas permutaciones tienen k "ascensos".

$$\left\langle\!\left\langle \begin{matrix} 1 \\ 0 \end{matrix} \right\rangle\!\right\rangle = 1$$

$$\left\langle\!\left\langle \begin{matrix} n \\ k \end{matrix} \right\rangle\!\right\rangle = (n-k)\left\langle\!\left\langle \begin{matrix} n-1 \\ k-1 \end{matrix} \right\rangle\!\right\rangle + (k+1)\left\langle\!\left\langle \begin{matrix} n-1 \\ k \end{matrix} \right\rangle\!\right\rangle \qquad , \quad n \geq 2$$

$$= \sum_{j=0}^{k} (-1)^j \binom{n+1}{j} (k+1-j)^n$$

$$\sum_{k=0}^{n-1} \left\langle\!\left\langle \begin{matrix} n \\ k \end{matrix} \right\rangle\!\right\rangle = n!$$

11.4 Números de Catalan

$$C_0 = 1$$

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \sum_{j=0}^{n-1} C_j C_{n-1-j}$$

$$\sum_{n=0}^{\infty} C_n x^n = \frac{1 - \sqrt{1-4x}}{2x}$$

11.5 Números de Bell

B_n representa el número de formas de particionar un conjunto de n elementos.

$$B_n = \sum_{k=0}^{n} \begin{Bmatrix} n \\ k \end{Bmatrix} = \sum_{k=0}^{n-1} \binom{n-1}{k} B_k$$

$$\sum_{n=0}^{\infty} \frac{B_n}{n!} x^n = e^{e^x - 1}$$

11.6 Números de Bernoulli

$${B_0}^+ = 1$$

$${B_n}^+ = 1 - \sum_{k=0}^{n-1} \binom{n}{k} \frac{{B_k}^+}{n-k+1}$$

$$\sum_{n=0}^{\infty} \frac{{B_n}^+ x^n}{n!} = \frac{x}{1-e^{-x}} = \frac{1}{\frac{1}{1!} - \frac{x}{2!} + \frac{x^2}{3!} - \frac{x^3}{4!} + \cdots}$$

11.7 Fórmula de Faulhaber

$$S_p(n) = \sum_{k=1}^{n} k^p = \frac{1}{p+1} \sum_{k=0}^{p} \binom{p+1}{k} B_k{}^+ n^{p+1-k}$$

11.8 Función Beta

$$B(x,y) = \frac{\Gamma(x)\Gamma(y)}{\Gamma(x+y)} = 2\int_0^{\pi/2} \sin^{2x-1}(\theta)\cos^{2x-1}(\theta)d\theta$$
$$= \int_0^1 t^{x-1}(1-t)^{y-1}dt = \int_0^\infty \frac{t^{x-1}}{(1+t)^{x+y}}dt$$

11.9 Función zeta de Riemann

La siguiente fórmula converge rápido para valores pequeños de n ($n \approx 20$):

$$\zeta(s) \approx \frac{1}{d_0(1-2^{1-s})} \sum_{k=1}^{n} \frac{(-1)^{k-1} d_k}{k^s}$$
$$d_k = \sum_{j=k}^{n} \frac{4^j}{n+j} \binom{n+j}{2j}$$

11.10 Funciones generadoras

$$\sum_{n=0}^{\infty} \left(\sum_{k=0}^{n} a_k \right) x^n = \frac{1}{1-x} \sum_{n=0}^{\infty} a_n x^n$$

$$\sum_{n=0}^{\infty} \binom{n+k-1}{k-1} x^n = \frac{1}{(1-x)^k}$$

$$\sum_{n=0}^{\infty} p_n x^n = \frac{1}{\prod_{k=1}^{\infty}(1-x^k)} = \frac{1}{\sum_{n=-\infty}^{\infty} (-1)^n x^{\frac{1}{2}n(3n+1)}}$$

$$\sum_{p=0}^{\infty} \frac{S_p(n)}{p!} x^p = \frac{e^{x(n+1)} - e^x}{e^x - 1}$$

$$\sum_{n=0}^{\infty} n^k x^n = \frac{\sum_{i=0}^{k-1} \left\langle\!\left\langle {k \atop i} \right\rangle\!\right\rangle x^{i+1}}{(1-x)^{k+1}} \quad , \quad k \geq 1$$

Sean $a_1, a_2, \ldots, a_n$ números complejos. Sean $p_m = \sum_{i=1}^{n} a_i^m$ y s_m el m-ésimo polinomio elemental simétrico de $a_1, a_2, \ldots, a_n$. Entonces se cumple que $xS'(x) + P(x)S(x) = 0$, donde $P(x) = \sum_{m=1}^{\infty} p_m x^m$ y $S(x) = \prod_{i=1}^{n}(1 - a_i x) = \sum_{m=0}^{n} (-1)^m s_m x^m$.

11.11 Números armónicos

$$H_n = \sum_{k=1}^{n} \frac{1}{k} \approx \ln(n) + \gamma + \frac{1}{2n} - \frac{1}{12n^2}$$
$$\gamma \approx 0.57721566490153286060606512$$

11.12 Aproximación de Stirling

$$\ln(n!) \approx n\ln(n) - n + \frac{1}{2}\ln(2\pi n)$$
$$\#\text{ de dígitos de } n! = 1 + \left\lfloor n \log\left(\frac{n}{e}\right) + \frac{1}{2}\log(2\pi n) \right\rfloor \quad (n \geq 30)$$

11.13 Ternas pitagóricas

- Una terna de enteros positivos (a, b, c) es pitagórica si $a^2 + b^2 = c^2$. Además es primitiva si $\gcd(a, b, c) = 1$.
- Generador de ternas primitivas:

$$a = m^2 - n^2$$
$$b = 2mn$$
$$c = m^2 + n^2$$

  donde $n \geq 1$, $m > n$, $\gcd(m, n) = 1$ y m, n tienen distinta paridad.

- Árbol de ternas pitagóricas primitivas: al multiplicar cualquiera de estas matrices:

$$\begin{pmatrix} 1 & -2 & 2 \\ 2 & -1 & 2 \\ 2 & -2 & 3 \end{pmatrix} \quad , \quad \begin{pmatrix} -1 & 2 & 2 \\ -2 & 1 & 2 \\ -2 & 2 & 3 \end{pmatrix} \quad , \quad \begin{pmatrix} 1 & 2 & 2 \\ 2 & 1 & 2 \\ 2 & 2 & 3 \end{pmatrix}$$

  por una terna primitiva $\mathbf{v^T}$, obtenemos otra terna primitiva diferente. En particular, si empezamos con $\mathbf{v} = (3, 4, 5)$, podremos generar todas las ternas primitivas.

11.14 Árbol de Stern–Brocot

Todos los racionales positivos se pueden representar como un árbol binario de búsqueda completo infinito con raíz $\frac{1}{1}$.

- Dado un racional $q = [a_0; a_1, a_2, \ldots, a_k]$ donde $a_k \neq 1$, sus hijos serán $[a_0; a_1, a_2, \ldots, a_k+1]$ y $[a_0; a_1, a_2, \ldots, a_k-1, 2]$, y su padre será $[a_0; a_1, a_2, \ldots, a_k-1]$.
- Para hallar el camino de la raíz $\frac{1}{1}$ a un racional q, se usa búsqueda binaria iniciando con $L = \frac{0}{1}$ y $R = \frac{1}{0}$. Para hallar M se supone que $L = \frac{a}{b}$ y $R = \frac{c}{d}$, entonces $M = \frac{a+c}{b+d}$.

11.15 Combinatoria

- Principio de las casillas: al colocar n objetos en k lugares hay al menos $\lceil \frac{n}{k} \rceil$ objetos en un mismo lugar.
- Número de funciones: sean A y B conjuntos con $m = |A|$ y $n = |B|$. Sea $f : A \to B$:
  - Si $m \leq n$, entonces hay $m!\binom{n}{m}$ funciones inyectivas f.
  - Si $m = n$, entonces hay $n!$ funciones biyectivas f.
  - Si $m \geq n$, entonces hay $n!\left\{{m \atop n}\right\}$ funciones suprayectivas f.
- Barras y estrellas: ¿cuántas soluciones en los enteros no negativos tiene la ecuación $\sum_{i=1}^{k} x_i = n$? Tiene $\binom{n+k-1}{k-1}$ soluciones.
- ¿Cuántas soluciones en los enteros positivos tiene la ecuación $\sum_{i=1}^{k} x_i = n$? Tiene $\binom{n-1}{k-1}$ soluciones.
- Desordenamientos: $a_0 = 1$, $a_1 = 0$, $a_n = (n-1)(a_{n-1} + a_{n-2}) = na_{n-1} + (-1)^n$.
- Sea $f(x)$ una función. Sea $g_n(x) = xg'_{n-1}(x)$ con $g_0(x) = f(x)$. Entonces $g_n(x) = \sum_{k=0}^{n} \left\{{n \atop k}\right\} x^k f^{(k)}(x)$.
- Supongamos que tenemos $m+1$ puntos: $(0, y_0)$, $(1, y_1)$, …, (m, y_m). Entonces el polinomio $P(x)$ de grado m que pasa por todos ellos es:

$$P(x) = \left[\prod_{i=0}^{m}(x-i)\right](-1)^m \sum_{i=0}^{m} \frac{y_i(-1)^i}{(x-i)i!(m-i)!}$$

- Sea $a_0, a_1, \ldots$ una recurrencia lineal homogénea de grado d dada por $a_n = \sum_{i=1}^{d} b_i a_{n-i}$ para $n \geq d$ con términos iniciales $a_0, a_1, \ldots, a_{d-1}$. Sean $A(x)$ y $B(x)$ las funciones generadoras de las sucesiones a_n y b_n respectivamente, entonces se cumple que $A(x) = \frac{A_0(x)}{1 - B(x)}$, donde $A_0(x) = \sum_{i=0}^{d-1} \left[a_i - \sum_{j=0}^{i-1} a_j b_{i-j}\right] x^i$.
- Si queremos obtener otra recurrencia c_n tal que $c_n = a_{kn}$, las raíces del polinomio característico de c_n se obtienen al elevar todas las raíces del polinomio característico de a_n a la k-ésima potencia; y sus términos iniciales serán $a_0, a_k, \ldots, a_{k(d-1)}$.

11.16 Grafos

- Sea d_n el número de grafos con n vértices etiquetados: $d_n = 2^{\binom{n}{2}}$.
- Sea c_n el número de grafos conexos con n vértices etiquetados. Tenemos la recurrencia: $c_1 = 1$ y $d_n = \sum_{k=1}^{n} \binom{n-1}{k-1} c_k d_{n-k}$. También se cumple, usando funciones generadoras exponenciales, que $C(x) = 1 + \ln(D(x))$.
- Sea t_n el número de torneos fuertemente conexos en n nodos etiquetados. Tenemos la recurrencia $t_1 = 1$ y $d_n = \sum_{k=1}^{n} \binom{n}{k} t_k d_{n-k}$. Usando funciones generadoras exponenciales, tenemos que $T(x) = 1 - \frac{1}{D(x)}$.
- Número de spanning trees en un grafo completo con n vértices etiquetados: n^{n-2}.
- Número de bosques etiquetados con n vértices y k componentes conexas: kn^{n-k-1}.
- Para un grafo no dirigido simple G con n vértices etiquetados de 1 a n, sea $Q = D - A$, donde D es la matriz diagonal de los grados de cada nodo de G y A es la matriz de adyacencia de G. Entonces el número de spanning trees de G es igual a cualquier cofactor de Q.
- Sea G un grafo. Se define al polinomio $P_G(x)$ como el polinomio cromático de G, en donde $P_G(k)$ nos dice cuántas k-coloraciones de los vértices admite G. Ejemplos comunes:
  - Grafo completo de n nodos: $P(x) = x(x-1)(x-2)\ldots(x-(n-1))$
  - Grafo vacío de n nodos: $P(x) = x^n$
  - Árbol de n nodos: $P(x) = x(x-1)^{n-1}$
  - Ciclo de n nodos: $P(x) = (x-1)^n + (-1)^n(x-1)$

11.17 Teoría de números

$$(f * e)(n) = f(n)$$
$$(\varphi * \mathbf{1})(n) = n$$
$$(\mu * \mathbf{1})(n) = e(n)$$
$$\varphi(n^k) = n^{k-1}\varphi(n)$$
$$\sum_{\substack{k=1 \\ \gcd(k,n)=1}}^{n} k = \frac{n\varphi(n)}{2} \quad , \quad n \geq 2$$
$$\sum_{k=1}^{n} \operatorname{lcm}(k,n) = \frac{n}{2} + \frac{n}{2}\sum_{d|n} d\varphi(d) = \frac{n}{2} + \frac{n}{2}\prod_{p^a|n} \frac{p^{2a+1}+1}{p+1}$$
$$\sum_{k=1}^{n} \gcd(k,n) = \sum_{d|n} d\varphi\left(\frac{n}{d}\right) = \prod_{p^a|n} p^{a-1}(1+(a+1)(p-1))$$

- Lifting the exponent: sea p un primo, x, y enteros y n un entero positivo tal que $p \mid x-y$ pero $p \nmid x$ ni $p \nmid y$. Entonces:
  - Si p es impar: $v_p(x^n - y^n) = v_p(x-y) + v_p(n)$
  - Si $p = 2$ y n es par: $v_p(x^n - y^n) = v_p(x-y) + v_p(n) + v_p(x+y) - 1$

  donde $v_p(n)$ es el exponente de p en la factorización en primos de n.

- Suma de dos cuadrados: sea $\chi_4(n)$ una función multiplicativa igual a 1 si $n \equiv 1 \mod 4$, -1 si $n \equiv 3 \mod 4$ y cero en otro caso. Entonces, el número de soluciones enteras (a,b) de la ecuación $a^2 + b^2 = n$ es $4(\chi_4 * 1)(n) = 4\sum_{d|n} \chi_4(d)$.

- Teorema de Lucas:

$$\binom{m}{n} \equiv \prod_{i=0}^{k} \binom{m_i}{k_i} \pmod{p}$$
$$m = \sum_{i=0}^{k} m_i p^i \quad , \quad n = \sum_{i=0}^{k} n_i p^i$$
$$0 \leq m_i, n_i < p$$

- Sean $a, b, c \in \mathbb{Z}$ con $a \neq 0$ y $b \neq 0$. La ecuación $ax + by = c$ tiene como soluciones:

$$x = \frac{x_0 c - bk}{d}$$
$$y = \frac{y_0 c + ak}{d}$$

para toda $k \in \mathbb{Z}$ si y solo si $d|c$, donde $ax_0 + by_0 = \gcd(a,b) = d$ (Euclides extendido). Si a y b tienen el mismo signo, hay exactamente $\max\left(\left\lfloor \frac{x_0 c}{|b|} \right\rfloor + \left\lfloor \frac{y_0 c}{|a|} \right\rfloor + 1, 0\right)$ soluciones no negativas. Si tienen el signo distinto, hay infinitas soluciones no negativas.

- Dada una función aritmética f con $f(1) \neq 0$, existe otra función aritmética g tal que $(f * g)(n) = e(n)$, dada por:

$$g(1) = \frac{1}{f(1)}$$
$$g(n) = -\frac{1}{f(1)} \sum_{d|n, d<n} f\left(\frac{n}{d}\right) g(d) \quad , \quad n > 1$$

- Sean $h(n) = \sum_{k=1}^{n} f\left(\left\lfloor \frac{n}{k} \right\rfloor\right) g(k)$, $G(n) = \sum_{k=1}^{n} g(k)$ y $m = \lfloor \sqrt{n} \rfloor$, entonces:

$$h(n) = \sum_{k=1}^{\lfloor n/m \rfloor} f\left(\left\lfloor \frac{n}{k} \right\rfloor\right) g(k) + \sum_{k=1}^{m-1} \left(G\left(\left\lfloor \frac{n}{k} \right\rfloor\right) - G\left(\left\lfloor \frac{n}{k+1} \right\rfloor\right)\right) f(k)$$

- Sean $F(n) = \sum_{k=1}^{n} f(k)$, $G(n) = \sum_{k=1}^{n} g(k)$, $h(n) = (f * g)(n) = \sum_{d|n} f(d) g\left(\frac{n}{d}\right)$ y $H(n) = \sum_{k=1}^{n} h(k)$, entonces:

$$H(n) = \sum_{k=1}^{n} f(k) G\left(\left\lfloor \frac{n}{k} \right\rfloor\right)$$

- Sean $\Phi_p(n) = \sum_{k=1}^{n} k^p \varphi(k)$ y $M_p(n) = \sum_{k=1}^{n} k^p \mu(k)$. Aplicando lo anterior, podemos calcular $\Phi_p(n)$ y $M_p(n)$ con complejidad $O(n^{2/3})$ si precalculamos con fuerza bruta los primeros $\lfloor n^{2/3} \rfloor$ valores, y para los demás, usamos las siguientes recurrencias (DP con `map`):

$$\Phi_p(n) = S_{p+1}(n) - \sum_{k=2}^{\lfloor n/m \rfloor} k^p \Phi_p\left(\left\lfloor \frac{n}{k} \right\rfloor\right) - \sum_{k=1}^{m-1} \left(S_p\left(\left\lfloor \frac{n}{k} \right\rfloor\right) - S_p\left(\left\lfloor \frac{n}{k+1} \right\rfloor\right)\right) \Phi_p(k)$$
$$M_p(n) = 1 - \sum_{k=2}^{\lfloor n/m \rfloor} k^p M_p\left(\left\lfloor \frac{n}{k} \right\rfloor\right) - \sum_{k=1}^{m-1} \left(S_p\left(\left\lfloor \frac{n}{k} \right\rfloor\right) - S_p\left(\left\lfloor \frac{n}{k+1} \right\rfloor\right)\right) M_p(k)$$

- En general, si queremos hallar $F(n)$ y existe una función mágica $g(n)$ tal que $G(n)$ y $H(n)$ se puedan calcular en $O(1)$, entonces:

$$F(n) = \frac{1}{g(1)}\left[H(n) - \sum_{k=2}^{\lfloor n/m \rfloor} g(k)F\left(\left\lfloor \frac{n}{k} \right\rfloor\right) - \sum_{k=1}^{m-1}\left(G\left(\left\lfloor \frac{n}{k} \right\rfloor\right) - G\left(\left\lfloor \frac{n}{k+1} \right\rfloor\right)\right)F(k)\right]$$

11.18 Primos

10^2+1, 10^3+9, 10^4+7, 10^5+3, 10^6+3, 10^7+19, 10^8+7, 10^9+7, $10^{10}+19$, $10^{11}+3$, $10^{12}+39$, $10^{13}+37$, $10^{14}+31$, $10^{15}+37$, $10^{16}+61$, $10^{17}+3$, $10^{18}+3$.
10^2-3, 10^3-3, 10^4-27, 10^5-9, 10^6-17, 10^7-9, 10^8-11, 10^9-63, $10^{10}-33$, $10^{11}-23$, $10^{12}-11$, $10^{13}-29$, $10^{14}-27$, $10^{15}-11$, $10^{16}-63$, $10^{17}-3$, $10^{18}-11$.

11.19 Números primos de Mersenne

Números primos de la forma $M_p = 2^p - 1$ con p primo. Todos los números perfectos pares son de la forma $2^{p-1}M_p$ y viceversa.
Los primeros 47 valores de p son: 2, 3, 5, 7, 13, 17, 19, 31, 61, 89, 107, 127, 521, 607, 1279, 2203, 2281, 3217, 4253, 4423, 9689, 9941, 11213, 19937, 21701, 23209, 44497, 86243, 110503, 132049, 216091, 756839, 859433, 1257787, 1398269, 2976221, 3021377, 6972593, 13466917, 20996011, 24036583, 25964951, 30402457, 32582657, 37156667, 42643801, 43112609.

11.20 Números primos de Fermat

Números primos de la forma $F_p = 2^{2^p} + 1$, solo se conocen cinco: 3, 5, 17, 257, 65537.
Un polígono de n lados es construible si y solo si n es el producto de algunas potencias de dos y distintos primos de Fermat.

11.21 Binomial Inversion

Sean f y g funciones definidas en los enteros no negativos. Entonces:

$$g(n) = \sum_{k=0}^{n} \binom{n}{k} f(k) \quad \Longleftrightarrow \quad f(n) = \sum_{k=0}^{n} (-1)^{n-k} \binom{n}{k} g(k)$$

11.22 Generating Functions

An ordinary generating function (OGF) is $F(x) = \sum_{n\ge 0} a_n x^n$; its answer is $a_n = [x^n]F(x)$. An exponential generating function (EGF) is $F(x) = \sum_{n\ge 0} a_n x^n/n!$; its answer is $a_n = n![x^n]F(x)$. An extra variable marks another quantity: $[x^n u^k]$ selects size n and statistic k.
Use formal power series: when only degrees through N matter, truncate after x^N. Positive weights make the infinite products well-defined coefficient by coefficient. Unless specified otherwise, $n \ge 0$, $k \ge 1$, $w_i > 0$, and impossible counting binomials are zero. Square roots below use the branch with constant term 1. The numbered entries correspond to the 81 formulas in this catalog.

Basic counting (OGF)

Every interpretation here is $[x^n]F(x)$.

1	$\dfrac{1}{1-x}$	1: one possibility for each nonnegative size.
2	$\dfrac{1}{1-cx}$	c^n: length-n strings over an alphabet of c symbols.
3	$(1+x)^m$	$\binom{m}{n}$: choose n of m distinct objects.
4	$\dfrac{1}{(1-x)^k}$	$\binom{n+k-1}{k-1}$: solutions of $a_1 + \cdots + a_k = n$ with $a_i \ge 0$.
5	$\dfrac{x^k}{(1-x)^k}$	$\binom{n-1}{k-1}$: solutions with every $a_i \ge 1$.
6	$\dfrac{x}{(1-x)^2}$	n: size-weighted counts and sums.
7	$\dfrac{x(1+x)}{(1-x)^3}$	n^2.
8	$\dfrac{x(1+4x+x^2)}{(1-x)^4}$	n^3.
9	$\dfrac{x^r}{(1-x)^{r+1}}$	$\binom{n}{r}$, for fixed $r \ge 0$. Multiply the GF by $r!$ to obtain the falling factorial $n(n-1)\cdots(n-r+1)$.
10	$\dfrac{-\log(1-x)}{1-x}$	Harmonic numbers $H_n = \sum_{j=1}^{n} 1/j$, with $H_0 = 0$.

More generally, for $r \ge 1$,

$$\sum_{n\ge 0} n^r x^n = \left(x\frac{d}{dx}\right)^r \frac{1}{1-x}.$$

Binomial generating-function identities.

Subset sum, knapsack, and compositions (OGF)

Let $S \subseteq \{1, 2, \ldots\}$ be allowed part sizes and $A(x) = \sum_{s\in S} x^s$. Products combine independent choices; powers and geometric series assemble ordered sequences.

11	$\prod_{i=1}^{m}(1+x^{w_i})$	Subsets of indexed items with total weight n. Equal-weight items remain distinct.
12	$\prod_{i=1}^{m}\dfrac{1}{1-x^{w_i}}$	Coin combinations totaling n, with unlimited copies of each type. Order does not matter.
13	$\prod_{i=1}^{m}\dfrac{1-x^{(b_i+1)w_i}}{1-x^{w_i}}$	Combinations totaling n, with at most b_i copies of type i, where $b_i \ge 0$.
14	$\prod_{i=1}^{m}(1+ux^{w_i})$	$[x^n u^k]$: subsets of weight n with exactly k selected items.
15	$\left(\dfrac{1-x^{b+1}}{1-x}\right)^k$	Ordered tuples $a_1 + \cdots + a_k = n$ with $0 \le a_i \le b$, where $b \ge 0$.
16	$A(x)^k$	Ordered sequences of exactly k allowed parts totaling n.
17	$\dfrac{1}{1-A(x)}$	Ordered sequences of any number of allowed parts, including the empty sequence.
18	$\dfrac{1-x}{1-2x}$	Compositions into arbitrary positive parts: coefficient 1 at $n = 0$, and 2^{n-1} for $n \ge 1$.

For coins $1, 3, 4$, compare

$$\frac{1}{(1-x)(1-x^3)(1-x^4)} \quad \text{unordered combinations,}$$
$$\frac{1}{1-x-x^3-x^4} \quad \text{ordered sequences.}$$

Entry 15 gives the bounded-sum formula

$$\sum_{j=0}^{k}(-1)^j\binom{k}{j}\binom{n-j(b+1)+k-1}{k-1}.$$

Product and sequence constructions.

Integer partitions (OGF)

Parts are positive and their order is ignored: $1+3$ and $3+1$ are the same partition. An infinite product need only include part sizes through N when computing through degree N.

19	$\prod_{j\ge 1}\dfrac{1}{1-x^j}$	$p(n)$: unrestricted integer partitions of n.

20 $\prod_{j\geq 1}(1+x^j)$ — Partitions of n into distinct parts.

21 $\prod_{j\geq 1}\frac{1}{1-x^{2j-1}}$ — Partitions into odd parts. This is the same series as entry 20.

22 $\prod_{j=1}^{k}\frac{1}{1-x^j}$ — Partitions with every part at most k; equivalently, partitions into at most k parts.

23 $\frac{x^k}{\prod_{j=1}^{k}(1-x^j)}$ — Partitions into exactly k positive parts.

24 $\prod_{j\geq 1}\frac{1}{1-ux^j}$ — $[x^n u^k]$: partitions of n into exactly k parts.

25 $\prod_{i=1}^{k}\frac{1-x^{m+i}}{1-x^i}$ — Gaussian binomial $\begin{bmatrix}k+m\\k\end{bmatrix}_x$: partitions fitting a $k\times m$ rectangle, with at most k parts, each at most m.

Unrestricted partitions; restricted partitions.

Recurrences, lattice paths, and ordered trees (OGF)

Write $C(x)=(1-\sqrt{1-4x})/(2x)$, with $C(0)=1$.

26 $\frac{x}{1-x-x^2}$ — Fibonacci numbers F_n, where $F_0=0$ and $F_1=1$.

27 $\frac{1}{1-x-x^2-\cdots-x^d}$ — Compositions with allowed parts $1,\ldots,d$, including the empty composition.

28 $\frac{1+x}{1-x-x^2}$ — Binary strings of length n without adjacent 1s: F_{n+2}.

29 $\frac{1}{\sqrt{1-4x}}$ — $\binom{2n}{n}$: walks with n up-steps and n down-steps, with no nonnegativity restriction.

30 $C(x)=\frac{1-\sqrt{1-4x}}{2x}$ — Catalan numbers $C_n=\binom{2n}{n}/(n+1)$: balanced parentheses with n pairs, Dyck paths, or full ordered binary trees with n internal nodes.

31 $C(x)^k$ — $\frac{k}{2n+k}\binom{2n+k}{n}$: ordered forests of k plane rooted trees with n total edges.

32 $T(x)=1+xT(x)^r$ — For $r\geq 2$ and $T(0)=1$, full ordered r-ary trees with n internal nodes: $\binom{rn}{n}/((r-1)n+1)$.

33 $\frac{1-x-\sqrt{1-2x-3x^2}}{2x^2}$ — Motzkin paths with n up, down, or horizontal steps, staying nonnegative and ending at height 0.

34 $\frac{1-x-\sqrt{1-6x+x^2}}{2x}$ — Large Schröder paths from $(0,0)$ to (n,n) with east, north, and diagonal steps, staying at or below $y=x$.

35 $\frac{1}{1-x-y}$ — $[x^a y^b]=\binom{a+b}{a}$: grid paths using east and north steps.

36 $\frac{1}{1-x-y-xy}$ — $[x^a y^b]$: Delannoy paths to (a,b) using east, north, and diagonal $(1,1)$ steps.

37 $\frac{1}{\sqrt{1-6x+x^2}}$ — Central Delannoy numbers: unrestricted east/north/diagonal paths from $(0,0)$ to (n,n).

38 $\left((I-xM)^{-1}\right)_{ij}$ — Length-n walks from i to j, where M_{ij} counts edges $i\to j$. Useful for automata and forbidden-pattern problems.

Entry 38 follows from $(I-xM)^{-1}=I+xM+x^2M^2+\cdots$. For $a_n=\sum_{i=1}^{d}c_i a_{n-i}$ when $n\geq d$, including initial conditions gives

$$F(x)=\frac{\sum_{j=0}^{d-1}\left(a_j-\sum_{i=1}^{j}c_i a_{j-i}\right)x^j}{1-\sum_{i=1}^{d}c_i x^i}.$$

Catalan numbers; other lattice-path numbers.

Inversions in permutations (polynomials)

Here the exponent of q is the inversion count, not the length. For fixed length m, let $[j]_q=1+q+\cdots+q^{j-1}$ and $[m]_q!=\prod_{j=1}^{m}[j]_q$.

39 $[m]_q!$ — $[q^r]$: permutations of m distinct elements with exactly r inversions. The same polynomial counts by major index.

40 $\frac{[m]_q!}{[m_1]_q!\cdots[m_t]_q!}$ — $[q^r]$: distinct permutations of a multiset with multiplicities $m_1,\ldots,m_t$, summing to m, with exactly r inversions.

Permutation-statistic generating functions.

Labeled permutations and cycles (EGF)

Read $a_n=n![x^n]F(x)$ throughout the EGF entries. For an extra marker, use $n![x^n u^k]F(x,u)$.

41 e^x — 1: one unordered set on a given collection of n labels.

42 e^{kx} — k^n: functions from n labeled objects to k labeled boxes; empty boxes are allowed.

43 $\dfrac{1}{1-x}$ — $n!$: all permutations. Contrast the OGF interpretation in entry 1.

44 $-\log(1-x)$ — $(n-1)!$ for $n \geq 1$: permutations consisting of one cycle. The constant coefficient is zero.

45 $\dfrac{e^{-x}}{1-x}$ — Derangements D_n: permutations without fixed points.

46 $\dfrac{e^{(u-1)x}}{1-x}$ — $n![x^n u^k]$: permutations with exactly k fixed points, equal to $\binom{n}{k} D_{n-k}$.

47 $e^{x+x^2/2}$ — Involutions: permutations with only cycles of lengths 1 and 2.

48 $e^{x^2/2}$ — Perfect matchings on n labeled vertices. For $n = 2m$, $(2m)!/(2^m m!)$; zero for odd n.

49 $\exp\left(\sum_{j \in S} \dfrac{x^j}{j}\right)$ — Permutations with every cycle length in $S \subseteq \{1, 2, \ldots\}$.

50 $(1-x)^{-u}$ — $n![x^n u^k] = \left[{n \atop k}\right]$: permutations with exactly k cycles, the unsigned first-kind Stirling numbers.

Equivalently, for fixed k cycles,

$$\sum_{n \geq 0} \left[{n \atop k}\right] \frac{x^n}{n!} = \frac{(-\log(1-x))^k}{k!}.$$

Permutation families; Stirling generating functions.

Set partitions and other labeled families (EGF)

Objects are labeled; blocks are unordered unless stated otherwise. The extraction rule is still $a_n = n![x^n]F(x)$.

51 $\dfrac{(e^x-1)^k}{k!}$ — $S(n,k)$: partitions of n labeled objects into k nonempty, unordered blocks (second-kind Stirling numbers).

52 $(e^x-1)^k$ — $k!S(n,k)$: surjections onto k labeled boxes, all nonempty.

53 e^{e^x-1} — Bell numbers: all set partitions of n labeled objects.

54 $\dfrac{1}{2-e^x}$ — Ordered Bell/Fubini numbers: partitions whose nonempty blocks are placed in an order.

55 $\exp\left(\sum_{j \in S} \dfrac{x^j}{j!}\right)$ — Set partitions with every block size in $S \subseteq \{1, 2, \ldots\}$.

56 $\left(\sum_{j \in S} \dfrac{x^j}{j!}\right)^k$ — Assignments of n labeled objects to k labeled boxes, with every occupancy in $S \subseteq \{0, 1, \ldots\}$. Include 0 to allow empty boxes.

57 $\sec x + \tan x$ — Alternating permutations $a_1 < a_2 > a_3 < \cdots$.

58 $\dfrac{1-u}{e^{(u-1)x}-u}$ — $n![x^n u^k]$: permutations with k descents (Eulerian numbers). At $u = 1$, take the limit $1/(1-x)$.

59 $\dfrac{x}{e^x-1}$ — Bernoulli numbers, with $B_1 = -1/2$. Used for sums of powers, rather than counting objects.

Two specializations of entry 55 are e^{e^x-1-x} for no singleton blocks, and $e^{x^r/r!}$ for all blocks of size r. Bell numbers; Stirling numbers; alternating permutations; Eulerian numbers; Bernoulli numbers.

Labeled trees and graphs (EGF)

Let $T(x)$ be the unique formal series with $T = xe^T$, $T(0) = 0$, and let $U(x) = T(x) - T(x)^2/2$. Continue to extract $n![x^n]$.

60 $T(x) = xe^{T(x)}$ — Rooted labeled trees: n^{n-1} for $n \geq 1$.

61 $U(x) = T(x) - \dfrac{T(x)^2}{2}$ — Unrooted labeled trees: n^{n-2} for $n \geq 2$, and one tree for $n = 1$.

62 $e^{T(x)}$ — Labeled forests with one chosen root per component: $(n+1)^{n-1}$ for $n \geq 1$, and one empty forest.

63 $\dfrac{1}{1-T(x)}$ — Functions from n vertices to themselves, equivalently directed functional graphs: n^n for $n \geq 1$, and one empty object.

64 $\log H(x)$ — Connected simple undirected graphs on n labeled vertices, where $H(x)$ is the EGF for all such graphs.

In entry 64,

$$H(x) = \sum_{n \geq 0} 2^{\binom{n}{2}} \frac{x^n}{n!}.$$

Every graph is an unordered set of its connected components, so $H(x) = \exp(\text{connected-graph EGF})$. The identity is formal; analytic convergence is unnecessary. Tree families; labeled constructions.

Divisor sums (Lambert series / OGF)

These series have no constant term; extract $[x^n]$ for $n \geq 1$. Here $d(n)$ counts divisors, $\sigma_r(n) = \sum_{d|n} d^r$, φ is Euler's totient, and μ is the Möbius function.

65	$\sum_{j\ge1} f(j)\dfrac{x^j}{1-x^j}$	$\sum_{d\mid n} f(d)$: a divisor sum of any arithmetic function f.
66	$\sum_{j\ge1} \dfrac{x^j}{1-x^j}$	$d(n)$: the number of positive divisors of n.
67	$\sum_{j\ge1} j^r\dfrac{x^j}{1-x^j}$	$\sigma_r(n)$: the sum of r-th powers of divisors. For $r=1$, the usual sum of divisors.
68	$\sum_{j\ge1} \varphi(j)\dfrac{x^j}{1-x^j}$	n: this series equals $x/(1-x)^2$, expressing $\sum_{d\mid n}\varphi(d)=n$.
69	$\sum_{j\ge1} \mu(j)\dfrac{x^j}{1-x^j}$	1 if $n=1$, otherwise 0: this series equals x, the basic Möbius-inversion identity.

Lambert-series identities.

Probability generating functions (PGF)

For $P(x)=\mathbb{E}[x^X]$, $[x^n]P(x)=\Pr(X=n)$. Products combine independent random variables. Write $q=1-p$; for geometric and negative-binomial distributions, $0<p\le1$.

70	$q+px$	Bernoulli: one trial with success probability p.
71	$(q+px)^m$	Binomial: successes in m independent trials, with probability $\binom{m}{n}p^nq^{m-n}$.
72	$\dfrac{p}{1-qx}$	Geometric: failures before the first success, with probability pq^n. Support starts at 0.
73	$\left(\dfrac{p}{1-qx}\right)^k$	Failures before the k-th success: probability $\binom{n+k-1}{k-1}p^kq^n$.
74	$e^{\lambda(x-1)}$	Poisson, $\lambda\ge0$: probability $e^{-\lambda}\lambda^n/n!$.
75	$\left(\dfrac{x+x^2+\cdots+x^d}{d}\right)^k$	The sum of k independent fair d-sided dice. Removing the factors $1/d$ gives the counting GF.

$$\mathbb{E}[X]=P'(1),\qquad \operatorname{Var}(X)=P''(1)+P'(1)-P'(1)^2.$$

Probability generating functions (KTH notes).

Multiplicative functions (Dirichlet series)

This is a separate representation: $D(s)=\sum_{n\ge1}a_n/n^s$, not a coefficient of x^n. Read the right column as the sequence a_n. Multiplication gives Dirichlet convolution. Identities may be read formally, or in a half-plane of convergence. Here ζ is the Riemann zeta function.

76	$\zeta(s)$	1.
77	$\zeta(s)^2$	$d(n)$: divisor count.
78	$\zeta(s)\zeta(s-r)$	$\sigma_r(n)$: sum of r-th powers of divisors.
79	$\dfrac{1}{\zeta(s)}$	Möbius function $\mu(n)$.
80	$\dfrac{\zeta(s-1)}{\zeta(s)}$	Euler's totient $\varphi(n)$.
81	$\dfrac{\zeta(s)}{\zeta(2s)}$	$\mu(n)^2$: the indicator that n is squarefree.

Dirichlet-series identities.

Transformations and coefficient recipes

In OGF rows, write $A(x)=\sum a_nx^n$ and $B(x)=\sum b_nx^n$. EGF rows use factorial-normalized coefficients instead.

R1	$x^rA(x)$	OGF shift: coefficient a_{n-r}; negative indices are zero.
R2	$A(cx)$	OGF scaling: coefficient c^na_n.
R3	$\dfrac{A(x)}{1-x}$	OGF prefix sums: coefficient $\sum_{i=0}^n a_i$.
R4	$(1-x)A(x)$	OGF differences: coefficient a_n-a_{n-1}, with $a_{-1}=0$.
R5	$xA'(x)$	OGF weighting by size: coefficient na_n.
R6	$A(x)B(x)$	OGF ordinary convolution: coefficient $\sum_{i=0}^n a_ib_{n-i}$.
R7	$A(x)B(x)$	EGF binomial convolution: represented count $\sum_{i=0}^n \binom{n}{i}a_ib_{n-i}$.
R8	$e^{A(x)}$	EGF for unordered sets of labeled components counted by A, requiring $A(0)=0$.
R9	$\dfrac{A(x)^k}{k!}$	EGF for exactly k unordered labeled components counted by A, requiring $A(0)=0$.
R10	$\dfrac{1}{1-x}A\left(\dfrac{x}{1-x}\right)$	OGF binomial transform: coefficient $\sum_{i=0}^n \binom{n}{i}a_i$.
R11	$\left.\dfrac{\partial F(x,u)}{\partial u}\right\vert_{u=1}$	If u marks a statistic, coefficients give its total over all objects of each size. Divide by the object count for an average; multiply EGF coefficients by $n!$ first.

Generating-function constructions and transformations.

Modular computations. EGF factorial inverses and formal-series operations such as exp, log, and integration divide by factorials or integers. The usual algorithms modulo a prime p assume all needed denominators are invertible; in particular, the standard factorial-inverse approach uses degrees below p.

12 More Topics

12.1 2D Prefix Sum

```
int b[MAXN][MAXN];
int a[MAXN][MAXN];
for (int i = 1; i <= N; i++) {
  for (int j = 1; j <= N; j++) {
    b[i][j] = a[i][j] + b[i - 1][j] +
              b[i][j - 1] - b[i - 1][j - 1];
  }
}
for (int q = 0; q < Q; q++) {
  int from_row, to_row, from_col, to_col;
  cin >> from_row >> from_col >> to_row >> to_col;
  cout << b[to_row][to_col] - b[from_row - 1][to_col] -
          b[to_row][from_col - 1] +
          b[from_row - 1][from_col - 1]
       << '\n';
}
```

12.2 CDQ D&C FFT

12.3 CDQ Divide and Conquer

```
// given x[i], y[i], z[i]
// count for each i number of j such that x[j] > x[i] && y[j] > x[i] &&
    z[j] > z[i]

#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

struct Pt {
  int x, y, z, i;
};

const int N = (int)1e5 + 5;

struct BIT {
  int b[N], n;

  void init(int _n) {
    n = _n;
    for(int i = 0 ; i <= n ; ++i) b[i] = 0;
  }
  inline int lowbit(int x) { return x & (-x); }
  void update(int x, int v) {
    for(int i = x ; i <= n ; i += lowbit(i)) b[i] += v;
  }
  int query(int x) {
    int ans = 0;
    for(int i = x ; i > 0 ; i -= lowbit(i)) ans += b[i];
    return ans;
  }
} bit;

vector<Pt> v;
int n, ans[N];

void cdq(int l, int r) {
  if(l + 1 == r) return;
  int m = (l + r) >> 1;
  cdq(l, m); cdq(m, r);
  int a = l, b = m, sum = 0;
  // need to record the modifications on BIT in order to reset it. The
      complexity will be $O(N^2)$ if we resetting it brute-forcely.
  vector<int> record;
  // temporary array for merge sort
  vector<Pt> tmp;
  while(a < m && b < r) {
    if(v[a].y > v[b].y) bit.update(v[a].z, 1), sum++, record.push_back(v
        [a].z), tmp.push_back(v[a++]);
    else ans[v[b].i] += sum - bit.query(v[b].z), tmp.push_back(v[b++]);
  }
  while(a < m) tmp.push_back(v[a++]);
  while(b < r) ans[v[b].i] += sum - bit.query(v[b].z), tmp.push_back(v[b
      ++]);
  for(int i = l ; i < r ; ++i) v[i] = tmp[i - l];
  // reset BIT
  for(auto i : record) bit.update(i, -1);
  // release used memory
  vector<int> ().swap(record);
  vector<Pt> ().swap(tmp);
}

```

```
void init() {
  cin >> n;
  for(int i = 0 ; i < n ; ++i) {
    int a, b, c; cin >> a >> b >> c;
    v.push_back({a, b, c, i});
  }
  bit.init(n);
}
void solve() {
  // As we require > not >=
  sort(v.begin(), v.end(), [&](auto a, auto b) {
      return (a.x == b.x ? (a.y == b.y ? a.z < b.z : a.y < b.y) : a.x >
          b.x);
  });
  cdq(0, n);
  for(int i = 0 ; i < n ; ++i) cout << ans[i] << '\n';
}

int main() {
  ios_base::sync_with_stdio(0), cin.tie(0);
  init();
  solve();
}
```

```
#include <iostream>
#include <vector>
#define F first
#define S second
using namespace std;
typedef long long ll;

const int N = (int)2e6 + 5;

namespace bit {
  ll b[N], n;

  void init(int _n) {
    n = _n;
    fill(b, b + n + 1, 0);
  }
  inline int lowbit(int x) { return x & (-x); }
  void update(int x, int v) {
    if(x == 0) return;
    for(int i = x ; i <= n ; i += lowbit(i)) b[i] += v;
  }
  ll query(int x) {
    ll ans = 0;
    for(int i = x ; i > 0 ; i -= lowbit(i)) ans += b[i];
    return ans;
  }
}
struct Node {
  int t, x, y, v, i, sgn;
};

vector<Node> v;
vector<ll> ans;
int s, w;

void cdq(int l, int r) {
  if(l + 1 == r) return;
  int m = (l + r) >> 1;
  cdq(l, m); cdq(m, r);
  vector<Node> tmp;
  vector<pair<int, ll> > his;
  int a = l, b = m;
  while(a < m && b < r) {
    if(v[a].x <= v[b].x) {
      bit::update(v[a].y, v[a].v);
      his.push_back({v[a].y, -v[a].v});
      tmp.push_back(v[a++]);
    }
    else {
      ans[v[b].i] += v[b].sgn * (bit::query(v[b].y) + (ll)s * v[b].x * v
          [b].y);
      tmp.push_back(v[b++]);
    }
  }
  while(a < m) tmp.push_back(v[a++]);
  while(b < r) {
    ans[v[b].i] += v[b].sgn * (bit::query(v[b].y) + (ll)s * v[b].x * v[b
```

```
        ].y);
    tmp.push_back(v[b++]);
  }
  for(int i = l ; i < r ; ++i) v[i] = tmp[i - l];
  for(int i = 0 ; i < (int)his.size() ; ++i) bit::update(his[i].F, his[i
      ].S);
  vector<Node> ().swap(tmp);
  vector<pair<int ,ll> > ().swap(his);
}

void init() {
  cin >> s >> w;
  bit::init(w);
  v.clear(); ans.clear();
  ans.push_back(0);
  int op, i = 0; cin >> op;
  while(op != 3) {
    if(op == 1) {
      int x, y, a; cin >> x >> y >> a;
      v.push_back({i, x, y, a, 0, 0});
    }
    else {
      int x1, y1, x2, y2; cin >> x1 >> y1 >> x2 >> y2;
      v.push_back({i, x1 - 1, y1 - 1, 0, (int)ans.size(), 1});
      v.push_back({i, x1 - 1, y2, 0, (int)ans.size(), -1});
      v.push_back({i, x2, y1 - 1, 0, (int)ans.size(), -1});
      v.push_back({i, x2, y2, 0, (int)ans.size(), 1});
      ans.push_back(0);
    }
    cin >> op, ++i;
  }
}
void solve() {
  cdq(0, (int)v.size());
  for(int i = 1 ; i < (int)ans.size() ; ++i) cout << ans[i] << '\n';
}


int main() {
  ios_base::sync_with_stdio(0), cin.tie(0);
  init();
  solve();
}
```

12.4 Count Segments Around Index

```
// count of segments that include i with at most size k, at most l to
    the left and at most r to the right

auto calc = [&](ll val, ll mxx, ll left, ll right) -> ll{
    left = min(left, mxx - 1);
    right = min(right, mxx - 1);
    ll mn = min(left, right);
    ll mx = max(left, right);
    mn++;
    ll res = (mn * (mn + 1)) / 2;
    mn--;
    ll dif = mx - mn;
    res += dif * (mn + 1);
    mxx = min(mxx, left + right + 1);
    ll dif1 = mxx - mx - 1;
    res += (mn * (mn + 1)) / 2;
    dif1 = mn - dif1;
    res -= (dif1 * (dif1 + 1)) / 2;
    return res;
  };
```

12.5 Day of the Week

```
int dayOfWeek(int d, int m, lli y){
  if(m == 1 || m == 2){
    m += 12;
    --y;
  }
  int k = y % 100;
  lli j = y / 100;
  return (d + 13*(m+1)/5 + k + k/4 + j/4 + 5*j) % 7;
}
```

12.6 Directed MST

```
#include<bits/stdc++.h>
using namespace std;

const int N = 3e5 + 9;

const long long inf = 1e18;

```

```cpp
template<typename T> struct PQ {
  long long sum = 0;
  priority_queue<T, vector<T>, greater<T>> Q;
  void push(T x) { x.w -= sum; Q.push(x); }
  T pop() { auto ans = Q.top(); Q.pop(); ans.w += sum; return ans; }
  int size() { return Q.size(); }
  void add(long long x) { sum += x; }
  void merge(PQ &&x) {
    if (size() < x.size()) swap(sum, x.sum), swap(Q, x.Q);
    while (x.size()) {
      auto tmp = x.pop();
      tmp.w -= sum;
      Q.push(tmp);
    }
  }
};
struct edge {
  int u, v; long long w;
  bool operator > (const edge &rhs) const { return w > rhs.w; }
};
struct DSU {
  vector<int> par;
  DSU (int n) : par(n, -1) {}
  int root(int i) { return par[i] < 0 ? i : par[i] = root(par[i]); }
  void set_par(int c, int p) { par[c] = p; }
};
// returns parents of each vertex
// each edge should be distinct
// it assumes that a solution exists (all vertices are reachable from
    root)
// 0 indexed
// Takes ~300ms for n = 2e5
vector<int> DMST(int n, int root, const vector<edge> &edges) {
  vector<int> u(2 * n - 1, -1), par(2 * n - 1, -1);
  edge par_edge[2 * n - 1];
  vector<int> child[2 * n - 1];

  PQ<edge> Q[2 * n - 1];
  for (auto e : edges) Q[e.v].push(e);
  for (int i = 0; i < n; i++) Q[(i + 1) % n].push({i, (i + 1) % n, inf})
      ;

  int super = n;
  DSU dsu(2 * n - 1);
  int head = 0;
  while (Q[head].size()) {
    auto x = Q[head].pop();
    int nxt_root = dsu.root(x.u);
    if (nxt_root == head) continue;
    u[head] = nxt_root;
    par_edge[head] = x;
    if (u[nxt_root] == -1) head = nxt_root;
    else {
      int j = nxt_root;
      do {
        Q[j].add(-par_edge[j].w);
        Q[super].merge(move(Q[j]));
        assert(u[j] != -1);
        child[super].push_back(j);
        j = dsu.root(u[j]);
      } while (j != nxt_root);
      for (auto u : child[super]) par[u] = super, dsu.set_par(u, super);
      head = super++;
    }
  }
  vector<int> res(2 * n - 1, -1);
  queue<int> q; q.push(root);
  while (q.size()) {
    int u = q.front();
    q.pop();
    while (par[u] != -1) {
      for (auto v : child[par[u]]) {
        if (v != u) {
          res[par_edge[v].v] = par_edge[v].u;
          q.push(par_edge[v].v);
          par[v] = -1;
        }
      }
      u = par[u];
    }
  }
  res[root] = root; res.resize(n);
  return res;
}
int32_t main() {
  ios_base::sync_with_stdio(0);
```

```
  cin.tie(0);
  int n, m, root; cin >> n >> m >> root;
  vector<edge> edges(m);
  for (auto &e : edges) cin >> e.u >> e.v >> e.w;
  auto res = DMST(n, root, edges);

  unordered_map<int, int> W[n];
  for (auto u : edges) W[u.v][u.u] = u.w;

  long long ans = 0;
  for (int i = 0; i < n; i++) if (i != root) ans += W[i][res[i]];
  cout << ans << '\n';
  for (auto x : res) cout << x << ' '; cout << '\n';
  return 0;
}
// https://judge.yosupo.jp/problem/directedmst
// http://www.cs.tau.ac.il/~zwick/grad-algo-13/directed-mst.pdf
```

12.7 GCD Convolution

```
/*
  GCD Convolution via Multiple Zeta and Mobius Transforms
  -------------------------------------------------------
  Given two sequences A[0..n] and B[0..n], computes C where
    C[d] = sum_{i, j: gcd(i, j) = d} A[i] * B[j]
  Indexing: 1-based, vectors of size n+1 (ignore index 0)
  Bounds:
    - i, j, d in [1..n]
  Time Complexity: O(n log log n + n log n) per convolution
  Space Complexity: O(n)
  Steps:
    1. Multiple Zeta Transform: for each prime p, v[i] += v[i*p]
    2. Pointwise multiply transformed A and B
    3. Multiple Mobius Transform: for each prime p, v[i] -= v[i*p]
*/
// Enumerate primes up to n in O(n)
vector<int> PrimeEnumerate(int n) {
  vector<bool> isPrime(n+1, true);
  vector<int> P;
  for (int i = 2; i <= n; i++) {
    if (isPrime[i]) P.push_back(i);
    for (int p : P) {
      if (i * p > n) break;
      isPrime[i*p] = false;
      if (i % p == 0) break;
    }
  }
  return P;
}
// Multiple Zeta Transform (over divisibility poset)
template<typename T>
void MultipleZetaTransform(vector<T>& v) {
  int n = (int)v.size() - 1;
  for (int p : PrimeEnumerate(n)) {
    for (int i = n/p; i >= 1; i--) {
      v[i] += v[i * p];
    }
  }
}
// Multiple Mobius Transform (inverse of zeta)
template<typename T>
void MultipleMobiusTransform(vector<T>& v) {
  int n = (int)v.size() - 1;
  for (int p : PrimeEnumerate(n)) {
    for (int i = 1; i * p <= n; i++) {
      v[i] -= v[i * p];
    }
  }
}
// GCD convolution of A and B, both size n+1
template<typename T>
vector<T> GCDConvolution(vector<T> A, vector<T> B) {
  int n = (int)A.size() - 1;
  MultipleZetaTransform(A);
  MultipleZetaTransform(B);
  for (int i = 1; i <= n; i++) {
    A[i] *= B[i];  // pointwise multiplication
  }
  MultipleMobiusTransform(A);
  return A;  // result C[0..n], where C[d] = sum_{gcd(i,j)=d} A[i]*B[j]
}
// Example usage
vector<long long> A(n+1), B(n+1);
for (int i = 1; i <= n; i++) cin >> A[i];
for (int i = 1; i <= n; i++) cin >> B[i];
auto C = GCDConvolution<long long>(A, B);
```

```
for (int d = 1; d <= n; d++){
  cout << C[d] << (d==n?'\n':'␣');
}
```

12.8 int128

```
//cout for __int128
ostream &operator<<(ostream &os, const __int128 & value){
  char buffer[64];
  char *pos = end(buffer) - 1;
  *pos = '\0';
  __int128 tmp = value < 0 ? -value : value;
  do{
    --pos;
    *pos = tmp % 10 + '0';
    tmp /= 10;
  }while(tmp != 0);
  if(value < 0){
    --pos;
    *pos = '-';
  }
  return os << pos;
}

//cin for __int128
istream &operator>>(istream &is, __int128 & value){
  char buffer[64];
  is >> buffer;
  char *pos = begin(buffer);
  int sgn = 1;
  value = 0;
  if(*pos == '-'){
    sgn = -1;
    ++pos;
  }else if(*pos == '+'){
    ++pos;
  }
  while(*pos != '\0'){
    value = (value << 3) + (value << 1) + (*pos - '0');
    ++pos;
  }
  value *= sgn;
  return is;
}


ll mult(__int128 a, __int128 b){ return ((a*1LL*b)%MOD + MOD)%MOD; }
```

12.9 Iterating Over All Subsets

```
for (int mk = 0; mk < (1 << k); mk++) {
  Ap[mk] = 0;
  for (int s = mk;; s = (s - 1) & mk) {
    Ap[mk] += A[s];
    if (!s) break;
  }
}
```

12.10 LCM Convolution

```
/*
  LCM Convolution via Divisor Zeta and Mobius Transforms
  ------------------------------------------------------
  Given two sequences A[1..n] and B[1..n], computes C where
    C[d] = sum_{i, j: lcm(i, j) = d} A[i] * B[j]
  Indexing: 1-based (vectors of size n+1, ignore index 0)
  Bounds:
    - i, j, d in [1..n]
  Time Complexity: O(n log log n + n log n)
  Space Complexity: O(n)
  Steps:
    1. Divisor Zeta Transform: for each prime p, v[i*p] += v[i]
    2. Pointwise multiply transformed A and B
    3. Divisor Mobius Transform: for each prime p, v[i*p] -= v[i]
*/
// Sieve primes up to n in O(n)
vector<int> PrimeEnumerate(int n) {
  vector<bool> isPrime(n+1, true);
  vector<int> P;
  for (int i = 2; i <= n; i++) {
    if (isPrime[i]) P.push_back(i);
    for (int p : P) {
      if (i * p > n) break;
      isPrime[i*p] = false;
      if (i % p == 0) break;
    }
  }
```

```
  return P;
}
// Divisor Zeta Transform: v[d] = sum_{d|k} original_v[k]
template<typename T>
void DivisorZetaTransform(vector<T>& v) {
  int n = (int)v.size() - 1;
  auto primes = PrimeEnumerate(n);
  for (int p : primes) {
    for (int i = 1; i * p <= n; i++) {
      v[i * p] += v[i];
    }
  }
}
// Divisor Mobius Transform (inverse of zeta)
template<typename T>
void DivisorMobiusTransform(vector<T>& v) {
  int n = (int)v.size() - 1;
  auto primes = PrimeEnumerate(n);
  for (int p : primes) {
    for (int i = n / p; i >= 1; i--) {
      v[i * p] -= v[i];
    }
  }
}
// LCM convolution of A and B (each size n+1)
template<typename T>
vector<T> LCMConvolution(vector<T> A, vector<T> B) {
  int n = (int)A.size() - 1;
  DivisorZetaTransform(A);
  DivisorZetaTransform(B);
  for (int d = 1; d <= n; d++) {
    A[d] *= B[d];  // pointwise multiplication
  }
  DivisorMobiusTransform(A);
  return A;  // result C[1..n], C[d] = sum_{lcm(i,j)=d} A[i]*B[j]
}
// Example usage
vector<long long> A(n+1), B(n+1);
for (int i = 1; i <= n; i++) cin >> A[i];
for (int i = 1; i <= n; i++) cin >> B[i];
auto C = LCMConvolution<long long>(A, B);
for (int d = 1; d <= n; d++){
  cout << C[d] << (d==n?'\n':' ');
}
```

12.11 Manhattan MST

```
/*
  Manhattan MST Edge Generation
  -----------------------------
  Given n points ps[0..n-1], returns a list of candidate edges
    (weight, u, v) for the Manhattan MST, where
    weight = |x[u]-x[v]| + |y[u]-y[v]|.
  Indexing: 0-based for points and returned edges
  Time Complexity: O(n log n)
*/
struct Point {
  ll x, y;
};

vector<tuple<ll,int,int>> manhattan_mst_edges(vector<Point> ps) {
  int n = ps.size();
  vector<int> order(n);
  iota(order.begin(), order.end(), 0);
  vector<tuple<ll,int,int>> edges;
  edges.reserve(n * 4);
  // Repeat for 4 orientations
  for (int r = 0; r < 4; r++) {
    // Sort by x+y ascending
    sort(order.begin(), order.end(), [&](int i, int j) {
      return ps[i].x + ps[i].y < ps[j].x + ps[j].y;
    });
    // Active map: key = x-coordinate, value = point index
    map<ll,int,greater<ll>> active;
    for (int i : order) {
      // For all active points with x >= ps[i].x
      for (auto it = active.lower_bound(ps[i].x); it != active.end();
          active.erase(it++)) {
        int j = it->second;
        if (ps[i].x - ps[i].y > ps[j].x - ps[j].y) break;
        assert(ps[i].x >= ps[j].x && ps[i].y >= ps[j].y);
        edges.emplace_back({(ps[i].x - ps[j].x) + (ps[i].y - ps[j].y), i
            , j});
      }
      active[ps[i].x] = i;
    }
}
```

```
    // Rotate points 90 degrees: (x,y) -> (y,-x)
    for (auto &p : ps) {
      ll tx = p.x;
      p.x = p.y;
      p.y = -tx;
    }
  }
  return edges;
}
```

12.12 Max Manhattan Distance

```
/*
  Max Manhattan distance
  ----------------------------------
  Finds maximum manhattan distance for any two points with d dimensions.
  Generalized code for d dimensions.
  Time Complexity: (n * 2 ^ d * d)
*/


long long ans = 0;
for (int msk = 0; msk < (1 << d); msk++) {
    long long mx = LLONG_MIN, mn = LLONG_MAX;
    for (int i = 0; i < n; i++) {
        long long cur = 0;
        for (int j = 0; j < d; j++) {
            if (msk & (1 << j)) cur += p[i][j];
            else cur -= p[i][j];
        }
        mx = max(mx, cur);
        mn = min(mn, cur);
    }
    ans = max(ans, mx - mn);
}
```

12.13 Matroid Intersection

```
template <class M1, class M2> vector<bool> MatroidIsect(M1 m1, M2 m2,
    int n) {
  vector<bool> in_S(n);
  auto augment = [&] {
    vector<int> S, E_S;  // E_S has indices of all edges not in S
    for (int i = 0; i < n; i++) {
      if (in_S[i]) S.push_back(i);
      else E_S.push_back(i);
    }
    // build exchange graph
    vector<vector<int>> adj(n);
    for (int y : S) {
      m1.clear(), m2.clear();
      for (int i : S) {
        if (i != y) m1.add(i), m2.add(i);
      }
      // optional precomputation, can be an empty function
      m1.precomp(), m2.precomp();
      for (int x : E_S) {
        if (m1.check(x)) adj[y].push_back(x);
        if (m2.check(x)) adj[x].push_back(y);
      }
    }
    // identify sources and sinks for BFS
    vector<bool> is_sink(n);
    queue<int> q;
    vector<bool> vis(n);

    m1.clear(), m2.clear();
    for (int y : S) m1.add(y), m2.add(y);
    m1.precomp(), m2.precomp();  // optional precomp
    for (int x : E_S) {
      if (m1.check(x)) vis[x] = 1, q.push(x);  // x is a source
      is_sink[x] = m2.check(x);
    }
    // BFS to find augmenting path from source to sink
    vector<int> prev(n, -1);
    while (!q.empty()) {
      int u = q.front();
      q.pop();
      if (is_sink[u]) {
        // augmenting path found
        for (int v = u; v != -1; v = prev[v]) { in_S[v] = !in_S[v]; }
        return true;
      }
      for (int v : adj[u]) {
        if (vis[v]) continue;
        q.push(v);
        vis[v] = 1;
```

```
            prev[v] = u;
          }
        }
        return false;
      };
      while (augment());
      return in_S;
    }
```

12.14 Matroid Bipartite

Given a bipartite graph $G = (U, V)$, the ground set is U. A subset of U is independent if it can be perfectly matched to some subset of V.
Note: DFS arrays are managed using a timer to avoid $\mathcal{O}(|V|)$ `fill()` operations on every `check()` call, which is critical since `check()` is called $|S| \times |E \setminus S|$ times per step.

```
struct TransversalMatroid {
    int n, m, timer; // n = |U| (ground set), m = |V|
    vector<vector<int>> adj;
    vector<int> matchL, matchR, vis;

    TransversalMatroid(int n, int m, const vector<vector<int>> &adj)
        : n(n), m(m), timer(0), adj(adj), matchL(n, -1), matchR(m, -1),
            vis(m, 0) {}

    void clear() {
        fill(begin(matchL), end(matchL), -1);
        fill(begin(matchR), end(matchR), -1);
        timer = 0;
        fill(begin(vis), end(vis), 0);
    }

    bool dfs_check(int u) {
        for (int v : adj[u]) {
            if (vis[v] == timer) continue;
            vis[v] = timer;
            if (matchR[v] == -1 || dfs_check(matchR[v])) return true;
        }
        return false;
    }

    bool check(int x) {
        if (matchL[x] != -1) return true;
        ++timer;
        return dfs_check(x);
    }

    bool dfs_add(int u) {
        for (int v : adj[u]) {
            if (vis[v] == timer) continue;
            vis[v] = timer;
            if (matchR[v] == -1 || dfs_add(matchR[v])) {
                matchR[v] = u;
                matchL[u] = v;
                return true;
            }
        }
        return false;
    }

    void add(int x) {
        if (matchL[x] != -1) return;
        ++timer;
        dfs_add(x);
    }

    void precomp() {}
};
```

12.15 Matroid Colorful

Ground Set: A set of elements where each element is assigned exactly one color.
Independent Set: A subset of elements where all chosen elements have strictly distinct colors (at most one element per color).
Implementation: Maintained with a boolean array tracking active colors. To `check()` if an element can be added, verify its color is currently marked as false (unused).

```
struct ColorfulMatroid {
    vector<int> color;
    vector<bool> used;

    ColorfulMatroid(int num_colors, const vector<int>& color)
        : color(color), used(num_colors, false) {}

    void clear() {
```

```
        fill(begin(used), end(used), false);
    }

    bool check(int x) {
        return !used[color[x]];
    }

    void add(int x) {
        used[color[x]] = true;
    }

    void precomp() {}
};
```

12.16 Matroid Graphic

Ground Set: The edges of an undirected graph.
Independent Set: A subset of edges that contains no cycles (i.e., forms a forest).
Implementation: Maintained using a Disjoint Set Union (DSU). To `check()` if adding an edge maintains independence, verify its endpoints belong to different DSU components.

```
struct DSU {
  vector<int> dsu, sz;
  DSU(int n) : dsu(n), sz(n) { iota(begin(dsu), end(dsu), 0); }
  int find(int x) {
    if (x == dsu[x]) return x;
    return dsu[x] = find(dsu[x]);
  }
  void merge(int x, int y) {
    x = find(x), y = find(y);
    if (x == y) return;
    if (sz[x] < sz[y]) swap(x, y);
    dsu[y] = x;
    sz[x] += sz[y];
  }
};

struct GraphicMatroid {
  int n;  // nodes of the graph must be in [0..n-1]
  DSU dsu;
  vector<pair<int, int>> edges;
  GraphicMatroid(int n, const vector<pair<int, int>> &edges)
      : n(n), dsu(n), edges(edges) {
    clear();
  }
  void clear() { dsu = DSU(n); }
  bool check(int x) { return dsu.find(edges[x].first) != dsu.find(edges[
      x].second); }
  void add(int x) { dsu.merge(edges[x].first, edges[x].second); }
  void precomp() {}
};
```

12.17 Matroid Partition

Ground Set: A set of elements partitioned into disjoint categories $G_1, G_2, \ldots, G_k$, each with a defined maximum capacity C_i.
Independent Set: A subset of elements where the number of chosen elements belonging to category G_i is less than or equal to C_i.
Implementation: Maintained with an integer frequency array. To `check()` if an element can be added, verify the current frequency of its category is strictly less than its capacity limit.

```
struct PartitionMatroid {
  vector<int> cap, cat, used;
  PartitionMatroid(const vector<int> &cap, const vector<int> &cat)
      : cap(cap), cat(cat), used(size(cap)) {}
  void clear() { fill(begin(used), end(used), 0); }
  bool check(int x) { return used[cat[x]] < cap[cat[x]]; }
  void add(int x) { used[cat[x]]++; }
  void precomp() {}
};
```

12.18 Matroid Prime Field

A set of vectors is independent if they are linearly independent over a prime field. Why it's useful in CP: If you map the edges of a graph to generic random vectors over $\mathbb{Z}_p$, then independence of edges is equivalent to independence of the corresponding vectors. This allows you to reduce problems about graphic matroids to problems about linear algebra over a prime field.

```
struct LinearPrimeMatroid {
    int D; // Vector dimension
    long long MOD;
    vector<vector<long long>> vecs;
    vector<vector<long long>> basis;
```

```
    LinearPrimeMatroid(int D, long long MOD, const vector<vector<long
        long>>& vecs)
        : D(D), MOD(MOD), vecs(vecs) {}

    long long power(long long base, long long exp) {
        long long res = 1;
        base %= MOD;
        while (exp > 0) {
            if (exp % 2 == 1) res = (res * base) % MOD;
            base = (base * base) % MOD;
            exp /= 2;
        }
        return res;
    }

    long long modInverse(long long n) {
        return power(n, MOD - 2);
    }

    void clear() {
        basis.clear();
    }

    bool check(int x) {
        vector<long long> v = vecs[x];
        for (const auto& b : basis) {
            int pivot = -1;
            for (int i = 0; i < D; i++) {
                if (b[i] != 0) { pivot = i; break; }
            }
            if (pivot != -1 && v[pivot] != 0) {
                long long f = (v[pivot] * modInverse(b[pivot])) % MOD;
                for (int i = pivot; i < D; i++) {
                    v[i] = (v[i] - f * b[i]) % MOD;
                    if (v[i] < 0) v[i] += MOD;
                }
            }
        }
        for (int i = 0; i < D; i++) {
            if (v[i] != 0) return true;
        }
        return false;
    }

    void add(int x) {
        vector<long long> v = vecs[x];
        for (const auto& b : basis) {
            int pivot = -1;
            for (int i = 0; i < D; i++) {
                if (b[i] != 0) { pivot = i; break; }
            }
            if (pivot != -1 && v[pivot] != 0) {
                long long f = (v[pivot] * modInverse(b[pivot])) % MOD;
                for (int i = pivot; i < D; i++) {
                    v[i] = (v[i] - f * b[i]) % MOD;
                    if (v[i] < 0) v[i] += MOD;
                }
            }
        }
        for (int i = 0; i < D; i++) {
            if (v[i] != 0) {
                basis.push_back(v);
                return;
            }
        }
    }

    void precomp() {}
};
```

12.19 Matroid Truncated

This is a highly composable utility. If you need a matroid that behaves exactly like M but is restricted to a maximum rank (size) K, you can wrap M. This is useful for problems like "find a spanning tree with at most K red edges" (intersect Graphic with a Truncated Colorful Matroid).

```
template<typename M>
struct TruncatedMatroid {
    M m;
    int k, cnt;

    TruncatedMatroid(M m, int k) : m(m), k(k), cnt(0) {}

    void clear() {
```

```
        m.clear();
        cnt = 0;
    }

    bool check(int x) {
        return cnt < k && m.check(x);
    }

    void add(int x) {
        m.add(x);
        cnt++;
    }

    void precomp() {
        m.precomp();
    }
};
```

12.20 Matroid XOR

A subset of integers is independent if no subset XORs to 0. This is commonly intersected with Graphic or Partition matroids for problems involving basis selection under constraints.

```
struct LinearGF2Matroid {
    vector<long long> vals;
    vector<long long> basis;

    LinearGF2Matroid(const vector<long long>& vals) : vals(vals) {}

    void clear() {
        basis.clear();
    }

    bool check(int x) {
        long long v = vals[x];
        for (long long b : basis) v = min(v, v ^ b);
        return v > 0;
    }

    void add(int x) {
        long long v = vals[x];
        for (long long b : basis) v = min(v, v ^ b);
        if (v > 0) {
            basis.push_back(v);
            // Sorting ensures MSBs remain strictly descending for
                correct basis reduction
            sort(basis.rbegin(), basis.rend());
        }
    }

    void precomp() {}
};
```

12.21 Mo

```
/*
  Mo's Algorithm (Sqrt Decomposition for Offline Queries)
  ------------------------------------------------------
  Problem: Answer q range queries [L, R] over array of length n
           using add/remove operations efficiently
  Indexing: 0-based
  Bounds:
    - arr[0..n-1]
    - queries input as 1-based and converted to 0-based
  Time Complexity: O((n + q) * sqrt(n)) per query batch
  Space Complexity: O(n + q)
  Usage:
    - Fill in logic for add/remove operations (update cur)
    - Fill answers[] indexed by original query order
*/

const int MAXN = 200500;
ll n, q;
ll arr[MAXN];      // input array
ll cnt[1000005];   // frequency count
ll answers[MAXN];  // output array
ll cur = 0;        // current query result
ll BLOCK_SIZE;

pair< pair<ll, ll>, ll> queries[MAXN]; // {{L, R}, query_index}

// Sort by block and then by R
inline bool cmp(const pair< pair<ll, ll>, ll> &x, const pair< pair<ll,
    ll>, ll> &y) {
```

```
  ll block_x = x.first.first / BLOCK_SIZE;
  ll block_y = y.first.first / BLOCK_SIZE;
  if (block_x != block_y) return block_x < block_y;
  return x.first.second < y.first.second;
}

int main() {
  cin >> n >> q;
  BLOCK_SIZE = sqrt(n);
  for (int i = 0; i < n; i++) cin >> arr[i];
  for (int i = 0; i < q; i++) {
    int l, r;
    cin >> l >> r;
    --l; --r; // convert to 0-based
    queries[i] = {{l, r}, i};
  }
  sort(queries, queries + q, cmp);
  ll l = 0, r = -1;
  for (int i = 0; i < q; i++) {
    int left = queries[i].first.first;
    int right = queries[i].first.second;
    // Expand to right
    while (r < right) {
      r++;
      // Operations to add arr[r], implement exactly here
    }
    // Shrink from right
    while (r > right) {
      // Operations to remove arr[r], implement exactly here
      r--;
    }
    // Expand to left
    while (l < left) {
      // Operations to remove arr[l], implement exactly here
      l++;
    }
    // Shrink from left
    while (l > left) {
      l--;
      // Operations to add arr[l], implement exactly here
    }
    answers[queries[i].second] = cur; // Current answer
  }
  for (int i = 0; i < q; i++) cout << answers[i] << '\n';
}
```

12.22 MOD INT

```
template<int MOD>
struct ModInt {
  ll v;
  ModInt(ll _v = 0) {v = (-MOD < _v && _v < MOD) ? _v : _v % MOD; if (v < 0) v += MOD;}
  ModInt& operator += (const ModInt &other) {v += other.v; if (v >= MOD) v -= MOD; return *this;}
  ModInt& operator -= (const ModInt &other) {v -= other.v; if (v < 0) v += MOD; return *this;}
  ModInt& operator *= (const ModInt &other) {v = v * other.v % MOD; return *this;}
  ModInt& operator /= (const ModInt &other) {return *this *= inverse(other);}
  bool operator == (const ModInt &other) const {return v == other.v;}
  bool operator != (const ModInt &other) const {return v != other.v;}
  friend ModInt operator + (ModInt a, const ModInt &b) {return a += b;}
  friend ModInt operator - (ModInt a, const ModInt &b) {return a -= b;}
  friend ModInt operator * (ModInt a, const ModInt &b) {return a *= b;}
  friend ModInt operator / (ModInt a, const ModInt &b) {return a /= b;}
  friend ModInt operator - (const ModInt &a) {return 0 - a;}
  friend ModInt power(ModInt a, ll b) {ModInt ret(1); while (b > 0) {if (b & 1) ret *= a; a *= a; b >>= 1;} return ret;}
  friend ModInt inverse(ModInt a) {return power(a, MOD - 2);}
  friend istream& operator >> (istream &is, ModInt &m) {is >> m.v; m.v = (-MOD < m.v && m.v < MOD) ? m.v : m.v % MOD; if (m.v < 0) m.v += MOD; return is;}
  friend ostream& operator << (ostream &os, const ModInt &m) {return os << m.v;}
};
```

12.23 Next Permutation

```
sort(v.begin(),v.end());
while(next_permutation(v.begin(),v.end())){
  for(auto u:v){
    cout<<u<<"␣";
  }
  cout<<endl;
}
```

```

string s="asdfassd";
sort(s.begin(),s.end());
while(next_permutation(s.begin(),s.end())){
  cout<<s<<endl;
}
```

12.24 Next and Previous Smaller/Greater Element

```
/*
  Next and Previous Smaller / Greater Elements
  ---------------------------------
  Given: array a[0..n-1]
  nextSmaller[i]: index of next element smaller than a[i], or n if none
  prevSmaller[i]: index of previous element smaller than a[i], or -1 if
      none
  For GREATER, replace a[s.top()] < a[i] with a[s.top()] > a[i]
  Indexing: 0-based
*/
vector<int> nextSmaller(vector<int> a, int n) {
  stack<int> s;
  vector<int> res(n, n); // default: no smaller to right
  for (int i = 0; i < n; i++) {
    while (!s.empty() && a[s.top()] > a[i]) {
      res[s.top()] = i;
      s.pop();
    }
    s.push(i);
  }
  return res;
}
vector<int> prevSmaller(vector<int> a, int n) {
  stack<int> s;
  vector<int> res(n, -1); // default: no smaller to left
  for (int i = n - 1; i >= 0; i--) {
    while (!s.empty() && a[s.top()] > a[i]) {
      res[s.top()] = i;
      s.pop();
    }
    s.push(i);
  }
  return res;
}
```

12.25 Parallel Binary Search

```
/*
  Parallel Binary Search
  ----------------------
  Solves: k independent queries where each answer lies in range [lo, hi]
  - At each iteration, test the midpoint for a batch of queries
  Indexing: 0-based
  Bounds:
    - i in [0, m-1] for the structure version
    - q in [0, k-1] for each of the k queries
  Time Complexity: O((m + k) * log M)
  Space Complexity: O(k + m)
  You must:
    - Reset data structures before each iteration
    - Implement apply_change(i) and check_query(q)
*/
int lo[MAXN], hi[MAXN];     // binary search bounds for each query
vector<int> tocheck[MAXN];  // tocheck[mid] = list of queries to check
    at mid

bool done = false;
while (!done) {
  done = true;
   // Reset changes of structure to 0 before each iteration
  // Assign queries to current midpoint
  for (int q = 0; q < k; q++) {
    if (lo[q] < hi[q]) {
      tocheck[(lo[q] + hi[q]) / 2].push_back(q);
    }
  }
  // Apply changes and evaluate queries
  for (int i = 0; i < m; i++) {
    // Apply change for ith query to your data structure
    apply_change(i);
    for (int q : tocheck[i]) {
      done = false; // at least one query is not done
      // Evaluate query q using the updated structure
      if (operationToCheck(q)) {
        hi[q] = i; // condition is true, try earlier
      } else {
        lo[q] = i + 1; // condition false, try later
      }
```

```
    }
  }
}
```

12.26 Random Number Generators

```
//to avoid hacks
mt19937 mt(chrono::steady_clock::now().time_since_epoch().count());
mt19937_64 mt(chrono::steady_clock::now().time_since_epoch().count());
//you can also just write seed_value if hacks are not an issue

// rng() for generating random numbers between 0 and 2<<31-1

// for generating numbers with uniform probability in range
uniform_int_distribution<int> rng(0, numeric_limits<int>::max());

// then for generating random values just do
int random_value = rng(mt);

// if you do not want to create variable
uniform_int_distribution<int>(0, n)(mt)

// other distributions
// Normal distribution: mean = 0.0, stddev = 1.0
normal_distribution<double> normal_dist(0.0, 1.0);

// Exponential distribution: lambda = 1.5
exponential_distribution<double> exp_dist(1.5);

// for shuffling array
shuffle(permutation.begin(), permutation.end(), rng);
```

12.27 Ternary Search

```
/*
  Ternary Search (for continuous unimodal functions)
  --------------------------------------------------
  Finds the maximum (or minimum) value of a unimodal function f(x) in [l
      , r]
  Indexing: continuous domain (double)
  Bounds:
    - Search range is [l, r]
    - Precision controlled by eps
  Time Complexity: O(log((r - l) / eps))
  - For minimum, reverse the condition (if f1 > f2 then l = m1)
*/
double ternary_search(double l, double r) {
  double eps = 1e-9; // precision error
  while (r - l > eps) {
    double m1 = l + (r - l) / 3;
    double m2 = r - (r - l) / 3;
    double f1 = f(m1); // evaluate function at m1
    double f2 = f(m2); // evaluate function at m2
    if (f1 < f2)
      l = m1;  // move left bound up if increasing
    else
      r = m2;  // move right bound down if decreasing
  }
  return f(l); // return value at the best point
}
```

12.28 Ternary Search Int

```
int lo = -1, hi = n;
while (hi - lo > 1){
    int mid = (hi + lo)>>1;
    if (f(mid) > f(mid + 1)) hi = mid;
    else lo = mid;
}
//lo + 1 is the answer
```

12.29 XOR Convolution

```
/*
  Fast Walsh-Hadamard Transform (FWHT) for XOR Convolution
  --------------------------------------------------------
  Given two arrays A[0..N-1], B[0..N-1], with N = 1<<k (power of two).
  Computes C[d] = sum_{i xor j = d} A[i] * B[j].
  Indexing: 0-based
  Bounds:
    - N must be a power of two
  Time Complexity: O(N * log(N))
  Steps:
    1. fwht(A, false); fwht(B, false); // forward transform
    2. for i in [0..N): C[i] = A[i] * B[i]
    3. fwht(C, true);                  // inverse transform
*/
void FWHT (int A[], int k, int inv) {
```

```
  for (int j = 0; j < k; j++)
    for (int i = 0; i < (1 << k); i++)
      if (~i & (1 << j)) {
        int p0 = A[i];
        int p1 = A[i | (1 << j)];
        A[i] = p0 + p1;
        A[i | (1 << j)] = p0 - p1;
        if (inv) {
          A[i] /= 2;
          A[i | (1 << j)] /= 2;
        }
      }
}
void XOR_conv (int A[], int B[], int C[], int k) {
  FWHT(A, k, false);
  FWHT(B, k, false);
  for (int i = 0; i < (1 << k); i++)
    C[i] = A[i] * B[i];
  FWHT(A, k, true);
  FWHT(B, k, true);
  FWHT(C, k, true);
}
// Example usage:
int A[1 << 20], B[1 << 20], C[1 << 20];
XOR_conv(A, B, C, 20);
for (int d = 0; d < (1<<20); d++){
  cout << C[d] << (d==(1<<20)?'\n':' ');
}
```

```
#include <bits/stdc++.h>

using namespace std;
#define int long long

const int mod = 998244353;

int inverse(int x, int mod) {
  return x == 1 ? 1 : mod - mod / x * inverse(mod % x, mod) % mod;
}

void xormul(vector<int> a, vector<int> b, vector<int> &c) {
  int m = (int) a.size();
  c.resize(m);
  for (int n = m / 2; n > 0; n /= 2)
    for (int i = 0; i < m; i += 2 * n)
      for (int j = 0; j < n; j++) {
        int x = a[i + j], y = a[i + j + n];
        a[i + j] = (x + y) % mod;
        a[i + j + n] = (x - y + mod) % mod;
      }
  for (int n = m / 2; n > 0; n /= 2)
    for (int i = 0; i < m; i += 2 * n)
      for (int j = 0; j < n; j++) {
        int x = b[i + j], y = b[i + j + n];
        b[i + j] = (x + y) % mod;
        b[i + j + n] = (x - y + mod) % mod;
      }
  for (int i = 0; i < m; i++)
    c[i] = a[i] * b[i] % mod;
  for (int n = 1; n < m; n *= 2)
    for (int i = 0; i < m; i += 2 * n)
      for (int j = 0; j < n; j++) {
        int x = c[i + j], y = c[i + j + n];
        c[i + j] = (x + y) % mod;
        c[i + j + n] = (x - y + mod) % mod;
      }
  int mrev = inverse(m, mod);
  for (int i = 0; i < m; i++)
    c[i] = c[i] * mrev % mod;
}

int32_t main() {
  ios::sync_with_stdio(0); cin.tie(0);
  int n; cin >> n;
  vector<int> a(1 << n), b(1 << n), c;
  for (int i = 0; i < (1 << n); i++)
    cin >> a[i];
  for (int i = 0; i < (1 << n); i++)
    cin >> b[i];
  xormul(a, b, c);
  for (int i = 0; i < (1 << n); i++)
    cout << c[i] << ' ';
```

```
}
```

12.30 XOR Basis

```
/*
  XOR Basis (Linear Basis over GF(2))
  -----------------------------------
  Supports insertion of numbers into a basis and querying the maximum
  XOR of any subset with a given value.

  Bit Width: up to D bits (here D = 60 for 64-bit integers)
  Indexing: basis[0] is for bit 0 (least significant), basis[D-1] for
      bit D-1
  Bounds:
    - Values x inserted must satisfy 0 <= x < 2^D
  Time Complexity:
    - insert(x):   O(D)
    - getMax(x):   O(D)
  Space Complexity: O(D)

  Notes:
    - We maintain one basis vector per bit position.
    - On insert, we reduce x by existing basis vectors, then store it
      in the highest bit it still has.
    - To query max XOR, we greedily try to xor with basis vectors
      from highest bit down.
*/
const int D = 31; // Maximum number of bits in the numbers
int basis[D]; // basis[i] keeps the mask of the vector whose f value is
    i
int sz; // Current size of the basis

void insertVector(int mask) {
  //turn for around if you want max xor
  for (int i = 0; i < D; i++) {
    if ((mask & 1 << i) == 0) continue; // continue if i != f(mask)
    if (!basis[i]) { // If there is no basis vector with the ith bit set
        , then insert this vector into the basis
      basis[i] = mask;
      ++sz;
      return;
    }
    mask ^= basis[i]; // Otherwise subtract the basis vector from this
        vector
  }
}

// V2: If you dont need the basis sorted.
vector<ll> basis;
void add(ll x)
{
  for (int i = 0; i < basis.size(); i++)
  {
    x = min(x, x ^ basis[i]);
  }
  if (x != 0)
  {
    basis.pb(x);
  }
}
```

12.31 XOR Basis Online

```
int rebuild_and_delete (int id) {
  int pos = 0, mn = inf, p2 = 0;
  for (int i = 0; i < basis.size(); i++) {
      if (basis[i].id == id) {
          pos = i;
      }
  }
  int bits = 0;
  for (int i = 0; i < basis.size(); i++) {
      if (basis[i].mask & (1 << pos)) {
          if (mn > basis[i].high) {
              mn = basis[i].high;
              p2 = i;
          }
          bits ^= 1 << basis[i].high;
      }
  }

  if (p2 != pos) {
      swap(basis[p2].id, basis[pos].id);
      for (auto &i : basis) {
          swap_bits(i.mask, pos, p2); // just swaps pos-th and p2-th bit
              in i.mask
```

```
        }
        pos = p2;
    }
    for (int i = 0; i < basis.size(); i++) {
        if (i != pos) {
            if (basis[i].mask & (1 << pos)) {
                basis[i].val ^= basis[pos].val;
                basis[i].mask ^= basis[pos].mask;
            }
        }
    }
    int good = (1 << pos) - 1;
    int other = ((1 << len(basis)) - 1) ^ (good | (1 << pos));
    basis.erase(basis.begin() + pos);
    for (auto &i : basis) {
        i.mask = ((i.mask & other) >> 1) | (i.mask & good);
    }
    return bits;
}

C++
41 lines
1161 bytes


int get_the_same_high_bit (int bits, vector <int> &val) {
    for (auto &i : basis) {
        if (__builtin_popcount(val[i.id] & bits) & 1) {
            return i.id;
        }
    }
    return -1;
}

C++
8 lines
200 bytes

Thus, we have learned to delete in O(m2).

Here is full version of the code:
Code

//#pragma GCC optimize("Ofast", "unroll-loops")
//#pragma GCC target("sse", "sse2", "sse3", "ssse3", "sse4")

#include <bits/stdc++.h>

#define all(a) a.begin(),a.end()
#define len(a) (int)(a.size())
#define mp make_pair
#define pb push_back
#define fir first
#define sec second
#define fi first
#define se second

using namespace std;

typedef pair<int, int> pii;
typedef long long ll;
typedef long double ld;

template<typename T>
bool umin(T &a, T b) {
    if (b < a) {
        a = b;
        return true;
    }
    return false;
}
template<typename T>
bool umax(T &a, T b) {
    if (a < b) {
        a = b;
        return true;
    }
    return false;
}

#ifdef KIVI
#define DEBUG for (bool _FLAG = true; _FLAG; _FLAG = false)
#define LOG(...) print(#__VA_ARGS__" ::", __VA_ARGS__) << endl
template <class ...Ts> auto &print(Ts ...ts) { return ((cerr << ts << "␣
    "), ...); }
#else
```

```
#define DEBUG while (false)
#define LOG(...)
#endif

const int max_n = -1, inf = 1000111222;


const int C = 20;

struct node {
  int val, id, mask, high;
};


inline int get_high (int x) {
  if (x == 0) {
      return -1;
  }
  return 31 - __builtin_clz(x);
}

inline void swap_bits (int &x, int a, int b) {
  int x1 = bool(x & (1 << a));
  int x2 = bool(x & (1 << b));
  x ^= (x1 ^ x2) << a;
  x ^= (x1 ^ x2) << b;
}

struct xor_basis {
  vector <node> basis;

  inline bool add (int x, int id) {
      int mask = 0;
      for (auto &i : basis) {
          if (umin(x, x ^ i.val)) {
              mask ^= i.mask;
          }
      }
      if (x) {
          mask |= 1 << len(basis);
          for (auto &i : basis) {
              if (umin(i.val, i.val ^ x)) {
                  i.mask ^= mask;
              }
          }
          basis.pb(node{val: x, id: id, mask: mask, high: get_high(x)});
          return true;
      }
      return false;
  }



inline int get_the_same_high_bit (int bits, const vector <int> &val) {
    for (auto &i : basis) {
        if (__builtin_popcount(val[i.id] & bits) & 1) {
            return i.id;
        }
    }
    return -1;
}


inline int rebuild_and_delete (int id) {
    int pos = 0, mn = inf, p2 = 0;
    for (int i = 0; i < len(basis); i++) {
        if (basis[i].id == id) {
            pos = i;
        }
    }
    int bits = 0;
    for (int i = 0; i < len(basis); i++) {
        if (basis[i].mask & (1 << pos)) {
            if (umin(mn, basis[i].high)) {
                p2 = i;
            }
            bits ^= 1 << basis[i].high;
        }
    }

    if (p2 != pos) {
        swap(basis[p2].id, basis[pos].id);
        for (auto &i : basis) {
            swap_bits(i.mask, pos, p2);
        }
        pos = p2;
```

```
        }
        for (int i = 0; i < len(basis); i++) {
            if (i != pos) {
                if (basis[i].mask & (1 << pos)) {
                    basis[i].val ^= basis[pos].val;
                    basis[i].mask ^= basis[pos].mask;
                }
            }
        }
        int good = (1 << pos) - 1;
        int other = ((1 << len(basis)) - 1) ^ (good | (1 << pos));
        basis.erase(basis.begin() + pos);
        for (auto &i : basis) {
            i.mask = ((i.mask & other) >> 1) | (i.mask & good);
        }
        return bits;
    }

};

template<int max_bit> // not inclusive
struct xor_basis_online {

    vector <xor_basis> basises[max_bit + 1];

    int mx;

    vector <pii> where;
    vector <int> val;

    xor_basis_online() : mx(0), cur_id(0) {}

    int cur_id;

    inline int add (int x) {
        val.pb(x);
        where.pb(make_pair(-1, -1));
        int id = cur_id++;
        if (x == 0) {
            return id;
        }
        for (int i = mx; i >= 0; i--) {
            if (basises[i].empty()) {
                continue;
            }
            if (basises[i].back().add(x, id)) {
                basises[i + 1].pb(basises[i].back());
                basises[i].pop_back();
                umax(mx, i + 1);
                for (auto &j : basises[i + 1].back().basis) {
                    where[j.id] = make_pair(i + 1, len(basises[i + 1]) -
                        1);
                }
                return id;
            }
        }
        basises[1].pb(xor_basis());
        basises[1].back().add(x, id);
        where.back() = make_pair(1, len(basises[1]) - 1);
        umax(mx, 1);
        return id;
    }

    inline int move_back (int sz, int pos) {
        int to = len(basises[sz]) - 1;
        if (to == pos) {
            return pos;
        }
        for (auto &i : basises[sz][pos].basis) {
            where[i.id].second = to;
        }
        for (auto &i : basises[sz][to].basis) {
            where[i.id].second = pos;
        }
        swap(basises[sz][pos], basises[sz][to]);
        return to;
    }

    inline void del (int id) {
        if (val[id] == 0) {
            return;
        }
        int sz, pos;
        tie(sz, pos) = where[id];
        pos = move_back(sz, pos);
        while (true) {
```

```
            int bits = basises[sz].back().rebuild_and_delete(id);
            int i = sz - 1;
            while (i > 0 && basises[i].empty()) {
                --i;
            }
            int new_id = -1;
            if (i > 0) {
                new_id = basises[i].back().get_the_same_high_bit(bits, val
                    );
            }
            if (new_id == -1) {
                if (sz > 1) {
                    basises[sz - 1].pb(basises[sz].back());
                    for (auto &j : basises[sz - 1].back().basis) {
                        where[j.id] = make_pair(sz - 1, len(basises[sz -
                            1]) - 1);
                    }
                }
                basises[sz].pop_back();
                if (mx > 0 && basises[mx].empty()) {
                    --mx;
                }
                return;
            }
            int cur = val[new_id];
            assert(basises[sz].back().add(cur, new_id));
            int new_sz = i;
            int new_pos = len(basises[i]) - 1;
            where[new_id] = make_pair(sz, pos);
            id = new_id;
            sz = new_sz;
            pos = new_pos;
        }
    }

    inline int size () {
        return mx;
    }
};

int main() {
//    freopen("input.txt", "r", stdin);
//    freopen("output.txt", "w", stdout);


    ios_base::sync_with_stdio(0);
    cin.tie(0);


    // solution to https://basecamp.eolymp.com/uk/problems/11732
    int n, q, add;
    cin >> n >> add;
    vector <int> p(n);
    xor_basis_online<19> t;
    vector <int> now(n);
    for (int i = 0; i < n; i++) {
        cin >> p[i];
        now[i] = t.add(i ^ p[i]);
    }
    cin >> q;
    int ans = t.size();
    cout << ans << '\n';
    for (int i = 0, l, r; i < q; i++) {
        cin >> l >> r;
        l = (l + ans * add) % n;
        r = (r + ans * add) % n;
        if (l != r) {
            t.del(now[l]);
            t.del(now[r]);
            swap(p[l], p[r]);
            now[l] = t.add(l ^ p[l]);
            now[r] = t.add(r ^ p[r]);
        }
        ans = t.size();
        cout << ans << '\n';
    }
}
```

13 Polynomials

13.1 Berlekamp Massey

```
template<typename T>
vector<T> berlekampMassey(const vector<T> &s) {
    vector<T> c;    // the linear recurrence sequence we are building
    vector<T> oldC; // the best previous version of c to use (the one
        with the rightmost left endpoint)
```

```
    int f = -1;     // the index at which the best previous version of c failed on
    for (int i=0; i<(int)s.size(); i++) {
        // evaluate c(i)
        // delta = s_i - \sum_{j=1}^n c_j s_{i-j}
        // if delta == 0, c(i) is correct
        T delta = s[i];
        for (int j=1; j<=(int)c.size(); j++)
            delta -= c[j-1] * s[i-j];   // c_j is one-indexed, so we actually need index j - 1 in the code
        if (delta == 0)
            continue;   // c(i) is correct, keep going
        // now at this point, delta != 0, so we need to adjust it
        if (f == -1) {
            // this is the first time we're updating c
            // s_i was the first non-zero element we encountered
            // we make c of length i + 1 so that s_i is part of the base case
            c.resize(i + 1);
            mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
            for (T &x : c)
                x = rng();  // just to prove that the initial values don't matter in the first step, I will set to random values
            f = i;
        } else {
            // we need to use a previous version of c to improve on this one
            // apply the 5 steps to build d
            // 1. set d equal to our chosen sequence
            vector<T> d = oldC;
            // 2. multiply the sequence by -1
            for (T &x : d)
                x = -x;
            // 3. insert a 1 on the left
            d.insert(d.begin(), 1);
            // 4. multiply the sequence by delta / d(f + 1)
            T df1 = 0;  // d(f + 1)
            for (int j=1; j<=(int)d.size(); j++)
                df1 += d[j-1] * s[f+1-j];
            assert(df1 != 0);
            T coef = delta / df1;   // storing this in outer variable so it's O(n^2) instead of O(n^2 log MOD)
            for (T &x : d)
                x *= coef;
            // 5. insert i - f - 1 zeros on the left
            vector<T> zeros(i - f - 1);
            zeros.insert(zeros.end(), d.begin(), d.end());
            d = zeros;
            // now we have our new recurrence: c + d
            vector<T> temp = c; // save the last version of c because it might have a better left endpoint
            c.resize(max(c.size(), d.size()));
            for (int j=0; j<(int)d.size(); j++)
                c[j] += d[j];
            // finally, let's consider updating oldC
            if (i - (int) temp.size() > f - (int) oldC.size()) {
                // better left endpoint, let's update!
                oldC = temp;
                f = i;
            }
        }
    }
    return c;
}
```

13.2 Binary Exp FFT

```
#include <bits/stdc++.h>
using namespace std;

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;


typedef complex<double> C;
typedef vector<double> vd;
void fft(vector<C>& a) {
  int n = sz(a), L = 31 - __builtin_clz(n);
  static vector<complex<long double>> R(2, 1);
  static vector<C> rt(2, 1);  // (^ 10% faster if double)
```

```
    for (static int k = 2; k < n; k *= 2) {
      R.resize(n); rt.resize(n);
      auto x = polar(1.0L, acos(-1.0L) / k);
      rep(i,k,2*k) rt[i] = R[i] = i&1 ? R[i/2] * x : R[i/2];
    }
    vi rev(n);
    rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
    rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]);
    for (int k = 1; k < n; k *= 2)
      for (int i = 0; i < n; i += 2 * k) rep(j,0,k) {
        // C z = rt[j+k] * a[i+j+k]; // (25% faster if hand-rolled)
            /// include-line
        auto x = (double *)&rt[j+k], y = (double *)&a[i+j+k];
            /// exclude-line
        C z(x[0]*y[0] - x[1]*y[1], x[0]*y[1] + x[1]*y[0]);
            /// exclude-line
        a[i + j + k] = a[i + j] - z;
        a[i + j] += z;
      }
  }
  vd conv(const vd& a, const vd& b) {
    if (a.empty() || b.empty()) return {};
    vd res(sz(a) + sz(b) - 1);
    int L = 32 - __builtin_clz(sz(res)), n = 1 << L;
    vector<C> in(n), out(n);
    copy(all(a), begin(in));
    rep(i,0,sz(b)) in[i].imag(b[i]);
    fft(in);
    for (C& x : in) x *= x;
    rep(i,0,n) out[i] = in[-i & (n - 1)] - conj(in[i]);
    fft(out);
    rep(i,0,sz(res)) res[i] = imag(out[i]) / (4 * n);
    return res;
  }


  const int maxn = 1 << 20;


  int main(){
    ios_base::sync_with_stdio(false); cin.tie(NULL);
    int n, k; cin >> n >> k;
    vd a(1 << 10, 0);
    for(int i = 0; i < n; i++){
      int x; cin >> x;
      a[x] = 1;
    }
    vd res(1 << 10, 0);
    res[0] = 1;
    auto fix_sizes = [&]() -> void{
      while(__builtin_popcount(sz(a)) != 1){
        a.push_back(0);
      }
      while(__builtin_popcount(sz(res)) != 1){
        res.push_back(0);
      }
      while(sz(a) > sz(res)) res.push_back(0);
      while(sz(res) > sz(a)) a.push_back(0);
    };
    while(k){
      if(k % 2){
        res = conv(res, a);
      }
      a = conv(a, a);
      fix_sizes();
      for(int i = 0; i < sz(a); i++){
        if(llround(a[i]) > 0) a[i] = 1;
        else a[i] = 0;
        if(llround(res[i]) > 0) res[i] = 1;
        else res[i] = 0;
      }
      k /= 2;
    }
    for(int i = 0; i < sz(res); i++){
      ll x = llround(res[i]);
      if(x > 0){
        cout << i << "␣";
      }
    }
    cout << endl;
  }
```

13.3 FFT

```
using cd = complex<double>;
const double PI = acos(-1);
```

```
//declare size of vectors used like this
const int MAXN=2<<19;

void fft(vector<cd> & a, bool invert) {
  int n = (int)a.size();

  for (int i = 1, j = 0; i < n; i++) {
    int bit = n >> 1;
    for (; j & bit; bit >>= 1)
      j ^= bit;
    j ^= bit;

    if (i < j)
      swap(a[i], a[j]);
  }

  for (int len = 2; len <= n; len <<= 1) {
    double ang = 2 * PI / len * (invert ? -1 : 1);
    cd wlen(cos(ang), sin(ang));
    for (int i = 0; i < n; i += len) {
      cd w(1);
      for (int j = 0; j < len / 2; j++) {
  cd u = a[i+j], v = a[i+j+len/2] * w;
  a[i+j] = u + v;
  a[i+j+len/2] = u - v;
  w *= wlen;
      }
    }
  }

  if (invert) {
    for (cd & x : a)
      x /= n;
  }
}

vector<int> multiply(vector<int> const& a, vector<int> const& b) {
    vector<cd> fa(a.begin(), a.end()), fb(b.begin(), b.end());
    int n = 1;
    while (n < a.size() + b.size())
        n <<= 1;
    fa.resize(n);
    fb.resize(n);

    fft(fa, false);
    fft(fb, false);
    for (int i = 0; i < n; i++)
        fa[i] *= fb[i];
    fft(fa, true);

    vector<int> result(n);
    for (int i = 0; i < n; i++)
        result[i] = round(fa[i].real());
    return result;
}

//normalizing for when mult is between 2 big numbers and not polynomials
int carry = 0;
for (int i = 0; i < n; i++){
    result[i] += carry;
    carry = result[i] / 10;
    result[i] %= 10;
}
```

13.4 NTT

```
// number theory transform

const int MOD = 998244353, ROOT = 3;
// const int MOD = 7340033, ROOT = 5;
// const int MOD = 167772161, ROOT = 3;
// const int MOD = 469762049, ROOT = 3;

int power(int base, int exp) {
    int res = 1;
    while (exp) {
        if (exp % 2) res = 1LL * res * base % MOD;
        base = 1LL * base * base % MOD;
        exp /= 2;
    }
    return res;
}

void ntt(vector<int>& a, bool invert) {
  int n = a.size();
  for (int i = 1, j = 0; i < n; i++) {
```

```
    int bit = n >> 1;
    for (; j & bit; bit >>= 1) j ^= bit;
    j ^= bit;
    if (i < j) swap(a[i], a[j]);
  }
  for (int len = 2; len <= n; len <<= 1) {
    int wlen = power(ROOT, (MOD - 1) / len);
    if (invert) wlen = power(wlen, MOD - 2);
    for (int i = 0; i < n; i += len) {
      int w = 1;
      for (int j = 0; j < len / 2; j++) {
        int u = a[i + j], v = 1LL * a[i + j + len / 2] * w % MOD;
        a[i + j] = u + v < MOD ? u + v : u + v - MOD;
        a[i + j + len / 2] = u - v >= 0 ? u - v : u - v + MOD;
        w = 1LL * w * wlen % MOD;
      }
    }
  }
  if (invert) {
    int n_inv = power(n, MOD - 2);
    for (int& x : a) x = 1LL * x * n_inv % MOD;
  }
}

vector<int> multiply(vector<int>& a, vector<int>& b) {
  int n = 1;
  while (n < a.size() + b.size()) n <<= 1;
  a.resize(n), b.resize(n);
  ntt(a, false), ntt(b, false);
  for (int i = 0; i < n; i++) a[i] = 1LL * a[i] * b[i] % MOD;
  ntt(a, true);
  return a;
}
// usage
// vector<int> a = {1, 2, 3}, b = {4, 5, 6};
// vector<int> c = multiply(a, b);
// for (int x : c) cout << x << " ";
```

13.5 Roots NTT

```
1*2^0 + 1 = 2, 1, 1
1*2^1 + 1 = 3, 2, 2
1*2^2 + 1 = 5, 2, 3
2*2^3 + 1 = 17, 2, 9
1*2^4 + 1 = 17, 3, 6
3*2^5 + 1 = 97, 19, 46
3*2^6 + 1 = 193, 11, 158
2*2^7 + 1 = 257, 9, 200
1*2^8 + 1 = 257, 3, 86
15*2^9 + 1 = 7681, 62, 1115
12*2^10 + 1 = 15361, 49, 1254
6*2^11 + 1 = 12289, 7, 8778
3*2^12 + 1 = 12289, 41, 4496
5*2^13 + 1 = 40961, 12, 23894
4*2^14 + 1 = 65537, 15, 30584
2*2^15 + 1 = 65537, 9, 7282
1*2^16 + 1 = 65537, 3, 21846
6*2^17 + 1 = 786433, 8, 688129
3*2^18 + 1 = 786433, 5, 471860
11*2^19 + 1 = 5767169, 12, 3364182
7*2^20 + 1 = 7340033, 5, 4404020
11*2^21 + 1 = 23068673, 38, 21247462
25*2^22 + 1 = 104857601, 21, 49932191
20*2^23 + 1 = 167772161, 4, 125829121
10*2^24 + 1 = 167772161, 2, 83886081
5*2^25 + 1 = 167772161, 17, 29606852
7*2^26 + 1 = 469762049, 30, 15658735
15*2^27 + 1 = 2013265921, 137, 749463956
12*2^28 + 1 = 3221225473, 8, 2818572289
6*2^29 + 1 = 3221225473, 14, 1150437669
3*2^30 + 1 = 3221225473, 13, 1734506024
35*2^31 + 1 = 75161927681, 93, 44450602392
18*2^32 + 1 = 77309411329, 106, 5105338484
```

14 Strings

14.1 Hashing Toolkit: Choosing the Right Hash

C++17 (GCC/Clang). These implementations supplement the existing `HashedString` below. All new headers use namespace `hashing_ref`; keep their dependency headers in the same folder. The shared `HashCore.hpp` appears first. Ranges are zero-based $[l, r)$, including empty ranges. Actual string sizes and segment-tree index arithmetic must fit `int` and available memory. Represented lengths and multiset counts must fit `uint64_t`. The checked length operations throw `overflow_error` if not.

Need	Use
Order-sensitive string equality	`Hash`, `PrefixHash`
Concatenated / repeated strings	`operator*`, `repeated`
Same elements with multiplicities	`MultisetHash`, `MultisetPrefix`
A dynamic set of distinct keys	`SetHash` with membership tracking
A board/state changed in few slots	`ZobristContext`, `ZobristState`
Character replacements and palindromes	`DynamicStringHash`
Rotation-invariant candidate filter	`RotationTraceHash`; verify matching candidates

Shared parameters. String hashes use the prime $M = 2^{61} - 1$ and two separately sampled bases, fixed for the entire process. Multiplication uses unsigned 128-bit intermediates. A byte is encoded as its unsigned value plus one, in $[1, 256]$; zero bytes are supported. Each equality key includes length as well as both residues. Hashes from different processes, seeds or contexts are not interchangeable. Do not edit stored hash fields directly. The randomization is for contest fingerprints, not security.

```cpp
#pragma once
#include <bits/stdc++.h>
#include <cassert>

namespace hashing_ref {
using u64 = uint64_t;
using u128 = __uint128_t;
inline constexpr u64 MOD = (1ULL << 61) - 1;
inline u64 add(u64 a, u64 b) {
    u64 s = a + b;
    return s >= MOD ? s - MOD : s;
}
inline u64 sub(u64 a, u64 b) {
    return a >= b ? a - b : a + MOD - b;
}
inline u64 mul(u64 a, u64 b) {
    return (u128)a * b % MOD;
}
inline u64 power(u64 a, u64 e) {
    u64 r = 1;
    for (; e; e >>= 1, a = mul(a, a))
        if (e & 1) r = mul(r, a);
    return r;
}
inline std::mt19937_64& rng() {
    static std::mt19937_64 gen(
        std::chrono::steady_clock::now().time_since_epoch().count());
    return gen;
}
inline u64 splitmix64(u64 x) {
    x += 0x9e3779b97f4a7c15ULL;
    x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
    x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
    return x ^ (x >> 31);
}
inline u64 checked_length(u128 n) {
    if (n > std::numeric_limits<u64>::max())
        throw std::overflow_error("hash length overflow");
    return (u64)n;
}
} // namespace hashing_ref
```

14.1.1 Collision Guide: What the Probability Refers To

Unequal hashes prove inequality for the represented comparison; equal hashes generally mean "candidate match". Ordinary string, collection, and Zobrist equality methods are Monte Carlo. Verify against the original data if an exact answer is required. The rotation class provides such a verification method explicitly.

Ideal uniform-hash model. Suppose distinct objects receive independent uniform values among R possibilities. A fixed unequal pair collides with probability $1/R$. For Q specified comparisons, the union bound gives at most Q/R. Among all pairs of K objects, it gives at most $\binom{K}{2}/R$, capped at one. The exact birthday probability in this ideal model is

$$1 - \prod_{i=0}^{K-1} \left(1 - \frac{i}{R}\right) \approx 1 - \exp\left(-\frac{K(K-1)}{2R}\right).$$

Thus comparing a million hashes against one fixed object is a very different risk from deduplicating a million hashes among themselves. The number of potentially bad pairs matters.

Polynomial hashes are not independent random labels. For two fixed, different strings of the same length L, subtracting their polynomial hashes gives a nonzero polynomial of degree at most $L - 1$ over the prime field. A nonzero degree-d polynomial has at most d roots. Here each base is uniform over $R = M - 258$ choices, so a single-base collision has probability at most $\min(1, (L-1)/R)$. With two independently sampled bases, the fixed-pair bound is the square of that quantity. Different lengths are separated by the stored length. These are worst-case bounds,

not claims that every collision probability equals $1/M$ or $1/M^2$. For many pairs, use a union bound over a fixed set of possible comparisons; independence between collision events is unnecessary. Input objects must be independent of the sampled parameters for this argument. The degree bound can become weak for huge virtual strings, even when their short descriptions fit in memory.

Practical rules. Use the same sampled bases for all objects that will be compared. Keep both residues; compressing the pair to one machine word loses information. A fixed polynomial base can be targeted by specially constructed inputs. Plain overflow arithmetic modulo 2^{64} is not a prime field, so the polynomial root argument does not apply there. The unsigned wrapping in the multiset and Zobrist implementations below has a different purpose and analysis. The two seeded mixers in collection hashing are a practical fingerprint scheme; they are not a proof of independent random labels for every possible key. Matrix traces from one family also must not be counted as independent hashes.

14.2 Composable Double Hash

For a string s, each component represents $H(s) = \sum_{i=0}^{|s|-1} (\text{byte}(s_i) + 1)B^i \pmod M$. A `Hash` stores two values of H, their $B^{|s|}$ factors, the inverse factors, and the length. The empty string has hash zero, powers one and length zero.

Concatenation is a monoid operation. If a is followed by c, then $H(ac) = H(a) + B^{|a|}H(c)$. Powers and inverse powers multiply. This operation is associative, has the empty string as its identity, and preserves order: generally $a * c \neq c * a$. For example, concatenate the hashes of `"ab"` and `"cd"` to get the hash of `"abcd"`, without its text. Each concatenation costs $O(1)$.

Removing a known prefix. If a is a prefix of c, then $H(c \setminus a) = (H(c) - H(a))B^{-|a|}$. The inverse factor is already stored, so `c/a` costs $O(1)$. The operator assumes a real prefix; it does not check membership. Prefix tables guarantee this precondition when extracting substrings. `PrefixHash(s)` builds in $O(n)$ time and space; `get(l,r)` gives a normalized substring hash in $O(1)$.

Repetition. Binary exponentiation works on any associative operation with an identity. `repeated(h,k)` concatenates k copies in $O(\log(k+1))$ time and $O(1)$ extra space; $k = 0$ gives empty. It checks the represented length first and avoids an unused final squaring that could overflow even when the answer fits. A numerical ordering of hash residues is not lexicographic string ordering; use the comparison helper in the next subsection.

```cpp
#pragma once
#include "HashCore.hpp"

namespace hashing_ref {
struct HashParameters {
    std::array<u64, 2> base, inverse;
    HashParameters() {
        std::uniform_int_distribution<u64> dist(257, MOD - 2);
        for (int k = 0; k < 2; ++k) {
            base[k] = dist(rng());
            inverse[k] = power(base[k], MOD - 2);
        }
    }
};
inline const HashParameters& parameters() {
    static const HashParameters p;
    return p;
}

// Two independent bases in one prime field; byte values are 1..256.
// Same process-wide parameters for every object. Do not edit fields.
struct Hash {
    std::array<u64, 2> h{0, 0}, b{1, 1}, bi{1, 1};
    u64 len = 0; // Empty string is the identity.
    static Hash character(unsigned char c) {
        Hash r;
        r.len = 1;
        r.h.fill((u64)c + 1);
        r.b = parameters().base;
        r.bi = parameters().inverse;
        return r;
    }
    // Concatenate a then c. Order matters.
    friend Hash operator*(const Hash& a, const Hash& c) {
        Hash r;
        r.len = checked_length((u128)a.len + c.len);
        for (int k = 0; k < 2; ++k) {
            r.h[k] = add(a.h[k], mul(a.b[k], c.h[k]));
            r.b[k] = mul(a.b[k], c.b[k]);
            r.bi[k] = mul(a.bi[k], c.bi[k]);
        }
        return r;
    }
    // Remove prefix a from whole c. Caller guarantees it IS a prefix.
    friend Hash operator/(const Hash& c, const Hash& a) {
        assert(a.len <= c.len);
        Hash r;
        r.len = c.len - a.len;
        for (int k = 0; k < 2; ++k) {
```

```
            r.h[k] = mul(sub(c.h[k], a.h[k]), a.bi[k]);
            r.b[k] = mul(c.b[k], a.bi[k]);
            r.bi[k] = mul(c.bi[k], a.b[k]);
        }
        return r;
    }
    auto key() const { return std::make_tuple(len, h[0], h[1]); }
    friend bool operator==(const Hash& a, const Hash& b) {
        return a.len == b.len && a.h == b.h;
    }
    friend bool operator!=(const Hash& a, const Hash& b) {
        return !(a == b);
    }
};
inline Hash repeated(Hash a, u64 copies) {
    checked_length((u128)a.len * copies);
    Hash r;
    while (copies) {
        if (copies & 1) r = r * a;
        copies >>= 1;
        if (copies) a = a * a;
    }
    return r;
}

struct PrefixHash {
    std::string text;
    std::vector<Hash> pref{Hash{}};
    explicit PrefixHash(std::string_view s) : text(s) {
        assert(s.size() < (size_t)INT_MAX);
        pref.reserve(s.size() + 1);
        for (unsigned char c : s)
            pref.push_back(pref.back() * Hash::character(c));
    }
    int size() const { return (int)text.size(); }
    Hash get(int l, int r) const { // Zero-based [l,r), empty allowed.
        assert(0 <= l && l <= r && r <= size());
        return pref[r] / pref[l];
    }
    Hash whole() const { return pref.back(); }
};
} // namespace hashing_ref
```

14.3 Hashing Recipes: LCP, Comparison and Periods

LCP. Substring equality allows a binary search on the common prefix length. `lcp(a,i,b,j,limit)` returns at most `limit` matched bytes in $O(\log(n+1))$ hash queries. `compare(a,l,r,b,i,j)` then compares the first differing byte; if one substring is exhausted, the shorter one comes first. It returns -1, zero or 1, with the same Monte Carlo caveat as substring equality. Byte comparisons use unsigned character values.

Period versus a whole number of repeated blocks. For a nonempty substring $s[l..r)$ and $1 \le k \le r-l$, period k means $s[l..r-k) = s[l+k..r)$. This needs one hash comparison. To be a power of its first k bytes, also require $(r-l) \bmod k = 0$. For instance, `"abcab"` has period three but is not a whole number of copies of `"abc"`. `has_period` and `is_block_power` make this distinction in $O(1)$. They return false for invalid period sizes and for an empty range.

Huge repetitions. The first L bytes of the infinite repetition of nonempty A consist of $\lfloor L/|A| \rfloor$ full copies followed by its prefix of length $L \bmod |A|$. `periodic_prefix` hashes this in $O(\log(L+1))$. `compare_repeated(a,p,b,q)` binary-searches the LCP of A^p and B^q using these hashes, in $O(\log^2(L+2))$ after prefix preprocessing, where L is the shorter represented length. It never constructs the repeated strings. It remains probabilistic, and very large represented lengths weaken the polynomial degree bound.

Exact alternative for this particular comparison. The Fine–Wilf theorem implies that two infinite periodic strings with periods $u = |A|$ and $v = |B|$ are identical if they agree through $u + v - \gcd(u, v)$ positions. Therefore `compare_repeated_exact` scans at most this many bytes, or stops when one finite repetition ends. If there is no mismatch, the finite lengths decide. It takes $O(|A| + |B|)$ time and $O(1)$ extra space beyond the supplied prefix objects, regardless of p, q. Empty strings or zero exponents are handled before any modular index. This exact version is convenient when only two powers need comparison.

```
#pragma once
#include "ComposableHash.hpp"

namespace hashing_ref {
// Hash-based answers are Monte Carlo: collisions can affect results.
inline int lcp(const PrefixHash& a, int i,
               const PrefixHash& b, int j, int limit = INT_MAX) {
    assert(0 <= i && i <= a.size() && 0 <= j && j <= b.size());
    assert(limit >= 0);
    int lo = 0, hi = std::min({a.size() - i, b.size() - j, limit});
    while (lo < hi) {
        int mid = lo + (hi - lo + 1) / 2;
        if (a.get(i, i + mid) == b.get(j, j + mid)) lo = mid;
        else hi = mid - 1;
    }
```

```
    return lo;
}
inline int compare(const PrefixHash& a, int l, int r,
                   const PrefixHash& b, int i, int j) {
    assert(0 <= l && l <= r && r <= a.size());
    assert(0 <= i && i <= j && j <= b.size());
    int n = r - l, m = j - i, k = lcp(a, l, b, i, std::min(n, m));
    if (k == std::min(n, m)) return (n > m) - (n < m);
    unsigned char x = a.text[l + k], y = b.text[i + k];
    return (x > y) - (x < y);
}
inline bool has_period(const PrefixHash& s, int l, int r, int k) {
    assert(0 <= l && l <= r && r <= s.size());
    if (k <= 0 || k > r - l) return false;
    return s.get(l, r - k) == s.get(l + k, r);
}
inline bool is_block_power(const PrefixHash& s, int l, int r, int k) {
    return has_period(s, l, r, k) && (r - l) % k == 0;
}
// First L bytes of the infinite repetition of nonempty s.
inline Hash periodic_prefix(const PrefixHash& s, u64 L) {
    if (L == 0) return {};
    assert(s.size() > 0);
    return repeated(s.whole(), L / s.size()) *
           s.get(0, (int)(L % s.size()));
}
// Compare a^p and b^q without constructing them. Return -1, 0 or 1.
inline int compare_repeated(const PrefixHash& a, u64 p,
                            const PrefixHash& b, u64 q) {
    u64 n = checked_length((u128)a.size() * p);
    u64 m = checked_length((u128)b.size() * q);
    u64 lo = 0, hi = std::min(n, m);
    while (lo < hi) {
        u64 d = hi - lo, mid = lo + d / 2 + d % 2;
        if (periodic_prefix(a, mid) == periodic_prefix(b, mid)) lo = mid
            ;
        else hi = mid - 1;
    }
    if (lo == std::min(n, m)) return (n > m) - (n < m);
    unsigned char x = a.text[lo % a.size()];
    unsigned char y = b.text[lo % b.size()];
    return (x > y) - (x < y);
}
// Exact alternative for comparing just two powers (Fine-Wilf).
inline int compare_repeated_exact(const PrefixHash& a, u64 p,
                                  const PrefixHash& b, u64 q) {
    u64 n = checked_length((u128)a.size() * p);
    u64 m = checked_length((u128)b.size() * q);
    if (n == 0 || m == 0) return (n > m) - (n < m);
    u64 bound = (u64)a.size() + b.size() - std::gcd(a.size(), b.size());
    u64 scan = std::min({n, m, bound});
    for (u64 k = 0; k < scan; ++k) {
        unsigned char x = a.text[k % a.size()], y = b.text[k % b.size()
            ];
        if (x != y) return (x > y) - (x < y);
    }
    return (n > m) - (n < m);
}
} // namespace hashing_ref
```

14.4 Set and Multiset Fingerprints

Ignore order, preserve the right information. An integer key receives the same pair of seeded 64-bit mixer values throughout the process. For a *multiset*, add those values once per occurrence, using intentional unsigned wraparound modulo 2^{64}. Removing one occurrence subtracts its pair. Store the total count too, so different cardinalities cannot compare equal. `MultisetHash` supports insertion, valid deletion, multiset sum with `a+b`, and subtraction of a known submultiset with `a-b`, all in $O(1)$. These operations combine multisets with multiplicity, not distinct-set union. The caller must ensure that deleted occurrences exist.

Range composition and anagrams. `MultisetPrefix` builds cumulative multiset hashes in $O(n)$ time and space. `get(l,r)` subtracts the prefix before l, answering range-multiset equality queries in $O(1)$. It accepts either a `vector<long long>` or a byte string. Signed integer keys are converted to their unique unsigned bit patterns. In `"abccba"`, ranges $[0, 3)$ and $[3, 6)$ both contain one each of `a,b,c`, although their string hashes usually differ. For a small fixed alphabet, prefix frequency counts are an exact alternative costing $O(\sigma)$ per query.

A true set needs membership tracking. For distinct present keys, XOR their fingerprints. Insertion toggles only when a key was absent; deletion toggles only when it was present. `SetHash` maintains this membership in an unordered set, so operations take expected $O(1)$ time and $O(n)$ memory overall. The hash-table key function is randomized too. Raw prefix XOR of an array with duplicates records *odd multiplicity*, not its set of distinct values: two equal copies cancel. For example, `[a,a]` and `[b,b]` would both have XOR zero. Use the sum-based prefix class for multiset/anagram queries. Set union with overlapping keys also needs membership accounting; XOR alone represents symmetric difference.

```cpp
#pragma once
#include "HashCore.hpp"

namespace hashing_ref {
// Same key -> same pair of words for the lifetime of this process.
// Random fingerprints, not cryptographic hashes or independent oracles.
inline std::array<u64, 2> random_id(u64 key) {
    static const std::array<u64, 2> seed{rng()(), rng()()};
    return {splitmix64(key + seed[0]), splitmix64(key + seed[1])};
}

// Multiplicities matter. Unsigned sums wrap modulo 2^64 intentionally.
struct MultisetHash {
    std::array<u64, 2> h{0, 0};
    u64 count = 0;
    void insert(u64 x) {
        u64 next_count = checked_length((u128)count + 1);
        auto r = random_id(x);
        for (int k = 0; k < 2; ++k) h[k] += r[k];
        count = next_count;
    }
    // Precondition: this occurrence of x is present.
    void erase(u64 x) {
        assert(count > 0);
        auto r = random_id(x);
        for (int k = 0; k < 2; ++k) h[k] -= r[k];
        --count;
    }
    friend MultisetHash operator+(MultisetHash a, const MultisetHash& b)
        {
        a.count = checked_length((u128)a.count + b.count);
        for (int k = 0; k < 2; ++k) a.h[k] += b.h[k];
        return a;
    }
    friend MultisetHash operator-(MultisetHash a, const MultisetHash& b)
        {
        assert(b.count <= a.count); // b is a submultiset of a.
        a.count -= b.count;
        for (int k = 0; k < 2; ++k) a.h[k] -= b.h[k];
        return a;
    }
    friend bool operator==(const MultisetHash& a, const MultisetHash& b)
        {
        return a.count == b.count && a.h == b.h;
    }
};
struct MultisetPrefix {
    std::vector<MultisetHash> pref{MultisetHash{}};
    explicit MultisetPrefix(const std::vector<long long>& a) {
        for (long long x : a) {
            pref.push_back(pref.back());
            pref.back().insert((u64)x);
        }
    }
    explicit MultisetPrefix(std::string_view s) {
        for (unsigned char c : s) {
            pref.push_back(pref.back());
            pref.back().insert(c);
        }
    }
    MultisetHash get(int l, int r) const {
        assert(0 <= l && l <= r && (size_t)r < pref.size());
        return pref[r] - pref[l];
    }
};

// A real set: duplicate insertion and absent deletion are ignored.
// Tracking membership prevents XOR from becoming a parity hash.
struct SetHash {
    struct KeyHash {
        size_t operator()(u64 x) const { return random_id(x)[0]; }
    };
    std::unordered_set<u64, KeyHash> present;
    std::array<u64, 2> h{0, 0};
    bool insert(u64 x) {
        if (!present.insert(x).second) return false;
        toggle(x);
        return true;
    }
    bool erase(u64 x) {
        if (!present.erase(x)) return false;
        toggle(x);
        return true;
    }
    friend bool operator==(const SetHash& a, const SetHash& b) {
        return a.present.size() == b.present.size() && a.h == b.h;
```

```
    }
private:
    void toggle(u64 x) {
        auto r = random_id(x);
        for (int k = 0; k < 2; ++k) h[k] ^= r[k];
    }
};
} // namespace hashing_ref
```

14.5 Zobrist Hashing for Incremental States

Represent a state as fixed positions with values in $[0, k)$; -1 means empty. A `ZobristContext(n,k)` assigns two random words to every (position,value) pair, including emptiness. The state hash is the XOR of all positions' selected tokens. To replace a value, XOR out its old token and XOR in its new token.
`set(position,value)` takes $O(1)$; setting the same value again has no effect. A move from one slot to another is two calls, and undoing the calls restores the hash exactly. The context costs $O(nk)$ time/space to build; a state costs $O(n)$ time/space and shares that context. Only compare states using the *same context*. Keep it alive and unchanged while any state uses it; constructing a state directly from a temporary context is explicitly disabled. Copies of a state retain the same context. `key()` gives a pair of words suitable for a visited-state set or transposition table. Include every component that affects the actual state, such as whose turn it is, as another modeled slot. Matching fingerprints can still collide; a full-state check is the exact alternative for visited-state deduplication.

```
#pragma once
#include "HashCore.hpp"

namespace hashing_ref {
// Fixed positions, states 0..states-1; -1 means empty.
// All positions contribute, including their empty state.
struct ZobristContext {
    int positions, states;
    std::vector<std::vector<std::array<u64, 2>>> token;
    ZobristContext(int n, int k) : positions(n), states(k),
        token(n, std::vector<std::array<u64, 2>>(k + 1)) {
        assert(n >= 0 && k > 0);
        for (auto& row : token)
            for (auto& x : row) x = {rng()(), rng()()};
    }
};
struct ZobristState {
    // Context must outlive the state and remain unchanged.
    const ZobristContext* context;
    std::vector<int> value;
    std::array<u64, 2> h{0, 0};
    explicit ZobristState(const ZobristContext& c) : context(&c),
        value(c.positions, -1) {
        for (int i = 0; i < c.positions; ++i)
            for (int k = 0; k < 2; ++k) h[k] ^= c.token[i][0][k];
    }
    ZobristState(ZobristContext&&) = delete;
    void set(int position, int state) {
        assert(0 <= position && position < context->positions);
        assert(-1 <= state && state < context->states);
        auto& old = context->token[position][value[position] + 1];
        auto& next = context->token[position][state + 1];
        for (int k = 0; k < 2; ++k) h[k] ^= old[k] ^ next[k];
        value[position] = state;
    }
    auto key() const { return h; } // Use with this context only.
    friend bool operator==(const ZobristState& a, const ZobristState& b)
         {
        assert(a.context == b.context);
        return a.h == b.h;
    }
};
} // namespace hashing_ref
```

14.6 Dynamic String Hash: Replacements and Palindromes

A segment-tree node stores forward and reverse `Hash` values. If consecutive segments are a, c, their forward hash is $a * c$ and their reverse hash is $\mathrm{rev}(c) * \mathrm{rev}(a)$. An empty segment is the identity. This merge is associative but not commutative, so range queries keep separate left and right accumulators and merge them in their original order.
`DynamicStringHash(s)` builds in $O(n)$ time and space. `set(i,c)` replaces one byte in $O(\log(n+1))$. `get(l,r)` and `reversed(l,r)` return forward/reverse hashes in $O(\log(n+1))$; `palindrome(l,r)` compares them. An empty range is a palindrome. These hashes share the static `PrefixHash` parameters, so static and dynamic results may be compared directly. This class supports replacements, not insertion, deletion or rearrangement of positions. A reverse *hash* does not mean that the tree implements a range-reversal update. Each `Hash` stores six words and a length; the tree holds two hashes per node, so account for its memory constant.

```cpp
#pragma once
#include "ComposableHash.hpp"

namespace hashing_ref {
// Point replacement; forward/reverse hashes; zero-based [l,r).
// Hashes use the same parameters as PrefixHash and Hash.
class DynamicStringHash {
    struct Node { Hash forward, reverse; };
    int n, base = 1;
    std::vector<Node> tree;
    static Node merge(const Node& a, const Node& b) {
        return {a.forward * b.forward, b.reverse * a.reverse};
    }
public:
    explicit DynamicStringHash(std::string_view s) : n((int)s.size()) {
        while (base < n) base *= 2;
        tree.resize(2 * base);
        for (int i = 0; i < n; ++i) {
            Hash c = Hash::character((unsigned char)s[i]);
            tree[base + i] = {c, c};
        }
        for (int v = base - 1; v > 0; --v)
            tree[v] = merge(tree[2 * v], tree[2 * v + 1]);
    }
    void set(int position, unsigned char c) {
        assert(0 <= position && position < n);
        int v = base + position;
        Hash h = Hash::character(c);
        tree[v] = {h, h};
        for (v /= 2; v; v /= 2)
            tree[v] = merge(tree[2 * v], tree[2 * v + 1]);
    }
    Node query(int l, int r) const {
        assert(0 <= l && l <= r && r <= n);
        Node left, right;
        for (l += base, r += base; l < r; l /= 2, r /= 2) {
            if (l & 1) left = merge(left, tree[l++]);
            if (r & 1) right = merge(tree[--r], right);
        }
        return merge(left, right);
    }
    Hash get(int l, int r) const { return query(l, r).forward; }
    Hash reversed(int l, int r) const { return query(l, r).reverse; }
    bool palindrome(int l, int r) const {
        Node x = query(l, r);
        return x.forward == x.reverse;
    }
};
} // namespace hashing_ref
```

14.7 Rotation-Invariant Matrix Fingerprint

Purpose. This is the lecture's specialized matrix-trace idea, implemented as a candidate filter with an exact verification method. A rotation changes `"abcab"` into `"cabab"`; it does not allow arbitrary reordering of the letters.
Assign every byte an independently generated dense invertible 3×3 matrix over the prime field. Generate again if inversion fails. A string is represented by its ordered matrix product P. For a split $s = uv$ with products U, V, its rotation vu has product $VU = U^{-1}(UV)U$. Thus rotations have the same $\operatorname{tr}(P)$ and $\operatorname{tr}(P^2)$. The code includes the length in its fingerprint as well.
Range extraction. Store ordered prefix products A_i and their inverses I_i. Append a byte matrix R by $A_{i+1} = A_i R$ and $I_{i+1} = R^{-1} I_i$; note the reversed order in the inverse. The product for $[l, r)$ is $I_l A_r$. Each query performs a constant number of 3×3 multiplications. `fingerprint(l,r)` returns $(r - l, \operatorname{tr} P, \operatorname{tr}(P^2))$ in $O(1)$ for this fixed dimension, after $O(n)$ prefix preprocessing. The prefix arrays alone occupy about $144(n + 1)$ bytes: this is a large constant, and two million-character objects need roughly 288 MB before their text and other allocations.
What equality guarantees. Unequal fingerprints rule out rotation equivalence. Equal fingerprints are only a necessary condition. The two traces depend on the same random matrices; calling them two independent 61-bit hashes is unjustified. No universal collision bound for all nonrotation word pairs is claimed here. In particular, the nonzero-polynomial condition needed for a Schwartz–Zippel bound has not been established for every pair. This distinction also avoids importing the lecture's heuristic multi-trace estimates as guarantees.
`same_rotation(l,r,other,i,j)` first applies the fingerprint filter, then verifies a matching candidate with KMP: search one length-L string among the $2L - 1$ characters of the other's doubled cyclic sequence. It uses $O(L)$ time and space for the verification, and returns an *exact* boolean. Matching/actually equivalent ranges therefore take linear time; this is not an exact $O(1)$ rotation-equivalence data structure. `exact_rotation(a,b)` is available separately and does not require matrix preprocessing.

```cpp
#pragma once
#include "HashCore.hpp"

namespace hashing_ref {
struct Matrix3 {
    u64 a[3][3]{};
    static Matrix3 identity() {
        Matrix3 r;
        for (int i = 0; i < 3; ++i) r.a[i][i] = 1;
        return r;
    }
    friend Matrix3 operator*(const Matrix3& x, const Matrix3& y) {
        Matrix3 r;
        for (int i = 0; i < 3; ++i)
            for (int k = 0; k < 3; ++k)
                for (int j = 0; j < 3; ++j)
                    r.a[i][j] = add(r.a[i][j], mul(x.a[i][k], y.a[k][j])
                        );
        return r;
    }
    u64 trace() const { return add(add(a[0][0], a[1][1]), a[2][2]); }
    std::optional<Matrix3> inverse() const {
        u64 b[3][6]{};
        for (int i = 0; i < 3; ++i) {
            for (int j = 0; j < 3; ++j) b[i][j] = a[i][j];
            b[i][i + 3] = 1;
        }
        for (int c = 0; c < 3; ++c) {
            int pivot = c;
            while (pivot < 3 && b[pivot][c] == 0) ++pivot;
            if (pivot == 3) return std::nullopt;
            for (int j = 0; j < 6; ++j) std::swap(b[c][j], b[pivot][j]);
            u64 inv = power(b[c][c], MOD - 2);
            for (int j = 0; j < 6; ++j) b[c][j] = mul(b[c][j], inv);
            for (int i = 0; i < 3; ++i) if (i != c) {
                u64 factor = b[i][c];
                for (int j = 0; j < 6; ++j)
                    b[i][j] = sub(b[i][j], mul(factor, b[c][j]));
            }
        }
        Matrix3 r;
        for (int i = 0; i < 3; ++i)
            for (int j = 0; j < 3; ++j) r.a[i][j] = b[i][j + 3];
        return r;
    }
};
struct RotationParameters {
    std::array<Matrix3, 256> letter, inverse;
    RotationParameters() {
        std::uniform_int_distribution<u64> dist(0, MOD - 1);
        for (int c = 0; c < 256; ++c) {
            while (true) {
                for (auto& row : letter[c].a)
                    for (u64& x : row) x = dist(rng());
                auto inv = letter[c].inverse();
                if (inv) { inverse[c] = *inv; break; }
            }
        }
    }
};
inline const RotationParameters& rotation_parameters() {
    static const RotationParameters p;
    return p;
}
// Exact fallback, independent of the fingerprint. O(|a|+|b|).
inline bool exact_rotation(std::string_view a, std::string_view b) {
    if (a.size() != b.size()) return false;
    if (a.empty()) return true;
    assert(a.size() < (size_t)INT_MAX);
    int n = (int)a.size();
    std::vector<int> pi(n);
    for (int i = 1; i < n; ++i) {
        int j = pi[i - 1];
        while (j && b[i] != b[j]) j = pi[j - 1];
        if (b[i] == b[j]) ++j;
        pi[i] = j;
    }
    int matched = 0;
    for (size_t i = 0; i < 2 * a.size() - 1; ++i) {
        char c = a[i % a.size()];
        while (matched && c != b[matched]) matched = pi[matched - 1];
        if (c == b[matched]) ++matched;
        if (matched == n) return true;
    }
    return false;
}
```

```cpp

// Rotation invariant FILTER, not an equivalence certificate.
// Same trace family components are not independent random hashes.
class RotationTraceHash {
    std::string text;
    std::vector<Matrix3> pref, inv;
public:
    explicit RotationTraceHash(std::string_view s) : text(s),
        pref(s.size() + 1, Matrix3::identity()), inv(pref) {
        assert(s.size() < (size_t)INT_MAX);
        const auto& p = rotation_parameters();
        for (int i = 0; i < (int)s.size(); ++i) {
            unsigned char c = s[i];
            pref[i + 1] = pref[i] * p.letter[c];
            inv[i + 1] = p.inverse[c] * inv[i];
        }
    }
    std::array<u64, 3> fingerprint(int l, int r) const {
        assert(0 <= l && l <= r && r <= (int)text.size());
        Matrix3 m = inv[l] * pref[r];
        return {(u64)(r - l), m.trace(), (m * m).trace()};
    }
    bool same_rotation(int l, int r, const RotationTraceHash& b,
                       int i, int j) const {
        if (fingerprint(l, r) != b.fingerprint(i, j)) return false;
        return exact_rotation(std::string_view(text).substr(l, r - l),
                              std::string_view(b.text).substr(i, j - i))
                                  ;
    }
};
} // namespace hashing_ref
```

14.8 Hash Toolkit: Runnable Examples

Build `strings/HashExamples.cpp` with C++17. All comments give the expected output for the shown inputs. Keep the original objects if hash matches need exact verification.

```cpp
#include "HashRecipes.hpp"
#include "CollectionHash.hpp"
#include "ZobristHash.hpp"
#include "DynamicStringHash.hpp"
#include "RotationTraceHash.hpp"
using namespace std;
using namespace hashing_ref;

int main() {
    PrefixHash a("ab"), b("abab"), s("abcabc");
    cout << (repeated(a.whole(), 2) == b.whole()) << '\n'; // 1
    cout << (s.get(3, 6) == s.get(0, 3)) << '\n';        // 1
    cout << lcp(s, 0, s, 3) << '\n';                     // 3
    cout << is_block_power(s, 0, 6, 3) << '\n';          // 1
    // Compare two virtual strings of length 2*10^18.
    cout << compare_repeated_exact(a, 1000000000000000000ULL,
                                   b, 5000000000000000000ULL) << '\n'; // 0

    MultisetPrefix p({4, 1, 4, 4, 4, 1});
    cout << (p.get(0, 3) == p.get(3, 6)) << '\n'; // 1: same counts
    SetHash x, y;
    x.insert(4); x.insert(4); y.insert(4);
    cout << (x == y) << '\n'; // 1: duplicate insertion is ignored

    ZobristContext context(8, 3); // Eight slots, states 0,1,2.
    ZobristState board(context), copy(context);
    board.set(2, 1); copy.set(2, 1);
    cout << (board == copy) << '\n'; // 1
    board.set(2, -1); board.set(5, 1); // Move, clearing the old slot.
    cout << (board == copy) << '\n'; // 0 (probabilistic)

    DynamicStringHash d("abca");
    cout << d.palindrome(0, 4) << '\n'; // 0
    d.set(2, 'b');
    cout << d.palindrome(0, 4) << '\n'; // 1: "abba"

    RotationTraceHash r("abcab"), t("cabab"), u("abacb");
    cout << r.same_rotation(0, 5, t, 0, 5) << '\n'; // 1, exact
    cout << r.same_rotation(0, 5, u, 0, 5) << '\n'; // 0, exact
}
```

Sources. Topics inspired by Lautaro Lasorsa, Hashing (2026). The implementations and examples here are independently written; the matrix API deliberately separates its fingerprint from exact verification. Further reading: CP-Algorithms: String Hashing, and Fine–Wilf periodicity in Combinatorics on Words, Periodicity Lemma.

14.9 Hashed String

```cpp

/*
                                    Hashed string
  ------------------------------------------------------------------
  Class for hashing string. Allows retrieval of hashes of any substring
      in the string.

  Double hash or use big mod values to avoid problems with collisions

  Important to take into account that the values used for the letters
      are not 0 to 26. it is ascii characters.

  Time Complexity(Construction): O(n)
  Space Complexity: O(n)
*/

const ll MOD = 212345678987654321LL;
const ll base = 33;

class HashedString {
  private:
  static const long long M = 1e9 + 9;
  static const long long B = 9973;

  static vector<long long> pow;

  vector<long long> p_hash;

  public:
  HashedString(const string &s) : p_hash(s.size() + 1) {
    while (pow.size() <= s.size()) { pow.push_back((pow.back() * B) % M)
        ; }

    p_hash[0] = 0;
    for (int i = 0; i < (int)s.size(); i++) {
      p_hash[i + 1] = ((p_hash[i] * B) % M + s[i]) % M;
    }
  }
  // substring [start, end]
  long long get_hash(int start, int end) {
    long long raw_val =
        (p_hash[end + 1] - (p_hash[start] * pow[end - start + 1]));
    return (raw_val % M + M) % M;
  }
    // substring [start, end] with l added at the end. Remember it is
        ascii characters, not the number from 1 to 26 for the hash.
    long long get_hash(int start, int end, int l){
      if(start > end) return l;
      long long val = ((p_hash[end + 1] * B) + l - (p_hash[start] * pow[
          end - start + 2]));
      val = (val % M + M) % M;
      return val;
    }
};
// you cant skip this
vector<long long> HashedString::pow = {1};
```

14.10 KMP

```cpp
/*
                                    KMP
  ------------------------------------------------------------------
  Computes the prefix function for a string.

  Maximum length of substring that ends at position i and is proper
      prefix (not equal to string itself) of string
  pf[i]  is the length of the longest proper prefix of the substrings
      [0......i]$  which is also a suffix of this substring.

  For matching, one can append the string with a delimiter like $
      between them

  Time Complexity: O(n)
  Space Complexity: O(n)
*/

vector<int> KMP(string s){
  int n=(int)s.length();
  vector<int> pf(n, 0);
```

```
  for(int i=1;i<n;i++){
    int j=pf[i-1];
    while(j>0 && s[i]!=s[j]){
      j=pf[j-1];
    }
    if(s[i]==s[j]){
      pf[i]=j+1;
    }
  }
  return pf;
}

// Counts how many times each prefix occurs
// Same thing can be done for two strings but only considering indices of second string
vector<int> count_occurrences_of_prefixes(vector<int> pf){
  int n=(int)pf.size();
  vector<int> ans(n + 1);
  for (int i = 0; i < n; i++)
    ans[pi[i]]++;
  for (int i = n-1; i > 0; i--)
    ans[pi[i-1]] += ans[i];
  for (int i = 0; i <= n; i++)
    ans[i]++;
}

// Computes automaton for string
// useful for not having to recalculate KMP of string s
// can be utilized when the second string (the one in which we are trying to count occurences)
// is very large
void compute_automaton(string s, vector<vector<int>>& aut) {
  s += '#';
  int n = s.size();
  vector<int> pi = KMP(s);
  aut.assign(n, vector<int>(26));
  for (int i = 0; i < n; i++) {
      for (int c = 0; c < 26; c++) {
          if (i > 0 && 'a' + c != s[i])
              aut[i][c] = aut[pi[i-1]][c];
          else
              aut[i][c] = i + ('a' + c == s[i]);
      }
  }
}
```

14.11 Least Rotation String

```
/*
                        Min cyclic shift
  ---------------------------------------------------------------
  Finds the lexicographically minimum cyclic shift of a string

  Time Complexity: O(n)
  Space Complexity: O(n)
*/

string least_rotation(string s)
{
  s += s;
  vector<int> f(s.size(), -1);
  int k = 0;
  for(int j = 1; j < s.size(); j++)
  {
    char sj = s[j];
    int i = f[j - k - 1];
    while(i != -1 && sj != s[k + i + 1])
    {
      if(sj < s[k + i + 1]){
        k = j - i - 1;
      }
      i = f[i];
    }
    if(sj != s[k + i + 1])
    {
      if(sj < s[k]){
        k = j;
      }
      f[j - k] = -1;
    }
    else
      f[j - k] = i + 1;
  }
  return s.substr(k, s.size() / 2);
}
```

14.12 Manacher

```cpp
/*
                                Manacher
  -----------------------------------------------------------------
  Computes the length of the longest palindrome centered at position i.

  p[i] is length of biggest palindrome centered in this position.
  Be careful with characters that are inserted to account for odd and
      even palindromes

  Time Complexity: O(n)
  Space Complexity: O(n)

*/

// Number of palindromes centered at each position

vector<int> manacher_odd(string s)
{
  int n = s.size();
  s = "$" + s + "^";
  vector<int> p(n + 2);
  int l = 1, r = 1;
  for (int i = 1; i <= n; i++)
  {
    p[i] = max(0, min(r - i, p[l + (r - i)]));
    while (s[i - p[i]] == s[i + p[i]])
    {
      p[i]++;
    }
    if (i + p[i] > r)
    {
      l = i - p[i], r = i + p[i];
    }
  }
  return vector<int>(begin(p) + 1, end(p) - 1);
}
vector<int> manacher(string s)
{
  string t;
  for (auto c : s)
  {
    t += string("#") + c;
  }
  auto res = manacher_odd(t + "#");
  return vector<int>(begin(res) + 1, end(res) - 1);
}

// example use. Longest palindrome that ends at each position
int main(){
  ios_base::sync_with_stdio(false); cin.tie(NULL);
  string s; cin >> s;
  int n = sz(s);
  auto man = manacher(s);
  vector<int> a(n), b(n);
  // a[i] is how many odd palindromes - 1 with this center
  // b[i] is how many even palindromes with this center + 1 as
  for(int i = 0; i < sz(man); i += 2){
    a[i / 2] = (man[i] - 1) / 2;
    if(i) b[i / 2] = (man[i - 1]) / 2;
  }
  vector<int> ans(n + 1, 1);
  for(int i = 0; i < n; i++){
    ans[i + a[i]] = max(ans[i + a[i]], a[i] * 2 + 1);
    if(b[i])ans[i + b[i] - 1] = max(ans[i + b[i] - 1], b[i] * 2);
  }
  for(int i = n - 2; i >= 0; i--){
    ans[i] = max(ans[i], ans[i + 1] - 2);
  }
  for(int i = 0; i < n; i++){
    cout << ans[i] << "␣";
  }
  cout << endl;
}

// usage
// vector<int> p = manacher("abacaba");
// this will return {2, 1, 4, 1, 2, 1, 8, 1, 2, 1, 4, 1, 2}
// vector<int> p = manacher("abaaba");
// this will return {2, 1, 4, 1, 2, 7, 2, 1, 4, 1, 2}
```

14.13 Suffix Array

```cpp
/*
                                Suffix Array
```

```
  ---------------------------------------------------------------
  Computes the suffix array of a string in O(n log n).
  Sorted array of all cyclic shifts of a string.

  If you want sorted suffixes append $ to the end of the string.
  lc is longest common prefix. Lcp of two substrings j > i is min(lc[i],
       ....., lc[j - 1]).

  To compute Largest common substring of multiple strings
  Join all strings separating them with special character like $ (it has
       to be different for each string)
  Sliding window on lcp array (all string have to appear on  the sliding
       window and
  the lcp of the interval will give the length of the substring that
      appears on all strings)

  Time Complexity: O(n log n)
  Space Complexity: O(n)

*/

struct SuffixArray
{
  int n;
  string t;
  vector<int> sa, rk, lc;
  SuffixArray(const std::string &s)
  {
    n = s.length();
    t = s;
    sa.resize(n);
    lc.resize(n - 1);
    rk.resize(n);
    std::iota(sa.begin(), sa.end(), 0);
    std::sort(sa.begin(), sa.end(), [&](int a, int b)
              { return s[a] < s[b]; });
    rk[sa[0]] = 0;
    for (int i = 1; i < n; ++i)
      rk[sa[i]] = rk[sa[i - 1]] + (s[sa[i]] != s[sa[i - 1]]);
    int k = 1;
    std::vector<int> tmp, cnt(n);
    tmp.reserve(n);
    while (rk[sa[n - 1]] < n - 1)
    {
      tmp.clear();
      for (int i = 0; i < k; ++i)
        tmp.push_back(n - k + i);
      for (auto i : sa)
        if (i >= k)
          tmp.push_back(i - k);
      std::fill(cnt.begin(), cnt.end(), 0);
      for (int i = 0; i < n; ++i)
        ++cnt[rk[i]];
      for (int i = 1; i < n; ++i)
        cnt[i] += cnt[i - 1];
      for (int i = n - 1; i >= 0; --i)
        sa[--cnt[rk[tmp[i]]]] = tmp[i];
      std::swap(rk, tmp);
      rk[sa[0]] = 0;
      for (int i = 1; i < n; ++i)
        rk[sa[i]] = rk[sa[i - 1]] + (tmp[sa[i - 1]] < tmp[sa[i]] || sa[i
             - 1] + k == n || tmp[sa[i - 1] + k] < tmp[sa[i] + k]);
      k *= 2;
    }
    for (int i = 0, j = 0; i < n; ++i)
    {
      if (rk[i] == 0)
      {
        j = 0;
      }
      else
      {
        for (j -= j > 0; i + j < n && sa[rk[i] - 1] + j < n && s[i + j]
            == s[sa[rk[i] - 1] + j];)
          ++j;
        lc[rk[i] - 1] = j;
      }
    }
  }

  // Finds if string p appears as substring in the string
  // might now work perfectly
  int search(string &p){
      int tam = p.size();
      int l = 0, r = n;

```

```
        string tmp = "";
        while(r > l) {
            int m = l + (r-l)/2;
            tmp = t.substr(sa[m], min(n-sa[m], tam));
            if(tmp >= p){
                r = m;
            } else {
                l = m + 1;
            }
        }
        if(l < n) {
            tmp = t.substr(sa[l], min(n-sa[l], tam));
        } else{
            return -1;
        }
        if(tmp == p){
            return l;
        } else {
            return -1;
        }
    }


    // Counts number of times a string p appears as substring in string
    int count(string &p) {
        int x = search(p);
        if(x == -1) return 0;
        int cnt = 0;
        int tam = p.size();
        int maxx = 0;
        while((1 << maxx) + x < n) maxx++;
        int y = x;
        for(int i = maxx-1; i >= 0; i--) {
            if(x + (1 << i) >= n) continue;
            string tmp = t.substr(sa[x + (1 << i)], min(n-sa[x + (1 << i)
                ], tam));
            if(tmp == p) x += (1 << i);
        }
        return x-y+1;
    }
};
int main() {
    cin.tie(0)->sync_with_stdio(0);
    string s; cin >> s;
    SuffixArray SA(s);

    int q; cin >> q;
    for(int t = 0; t < q; t++) {
        string tmp; cin >> tmp;
        cout << SA.count(tmp) << endl;
    }

    return 0;
}
```

14.14 Suffix Automaton

```
/*
                                Suffix Automaton
  ------------------------------------------------------------
  Constructs suffix automaton for a given string.
  Be careful with overlapping substrings.

  Firstposition if first position string ends in.
  If you want starting index you need to substract length of the string
      being searched.

  len is length of longest string of state

  Time Complexity(Construction): O(n)
  Space Complexity: O(n)
*/


struct state {
  int len, link, firstposition;
  vector<int> inv_link; // can skip for almost everything
  map<char, int> next;
};

const int MAXN = 100000;
state st[MAXN * 2];
ll cnt[MAXN*2], cntPaths[MAXN*2], cntSum[MAXN*2], cnt1[2 * MAXN];
int sz, last;

// call this first
```

```cpp
void initSuffixAutomaton() {
  st[0].len = 0;
  st[0].link = -1;
  sz++;
  last = 0;
}

// construction is O(n)
void insertChar(char c) {
  int cur = sz++;
  st[cur].len = st[last].len + 1;
  st[cur].firstposition=st[last].len;
  int p = last;
  while (p != -1 && !st[p].next.count(c)) {
      st[p].next[c] = cur;
      p = st[p].link;
  }
  if (p == -1) {
      st[cur].link = 0;
  } else {
      int q = st[p].next[c];
      if (st[p].len + 1 == st[q].len) {
          st[cur].link = q;
      } else {
          int clone = sz++;
          st[clone].len = st[p].len + 1;
          st[clone].next = st[q].next;
          st[clone].link = st[q].link;
          st[clone].firstposition=st[q].firstposition;
          while (p != -1 && st[p].next[c] == q) {
              st[p].next[c] = clone;
              p = st[p].link;
          }
          st[q].link = st[cur].link = clone;
      }
  }
  last = cur;
  cnt[last]=1;
}

// searches for the starting position in O(len(s)). Returns starting
    index of first ocurrence or -1 if it does not appear.
int search(string s){
  int cur=0, i=0, n=(int)s.length();
  while(i<n){
    if(!st[cur].next.count(s[i])) return -1;
    cur=st[cur].next[s[i]];
    i++;
  }
  //sumar 2 si se quiere 1 indexado
  return st[cur].firstposition-n+1;
}

void dfs(int cur){
  cntPaths[cur]=1;
  for(auto [x, y]:st[cur].next){
    if(cntPaths[y]==0) dfs(y);
    cntPaths[cur]+=cntPaths[y];
  }
}

// Counts how many paths exist from state. How many substrings exist
    from a specific state.
// Stored in cntPaths
void countPaths(){
  dfs(0);
}

// Computes the number of times each state appears
void countOcurrences(){
  vector<pair<int, int>> a;
  for(int i=sz-1;i>0;i--){
    a.push_back({st[i].len, i});
  }
  sort(a.begin(), a.end());
  for(int i=sz-2;i>=0;i--){
    cnt[st[a[i].second].link]+=cnt[a[i].second];
  }
}

void dfs1(int cur){
  for(auto [x, y]:st[cur].next){
    if(cntSum[y]==cnt[y]) dfs1(y);
    cntSum[cur]+=cntSum[y];
  }
}
```

```cpp

// Computes the number of times each state or any of its children appear in the string.
void countSumOcurrences(){
  for(int i=0;i<sz;i++){
    cntSum[i]=cnt[i];
  }
  dfs1(0);
}


// Counts number of paths that can reach specific state.
void countPathsReverse(){
  cnt1[0]=1;
  queue<int> q;
  q.push(0);
  vector<int> in(2*MAXN, 0);
  for(int i=0;i<sz;i++){
    for(auto [x, y]:st[i].next){
      in[y]++;
    }
  }
  while((int)q.size()){
    int cur=q.front();
    q.pop();
    for(auto [x, y]:st[cur].next){
      cnt1[y]+=cnt1[cur];
      in[y]--;
      if(in[y]==0){
        q.push(y);
      }
    }
  }
}

// Computes the kth smallest string that appears on the string (counting repetitions)
string kthSmallest(ll k){
  string s="";
  int cur=0;
  while(k>0){
    for(auto [c, y]:st[cur].next){
      if(k>cntSum[y]) k-=cntSum[y];
      else{
        k-=cnt[y];
        s+=c;
        cur=y;
        break;
      }
    }
  }
  return s;
}


// Computes the kth smallest string that appears on the string (without counting repetitions)
string kthSmallestDistinct(ll k){
  string s="";
  int cur=0;
  while(k>0){
    for(auto [c, y]:st[cur].next){
      if(k>cntPaths[y]) k-=cntPaths[y];
      else{
        k--;
        s+=c;
        cur=y;
        break;
      }
    }
  }
  return s;
}


// Precomputation to find all occurrences of a substring
void precoumpte_for_all_ocurrences(){
  for (int v = 1; v < sz; v++) {
    st[st[v].link].inv_link.push_back(v);
  }
}

// Finding all occurences of substring in string
// P_length is length of substring
// v is state where first ocurrence happens
// be careful as indices can appear multiple times due to clone states
```

```cpp
// if you want to avoid duplicate positions utilize set or have a flag
    for each state to know if it is clone or not
void output_all_occurrences(int v, int P_length) {
  cout << st[v].firstposition - P_length + 1 << endl;
  for (int u : st[v].inv_link)
    output_all_occurrences(u, P_length);
}


//longest common substring
//build automaton for s first
string lcs (string S, string T) {
    int v = 0, l = 0, best = 0, bestpos = 0;
    for (int i = 0; i < T.size(); i++) {
        while (v && !st[v].next.count(T[i])) {
            v = st[v].link ;
            l = st[v].len;
        }
        if (st[v].next.count(T[i])) {
            v = st [v].next[T[i]];
            l++;
        }
        if (l > best) {
            best = l;
            bestpos = i;
        }
    }
    return T.substr(bestpos - best + 1, best);
}


int main(){
  ios_base::sync_with_stdio(false); cin.tie(NULL);
  string s; cin >> s;
  initSuffixAutomaton();
  for(char c:s){
    insertChar(c);
  }
}
```

14.15 Suffix Automaton Extended

C++17. Alternative implementation with occurrence counts, lexicographic queries, matching, and substring statistics. Positions are zero-based; `kthSmallest` uses one-based k. Compile each version separately.

```cpp
// Quick example (inside main):
SuffixAutomaton sam("ababa");
sam.search("aba");               // 0
sam.occurrences("aba");          // 2
sam.allOccurrences("aba");       // {0, 2}
sam.kthSmallestDistinct(3);      // "aba"
sam.lcs("cabac");                // "aba"
```

```cpp
#pragma once
#include <bits/stdc++.h>
using namespace std;

struct SuffixAutomaton {
    struct State {
        int len = 0, link = -1, firstposition = -1;
        bool isClone = false;
        map<char, int> next;
        vector<int> inv_link;
    };

    string text;
    vector<State> st;
    vector<long long> occurrenceCount;       // Occurrences of a state's
        substrings.
    vector<long long> pathCountFromState;    // Paths starting here,
        including the empty path.
    vector<long long> extensionOccurrenceSum; // Occurrences of the
        current substring and extensions.
    vector<long long> pathCountFromRoot;     // Paths from the root to
        each state.
    int sz = 0, last = 0;
    bool ready = false;

    SuffixAutomaton(const string& s = "") {
        build(s);
    }

    void build(string s) {
        text.clear();
        text.reserve(s.size());
        st.clear();
        st.reserve(2 * s.size() + 1);
```

```
        st.emplace_back();
        sz = 1;
        last = 0;
        ready = false;
        occurrenceCount.clear();
        pathCountFromState.clear();
        extensionOccurrenceSum.clear();
        pathCountFromRoot.clear();
        for (char c : s) insertChar(c);
    }

    void insertChar(char c) {
        ready = false;
        int cur = sz++;
        st.emplace_back();
        st[cur].len = st[last].len + 1;
        st[cur].firstposition = st[cur].len - 1;

        int p = last;
        while (p != -1 && !st[p].next.count(c)) {
            st[p].next[c] = cur;
            p = st[p].link;
        }

        if (p == -1) {
            st[cur].link = 0;
        } else {
            int q = st[p].next[c];
            if (st[p].len + 1 == st[q].len) {
                st[cur].link = q;
            } else {
                int clone = sz++;
                State copy;
                copy.len = st[p].len + 1;
                copy.link = st[q].link;
                copy.firstposition = st[q].firstposition;
                copy.next = st[q].next;
                copy.isClone = true;
                st.push_back(move(copy));

                while (p != -1) {
                    auto it = st[p].next.find(c);
                    if (it == st[p].next.end() || it->second != q) break
                        ;
                    it->second = clone;
                    p = st[p].link;
                }
                st[q].link = st[cur].link = clone;
            }
        }
        last = cur;
        text += c;
    }

    int findState(const string& s) const {
        int v = 0;
        for (char c : s) {
            auto it = st[v].next.find(c);
            if (it == st[v].next.end()) return -1;
            v = it->second;
        }
        return v;
    }

    // First occurrence, zero-based. Returns -1 if absent.
    int search(const string& s) const {
        if (s.empty()) return 0;
        int v = findState(s);
        return v == -1 ? -1 : st[v].firstposition - (int)s.size() + 1;
    }

    // State indices sorted by increasing len, in O(n).
    vector<int> statesByLength() const {
        vector<int> bucket(st[last].len + 1, 0), order(sz);
        for (const auto& state : st) ++bucket[state.len];
        for (int i = 1; i < (int)bucket.size(); ++i)
            bucket[i] += bucket[i - 1];
        for (int v = sz - 1; v >= 0; --v)
            order[--bucket[st[v].len]] = v;
        return order;
    }

    // Computes all counters and inverse suffix links in O(n).
    // Calling it repeatedly without an insertion does no extra work.
    void precompute() {
        if (ready) return;
```

```
        occurrenceCount.assign(sz, 0);
        pathCountFromState.assign(sz, 1);
        extensionOccurrenceSum.assign(sz, 0);
        pathCountFromRoot.assign(sz, 0);
        for (auto& state : st) state.inv_link.clear();

        for (int v = 1; v < sz; ++v) {
            occurrenceCount[v] = !st[v].isClone;
            st[st[v].link].inv_link.push_back(v);
        }

        vector<int> order = statesByLength();

        // Propagate occurrence counts through suffix links.
        for (int i = sz - 1; i > 0; --i) {
            int v = order[i];
            occurrenceCount[st[v].link] += occurrenceCount[v];
        }

        // Transitions always lead to states with greater len.
        for (int i = sz - 1; i >= 0; --i) {
            int v = order[i];
            extensionOccurrenceSum[v] = v == 0 ? 0 : occurrenceCount[v];
            for (auto [c, u] : st[v].next) {
                pathCountFromState[v] += pathCountFromState[u];
                extensionOccurrenceSum[v] += extensionOccurrenceSum[u];
            }
        }

        pathCountFromRoot[0] = 1;
        for (int v : order)
            for (auto [c, u] : st[v].next)
                pathCountFromRoot[u] += pathCountFromRoot[v];

        ready = true;
    }

    // Counts overlapping occurrences too.
    long long occurrences(const string& s) {
        if (s.empty()) return (long long)st[last].len + 1;
        int v = findState(s);
        if (v == -1) return 0;
        precompute();
        return occurrenceCount[v];
    }

    // k is one-based. Returns "" for an invalid k.
    // By default, equal substrings appear once per occurrence.
    string kthSmallest(long long k, bool countRepetitions = true) {
        precompute();
        const auto& ways = countRepetitions ? extensionOccurrenceSum :
            pathCountFromState;
        long long total = countRepetitions ? extensionOccurrenceSum[0] :
             pathCountFromState[0] - 1;
        if (k < 1 || k > total) return "";

        string answer;
        int v = 0;
        while (true) {
            for (auto [c, u] : st[v].next) {
                if (k > ways[u]) {
                    k -= ways[u];
                } else {
                    answer += c;
                    long long copies = countRepetitions ?
                        occurrenceCount[u] : 1;
                    if (k <= copies) return answer;
                    k -= copies;
                    v = u;
                    break;
                }
            }
        }
    }

    string kthSmallestDistinct(long long k) {
        return kthSmallest(k, false);
    }

    // Returns sorted, zero-based starting positions without duplicates.
    vector<int> allOccurrences(const string& s) {
        if (s.empty()) {
            vector<int> result(st[last].len + 1);
            iota(result.begin(), result.end(), 0);
            return result;
```

```
        }
        int v = findState(s);
        if (v == -1) return {};
        precompute();

        vector<int> result, pending = {v};
        while (!pending.empty()) {
            int u = pending.back();
            pending.pop_back();
            if (!st[u].isClone)
                result.push_back(st[u].firstposition - (int)s.size() +
                    1);
            for (int child : st[u].inv_link) pending.push_back(child);
        }
        sort(result.begin(), result.end());
        return result;
    }

    // answer[i] = longest substring ending at t[i] that occurs in the
        built string.
    vector<int> longestEndingAtEachIndex(const string& t) const {
        vector<int> answer(t.size());
        int v = 0, length = 0;

        for (int i = 0; i < (int)t.size(); ++i) {
            char c = t[i];

            while (v && !st[v].next.count(c)) {
                v = st[v].link;
                length = st[v].len;
            }

            auto it = st[v].next.find(c);
            if (it != st[v].next.end()) {
                v = it->second;
                ++length;
            } else {
                v = 0;
                length = 0;
            }

            answer[i] = length;
        }
        return answer;
    }

    // Longest substring with at least k occurrences; overlaps are
        allowed.
    // O(n) after precompute(). Ties may return any longest substring.
    string longestRepeatedSubstring(int k = 2) {
        precompute();
        int best = 0;
        for (int v = 1; v < sz; ++v) {
            if (occurrenceCount[v] >= k && st[v].len > st[best].len)
                best = v;
        }
        if (best == 0) return "";
        return text.substr(st[best].firstposition - st[best].len + 1, st
            [best].len);
    }

    // answer[k] = number of distinct nonempty substrings occurring
        exactly k times.
    // answer[0] = 0. O(n) after precompute().
    vector<long long> substringFrequencyHistogram() {
        precompute();
        vector<long long> answer(text.size() + 1, 0);
        for (int v = 1; v < sz; ++v)
            answer[occurrenceCount[v]] += st[v].len - st[st[v].link].len
                ;
        return answer;
    }

    // answer[length] = number of distinct substrings of that length.
    // answer[0] = 0. O(n); does not need precompute().
    vector<long long> distinctSubstringCountsByLength() const {
        vector<long long> answer(text.size() + 2, 0);
        for (int v = 1; v < sz; ++v) {
            ++answer[st[st[v].link].len + 1];
            --answer[st[v].len + 1];
        }
        for (int length = 1; length <= (int)text.size(); ++length)
            answer[length] += answer[length - 1];
        answer.pop_back();
        return answer;
```

```
    }

    // answer[i] = longest substring starting at t[i] that occurs in the built string.
    // Builds a reversed automaton each call: O(n + |t|) for a fixed alphabet.
    vector<int> longestStartingAtEachIndex(const string& t) const {
        SuffixAutomaton reversed(string(text.rbegin(), text.rend()));
        vector<int> answer = reversed.longestEndingAtEachIndex(string(t.rbegin(), t.rend()));
        reverse(answer.begin(), answer.end());
        return answer;
    }

    // Shortest absent nonempty string over alphabet. Ties use lexicographic order.
    // Returns "" for an empty alphabet. O(n * |alphabet|) for a fixed alphabet.
    string shortestAbsentSubstring(string alphabet = "abcdefghijklmnopqrstuvwxyz") const {
        sort(alphabet.begin(), alphabet.end());
        alphabet.erase(unique(alphabet.begin(), alphabet.end()), alphabet.end());
        if (alphabet.empty()) return "";

        vector<int> order = statesByLength(), distance(sz, INT_MAX);
        vector<char> choice(sz);
        for (int i = sz - 1; i >= 0; --i) {
            int v = order[i];
            for (char c : alphabet) {
                auto it = st[v].next.find(c);
                int candidate = it == st[v].next.end() ? 1 : 1 + distance[it->second];
                if (candidate < distance[v]) {
                    distance[v] = candidate;
                    choice[v] = c;
                }
            }
        }

        string answer;
        int v = 0;
        while (true) {
            char c = choice[v];
            answer += c;
            auto it = st[v].next.find(c);
            if (it == st[v].next.end()) return answer;
            v = it->second;
        }
    }

    // Shortest substring occurring exactly once. Returns "" for an empty text.
    // O(n) after precompute(); ties may return any shortest substring.
    string shortestUniqueSubstring() {
        precompute();
        int bestState = 0, bestLength = (int)text.size() + 1;
        for (int v = 1; v < sz; ++v) {
            int length = st[st[v].link].len + 1;
            if (occurrenceCount[v] == 1 && length < bestLength) {
                bestState = v;
                bestLength = length;
            }
        }
        if (bestState == 0) return "";
        return text.substr(st[bestState].firstposition - bestLength + 1, bestLength);
    }

    // Helper: longest matching substring length in each state, including suffixes.
    vector<int> matchLengthsByState(const string& t, const vector<int>& order) const {
        vector<int> matches(sz, 0);
        int v = 0, length = 0;
        for (char c : t) {
            while (v && !st[v].next.count(c)) {
                v = st[v].link;
                length = st[v].len;
            }
            auto it = st[v].next.find(c);
            if (it != st[v].next.end()) {
                v = it->second;
                ++length;
                matches[v] = max(matches[v], length);
            } else {
```

```
                v = 0;
                length = 0;
            }
        }
        for (int i = sz - 1; i > 0; --i) {
            int u = order[i], p = st[u].link;
            matches[p] = max(matches[p], min(matches[u], st[p].len));
        }
        return matches;
    }

    // Number of distinct nonempty substrings shared by text and t.
    // O(n + |t|) for a fixed alphabet.
    long long countCommonDistinctSubstrings(const string& t) const {
        vector<int> order = statesByLength();
        vector<int> matches = matchLengthsByState(t, order);
        long long answer = 0;
        for (int v = 1; v < sz; ++v)
            answer += max(0, matches[v] - st[st[v].link].len);
        return answer;
    }

    // answer[length] = number of distinct shared substrings of that
        length.
    // answer has text.size() + 1 entries; answer[0] = 0.
    // O(n + |t|) for a fixed alphabet.
    vector<long long> countCommonDistinctSubstringsByLength(const string
        & t) const {
        vector<int> order = statesByLength();
        vector<int> matches = matchLengthsByState(t, order);
        vector<long long> answer(text.size() + 2, 0);
        for (int v = 1; v < sz; ++v) {
            int minimumLength = st[st[v].link].len + 1;
            if (matches[v] < minimumLength) continue;
            ++answer[minimumLength];
            --answer[matches[v] + 1];
        }
        for (int length = 1; length <= (int)text.size(); ++length)
            answer[length] += answer[length - 1];
        answer.pop_back();
        return answer;
    }

    // Shortest nonempty substring of text absent from t; "" if none
        exists.
    // O(n + |t|) for a fixed alphabet. Ties may return any shortest
        substring.
    string shortestDistinguishingSubstring(const string& t) const {
        vector<int> order = statesByLength();
        vector<int> matches = matchLengthsByState(t, order);
        int bestState = 0, bestLength = 0;
        for (int v = 1; v < sz; ++v) {
            if (matches[v] == st[v].len) continue;
            // This state's lengths above matches[v] are absent from t.
            int length = max(st[st[v].link].len + 1, matches[v] + 1);
            if (bestState == 0 || length < bestLength) {
                bestState = v;
                bestLength = length;
            }
        }
        if (bestState == 0) return "";
        return text.substr(st[bestState].firstposition - bestLength + 1,
             bestLength);
    }

    // Minimum pieces whose concatenation is t, each a substring of text
        .
    // Returns 0 for empty t, -1 if impossible. O(|t|) for a fixed
        alphabet.
    int minimumSubstringPieces(const string& t) const {
        if (t.empty()) return 0;
        // Greedy longest pieces are optimal because valid pieces are
            suffix-closed.
        int pieces = 1, v = 0;
        for (char c : t) {
            auto it = st[v].next.find(c);
            if (it == st[v].next.end()) {
                it = st[0].next.find(c);
                if (it == st[0].next.end()) return -1;
                ++pieces;
            }
            v = it->second;
        }
        return pieces;
    }
```

```
    // Sum of occurrences_in_text(p) * occurrences_in_t(p) over all nonempty p.
    // Overlaps count. The result can exceed 64 bits, so it returns __int128.
    // O(n + |t|) for a fixed alphabet, including precompute().
    __int128 countCommonSubstringOccurrencePairs(const string& t) {
        precompute();
        vector<int> order = statesByLength();
        vector<long long> suffixOccurrenceSum(sz, 0);
        for (int v : order) {
            if (v == 0) continue;
            int p = st[v].link;
            suffixOccurrenceSum[v] = suffixOccurrenceSum[p]
                + occurrenceCount[v] * (st[v].len - st[p].len);
        }

        __int128 answer = 0;
        int v = 0, length = 0;
        for (char c : t) {
            while (v && !st[v].next.count(c)) {
                v = st[v].link;
                length = st[v].len;
            }
            auto it = st[v].next.find(c);
            if (it != st[v].next.end()) {
                v = it->second;
                ++length;
                int p = st[v].link;
                // Include all shorter states and only the matched part of v.
                answer += suffixOccurrenceSum[p]
                    + occurrenceCount[v] * (length - st[p].len);
            } else {
                v = 0;
                length = 0;
            }
        }
        return answer;
    }

    // Total occurrences in text of all DISTINCT cyclic rotations of pattern.
    // Returns 0 for an empty pattern. Overlapping occurrences count.
    // O(n + |pattern|) for a fixed alphabet, including precompute().
    long long countCyclicRotationMatches(const string& pattern) {
        if (pattern.empty() || pattern.size() > text.size()) return 0;
        precompute();
        vector<char> seen(sz, false);
        int m = (int)pattern.size(), v = 0, length = 0;
        long long answer = 0;

        // The first 2*m-1 characters of pattern + pattern cover all rotations.
        for (int pass = 0; pass < 2; ++pass) {
            int limit = m - (pass == 1);
            for (int i = 0; i < limit; ++i) {
                char c = pattern[i];
                while (v && !st[v].next.count(c)) {
                    v = st[v].link;
                    length = st[v].len;
                }
                auto it = st[v].next.find(c);
                if (it == st[v].next.end()) {
                    v = 0;
                    length = 0;
                    continue;
                }
                v = it->second;
                ++length;
                if (length < m) continue;

                length = m;
                // Use the state representing this length, not a longer match.
                while (st[st[v].link].len >= m) v = st[v].link;
                if (!seen[v]) {
                    seen[v] = true;
                    answer += occurrenceCount[v];
                }
            }
        }
        return answer;
    }

    // Longest substring shared by text and EVERY string in strings.
    // An empty vector returns text. An empty string in it returns "".
```

```
    // O((strings.size() + 1) * n + total input length) for a fixed alphabet.
    string longestCommonSubstringMultiple(const vector<string>& strings) const {
        vector<int> order = statesByLength(), common(sz);
        for (int v = 0; v < sz; ++v) common[v] = st[v].len;
        for (const string& t : strings) {
            if (t.empty()) return "";
            vector<int> matches = matchLengthsByState(t, order);
            for (int v = 0; v < sz; ++v) common[v] = min(common[v], matches[v]);
        }

        int bestState = 0, bestLength = 0;
        for (int v = 1; v < sz; ++v) {
            if (common[v] > bestLength) {
                bestState = v;
                bestLength = common[v];
            }
        }
        if (bestLength == 0) return "";
        return text.substr(st[bestState].firstposition - bestLength + 1, bestLength);
    }

    // Longest substring with two occurrences that do not overlap. O(n).
    // Returns "" if none exists; ties may return any longest substring.
    string longestNonOverlappingRepeat() const {
        vector<int> order = statesByLength();
        vector<int> minEnd(sz, text.size()), maxEnd(sz, -1);
        for (int v = 1; v < sz; ++v) {
            if (!st[v].isClone)
                minEnd[v] = maxEnd[v] = st[v].firstposition;
        }
        for (int i = sz - 1; i > 0; --i) {
            int v = order[i], p = st[v].link;
            minEnd[p] = min(minEnd[p], minEnd[v]);
            maxEnd[p] = max(maxEnd[p], maxEnd[v]);
        }

        int bestState = 0, bestLength = 0;
        for (int v = 1; v < sz; ++v) {
            int length = min(st[v].len, maxEnd[v] - minEnd[v]);
            if (length > st[st[v].link].len && length > bestLength) {
                bestState = v;
                bestLength = length;
            }
        }
        if (bestLength == 0) return "";
        return text.substr(st[bestState].firstposition - bestLength + 1, bestLength);
    }

    // Longest common substring of the built string and t.
    string lcs(const string& t) const {
        int v = 0, length = 0, best = 0;
        size_t bestpos = 0;
        for (size_t i = 0; i < t.size(); ++i) {
            while (v && !st[v].next.count(t[i])) {
                v = st[v].link;
                length = st[v].len;
            }
            auto it = st[v].next.find(t[i]);
            if (it != st[v].next.end()) {
                v = it->second;
                ++length;
            } else {
                v = 0;
                length = 0;
            }
            if (length > best) {
                best = length;
                bestpos = i;
            }
        }
        return best == 0 ? string{} : t.substr(bestpos - (best - 1), best);
    }
};
```

14.16 Trie Ahocorasick

```
/*
                                Trie - AhoCorasick
 ------------------------------------------------------------------------
```

```
  Builds a trie for subset of strings and computes suffix links.
  KATCL implementation is cleaner.

  Time Complexity(Construction): O(m) where m is sum
   of lengths of strings
  Space Complexity: O(m)
*/



const int K = 26;

struct Vertex {
    int next[K];
    bool output = false;
    int p = -1;
    char pch;
    int link = -1;
    int go[K];

    Vertex(int p=-1, char ch='$') : p(p), pch(ch) {
        fill(begin(next), end(next), -1);
        fill(begin(go), end(go), -1);
    }
};

vector<Vertex> t(1);

void add_string(string const& s) {
    int v = 0;
    for (char ch : s) {
        int c = ch - 'a';
        if (t[v].next[c] == -1) {
            t[v].next[c] = t.size();
            t.emplace_back(v, ch);
        }
        v = t[v].next[c];
    }
    t[v].output = true;
}

int go(int v, char ch);

int get_link(int v) {
    if (t[v].link == -1) {
        if (v == 0 || t[v].p == 0)
            t[v].link = 0;
        else
            t[v].link = go(get_link(t[v].p), t[v].pch);
    }
    return t[v].link;
}

int go(int v, char ch) {
    int c = ch - 'a';
    if (t[v].go[c] == -1) {
        if (t[v].next[c] != -1)
            t[v].go[c] = t[v].next[c];
        else
            t[v].go[c] = v == 0 ? 0 : go(get_link(v), ch);
    }
    return t[v].go[c];
}
```

14.17 Z Function

```
/*
                              Z_function
  ----------------------------------------------------------------
  Computes the z_function for any string.
  ith element is equal to the greatest number of characters starting
      from the position i that coincide with the first characters of s

  z[i] length of the longest string that is, at the same time, a prefix
      of s and a prefix of the suffix of $s$ starting at  i.

  to compress string, one can run z_function and then find the smallest
      i that divides n such that i + z[i] = n

  Time Complexity: O(n)
  Space Complexity: O(n)
*/


vector<int> z_function(string s) {
  int n = s.size();
```

```
  vector<int> z(n);
  int l = 0, r = 0;
  for(int i = 1; i < n; i++) {
    if(i < r) {
      z[i] = min(r - i, z[i - l]);
    }
    while(i + z[i] < n && s[z[i]] == s[i + z[i]]) {
      z[i]++;
    }
    if(i + z[i] > r) {
      l = i;
      r = i + z[i];
    }
  }
  return z;
}

// usage
// vector<int> z = z_function("abacaba");
// this will return {0, 0, 1, 0, 3, 0, 1}
// vector<int> z = z_function("aaaaa");
// this will return {0, 4, 3, 2, 1}
// vector<int> z = z_function("aaabaab");
// this will return {0, 2, 1, 0, 2, 1, 0}
```

15 Trees

15.1 Centroid Decomposition

```
/*
                        Centroid Decomposition
  ------------------------------------------------------------------
  Finds the centroid decomposition of a given tree.
  Any vertex can have at most log n centroid ancestors

  The code below is the solution to Xenia and tree.
  Given tree, queries of two types:
  1) u - color vertex u
  2) v - print minimum distance of vertex v to any colored vertex before


  Time Complexity: O(n log n)
  Space Complexity: O(n log n)
*/
const int MAXN=200005;

vector<int> adj[MAXN];
vector<bool> is_removed(MAXN, false);
vector<int> subtree_size(MAXN, 0);
vector<int> dis(MAXN, 1e9);
vector<vector<pair<int, int>>> ancestor(MAXN);

int get_subtree_size(int node, int parent = -1) {
  subtree_size[node] = 1;
  for (int child : adj[node]) {
    if (child == parent || is_removed[child]) { continue; }
    subtree_size[node] += get_subtree_size(child, node);
  }
  return subtree_size[node];
}

int get_centroid(int node, int tree_size, int parent = -1) {
  for (int child : adj[node]) {
    if (child == parent || is_removed[child]) { continue; }
    if (subtree_size[child] * 2 > tree_size) {
      return get_centroid(child, tree_size, node);
    }
  }
  return node;
}

void getDist(int cur, int centroid, int p=-1, int dist=1){
    for (int child:adj[cur]){
        if(child==p ||is_removed[child])
            continue;
        dist++;
        getDist(child, centroid, cur, dist);
        dist--;
    }
    ancestor[cur].push_back(make_pair(centroid, dist));
}

void update(int cur){
    for (int i=0;i<ancestor[cur].size();i++){
        dis[ancestor[cur][i].first]=min(dis[ancestor[cur][i].first],
            ancestor[cur][i].second);
```

```
    }
    dis[cur]=0;
}

int query(int cur){
    int mini=dis[cur];
    for (int i=0;i<ancestor[cur].size();i++){
       mini=min(mini, ancestor[cur][i].second+dis[ancestor[cur][i].first
           ]);
    }
    return mini;
}

void build_centroid_decomp(int node = 1) {
  int centroid = get_centroid(node, get_subtree_size(node));

  for (int child : adj[centroid]) {
    if (is_removed[child]) { continue; }
    getDist(child, centroid, centroid);
  }

  is_removed[centroid] = true;

  for (int child : adj[centroid]) {
    if (is_removed[child]) { continue; }
    build_centroid_decomp(child);
  }
}
```

15.2 HLD subtree query

```
//hld subtree queries
// path is [in[nextv]....in[v]]
void dfs_sz(int v = 0) {
    sz[v] = 1;
    for(auto &u: g[v]) {
        dfs_sz(u);
        sz[v] += sz[u];
        if(sz[u] > sz[g[v][0]]) {
            swap(u, g[v][0]);
        }
    }
}

void dfs_hld(int v = 0) {
    in[v] = t++;
    for(auto u: g[v]) {
        nxt[u] = (u == g[v][0] ? nxt[v] : u);
        dfs_hld(u);
    }
    out[v] = t;
}
```

15.3 Heavy Light Decomposition

```
/*
                    Heavy Light Decomposition(HLD)
  ---------------------------------------------------------------------
  Constructs the heavy light decomposition of a tree

  Splits the tree into several paths so that we can reach the root
      vertex from any v  by traversing at most log n paths.
  In addition, none of these paths intersect with another.

  Time Complexity(Creation): O(n log n)
  Time Complexity(Query): O((log n) ^ 2) usually, depending on the query
       itself
  Space Complexity: O(n)
*/

//call dfs1 first
struct SegmentTree {
  vector<ll> a;
  int n;

  SegmentTree(int _n) : a(2 * _n, 0), n(_n) {}

  void update(int pos, ll val) {
    for (a[pos += n] = val; pos > 1; pos >>= 1) {
      a[pos / 2] = (a[pos]^a[pos ^ 1]);
    }
  }

  ll get(int l, int r) {
    ll res = 0;
    for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
```

```
      if (l & 1) {
        res^=a[l++];
      }
      if (r & 1) {
        res^=a[--r];
      }
    }
    return res;
  }
};


const int MAXN=500005;
vector<int> adj[MAXN];
SegmentTree st(MAXN);
int a[MAXN], sz[MAXN], to[MAXN], dpth[MAXN], s[MAXN], par[MAXN];
int cnt=0;

void dfs1(int cur, int p){
    sz[cur]=1;
    for(int x:adj[cur]){
        if(x==p) continue;
        dpth[x]=dpth[cur]+1;
        par[x]=cur;
        dfs1(x, cur);
        sz[cur]+=sz[x];
    }
}

void dfs(int cur, int p, int l){
    st.update(cnt, a[cur]);
    s[cur]=cnt++;
    to[cur]=l;
    int g=-1;
    for(int x:adj[cur]){
        if(x==p) continue;
        if(g==-1 || sz[g]<sz[x]){
            g=x;
        }
    }
    if(g==-1) return;
    dfs(g, cur, l);
    for(int x:adj[cur]){
        if(x==p || x==g) continue;
        dfs(x, cur, x);
    }
}

int query(int u, int v){
    int res=0;
    while(to[u]!=to[v]){
        if(dpth[to[u]]<dpth[to[v]]) swap(u, v);
        res^=st.get(s[to[u]], s[u]+1);
        u=par[to[u]];
    }
    if(dpth[u]>dpth[v]) swap(u, v);
    res^=st.get(s[u], s[v]+1);
    return res;
}

// dfs1(0, -1);
// dfs(0, -1, -1);



//alternate implementation
vector<int> parent, depth, heavy, head, pos;
int cur_pos;

int dfs(int v, vector<vector<int>> const& adj) {
    int size = 1;
    int max_c_size = 0;
    for (int c : adj[v]) {
        if (c != parent[v]) {
            parent[c] = v, depth[c] = depth[v] + 1;
            int c_size = dfs(c, adj);
            size += c_size;
            if (c_size > max_c_size)
                max_c_size = c_size, heavy[v] = c;
        }
    }
    return size;
}

void decompose(int v, int h, vector<vector<int>> const& adj) {
    head[v] = h, pos[v] = cur_pos++;
```

```
    if (heavy[v] != -1)
        decompose(heavy[v], h, adj);
    for (int c : adj[v]) {
        if (c != parent[v] && c != heavy[v])
            decompose(c, c, adj);
    }
}

void init(vector<vector<int>> const& adj) {
    int n = adj.size();
    parent = vector<int>(n);
    depth = vector<int>(n);
    heavy = vector<int>(n, -1);
    head = vector<int>(n);
    pos = vector<int>(n);
    cur_pos = 0;

    dfs(0, adj);
    decompose(0, 0, adj);
}

int query(int a, int b) {
    int res = 0;
    for (; head[a] != head[b]; b = parent[head[b]]) {
        if (depth[head[a]] > depth[head[b]])
            swap(a, b);
        int cur_heavy_path_max = segment_tree_query(pos[head[b]], pos[b
            ]);
        res = max(res, cur_heavy_path_max);
    }
    if (depth[a] > depth[b])
        swap(a, b);
    int last_heavy_path_max = segment_tree_query(pos[a], pos[b]);
    res = max(res, last_heavy_path_max);
    return res;
}
```

15.4 Kruskal Reconstruction Tree

```
#include <bits/stdc++.h>
using namespace std;

constexpr int MAX_N = 4e5 + 5;
constexpr int LG = 20;

int n, m, q, va[MAX_N], f[MAX_N], nx;
int trace(int v) { return f[v] == v ? v : f[v] = trace(f[v]); }

/** Implements LCA with binary lifting */
namespace LCA {
int lift[MAX_N][LG], ch[MAX_N][2], t, tin[MAX_N], tout[MAX_N], tour[2 *
    MAX_N];

bool is_ancestor(int u, int v) { return tin[u] <= tin[v] && tout[u] >=
    tout[v]; }

int lca(int u, int v) {
  if (is_ancestor(u, v)) { return u; }
  if (is_ancestor(v, u)) { return v; }

  for (int i = LG - 1; i >= 0; --i) {
    if (!is_ancestor(lift[u][i], v)) { u = lift[u][i]; }
  }

  return lift[u][0];
}

void dfs(int v) {
  if (v == -1) { return; }
  tour[t] = v;
  tin[v] = t;

  for (int i = 1; i < LG; i++) { lift[v][i] = lift[lift[v][i - 1]][i -
      1]; }

  dfs(ch[v][0]);
  dfs(ch[v][1]);
  tout[v] = t++;
}
}  // namespace LCA

/** Point Update / Range Min/Max Segment Tree */
namespace SGT {
int sg_min[1 << LG + 1], sg_max[1 << LG + 1];

void point_set(int i, int v) {
```

```
  sg_min[i + (1 << LG)] = sg_max[i + (1 << LG)] = v;
  for (int j = (i + (1 << LG)) / 2; j; j /= 2) {
    sg_min[j] = min(sg_min[2 * j], sg_min[2 * j + 1]),
    sg_max[j] = max(sg_max[2 * j], sg_max[2 * j + 1]);
  }
}

/** @return edge (LCA of nodes w/ lowest and highest traversal time in [l, r]) */
int query(int l, int r) {
  int lift = MAX_N, rt = -1;
  for (l += 1 << LG, r += 1 << LG; l < r; l >>= 1, r >>= 1) {
    if (l & 1) { lift = min(lift, sg_min[l]), rt = max(rt, sg_max[l++]); }
    if (r & 1) { lift = min(lift, sg_min[--r]), rt = max(rt, sg_max[r]); }
  }
  return va[LCA::lca(LCA::tour[lift], LCA::tour[rt])];
}
}  // namespace SGT

void solve() {
  cin >> n >> m >> q;

  iota(f, f + 2 * n, 0);
  nx = n;
  memset(LCA::ch, -1, sizeof(LCA::ch));

  for (int i = 0; i < m; i++) {
    int u, v;
    cin >> u >> v;
    // assign u and v to the top of their component
    u = trace(--u), v = trace(--v);

    // if the edge is within one component, no further work is needed
    // otherwise, create a new node that is the parent of both components

    if (u == v) { continue; }
    va[nx] = i, LCA::ch[nx][0] = u, LCA::ch[nx][1] = v;
    f[u] = f[v] = LCA::lift[u][0] = LCA::lift[v][0] = nx++;
  }

  LCA::lift[2 * n - 2][0] = 2 * n - 2;
  LCA::dfs(2 * n - 2);

  // update segment tree for range min/max of dfs-order traversal times
  for (int i = 0; i < 2 * n - 1; i++) { SGT::point_set(i, LCA::tin[i]); }

  while (q--) {
    int l, r;
    cin >> l >> r;
    if (l == r) {
      cout << "0␣";
    } else {
      cout << SGT::query(l - 1, r) + 1 << "␣";
    }
  }

  cout << "\n";
}

int main() {
  int test_num;
  cin >> test_num;
  for (int t = 0; t < test_num; t++) { solve(); }
}
```

15.5 Lowest Common Ancestor (LCA)

```
/*
                        LCA(Lowest Common Ancestor)
  ------------------------------------------------------------------------
  Computes the lowest common ancestor of two vertices in a tree.

  Be careful as implementation is indexed starting with 1

  Time Complexity(Creation): O(n log n)
  Time Complexity(Query): O(log n)
  Space Complexity: O(n log n)
*/

const int N=200005;
vector<int> adj[N];
vector<int> start(N), end1(N), depth(N);
```

```cpp
vector<vector<int>> t(N, vi(32));
int timer=0;
int n, l;
// l=(int)ceil(log2(n))
// call dfs(1, 1, 0)
// 1 indexed, dont use 0 indexing


void dfs(int cur, int p, int cnt){
    depth[cur]=cnt;
    t[cur][0]=p;
    start[cur]=timer++;
    for(int i=1;i<=l;i++){
        t[cur][i]=t[t[cur][i-1]][i-1];
    }
    for(int x:adj[cur]){
        if(x==p) continue;
        dfs(x, cur, cnt+1);
    }
    end1[cur]=++timer;
}

bool ancestor(int u, int v){
    return start[u]<=start[v] && end1[u]>=end1[v];
}

int lca(int u, int v){
    if(ancestor(u, v))
        return u;
    if (ancestor(v, u)){
        return v;
    }
    for(int i=l;i>=0;i--){
        if(!ancestor(t[u][i], v)){
            u=t[u][i];
        }
    }
    return t[u][0];
}
```

15.6 Tree Diameter

```cpp
/*
                    Tree Diameter
  ------------------------------------------------------------
  Finds the vertex most distant to vertex on which function is called.

  The first value is the vertex itself and the second value is the
      distance.

  To find diameter run algorithm twice, first on random vertex and then
      on the vertex that is farthest away.

  The vertex that is the farthest away from any vertex in tree must be
      an endpoint of the diameter.

  Time Complexity: O(n)
  Space Complexity: O(n)
*/

pair<int, int> dfs(const vector<vector<int>> &tree, int node = 1,
  int previous = 0, int length = 0) {
  pair<int, int> max_path = {node, length};
  for (const int &i : tree[node]) {
    if (i == previous) { continue; }
    pair<int, int> other = dfs(tree, i, node, length + 1);
    if (other.second > max_path.second) { max_path = other; }
  }
  return max_path;
}
```

15.7 Virtual Tree

```cpp
#include <bits/stdc++.h>
using namespace std;

const int MAXN = 2e5 + 10;
const int LOGN = 20;
const int MOD = 998244353;

vector<int> g[MAXN];          // Adjacency list of the given graph
vector<int> ver[MAXN];        // For each color c, ver[c] stores vertices
    with color c
vector<int> aux_g[MAXN];      // Adjacency list for the virtual tree
int col[MAXN];                // Color of each vertex
int tmr, n;                   // Global time counter and number of
```

```
    vertices
int up[LOGN][MAXN], dep[MAXN], tin[MAXN]; // For computing LCA and entry
    time

// DP arrays for dynamic programming on the virtual tree
int dp[MAXN][2], sum[MAXN];

// Preprocessing function: DFS to compute tin, up array, and dep
void dfs_precalc(int v, int p) {
    tin[v] = ++tmr;
    up[0][v] = p;
    dep[v] = dep[p] + 1;
    for (int i = 1; i < LOGN; ++i)
        up[i][v] = up[i - 1][up[i - 1][v]];
    for (auto to : g[v])
        if (to != p)
            dfs_precalc(to, v);
}

// Function to compute LCA using binary lifting
int getlca(int x, int y) {
    if (dep[x] < dep[y]) swap(x, y);
    for (int i = LOGN - 1; i >= 0; --i)
        if (dep[up[i][x]] >= dep[y])
            x = up[i][x];
    if (x == y) return x;
    for (int i = LOGN - 1; i >= 0; --i)
        if (up[i][x] != up[i][y]) {
            x = up[i][x];
            y = up[i][y];
        }
    return up[0][x];
}

// DFS on the virtual tree to perform DP.
// Parameter c - target color for which counting is performed.

void dfs_calc(int v, int p, int c, int &ans) {
    dp[v][0] = dp[v][1] = 0;
    sum[v] = 0;
    for(auto to : aux_g[v]) {
        if(to == p) continue;
        dfs_calc(to, v, c, ans);
        // DP transitions: combining current state with result from
            subtree.
        int nxt0 = (dp[v][0] + sum[to]) % MOD;
        int nxt1 = ((dp[v][0] + dp[v][1]) * 1ll * sum[to] % MOD + dp[v
            ][1]) % MOD;
        dp[v][0] = nxt0;
        dp[v][1] = nxt1;
    }
    sum[v] = (dp[v][0] + dp[v][1]) % MOD;
    if(col[v] == c) {
        // If the vertex has the target color, it can participate in a
            valid subtree.
        sum[v] = (sum[v] + 1) % MOD;
        ans = (ans + sum[v]) % MOD;
    } else {
        ans = (ans + dp[v][1]) % MOD;
    }
}

// Function to build a virtual tree for color c and perform DP.
void calc_aux(int c, int &ans) {
    auto p = ver[c];
    if (p.empty()) return;
    // Sort vertices by entry time (tin) - DFS traversal order.
    sort(p.begin(), p.end(), [&](const int a, const int b) { return tin[
        a] < tin[b]; });
    vector<int> verstk = {1}; // Initialize stack with the root of tree
        T (vertex 1).
    aux_g[1].clear();
    auto add = [&](int u, int v) {
        aux_g[u].push_back(v);
        aux_g[v].push_back(u);
    };
    // Process each vertex from set p, maintaining a stack to build the
        virtual tree.
    for (auto u : p) {
        if (u == 1) continue;
        int lca = getlca(u, verstk.back());
        if (lca != verstk.back()) {
            while (verstk.size() >= 2 && tin[lca] < tin[verstk[verstk.
                size() - 2]]) {
                add(verstk.back(), verstk[verstk.size() - 2]);
                verstk.pop_back();
```

```
            }
            if (verstk.size() >= 2 && tin[lca] != tin[verstk[verstk.size
                () - 2]]) {
                aux_g[lca].clear();
                add(verstk.back(), lca);
                verstk.back() = lca;
            } else {
                add(verstk.back(), lca);
                verstk.pop_back();
            }
        }
        aux_g[u].clear();
        verstk.push_back(u);
    }
    while (verstk.size() > 1) {
        add(verstk.back(), verstk[verstk.size() - 2]);
        verstk.pop_back();
    }
    // Perform DP on the virtual tree, starting from root 1.
    return dfs_calc(1, 0, c, ans);
}

int main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0);
    cin >> n;
    // Read colors and group vertices by color.
    for (int i = 1; i <= n; ++i) {
        cin >> col[i];
        ver[col[i]].push_back(i);
    }
    for (int i = 1; i < n; ++i) {
        int u, v;
        cin >> u >> v;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    dfs_precalc(1, 0);
    int ans = 0;
    // Process the corresponding virtual tree for each possible color.
    for (int i = 1; i <= n; ++i)
        calc_aux(i, ans);
    cout << ans;
    return 0;
}
```

16 Scripts

16.1 build.sh

This file should be called before stress.sh or validate.sh. build.sh name.cpp

```
g++ -static -DLOCAL -lm -s -x c++ -Wall -Wextra -O2 -std=c++17 -o $1 $1.
    cpp
```

16.2 stress.sh

Format is stress.sh Awrong Aslow Agen Numtests

```
#!/usr/bin/env bash

for ((testNum=0;testNum<$4;testNum++))
do
    ./$3 > input
    ./$2 < input > outSlow
    ./$1 < input > outWrong
    H1=`md5sum outWrong`
    H2=`md5sum outSlow`
    if !(cmp -s "outWrong" "outSlow")
    then
        echo "Error found!"
        echo "Input:"
        cat input
        echo "Wrong Output:"
        cat outWrong
        echo "Slow Output:"
        cat outSlow
        exit
    fi
done
echo Passed $4 tests
```

16.3 validate.sh

Format is validate.sh Awrong Avalidator Agen NumTests

```
#!/usr/bin/env bash
```

```
2
3  for ((testNum=0;testNum<$4;testNum++))
4  do
5      ./$3 > input
6      ./$1 < input > out
7      cat input out > data
8      ./$2 < data > res
9      result=$(cat res)
10     if [ "${result:0:2}" != "OK" ];
11     then
12         echo "Error␣found!"
13         echo "Input:"
14         cat input
15         echo "Output:"
16         cat out
17         echo "Validator␣Result:"
18         cat res
19         exit
20     fi
21 done
22 echo Passed $4 tests
```